

Databricks Certified Data Engineer Associate

Ultimate Preparation Course + Practice Exam





Ramesh Retnasamy

Data Engineer/ Machine Learning Engineer



<https://www.linkedin.com/in/ramesh-retnasamy/>

INSTRUCTOR

Ramesh Retnasamy

Senior Data Engineer/ Machine Learning Engineer

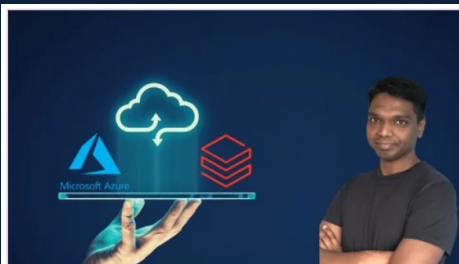
Udemy Instructor Partner

Total students

181,141

Reviews

39,927



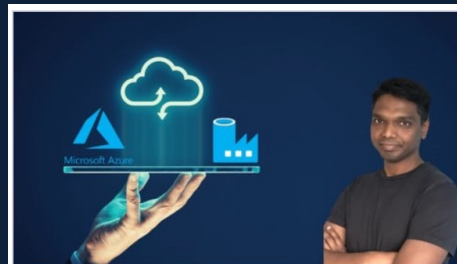
Azure Databricks & Spark For Data Engineers: Hands-on Project

Ramesh Retnasamy

4.6 ★★★★★ (20,616)

20 total hours · 190 lectures · All Levels

Bestseller



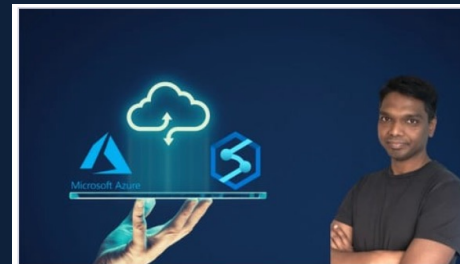
Azure Data Factory For Data Engineers - Project on Covid19

Ramesh Retnasamy

4.5 ★★★★★ (15,081)

13 total hours · 146 lectures · All Levels

Bestseller



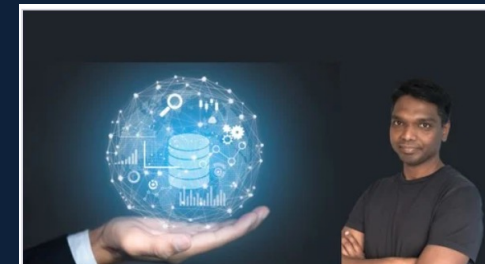
Azure Synapse Analytics For Data Engineers - Hands On Project

Ramesh Retnasamy

4.6 ★★★★★ (3,521)

13.5 total hours · 132 lectures · All Levels

Bestseller



Comprehensive SQL Course For Data Professionals [NEW]

Ramesh Retnasamy

4.7 ★★★★★ (420)

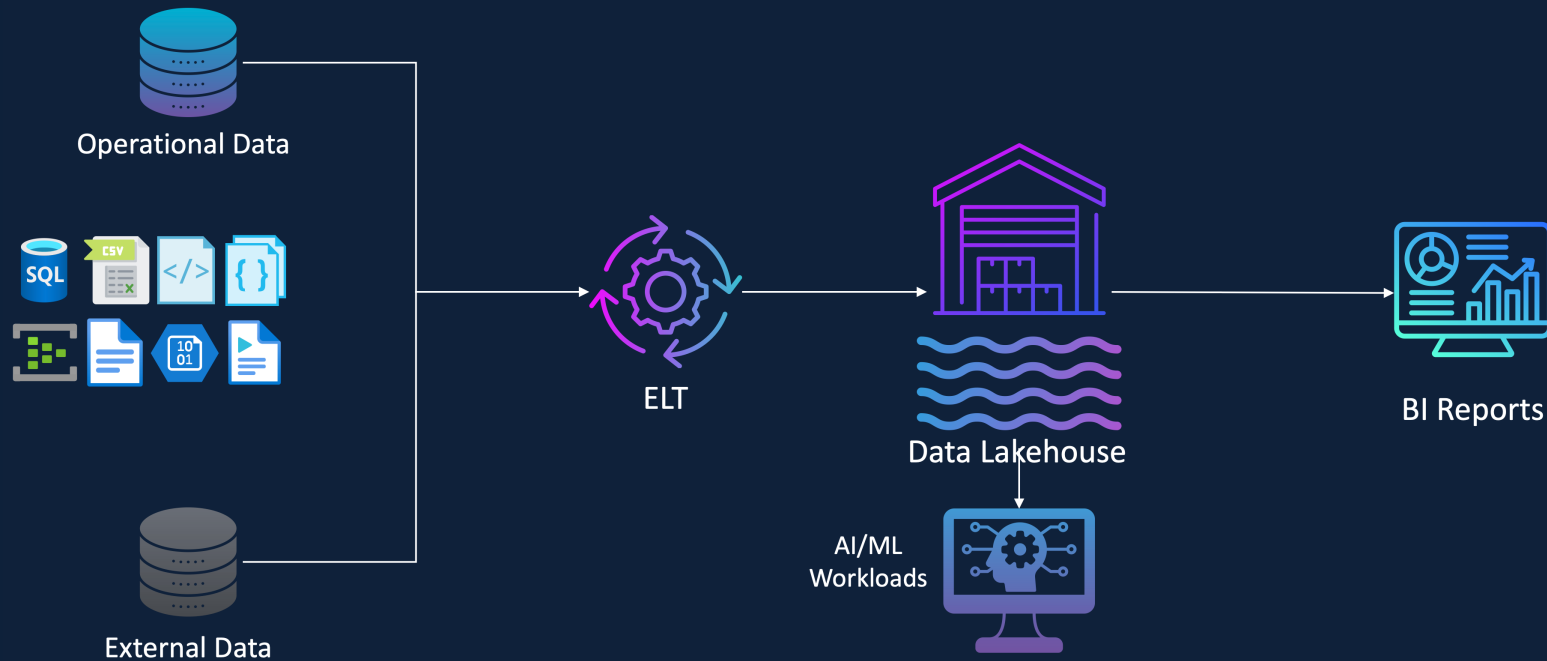
13.5 total hours · 130 lectures · All Levels

Databricks Certified Data Engineer Associate

Ultimate Preparation Course + Practice Exam



What is Data Lakehouse?



Handles All Types of Data

Cheap Cloud Object Storage

Uses Open File Format

Support for All Workloads

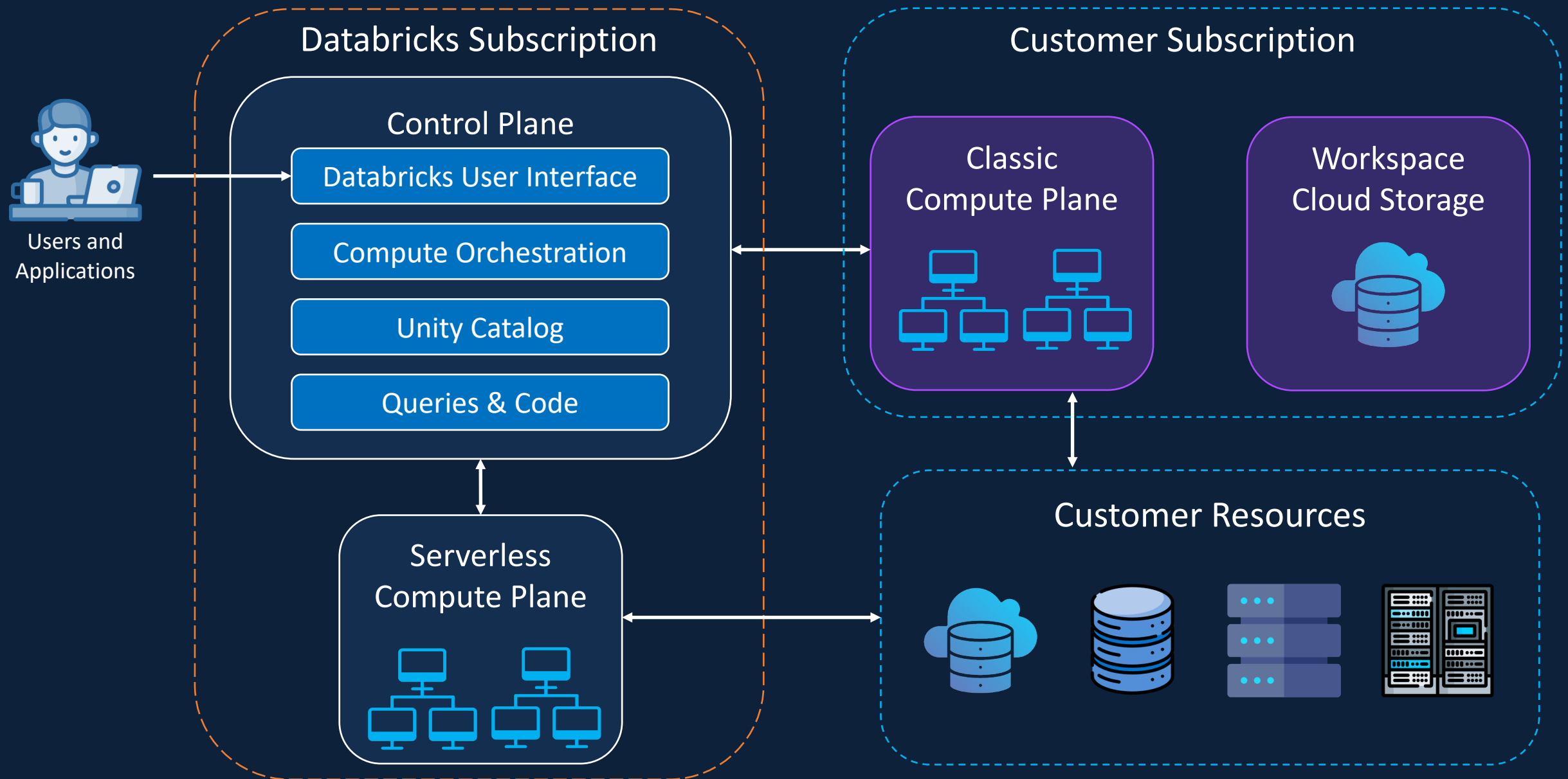
Direct BI Integration

ACID Support & Version History

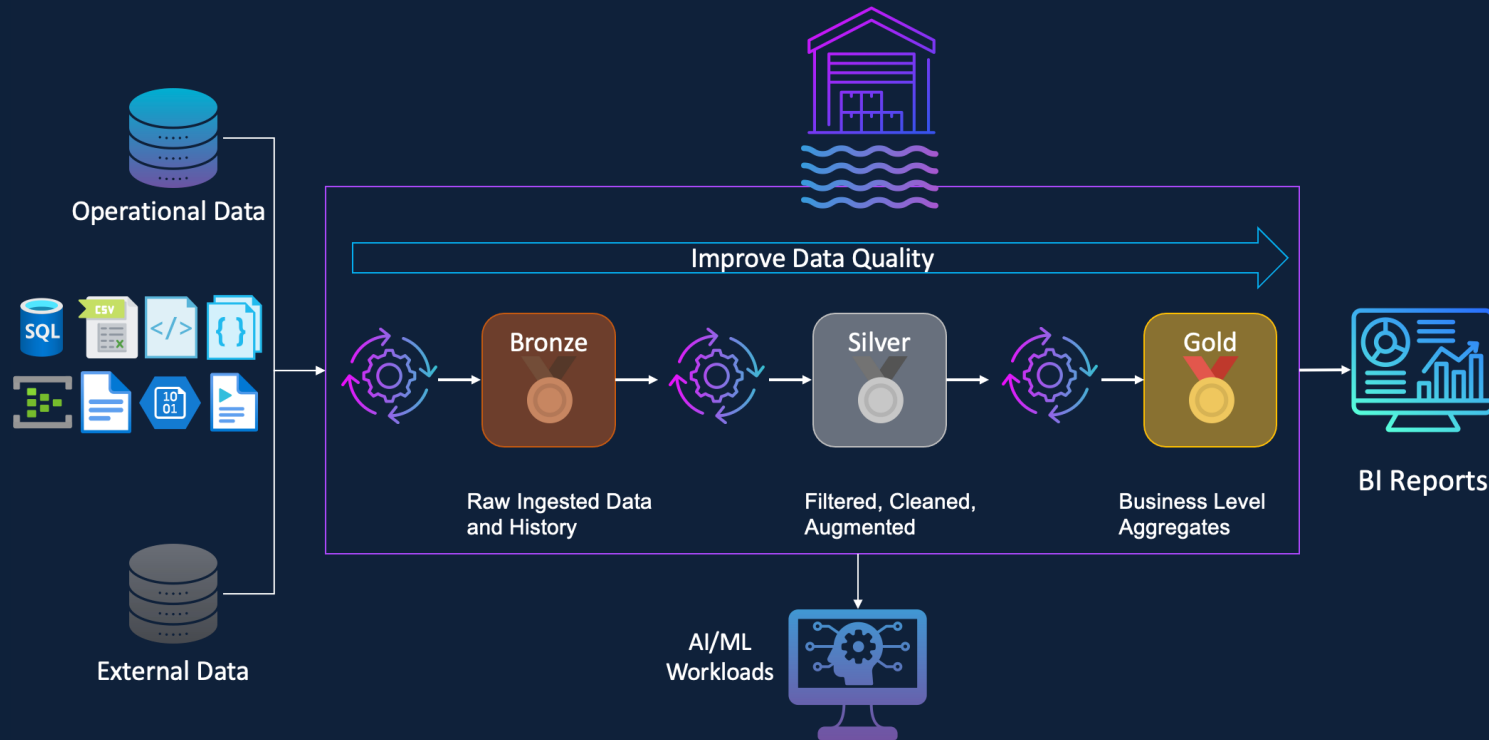
Improved Performance

Simple Architecture

Databricks Architecture



Medallion Architecture - Benefits



Data Lineage & Traceability

Data Governance & Compliance

Support for Incremental Processing

Enhanced Performance & Scalability

Data Security

Delta Live Tables (DLT) Architecture

Delta Live Tables (DLT)

User Interface

DLT Notebooks

SQL

Python



Data Transformations

Data Quality Expectations

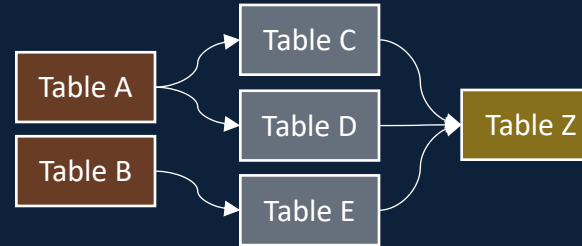
DLT Pipelines



Hive Metastore

Unity Catalog

DAG



Infrastructure



Metadata

Pipeline Status/ Health

Data Quality Checks

Data Lineage

Data Quality Checks

Dependency Management

Automated Checkpointing & Re-tries

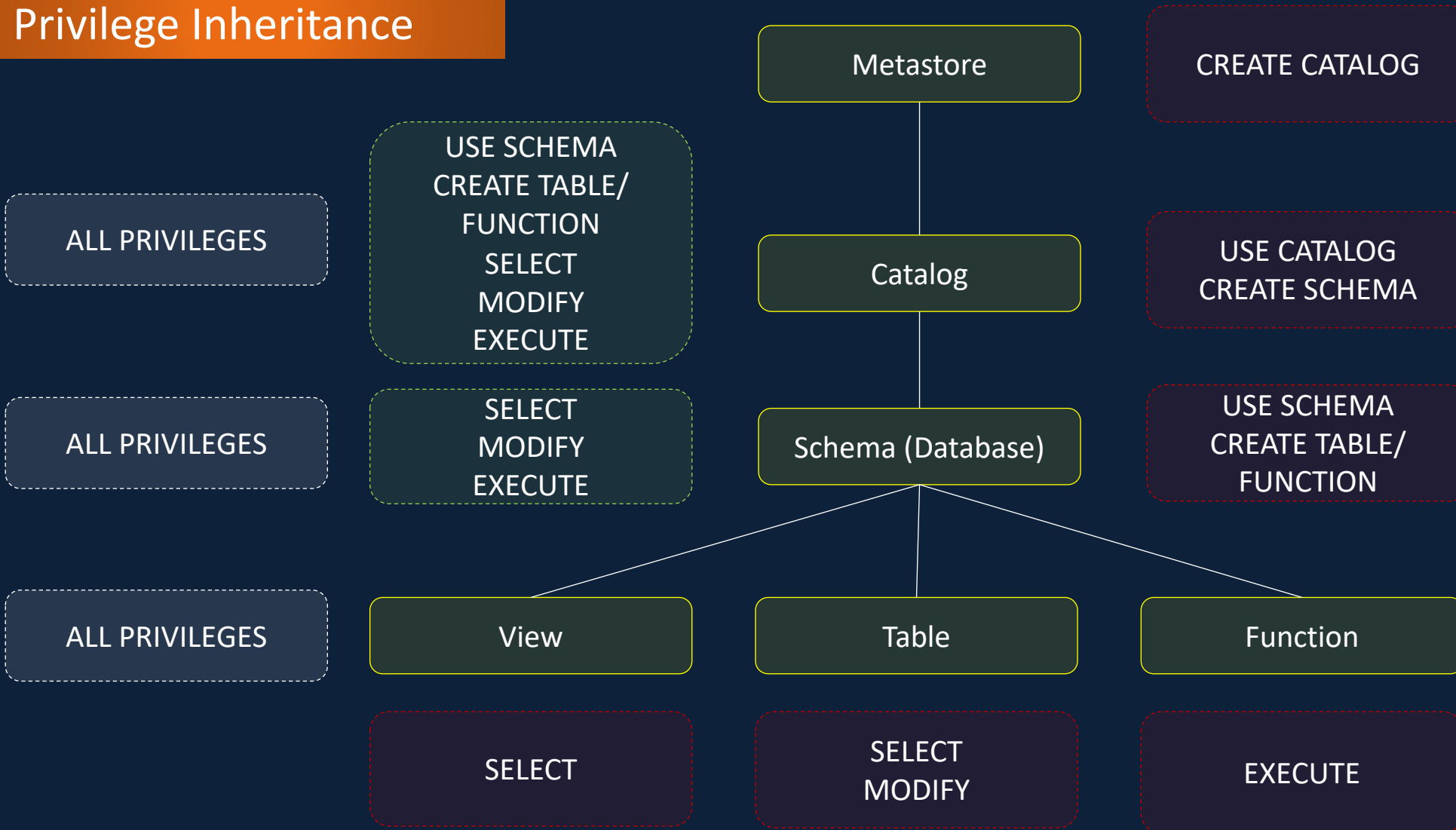
Incremental Processing

Infrastructure Management

Data Lineage

Unity Catalog - Security Model

Privilege Inheritance



Databricks SQL



Users and Applications

User Interface



SQL Editor



Dashboards



Genie



Alert

SQL Warehouse

Serverless



Provisioned



Data Lakehouse

Unity Catalog



Volume



Table



View



Function

Data Storage



ADLS



AWS S3



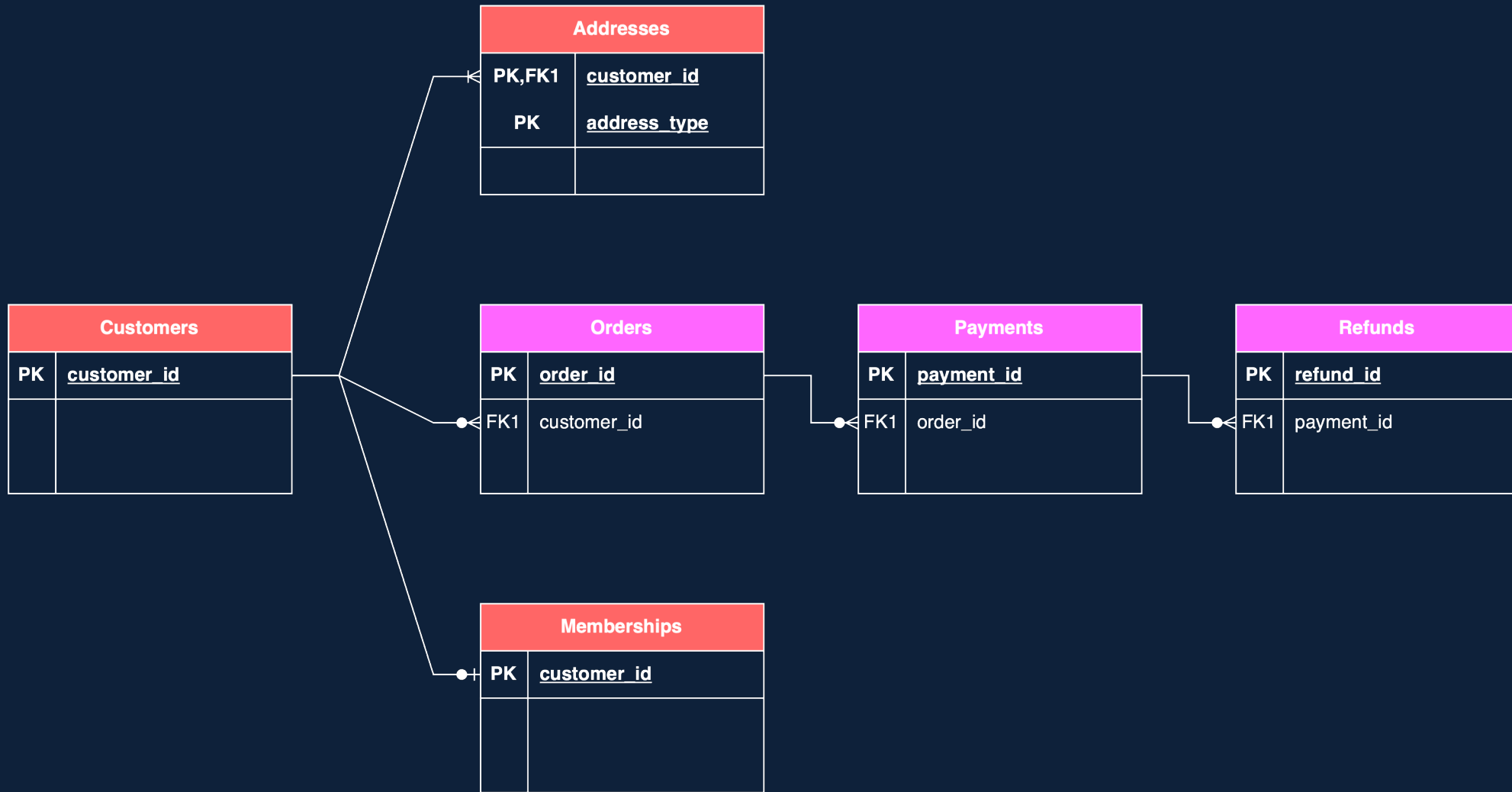
GCS



DELTA LAKE

**Tell me and I forget,
Teach me and I may remember,
Involve me and I learn!**
- Benjamin Franklin

Data Lakehouse Project - GizmoBox



Workspace

dea03-etl-with-apache-s...

01. Set-up Project Environment

02. Extract Customers Data - JSON

03. Extract Orders Data - Comple...

04. Extract Memberships Data - I...

05. Extract Addresses Data - CSV...

06. Extract Payments Data - CSV ...

07. Extract Refunds Data - via JD...

08. Extract Data using Python

09. Data Profiling

10. Transform Customers Data

11. Transform Payments Data

12. Transform Refunds Data

13. Transform Memberships Data

14. Transform Addresses Data

15. Query Orders Data - JSON Str...

16. Transform Orders Data - Strin...

17. Transform Orders Data - Explo...

18. Join Customer Address

19. Monthly Order Summary

20. User Defined Functions

Transform Addresses Data

Markdown

1. Create one record for each customer with 2 sets of address columns, 1 for shipping and 1 for billing address

2. Write transformed data to the Silver schema

▶ Dec 04, 2024 (3s) 2 SQL

SELECT customer_id, address_type, address_line_1, city, state, postcode FROM gizmo_box.bronze.v_addresses;

Table +

	customer_id	address_type	address_line_1	city	state	postcode	_rescue
1	9179	shipping	9713 Medina Mountains Apt. 813	Lake Erika	Mississippi	91945	null
2	9179	billing	084 Anne Hollow Apt. 064	East Jasontown	Minnesota	77329	null
3	5816	shipping	8400 Rebecca Lodge Suite 011	Lake Lisatown	Montana	11354	null
4	5816	billing	869 Linda Locks Apt. 105	West Melissabury	Michigan	37662	null
5	4858	shipping	4838 Jacob Neck	Jamesview	Wisconsin	58657	null
6							

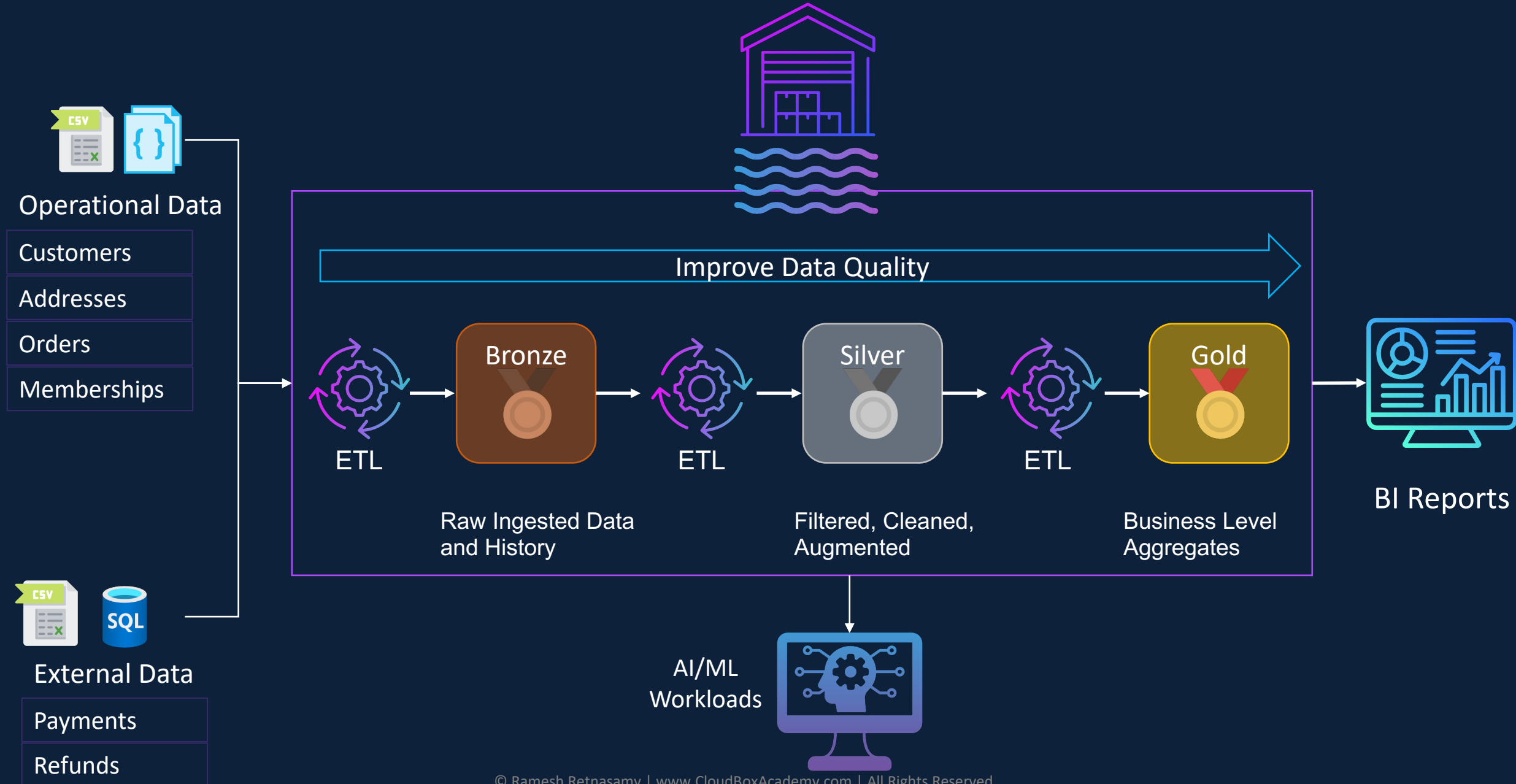
↓ 100 rows | 3.42s runtime

Refreshed 85 days ago

1. Create one record for each customer with both addresses, one for each address_type

[Documentation for PIVOT clause](#)

Data Architecture - GizmoBox



Workspace

dea03-etl-with-apache-s...

01. Set-up Project Environment

02. Extract Customers Data - JSON

03. Extract Orders Data - Comple...

04. Extract Memberships Data - I...

05. Extract Addresses Data - CSV...

06. Extract Payments Data - CSV ...

07. Extract Refunds Data - via JD...

08. Extract Data using Python

09. Data Profiling

10. Transform Customers Data

11. Transform Payments Data

12. Transform Refunds Data

13. Transform Memberships Data

14. Transform Addresses Data

15. Query Orders Data - JSON Str...

16. Transform Orders Data - Strin...

17. Transform Orders Data - Explo...

18. Join Customer Address

19. Monthly Order Summary

20. User Defined Functions

Markdown

Monthly Order Summary

For each of the customer, produce the following summary per month

1. total orders

2. total items bought

3. total amount spent

▶ Dec 30, 2024 (1s) 2

SQL

SELECT * FROM gizmobox.silver.orders WHERE customer_id = 5816 ;

▶ (2) Spark Jobs

Table

	order_id	order_status	payment_method	total_amount	transaction_timestamp	customer_id	ite
1	12	Cancelled	PayPal	999	2024-12-06 20:26:49	5816	
2	10	Pending	Credit Card	399	2024-10-07 22:09:27	5816	
3	11	Pending	Bank Transfer	897	2024-10-13 17:34:19	5816	

3 rows | 0.67s runtime

Refreshed 59 days ago

▶ Dec 30, 2024 (7s) 3

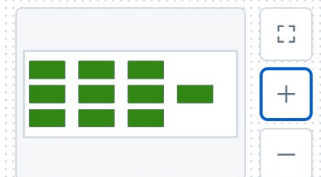
CREATE TABLE IF NOT EXISTS gizmobox.gold.order_summary_monthly AS SELECT date_format(transaction_timestamp, 'yyyy-MM') AS order_month, customer_id, COUNT(DISTINCT order_id) AS total_orders

Delta Live Tables (DLT) Project - CircuitBox



Select tables for refresh

List





Full Length Practice Exam

Exam mode



Practice Exam 1 - Databricks Certified Data Engineer Associate

45 questions | 1 hour 30 minutes | 70% correct required to pass

This exam consists of 45 questions across the 5 domains of the Databricks Certified Data Engineer Associate Certification Exam. You will have 90 minutes to complete this exam.

Passing Score for the actual exam is 70%. If you are scoring 85% or higher on these practice exams, you are ready to take and pass the real Databricks Certified Data Engineer Associate Certification Exam!

Good luck!

Instructions:

- You can pause the test at any time and resume later.
- You can retake the test as many times as you would like.
- The progress bar at the top of the screen will show your progress as well as the time remaining. If you run out of time, don't worry; you will still be able to finish the test.
- You can click the bookmark icon to mark a question for review or click "skip question" to skip it entirely.
- Click "finish test" to finish the test and see your results immediately.

Exam mode ⓘ

1/45

🕒 1:29:21 ||

Finish test

🔖 Question 1:

A **financial services company** processes large volumes of **transactional data** for fraud detection and compliance reporting.

- They need to run **scheduled ETL jobs** to clean, transform, and load data at regular intervals.
- The workloads are **not interactive** and do not require manual intervention.
- Cost efficiency is a priority, as clusters should automatically **terminate when the job is completed**.

Which type of **Databricks cluster** is best suited for this use case?

☐ All-Purpose Cluster

☐ Delta Cluster

☐ High-Concurrency Cluster

☐ NoSQL Cluster

☐ Job Cluster

Exam mode ⓘ

33/45

🕒 1:16:48 ⏸

Finish test

🔖 Question 33:

A **data analyst** has written the following SQL query to analyze sales data in Databricks:

```
1 | SELECT customer_id, SUM(total_amount) AS total_spent
2 | FROM sales_data
3 | GROUP BY customer_id
4 | ORDER BY total_spent DESC;
```

A **data engineer** wants to execute this SQL query inside a **PySpark notebook** while working with a **Spark DataFrame**.

Which of the following methods correctly allows the engineer to run this SQL command in PySpark?

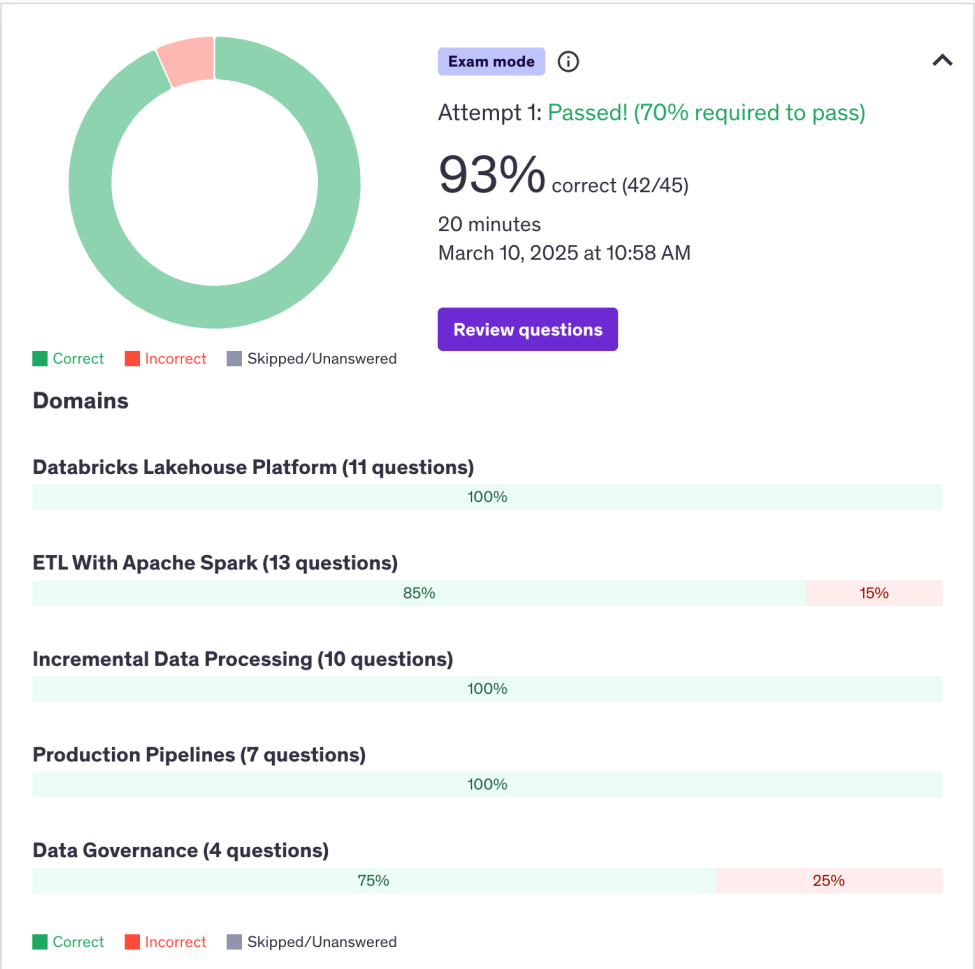
☐ 1 | df = sql("""
2 | SELECT customer_id, SUM(total_amount) AS total_spent
3 | FROM sales_data
4 | GROUP BY customer_id
5 | ORDER BY total_spent DESC
6 | """)

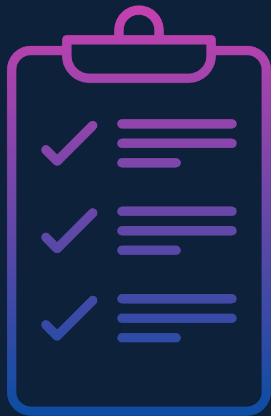
☐ 1 | df = spark.sql("""
2 | SELECT customer_id, SUM(total_amount) AS total_spent
3 | FROM sales_data
4 | GROUP BY customer_id
5 | ORDER BY total_spent DESC
6 | """)

☐ 1 | df = spark.execute_sql("""
2 | SELECT customer_id, SUM(total_amount) AS total_spent

Practice Exam 1 - Databricks Certified Data Engineer Associate - Results

45 questions | 1 hour 30 minutes | 70% correct required to pass





No Prior Databricks Knowledge Required

Basic SQL & Python Knowledge Required

Azure Cloud Subscription Required



Ask Questions, I will answer 😊

Udemy life time access

Udemy 30 day money back guarantee



Industry Recognition

Career Advancement

Skill Validation

Future-proof Your Career



ENROLL NOW!

DISCLAIMER

This course is independently created and is **not affiliated with or endorsed by Databricks, Inc.**

All content, explanations, and practice materials are original and created only for educational purposes.

This course doesn't contain actual exam questions.

All trademarks and product names are the property of their respective owners.

For official certification materials, please visit databricks.com.



Official Exam Guide - <https://www.databricks.com/learn/certification/data-engineer-associate>



Databricks Documentation- <https://docs.databricks.com/>



Course Structure

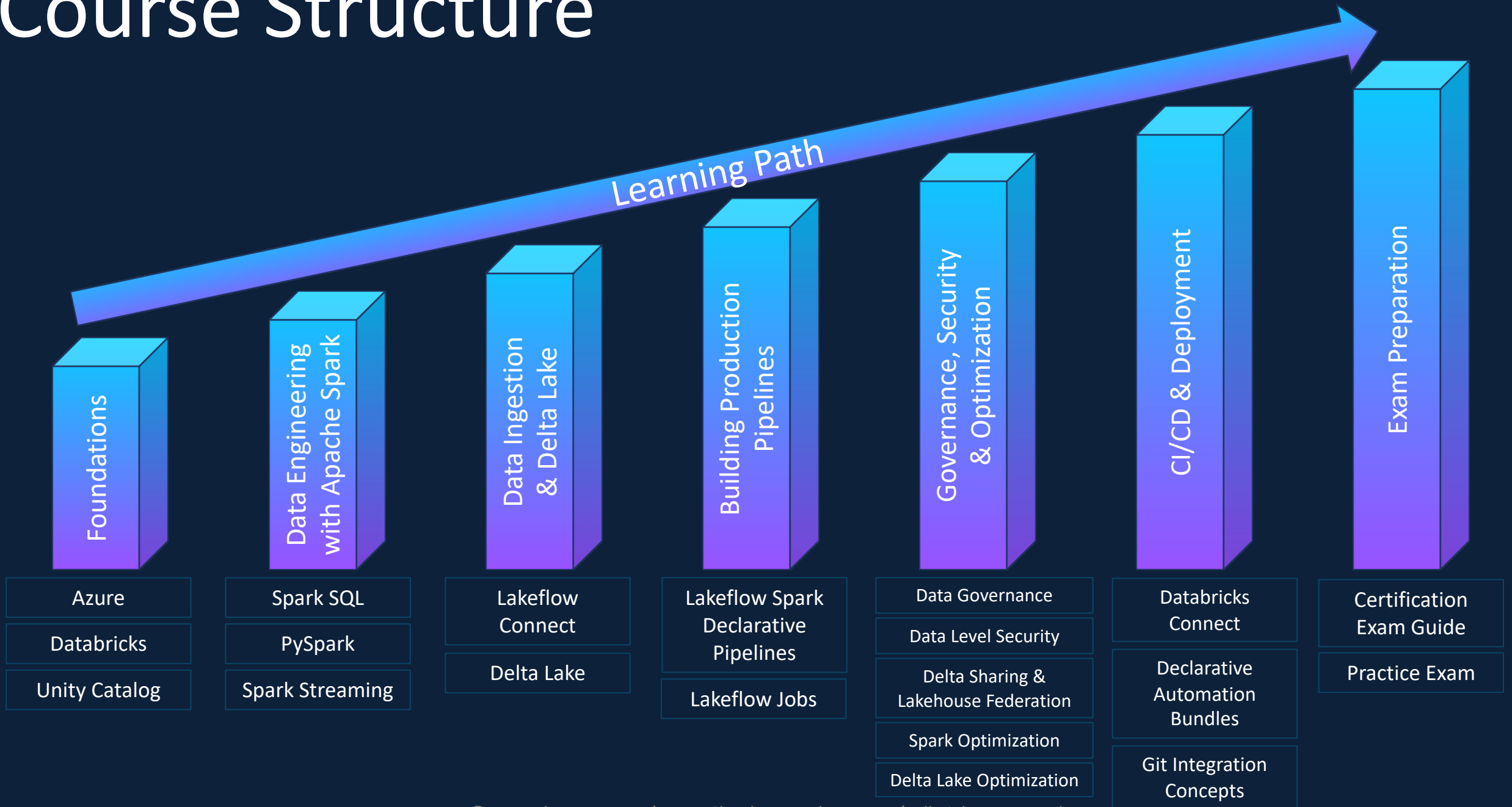
Course Structure



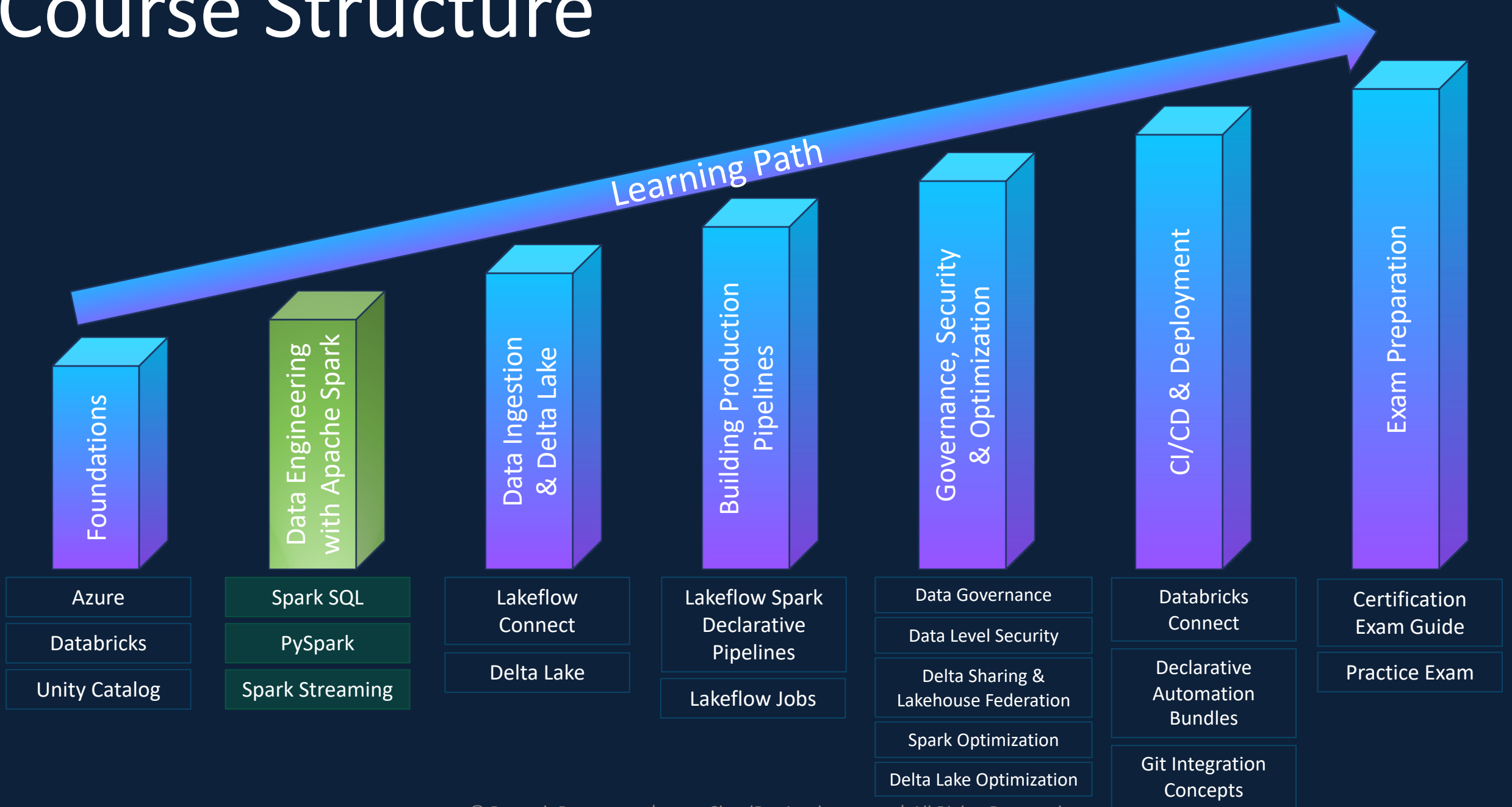
Aligned with Current Exam Guide

Build practical data engineering skills on Databricks

Course Structure



Course Structure

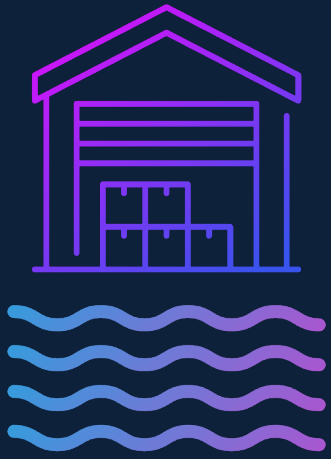




Creating Azure Free Subscription

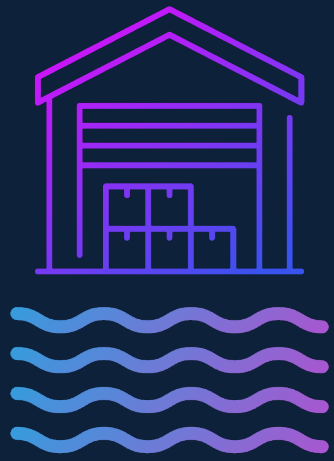


Azure Portal Overview



What is Data Lakehouse?

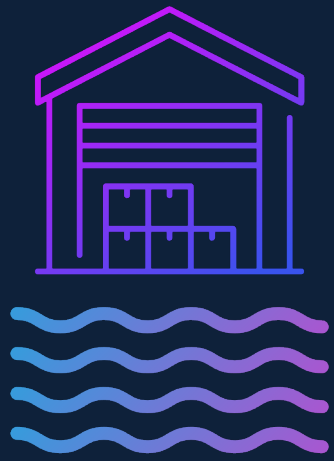
What is Data Lakehouse?



A Data Lakehouse is a new, open data management architecture that combines the flexibility, cost-efficiency, and scale of Data Lakes with the data management and ACID transactions of Data Warehouses, enabling business intelligence (BI) and machine learning (ML) on all data.

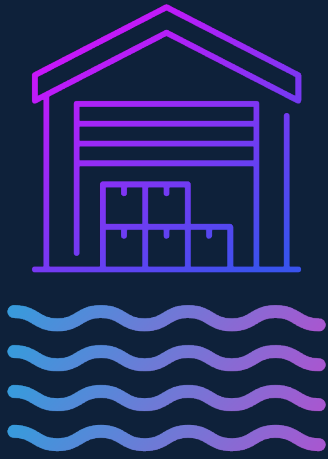
- *Databricks*

What is Data Lakehouse?



A Data Lakehouse is a new, open data management architecture that combines the flexibility, cost-efficiency, and scale of Data Lakes with the data management and ACID transactions of Data Warehouses, enabling business intelligence (BI) and machine learning (ML) on all data.

What is Data Lakehouse?



Data Lakehouse

=



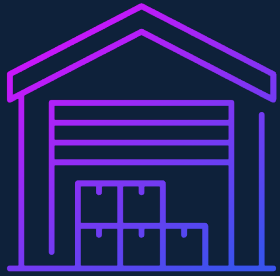
Data Lake

+



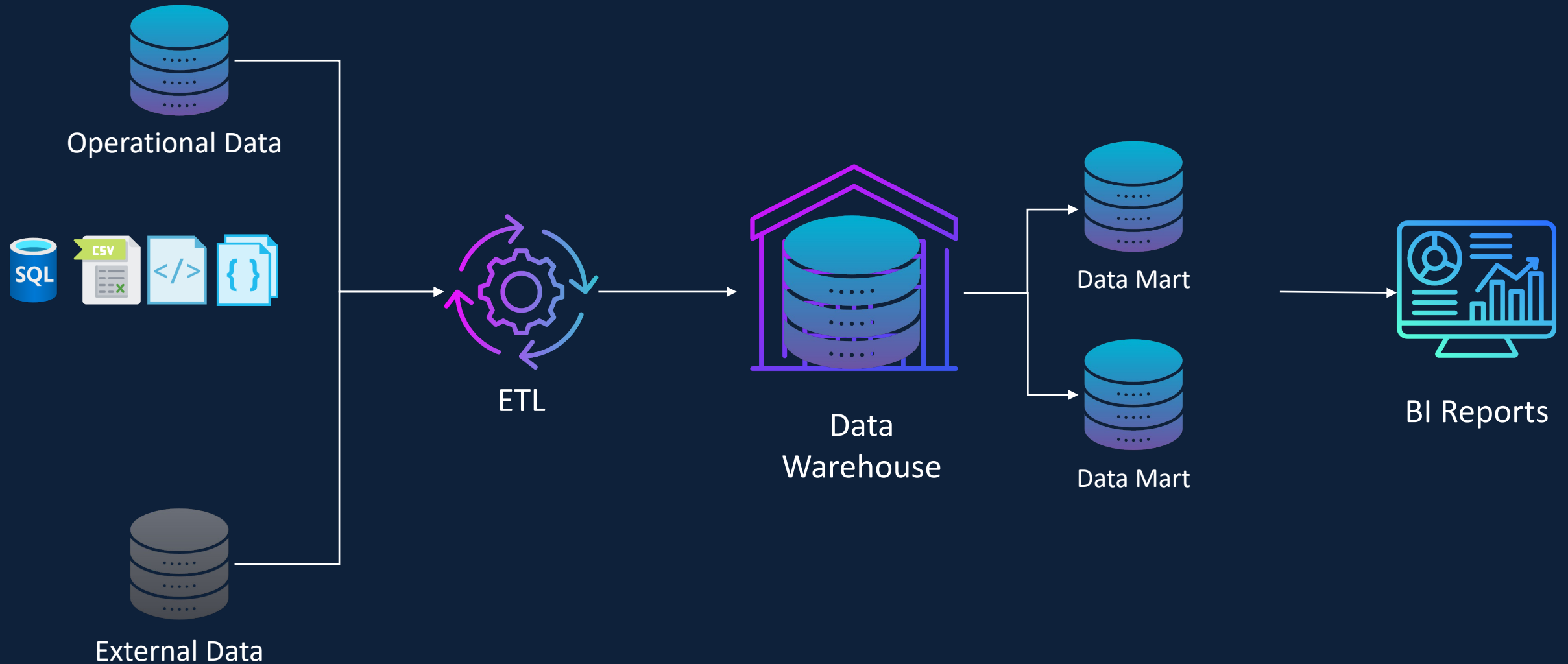
Data Warehouse

A Data Lakehouse is a new, open data management architecture that combines the flexibility, cost-efficiency, and scale of Data Lakes with the data management and ACID transactions of Data Warehouses, enabling business intelligence (BI) and machine learning (ML) on all data.

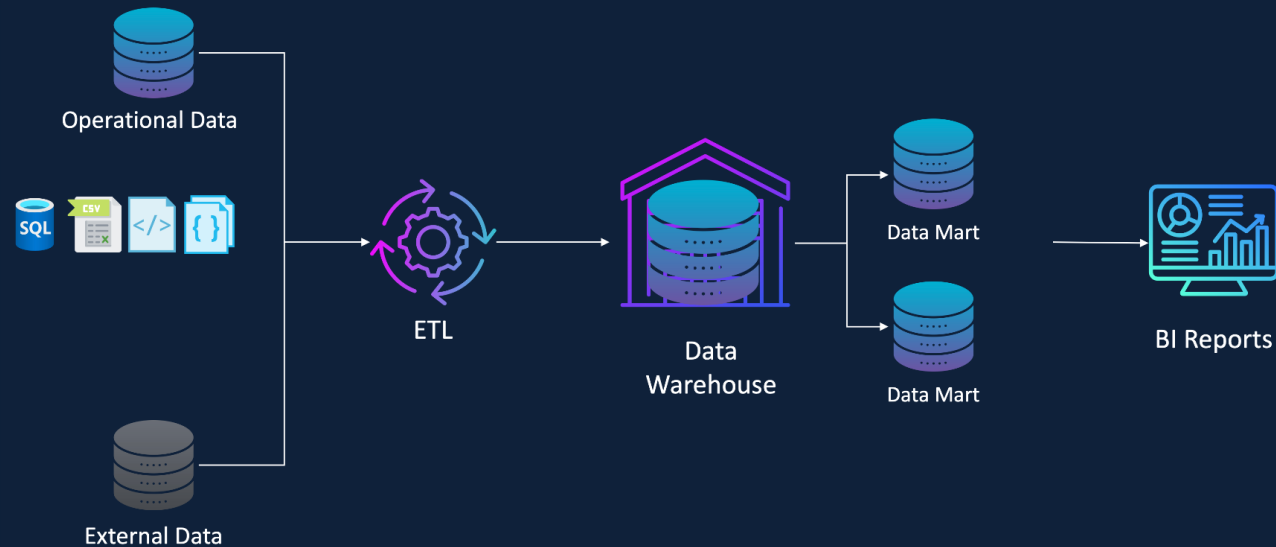


What is Data Warehouse?

What is Data Warehouse?



What is Data Warehouse?



Increased Data Volume & Variety

Longer Time to Ingest New Data

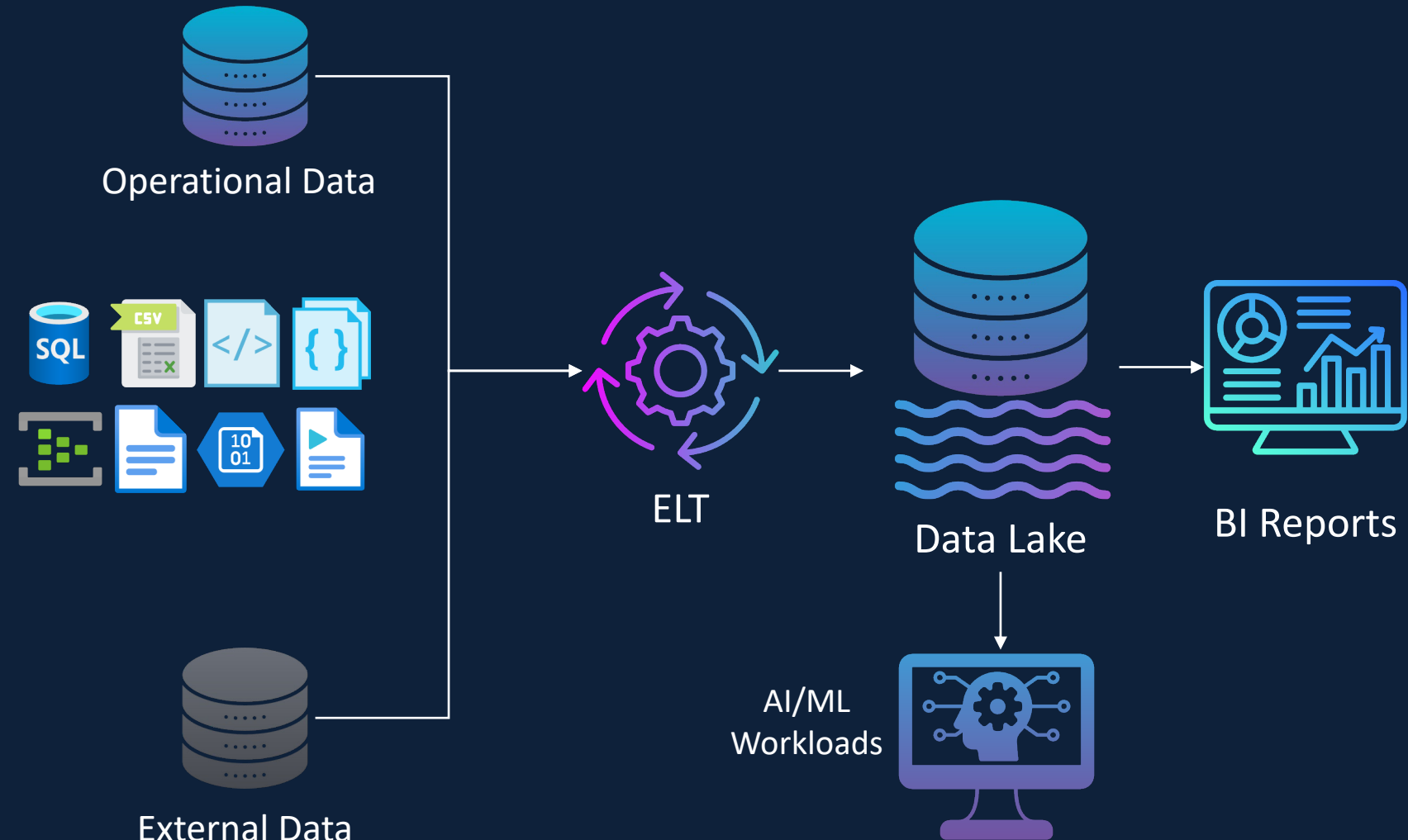
Proprietary Data Formats /Vendor Lock-in

Scalability Issues

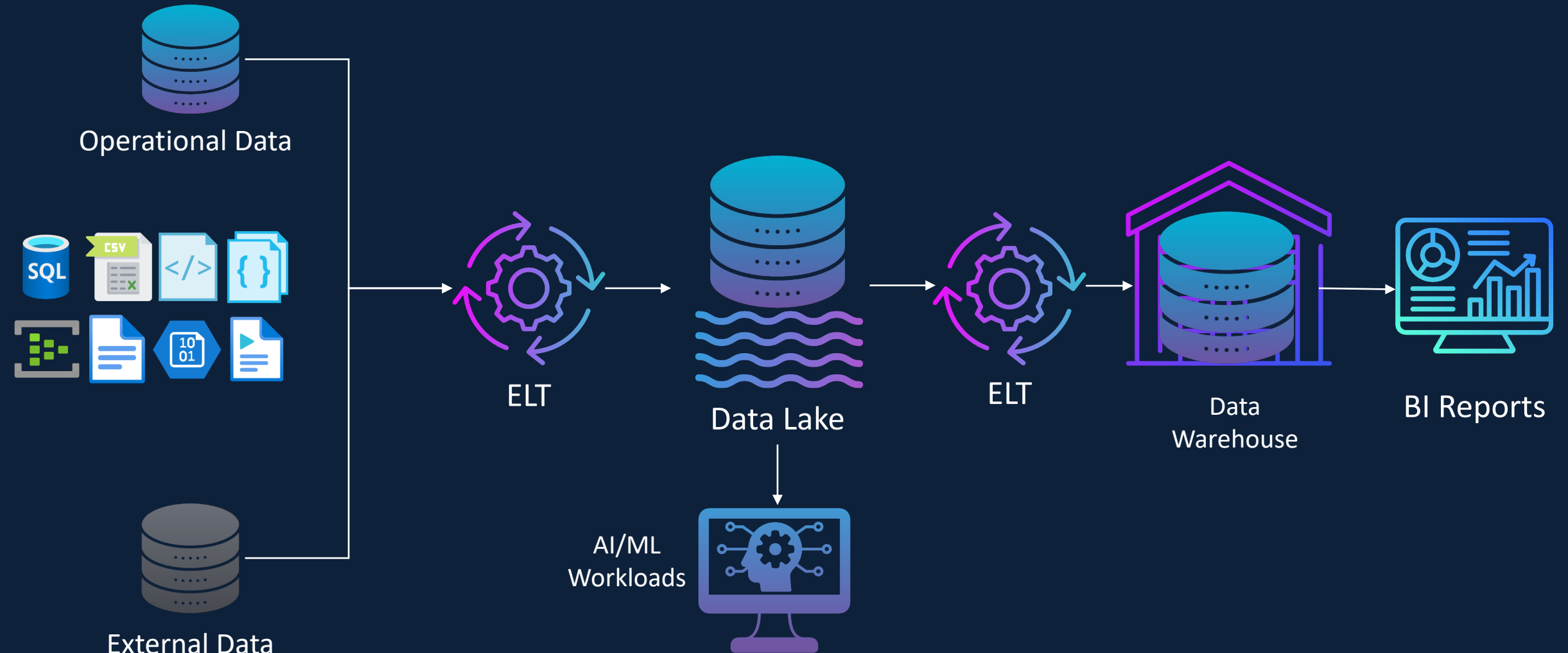
High Storage Costs

Limited Support for Advanced Analytics

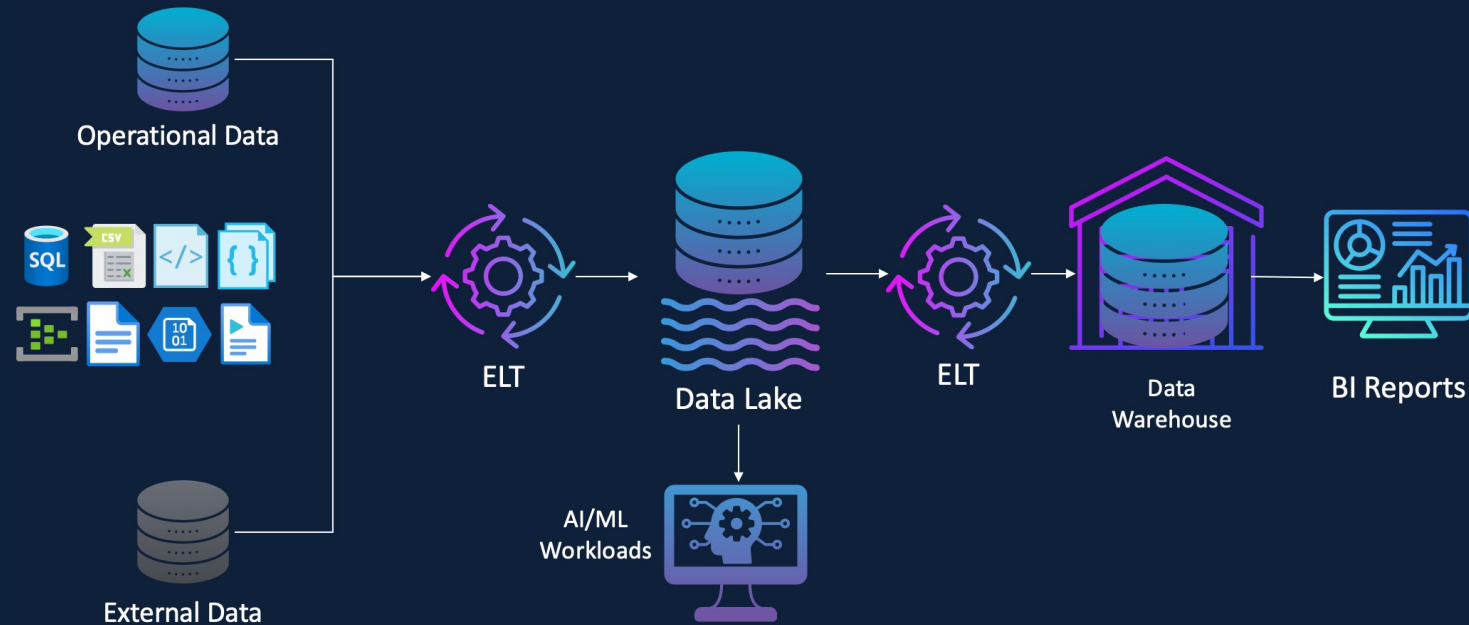
What is Data Lake?



What is Data Lake?



What is Data Lake?



No Support for ACID Transactions

Partial Data Loads

Inconsistent Reads

Complicated Data Corrections

No Rollback Capability

GDPR Challenges

No Data Versioning

Poor BI Performance and Support

Complex to set-up

Streaming vs. Batch Processing

What is Data Lakehouse?



Data Lake



Data Warehouse

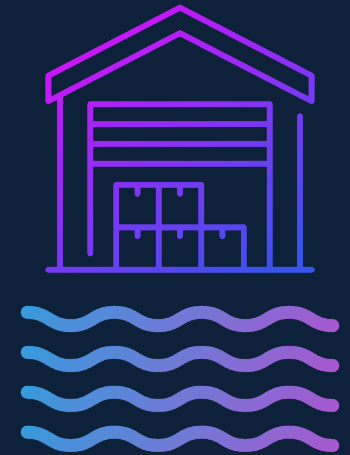
What is Data Lakehouse?



Data Lake

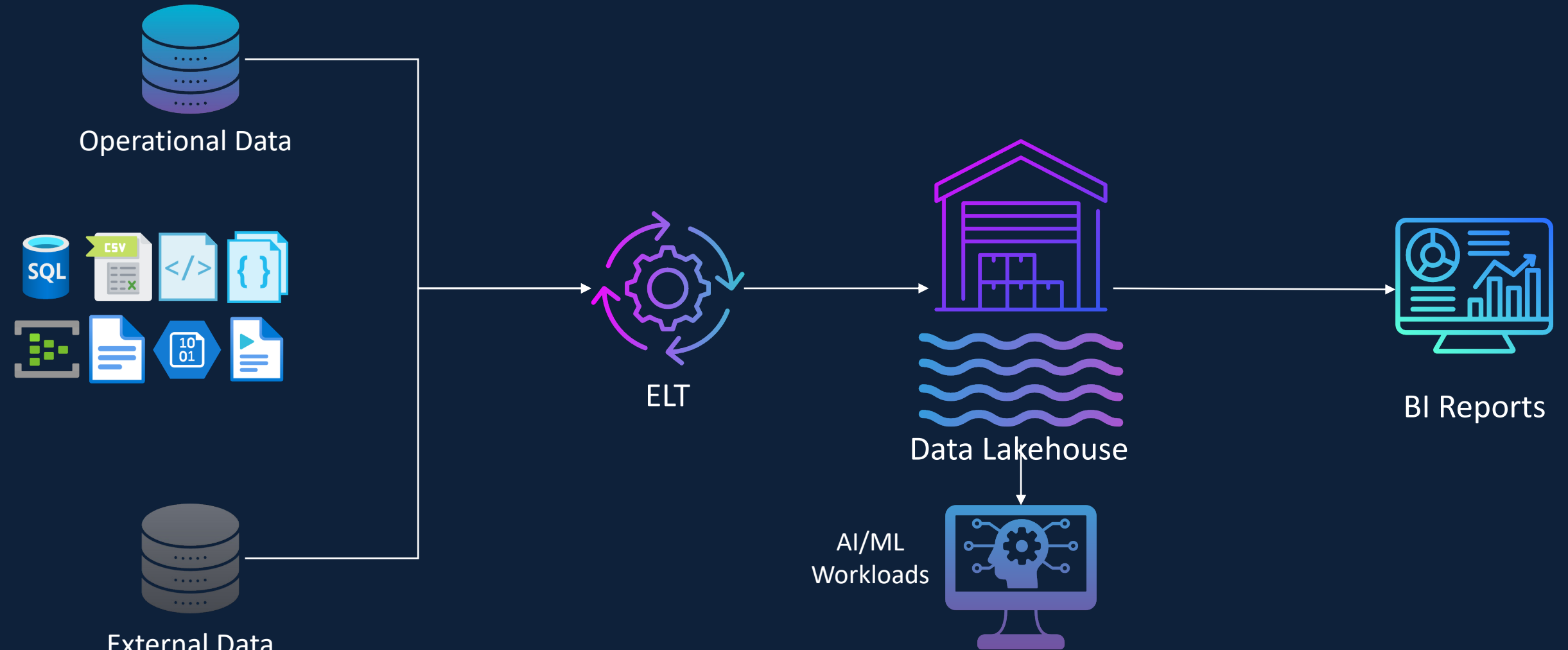


Data Warehouse

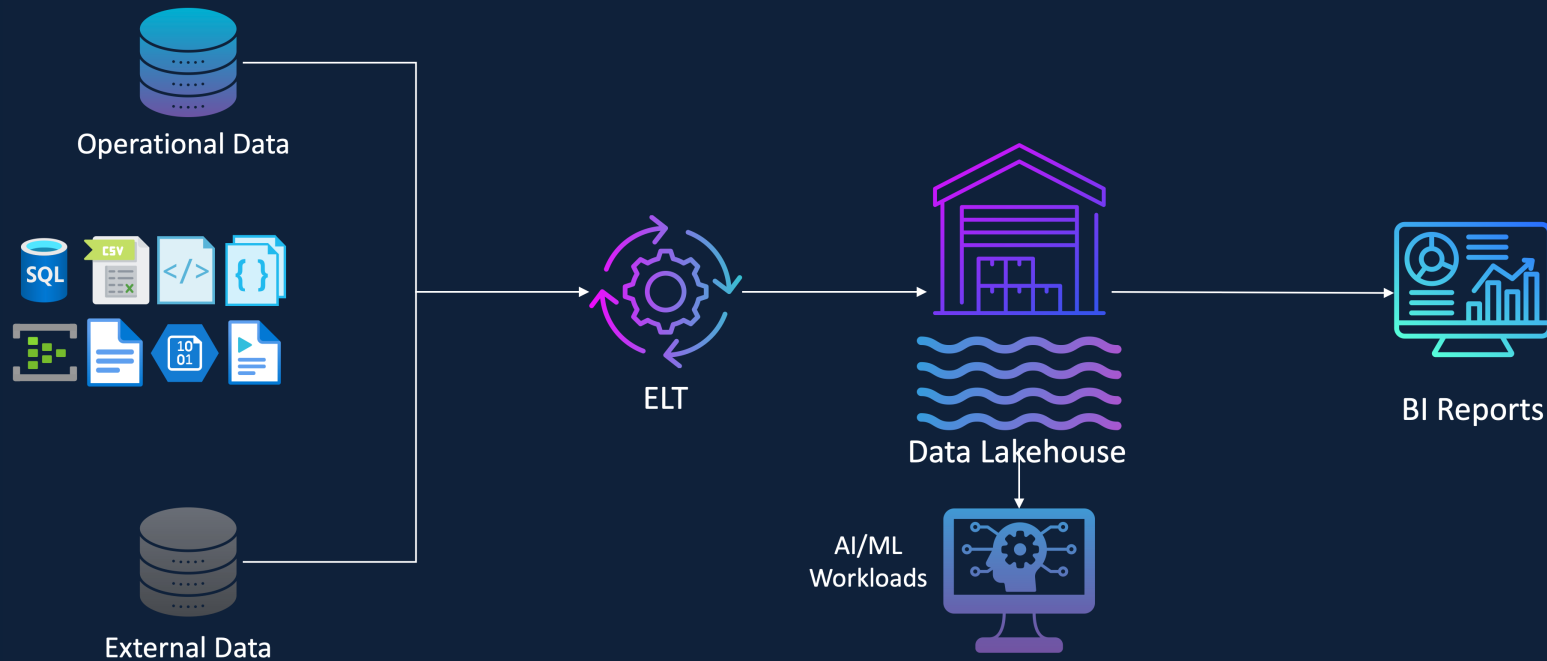


Data Lakehouse

What is Data Lakehouse?



What is Data Lakehouse?



Handles All Types of Data

Cheap Cloud Object Storage

Uses Open File Format

Support for All Workloads

Direct BI Integration

ACID Support & Version History

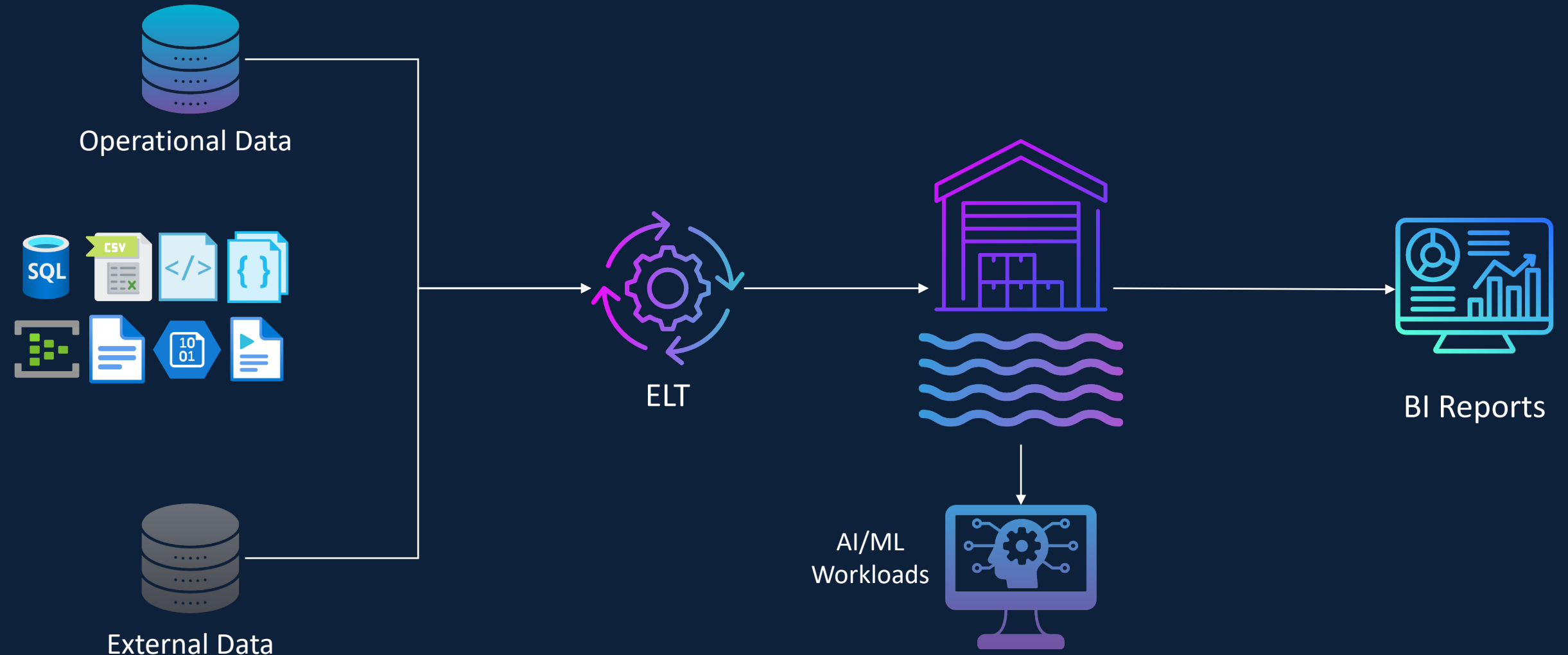
Improved Performance

Simple Architecture

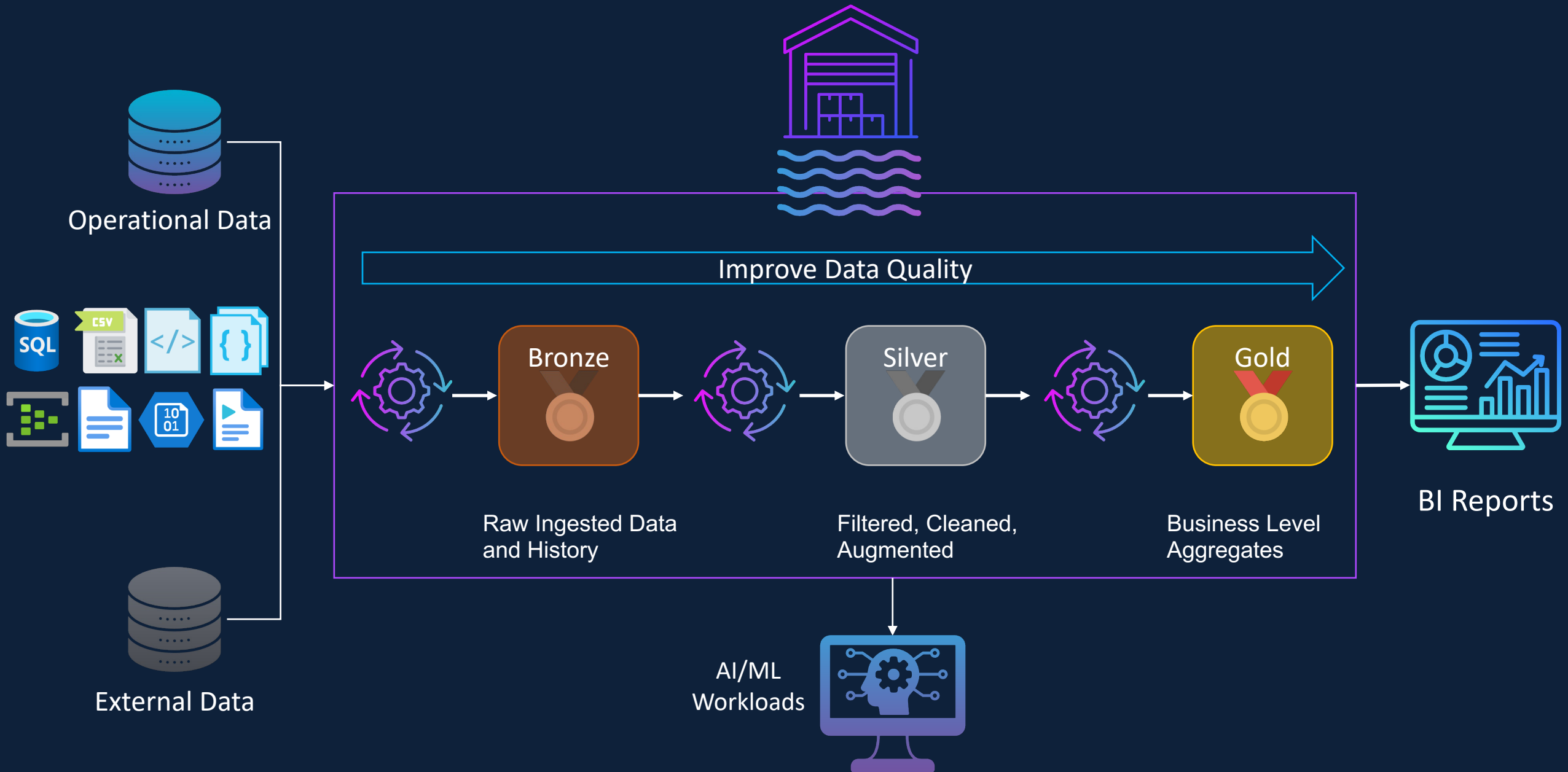


What is Medallion Architecture?

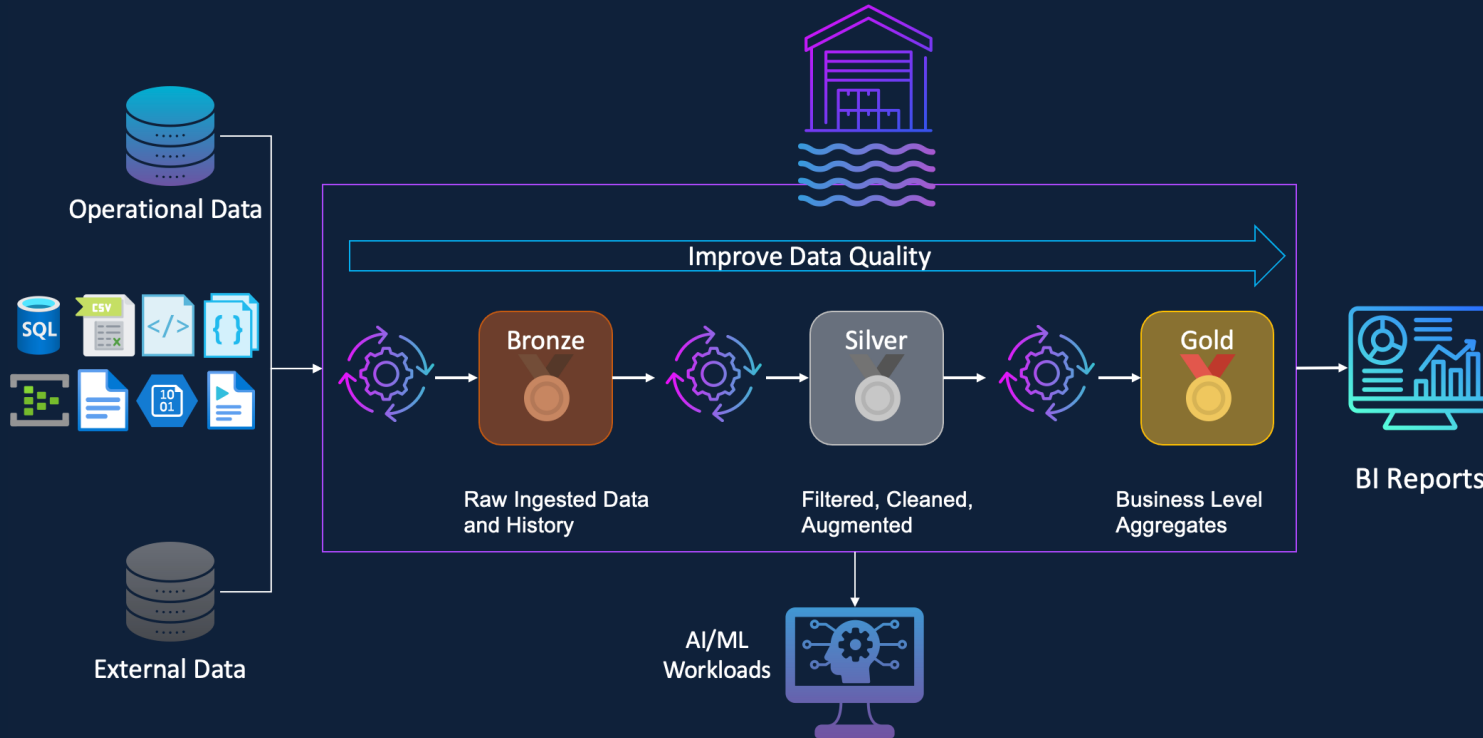
Medallion Architecture



Medallion Architecture



Medallion Architecture - Benefits



Data Lineage & Traceability

Data Governance & Compliance

Support for Incremental Processing

Enhanced Performance & Scalability

Data Security



Introduction to Databricks

Introduction to Databricks



Apache Spark

Apache Spark is a lightning-fast unified analytics engine for big data processing and machine learning



100% Open source under Apache License

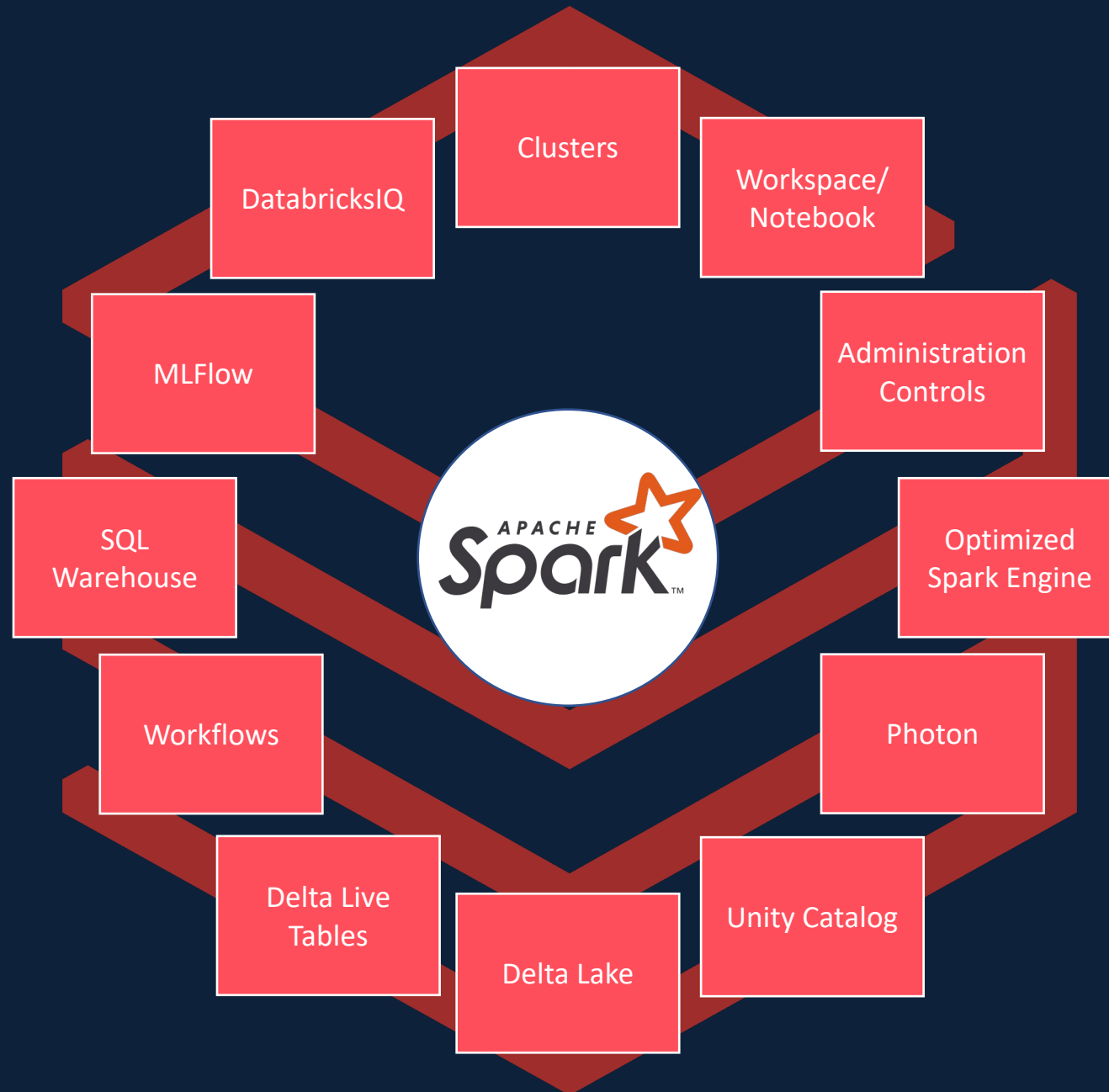
Simple and easy to use APIs

In-memory processing engine

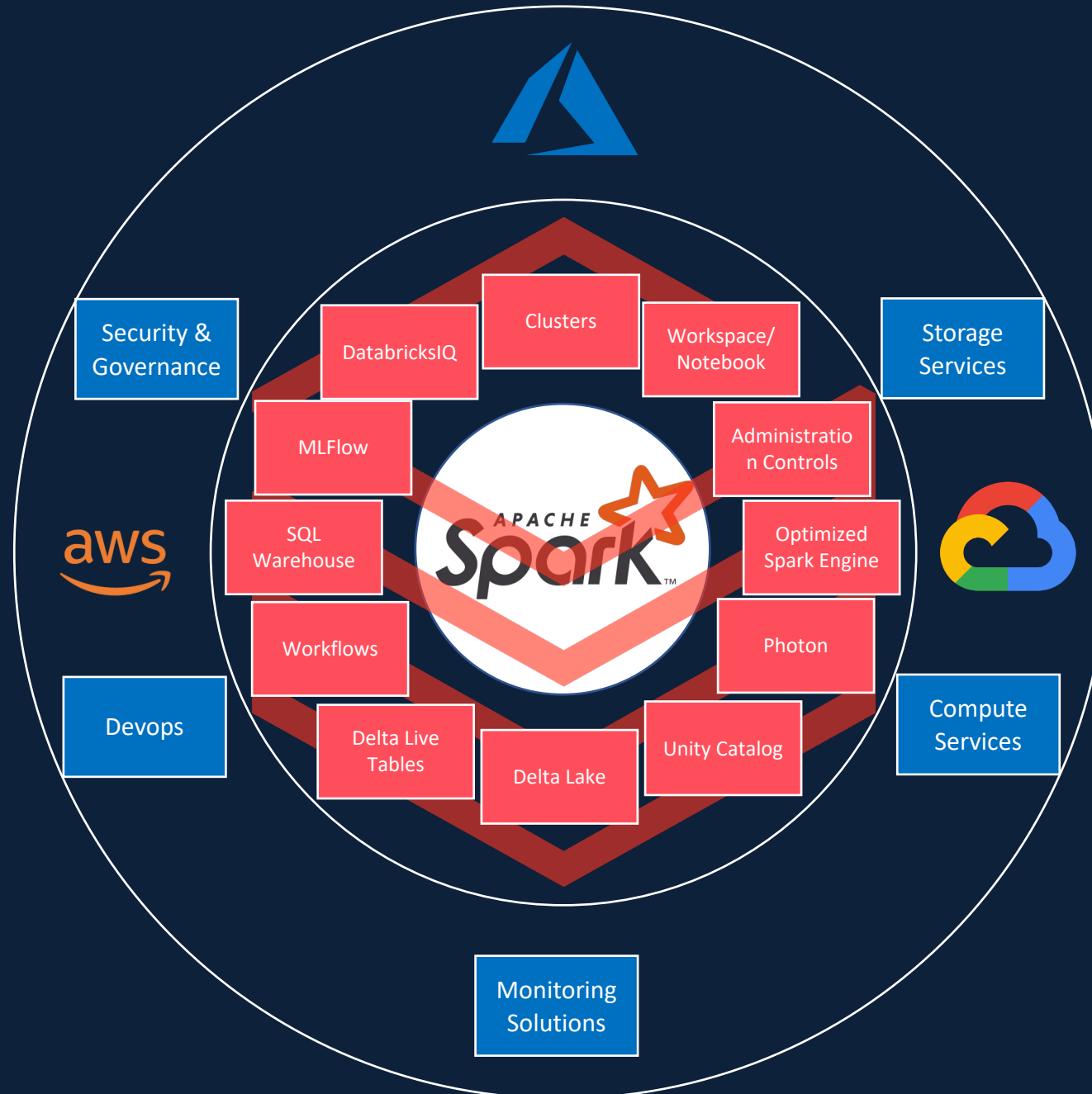
Distributed computing Platform

Unified engine which supports SQL, streaming, ML and graph processing

Databricks



Databricks





Creating a Databricks Service

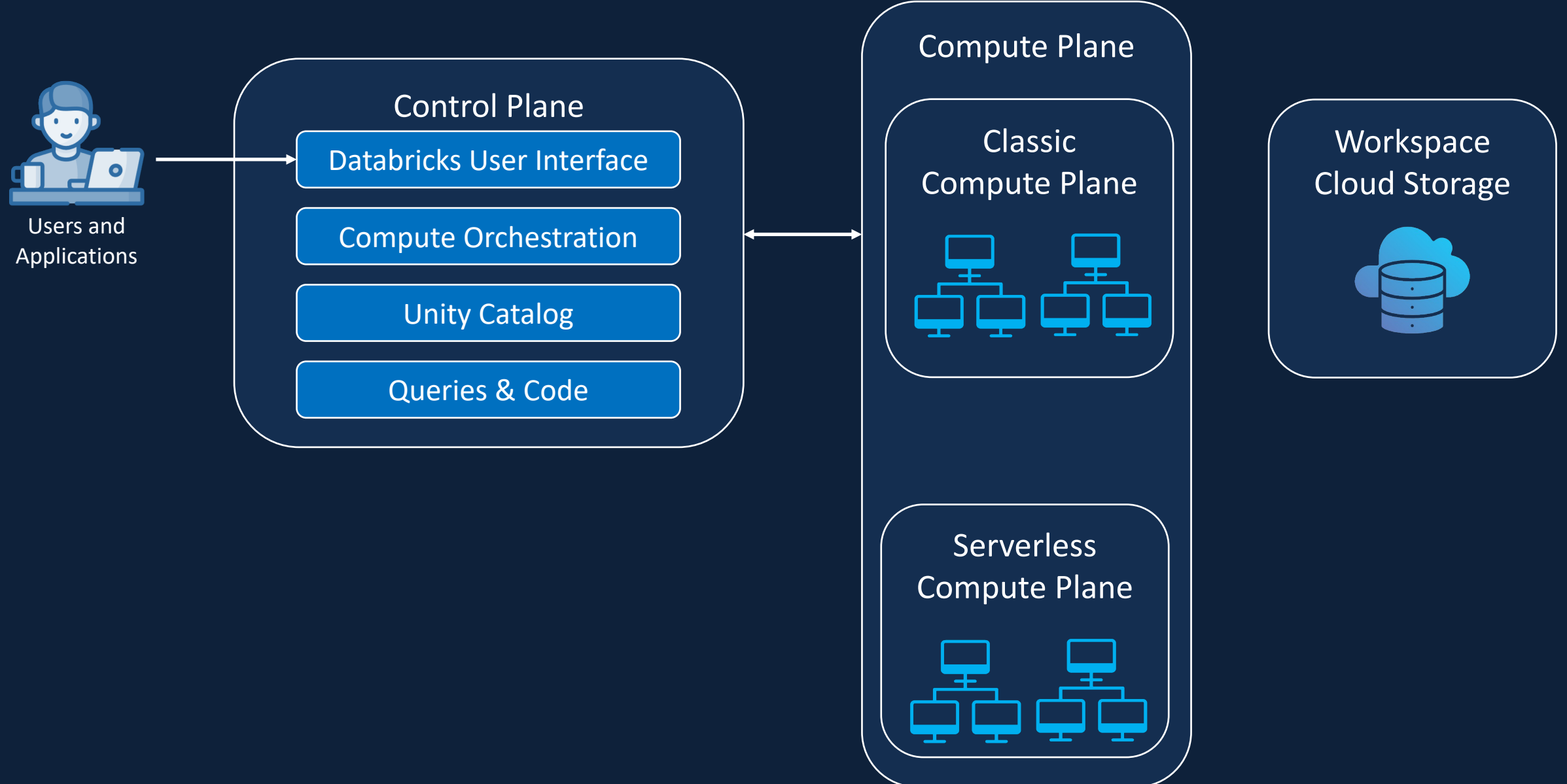


Databricks User Interface

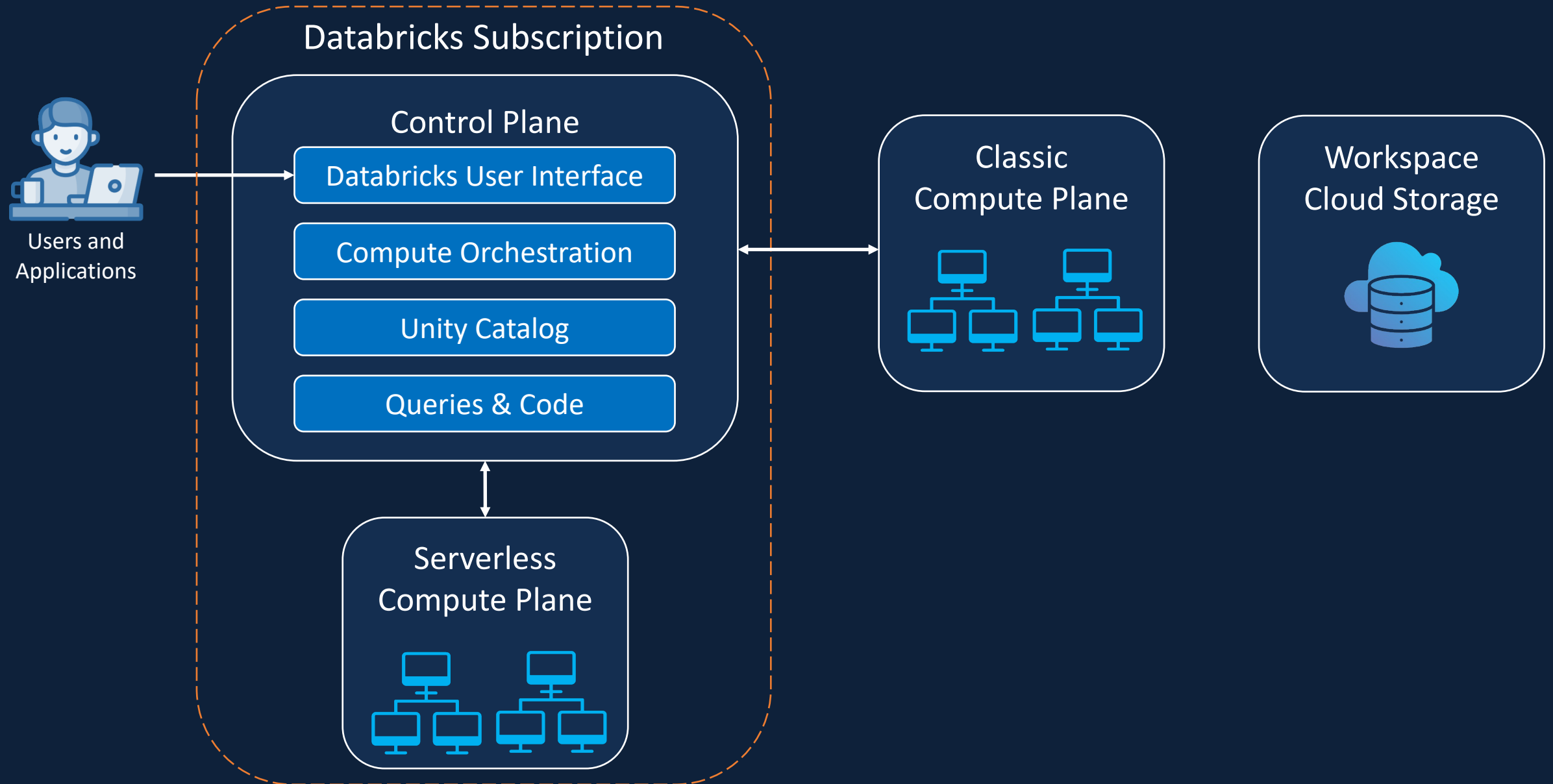


Databricks Architecture

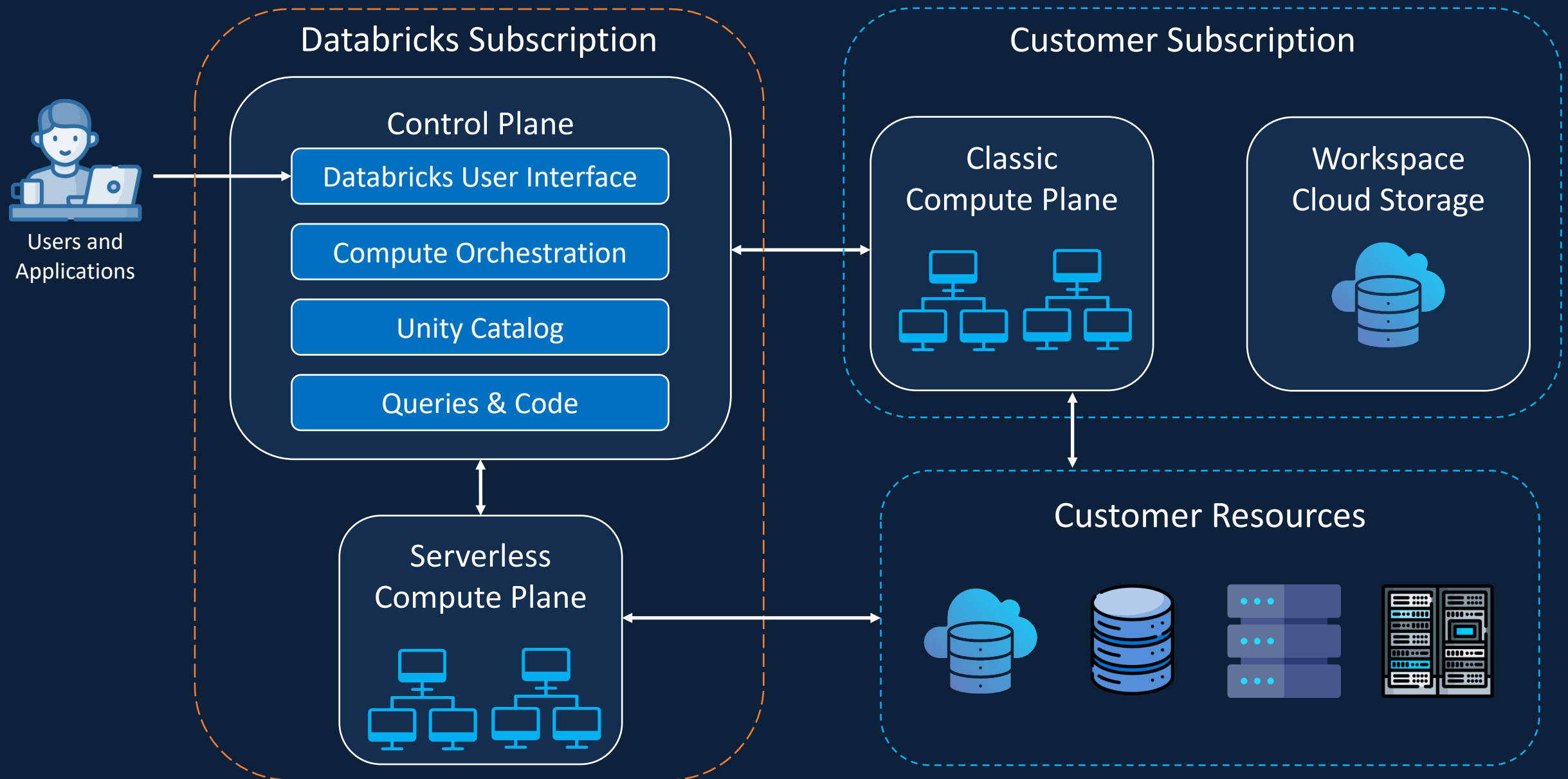
Databricks Architecture



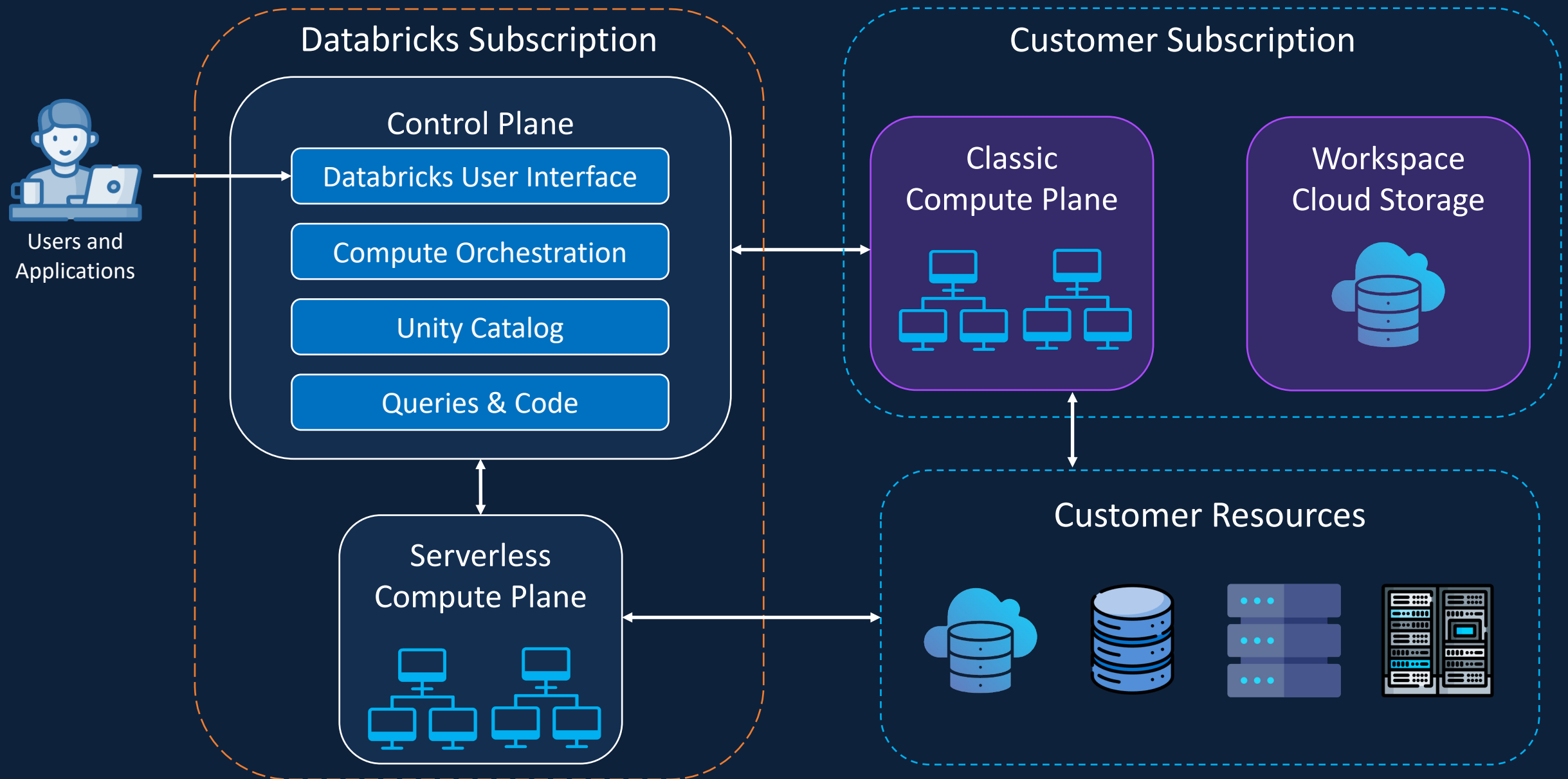
Databricks Architecture



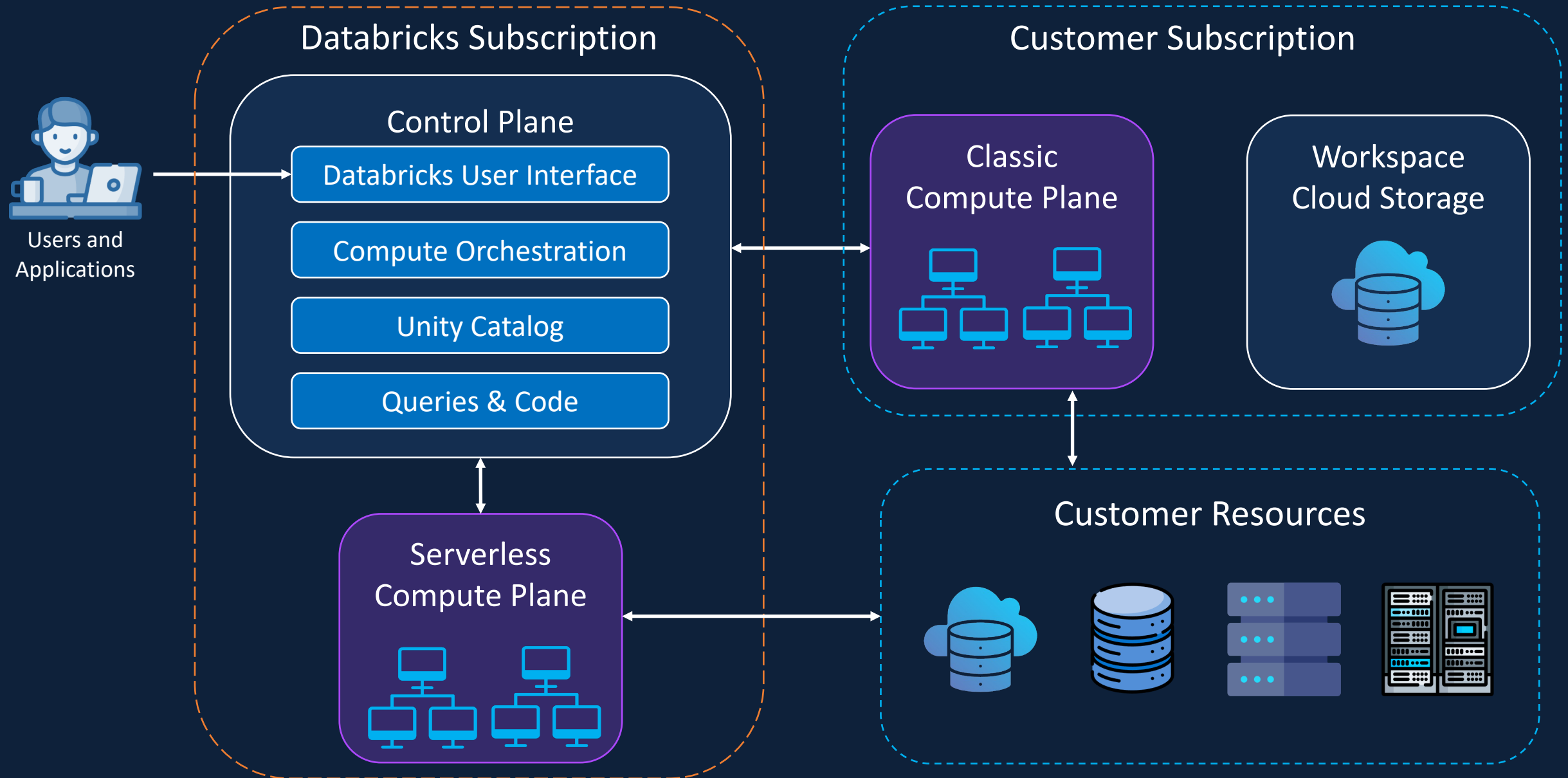
Databricks Architecture



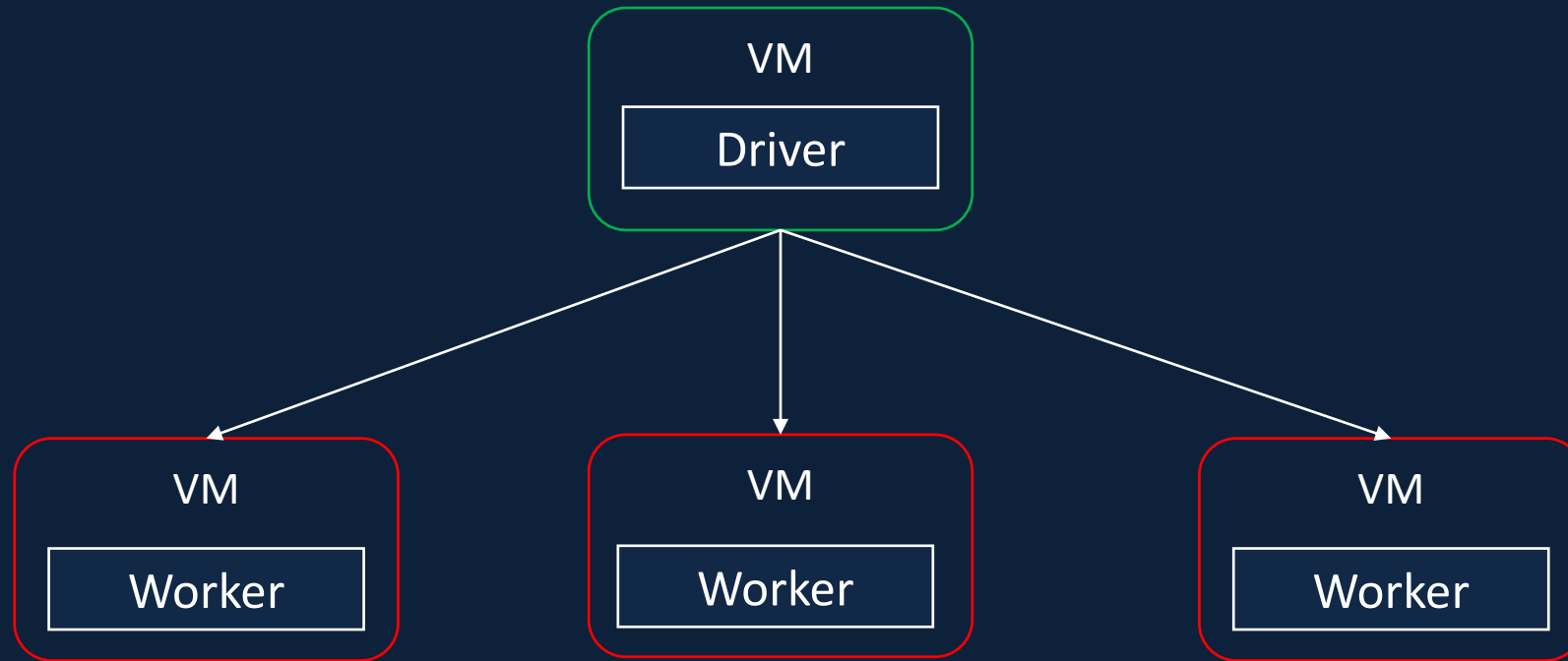
Databricks Architecture



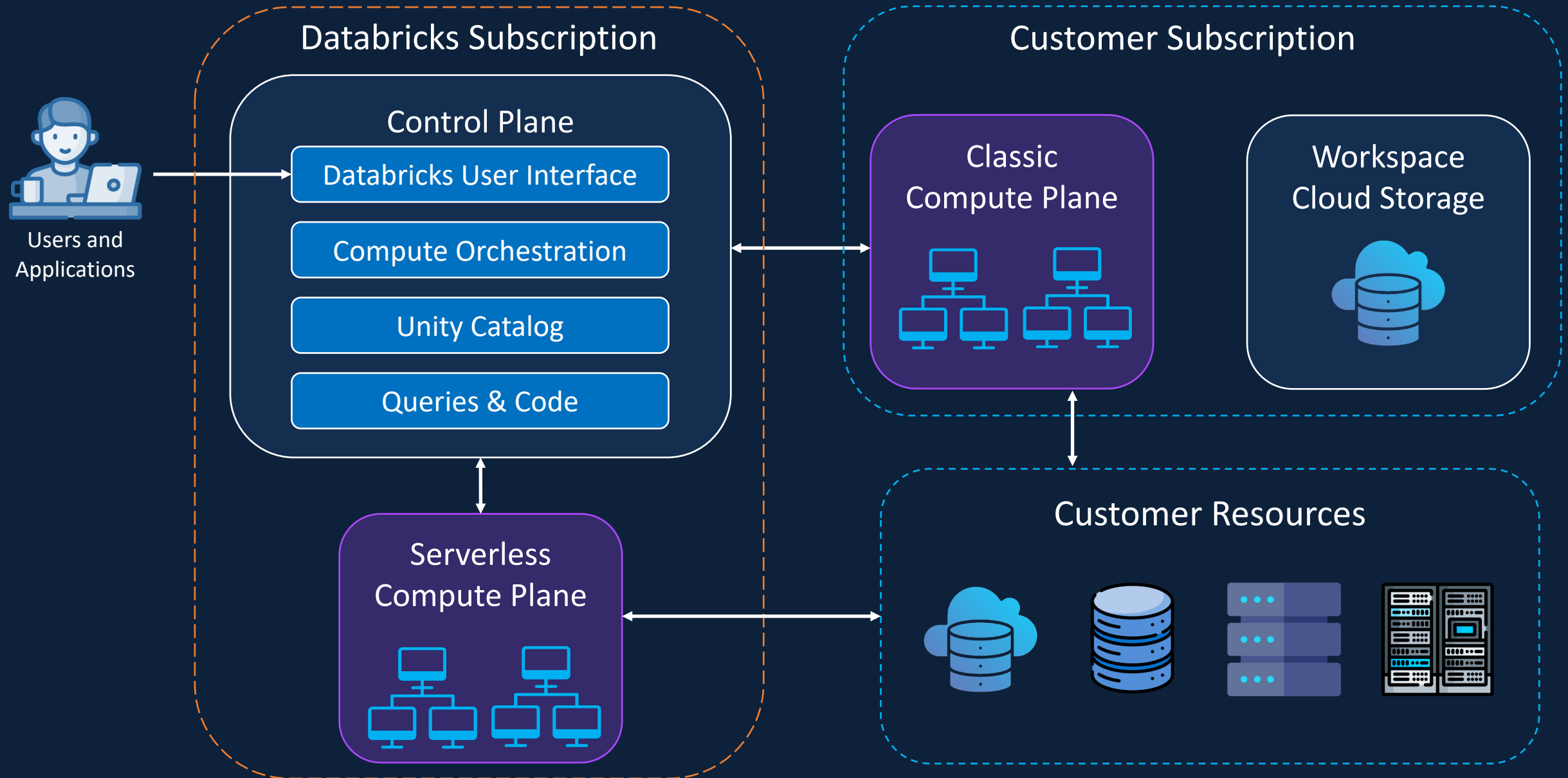
Databricks Compute



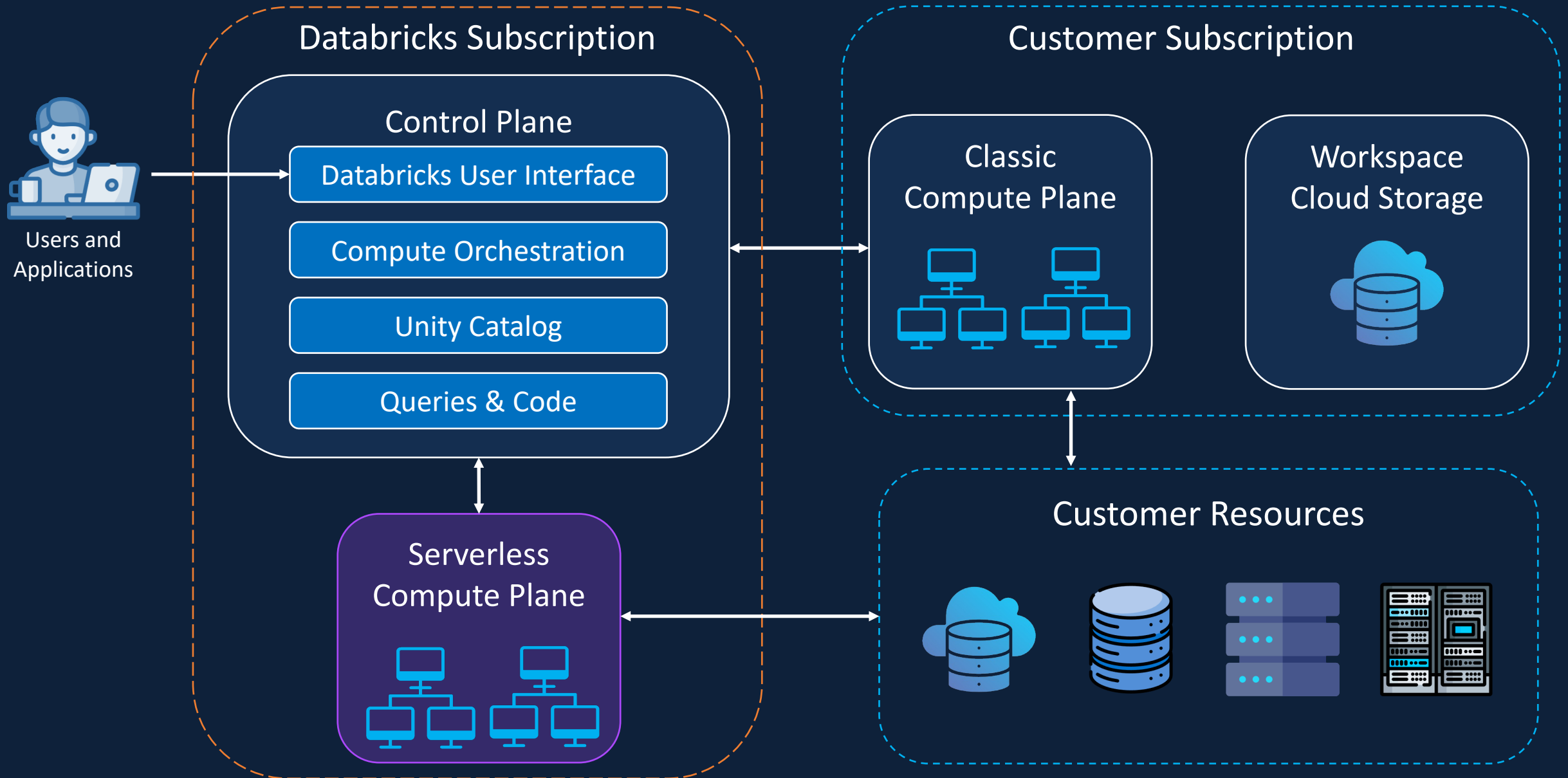
Databricks Compute



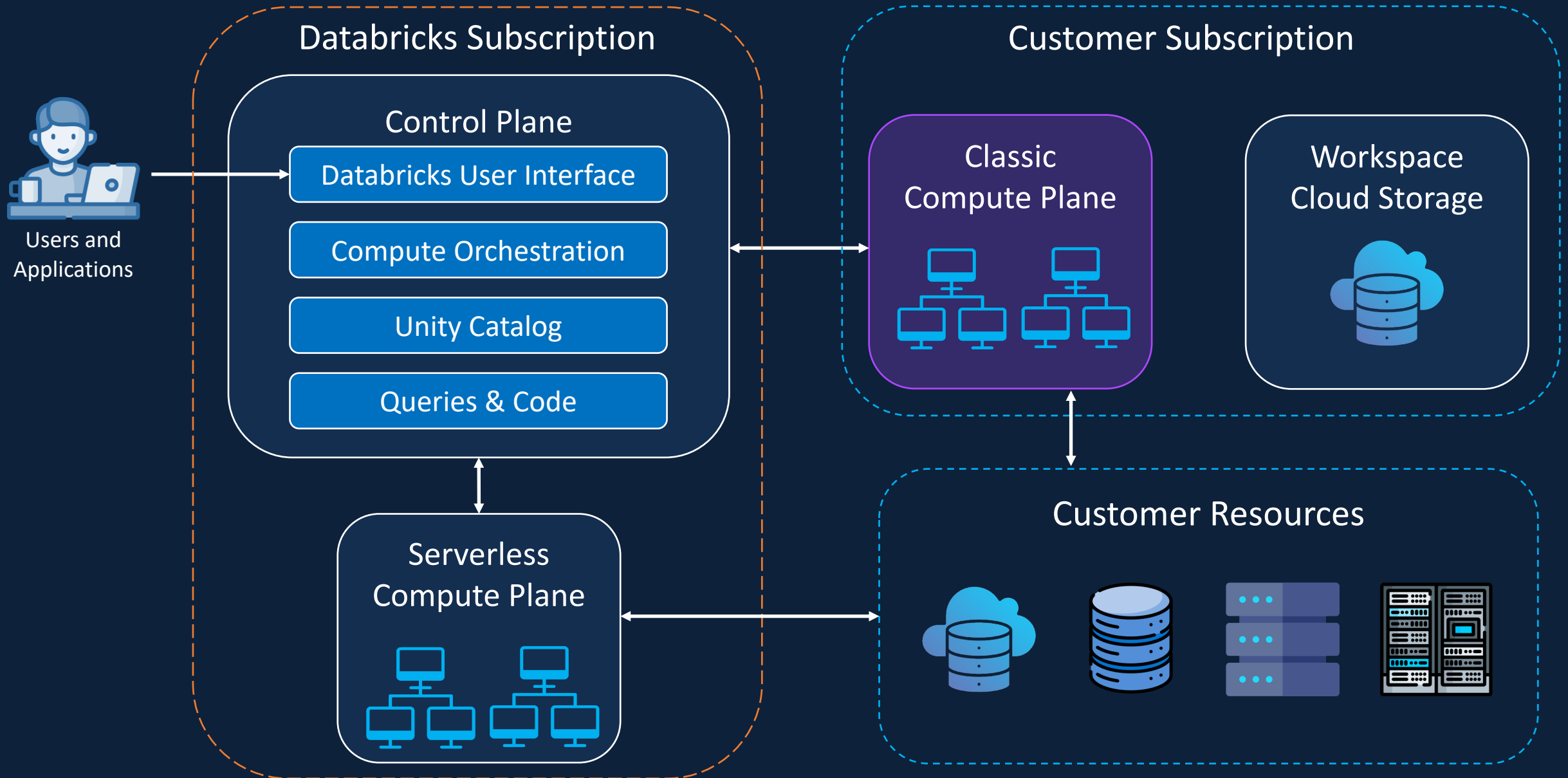
Databricks Compute



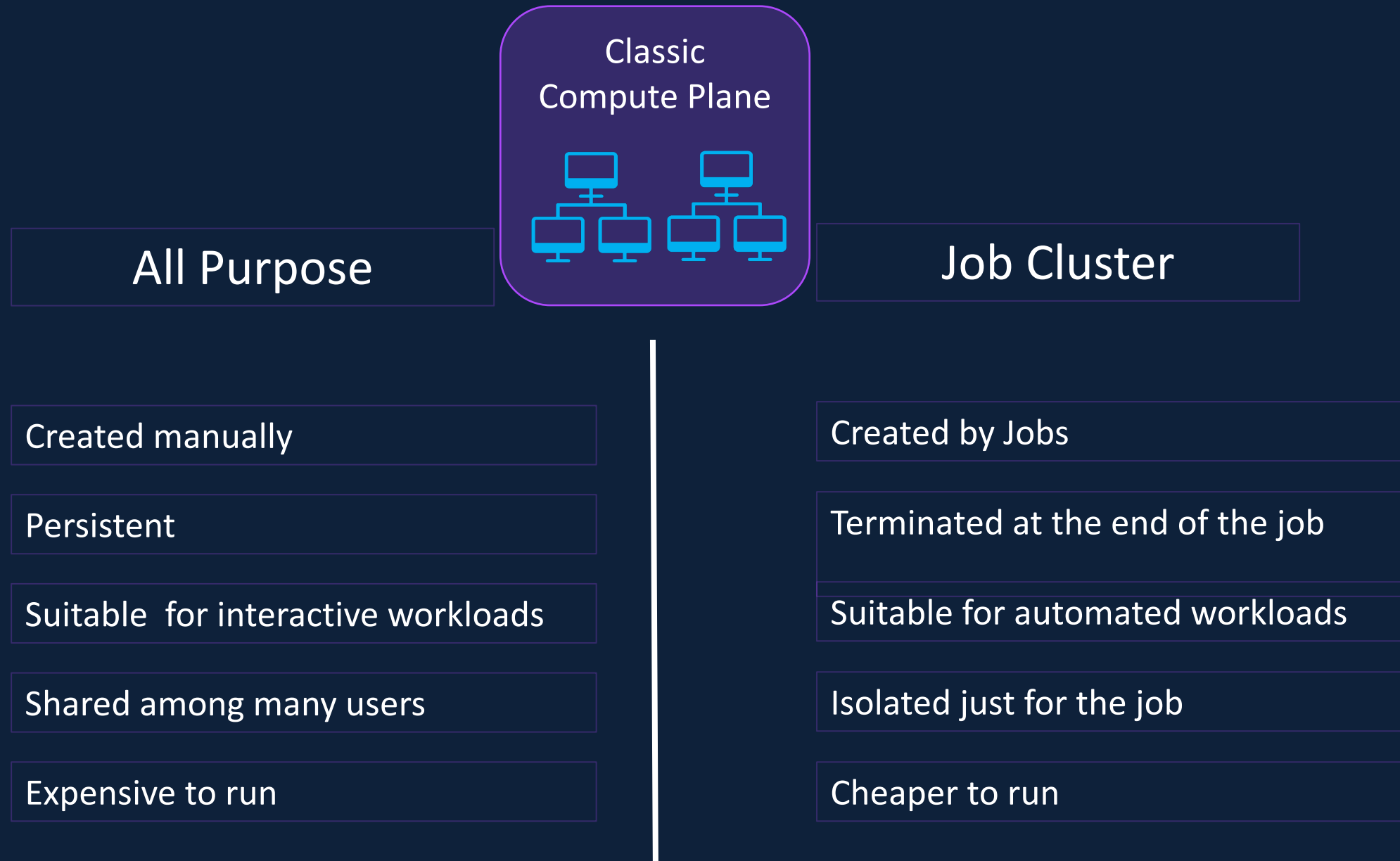
Databricks Compute



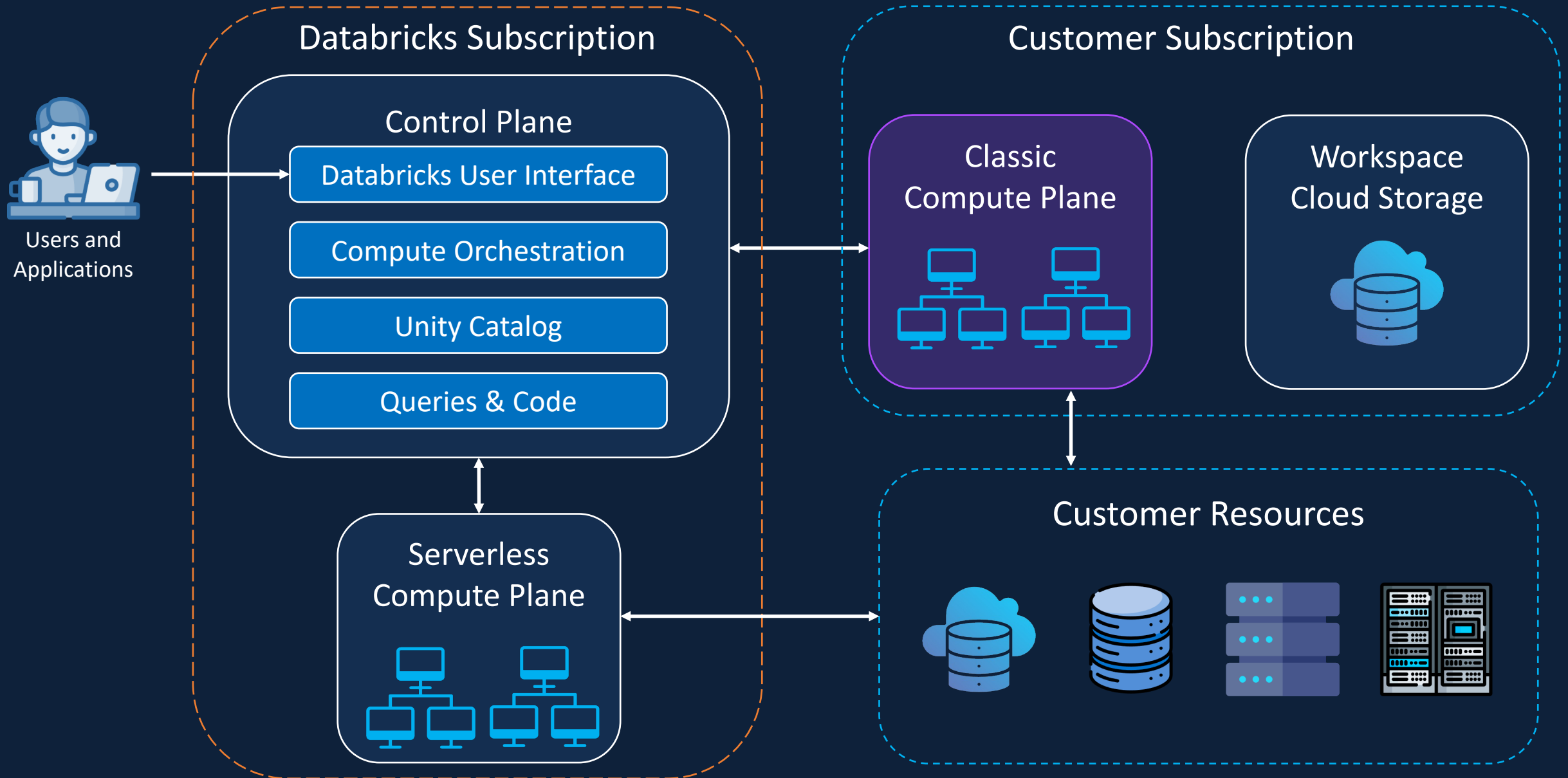
Databricks Compute



Databricks Classic Compute



Cluster Configuration



Cluster Configuration

Classic Compute Plane



Cluster Configuration

Policy

Unrestricted | 

☒ Multi node ☐ Single node

Access mode

Single user | 

Single user access

Ramesh Retnasamy (az.adm1... | 

Performance

Databricks runtime version

Runtime: 11.3 LTS (Scala 2.12, Spark 3.3.0) | 

☐ Use Photon Acceleration 

Worker type

Standard_DS3_v2 14 GB Memory, 4 Cores | 

Min workers

2

Max workers

8

 ☐ Spot instances 

Driver type

Same as worker 14 GB Memory, 4 Cores | 

☒ Enable autoscaling 

☒ Terminate after 120 minutes of inactivity 

Cluster Configuration

Single/ Multi Node

Multi Node

Single Node

Cluster Configuration

Single/ Multi Node

Access Mode

Dedicated (formerly Single User)

Only One User Access
Supports Python, SQL, Scala, R

Standard (formerly Shared)

Multiple User Access
Only available in Premium. Supports Python, SQL

No Isolation Shared

Multiple User Access
Supports Python, SQL, Scala, R

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Databricks Runtime

Apache
Spark

Supporting
Libraries

Photon

Databricks Runtime ML

Apache
Spark

Supporting
Libraries

Popular ML
Libraries

Photon

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Termination

- Terminates the cluster after X minutes of inactivity
- Default value for Single Node and Standard clusters is 120 minutes
- Users can specify a value between 10 and 43200 mins as the duration

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Auto Scaling

- User specifies the min and max worker nodes
- Auto scales between min and max based on the workload
- Users can opt for spot instances for the worker nodes

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Cluster VM Type/ Size

Memory Optimized

Compute Optimized

Storage Optimized

General Purpose

GPU Accelerated

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Cluster VM Type/ Size

Cluster Policy

Cluster Configuration

Policy 

Unrestricted 

☒ Multi node ☐ Single node

Access mode 

Single user access 

Single user 

Ramesh Retnasamy (az.adm1... 

Performance

Databricks runtime version 

Runtime: 11.3 LTS (Scala 2.12, Spark 3.3.0) 

☐ Use Photon Acceleration 

Worker type 

Standard_DS3_v2 14 GB Memory, 4 Cores 

Min workers 2

Max workers 8

 ☐ Spot instances 

Driver type

Same as worker 14 GB Memory, 4 Cores 

☒ Enable autoscaling 

☒ Terminate after 120 minutes of inactivity 

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Cluster VM Type/ Size

Cluster Policy

Cluster Configuration

Policy ?

Personal Compute | 

Single user access ?

Ramesh Retnasamy (az.adm1@outlook.com) | 

Performance

Databricks runtime version ?

Runtime: 11.3 LTS ML (Scala 2.12, Spark 3.3.0) | 

Node type ?

Standard_DS3_v2 14 GB Memory, 4 Cores | 

☒ Terminate after minutes of inactivity ?



Create Databricks Compute



Databricks Notebooks



Databricks Magic Commands

Databricks Magic Commands

%python, %sql, %scala, %r

Switch to a different language for a specific cell

%md

Markdown for documenting notebooks

%fs

Run file system commands

%sh

Run shell commands (Driver Node only)

%pip

Install Python libraries

%run

Include/ Import another notebook into the current notebook



Databricks Utilities

Databricks Utilities



File System Utilities

Secrets Utilities

Widget Utilities

Notebook Workflow Utilities



Databricks Git Folders / Repos

Git Folders is a visual Git client made available in the Databricks workspace to enable collaborative development amongst developers.

Notebook Version History



1.Notebooks Introduction

Python☆

FileEditViewRunHelpLast edit was 5 days ago

▶ Run all⌵ TerminatedScheduleShare⌵

You are viewing a version from **Nov 8 2024, 14:59 PM GMT**. [Exit](#)

%md

Introduction to Notebooks

Heading 2

- bullet 1

- bullet 2

This notebook gives you an introduction to databricks notebooks!

%md

Introduction to Notebooks

Heading 2

- bullet 1

- bullet 2

This notebook gives you an introduction to databricks notebooks!

Last execution failed

1print('hello')

2print('bye')

hello

bye

Assistant Quick Fix: ON

Last execution failed

1print('Hello Again')

Hello Again

Assistant Quick Fix: ON

Version history

[Clear history](#)

Nov 8 2024, 14:59 PM GMT

● Ramesh Retnasamy

[Save now](#)

Nov 8 2024, 14:55 PM GMT

● Ramesh Retnasamy

Nov 8 2024, 14:53 PM GMT

● Ramesh Retnasamy

Nov 8 2024, 14:52 PM GMT

● Ramesh Retnasamy

Nov 8 2024, 14:50 PM GMT

● Ramesh Retnasamy

Nov 8 2024, 14:48 PM GMT

● Ramesh Retnasamy

Nov 8 2024, 14:44 PM GMT

● Ramesh Retnasamy

Nov 8 2024, 14:39 PM GMT

● Ramesh Retnasamy

Nov 8 2024, 14:36 PM GMT

● Ramesh Retnasamy

© Ramesh Retnasamy | www.CloudBoxAcademy.com | All Rights Reserved

Notebook Version History

Limitations



Limited to Single Notebook

Basic Tracking Only / No Branching

Lack of integration with CI/CD pipelines

Databricks Git Folders / Repos



Holistic Version Control

Team Collaboration

Automated CI/CD Pipelines

Databricks Git Folders / Repos

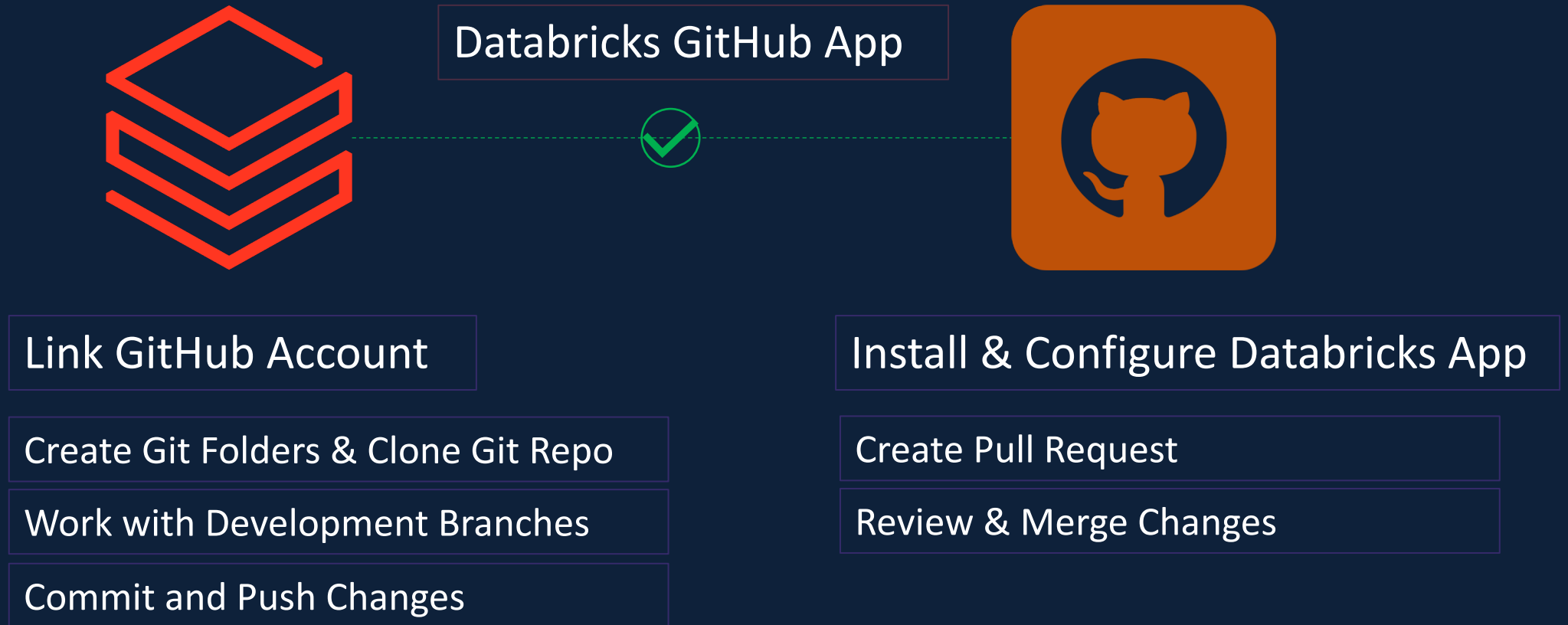


Databricks GitHub App

Personal Access Token



Databricks Git Folders / Repos



Databricks Git Folders / Repos - Demo



Databricks GitHub App



1

Link GitHub Account

3

Create Git Folders & Clone Git Repo

4

Work with Development Branches

5

Commit and Push Changes

2

Install & Configure Databricks App

6

Create Pull Request

7

Review & Merge Changes

Unity Catalog - Introduction



Unity Catalog Overview

Enable Unity Catalog Metastore

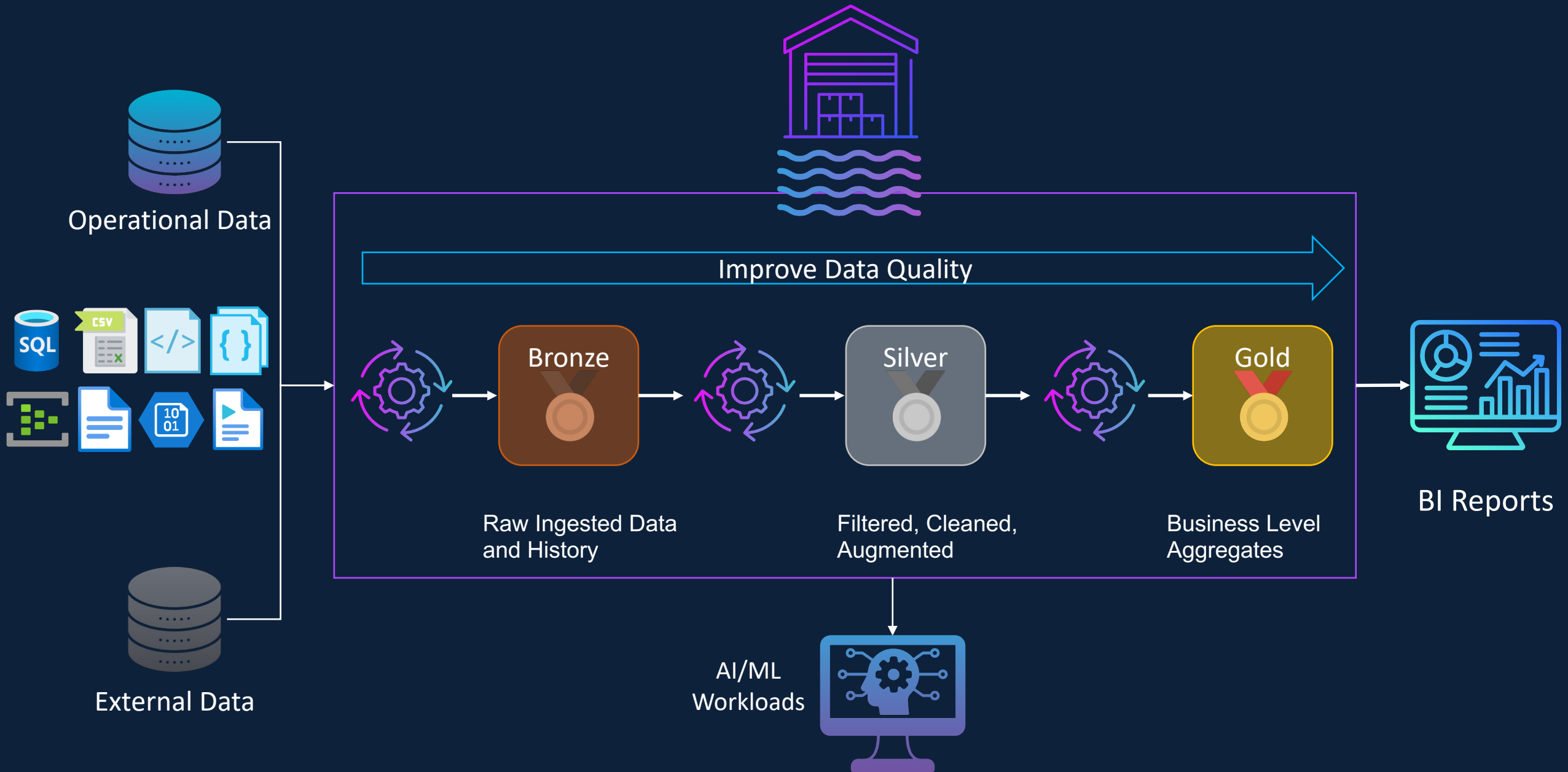
Cluster Configurations

Unity Catalog Object Model

Access External Data Lake

Introduction to Unity Catalog

Medallion Architecture



Data Access Patterns



Users and
Applications



Compute

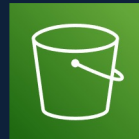
Read / Write



Data Storage



ADLS

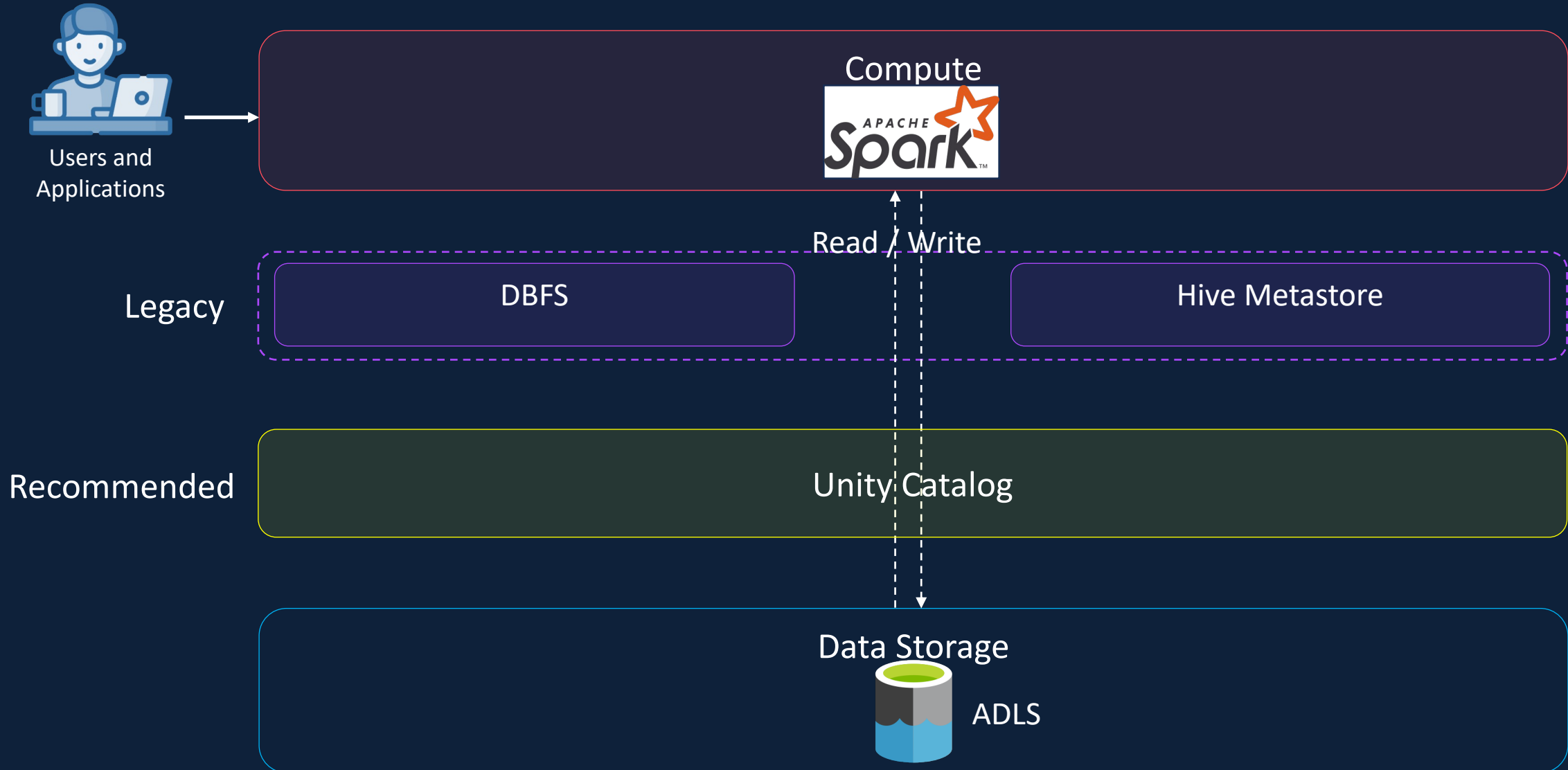


AWS S3

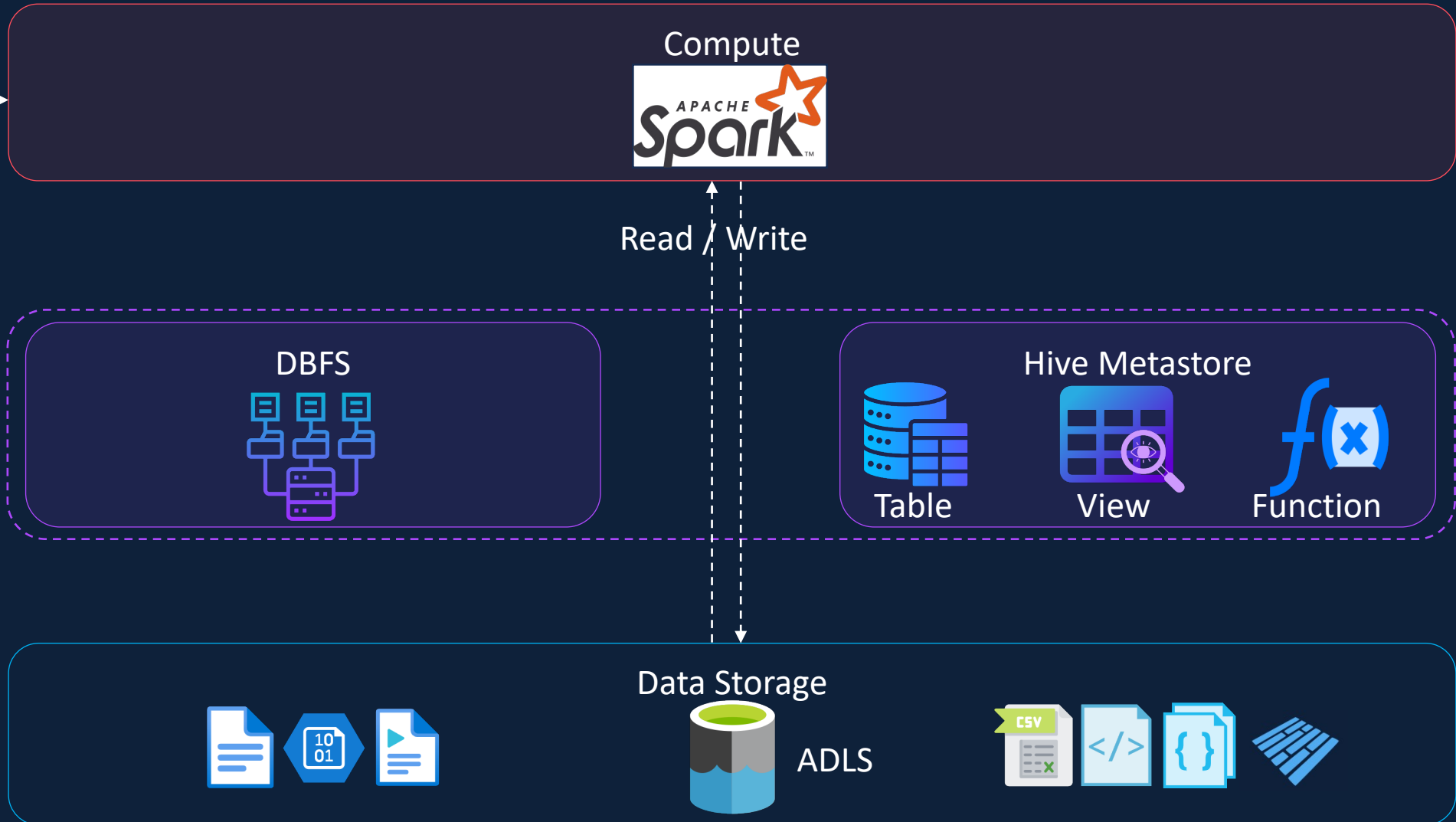


GCS

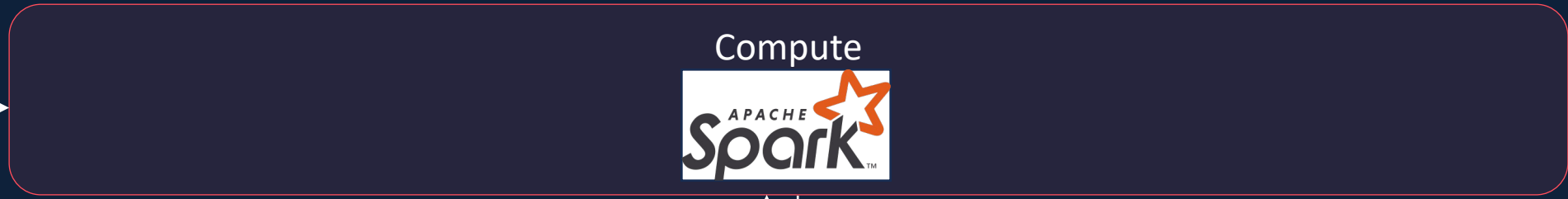
Data Access Patterns



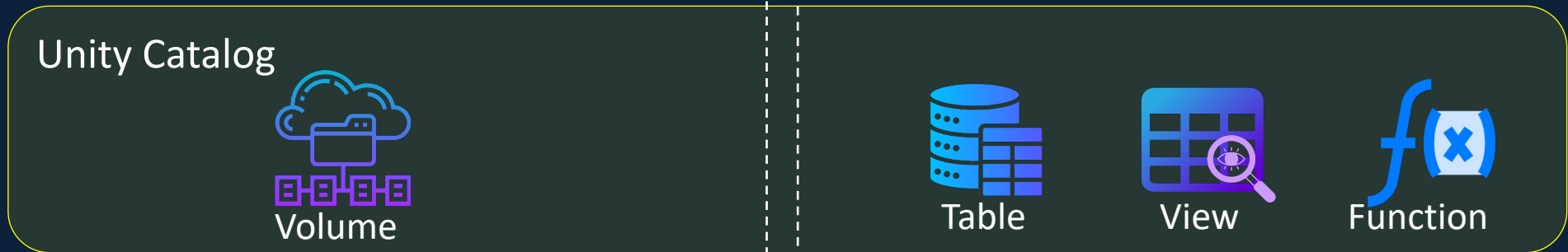
Data Access Patterns



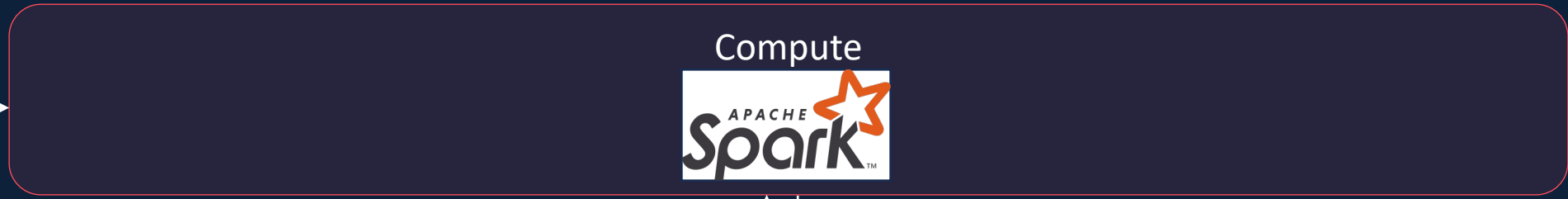
Data Access Patterns



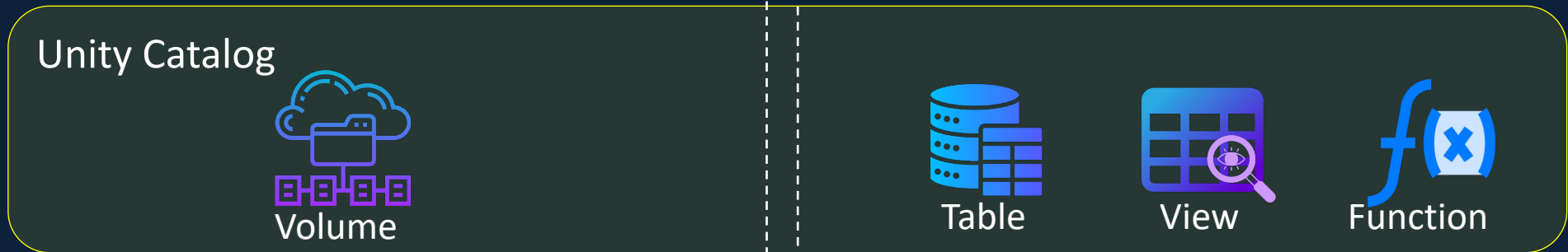
Read / Write



Unity Catalog Object Model



Read / Write



Unity Catalog Object Model

A dark green rounded rectangle with a thin yellow border, containing the word "Metastore" in white text.

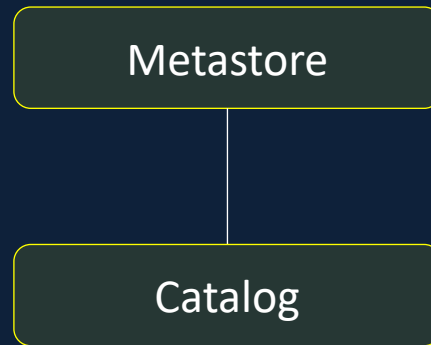
Metastore

Top level container

Only one per region

Paired with Default ADLS Storage

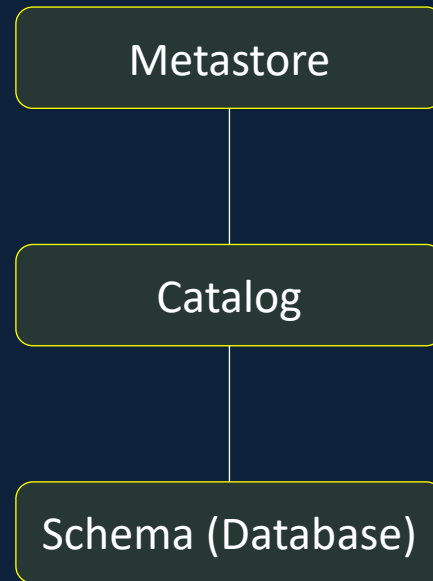
Unity Catalog Object Model



Newly added to Unity Catalog

Logical container within Metastore

Unity Catalog Object Model

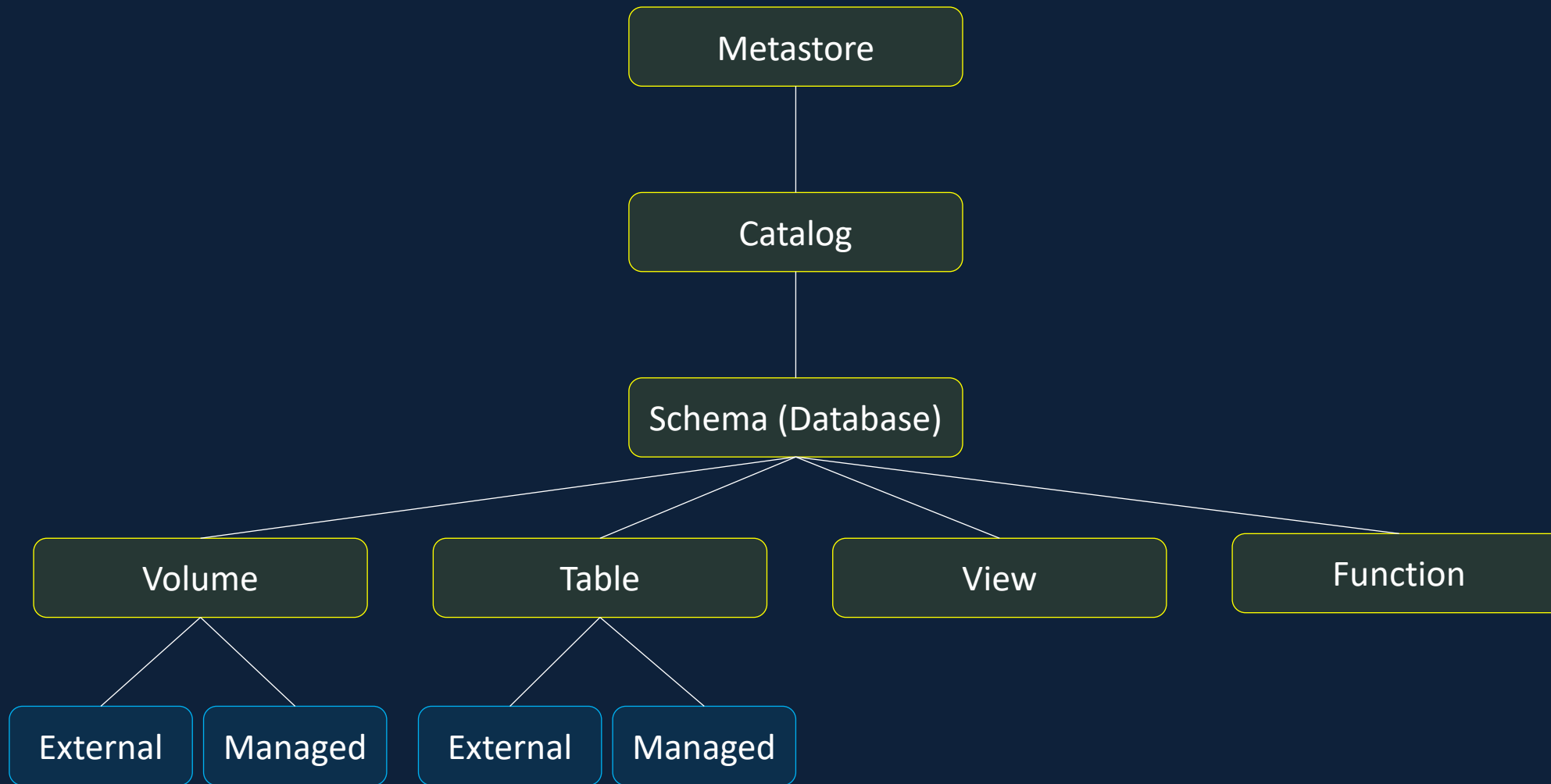


Next level container within Catalogs

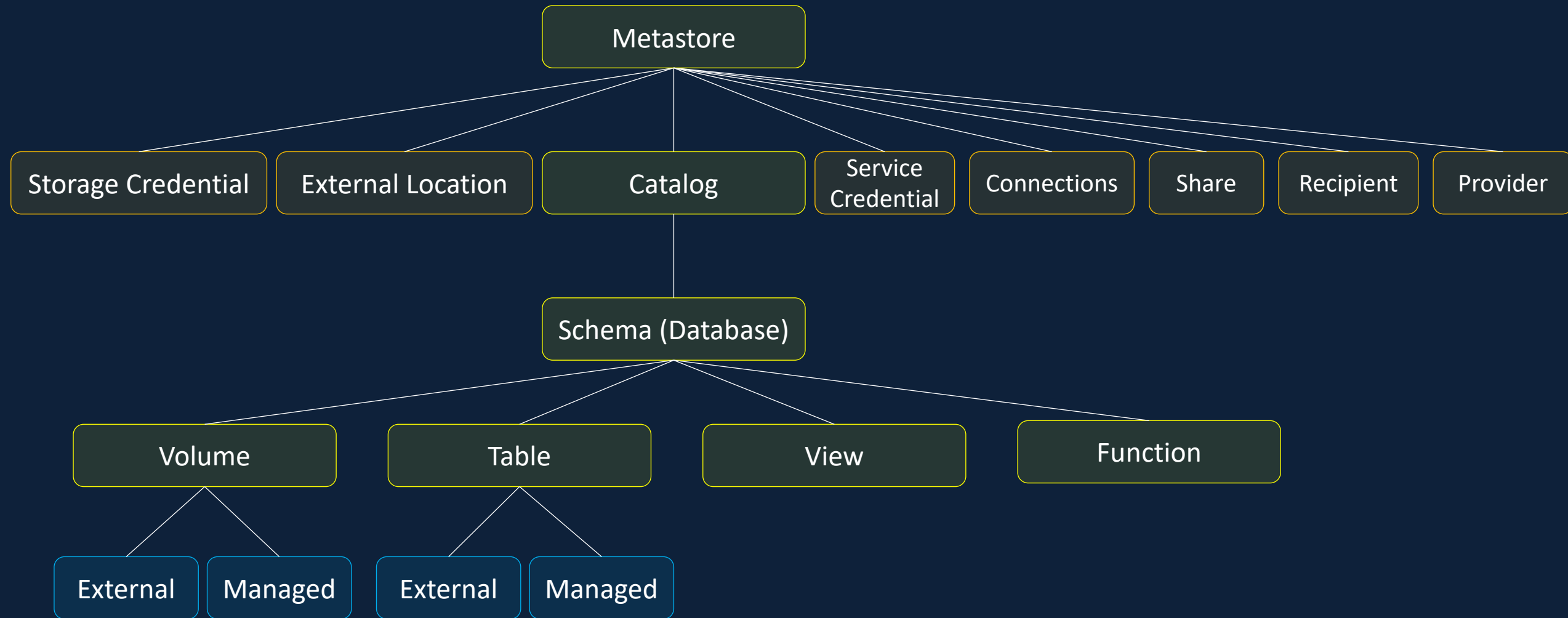
Schemas and Databases are the same

Use Schema instead of Database

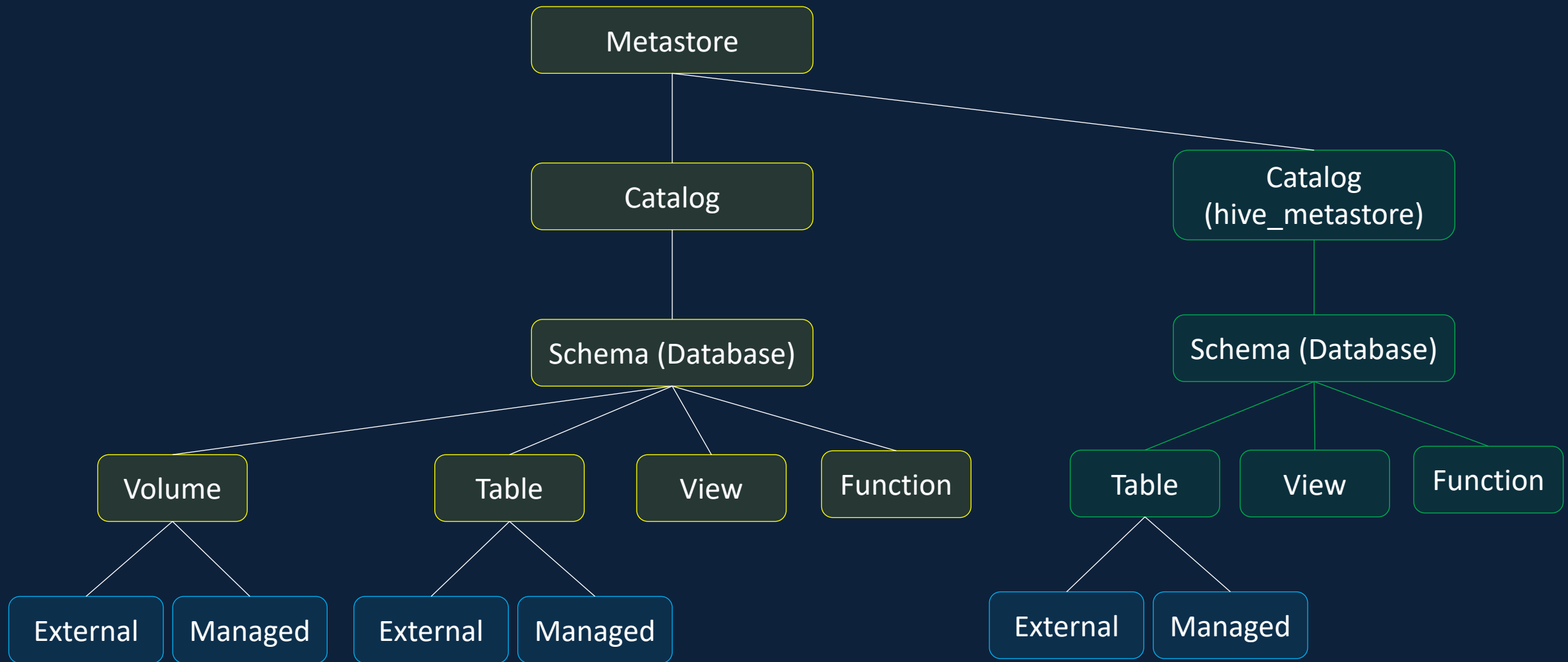
Unity Catalog Object Model



Unity Catalog Object Model

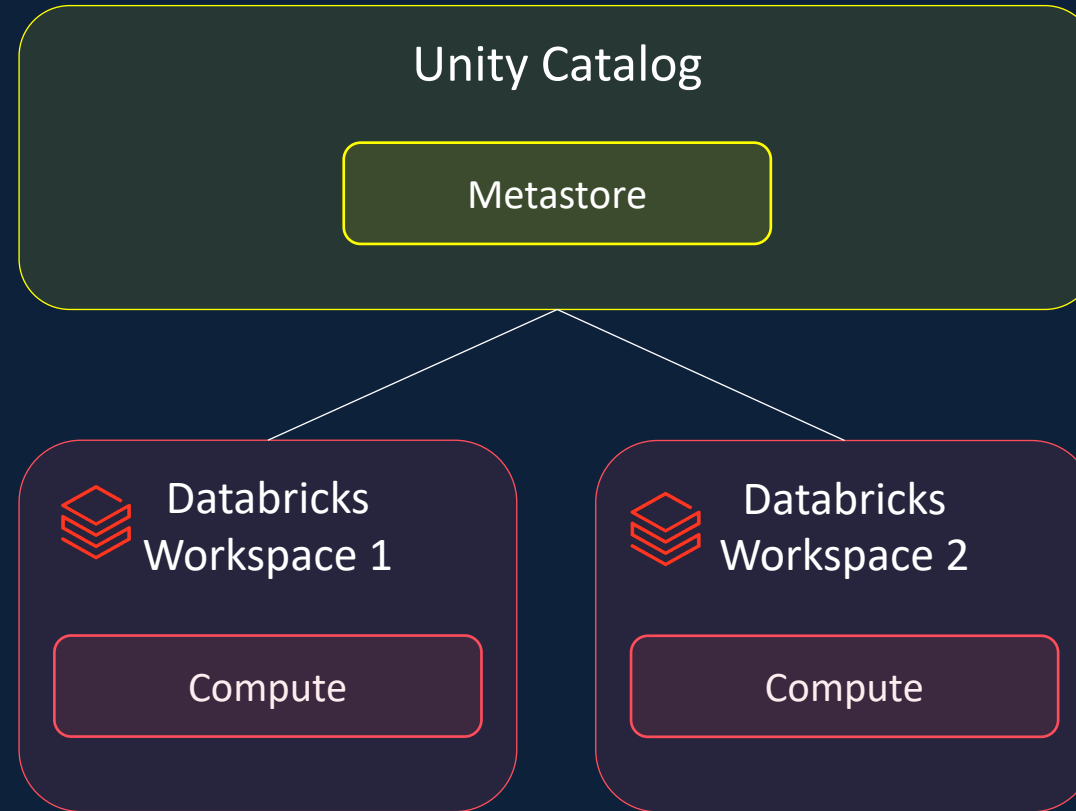


Unity Catalog / Hive Metastore Object Model

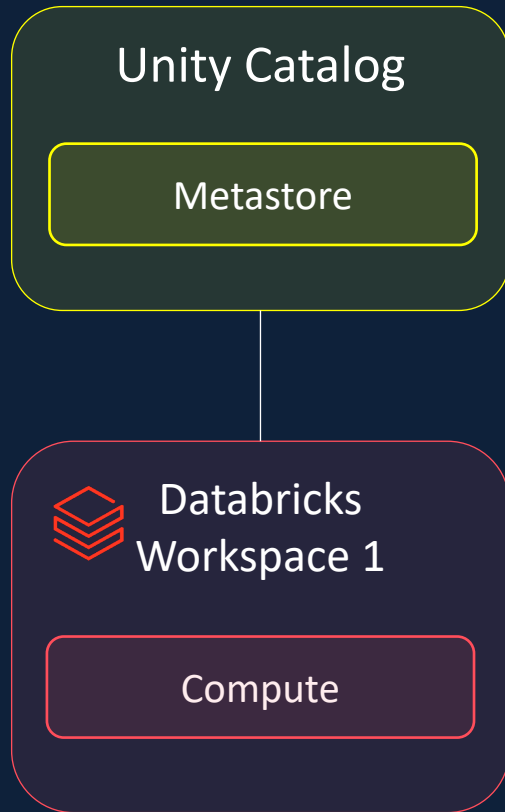


Unity Catalog Set-up

Unity Catalog Set-up



Unity Catalog Set-up



Login to Databricks Account Console

<https://accounts.azuredatabricks.net/>

User Must have Global Administrator Privileges

Create Unity Catalog Metastore

Assign Databricks Workspace to the Unity Catalog Metastore

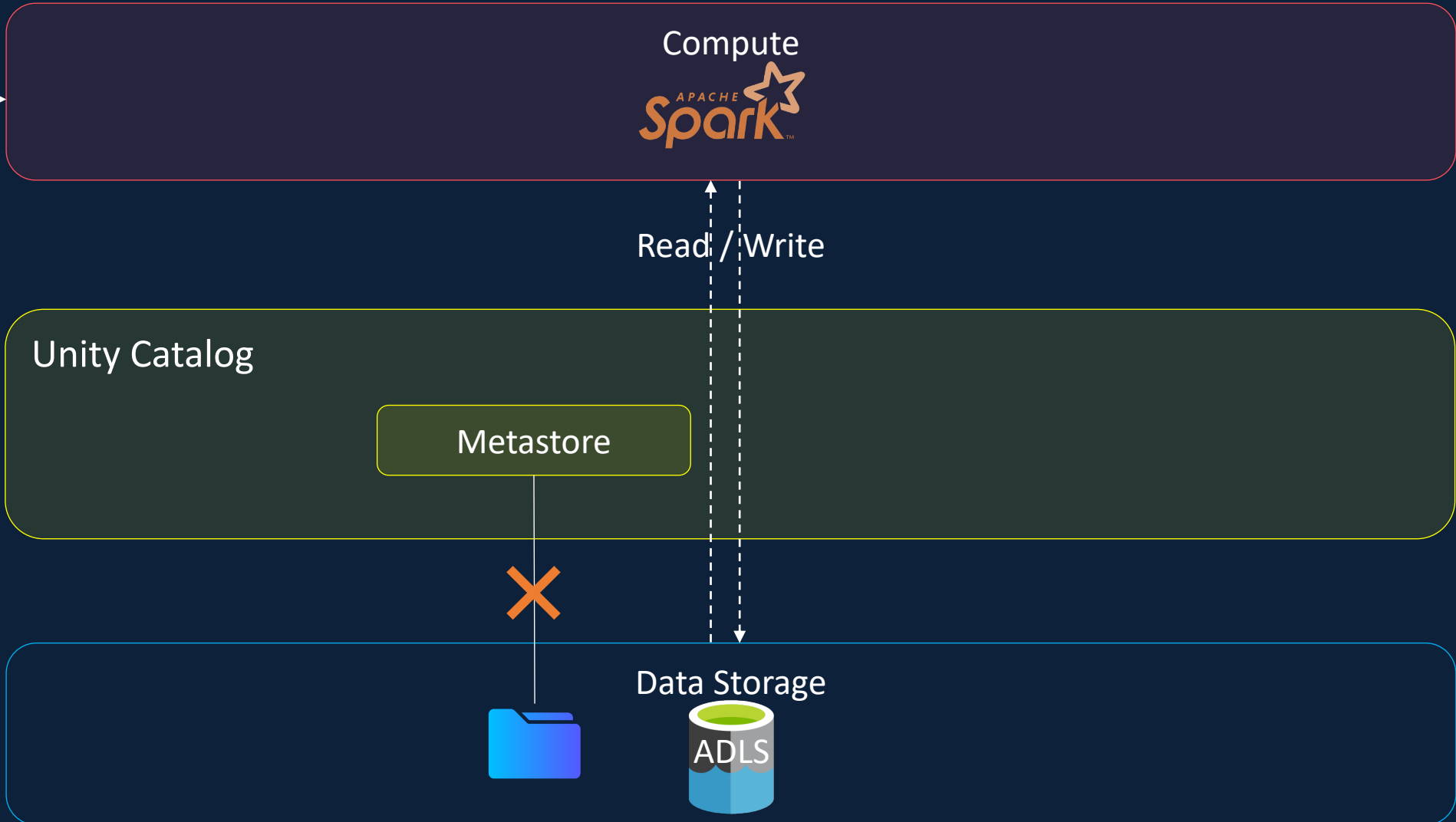
Azure Subscriptions Created on or after Nov 2023 have Unity Catalog Metastore Created by Default.

Cluster Configuration

Configure Access to Cloud Storage



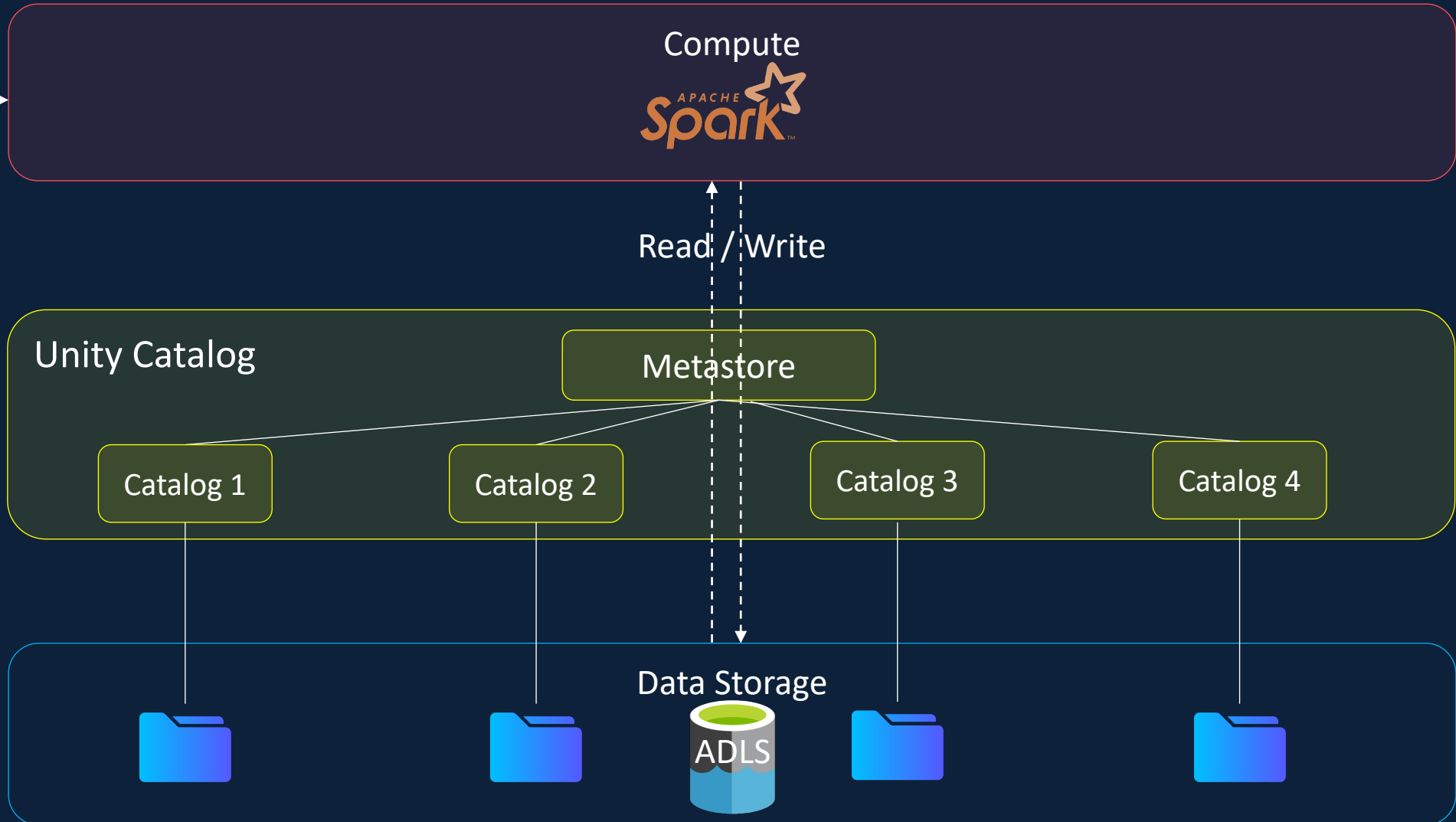
Users and Applications



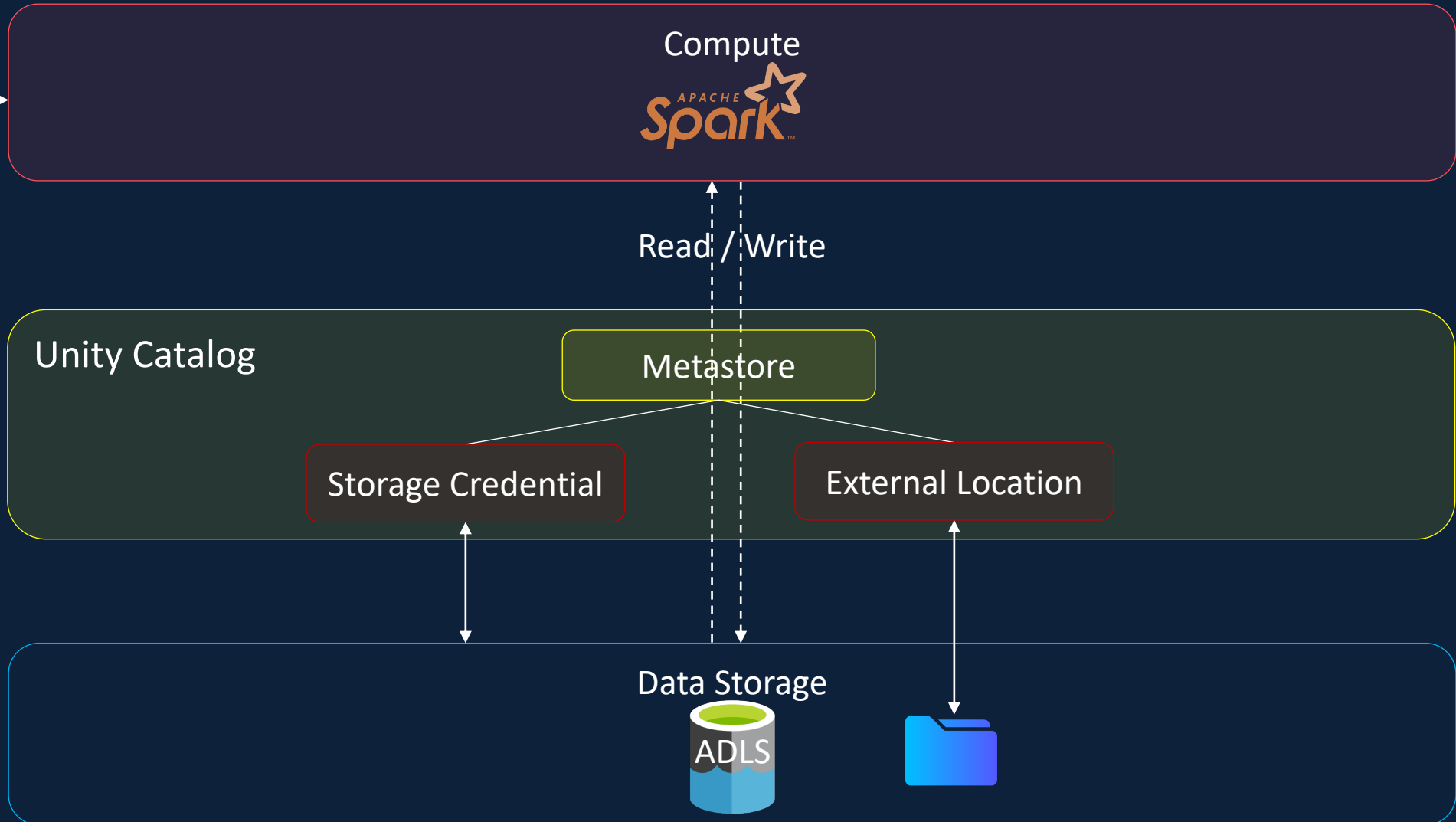
Configure Access to Cloud Storage



Users and Applications



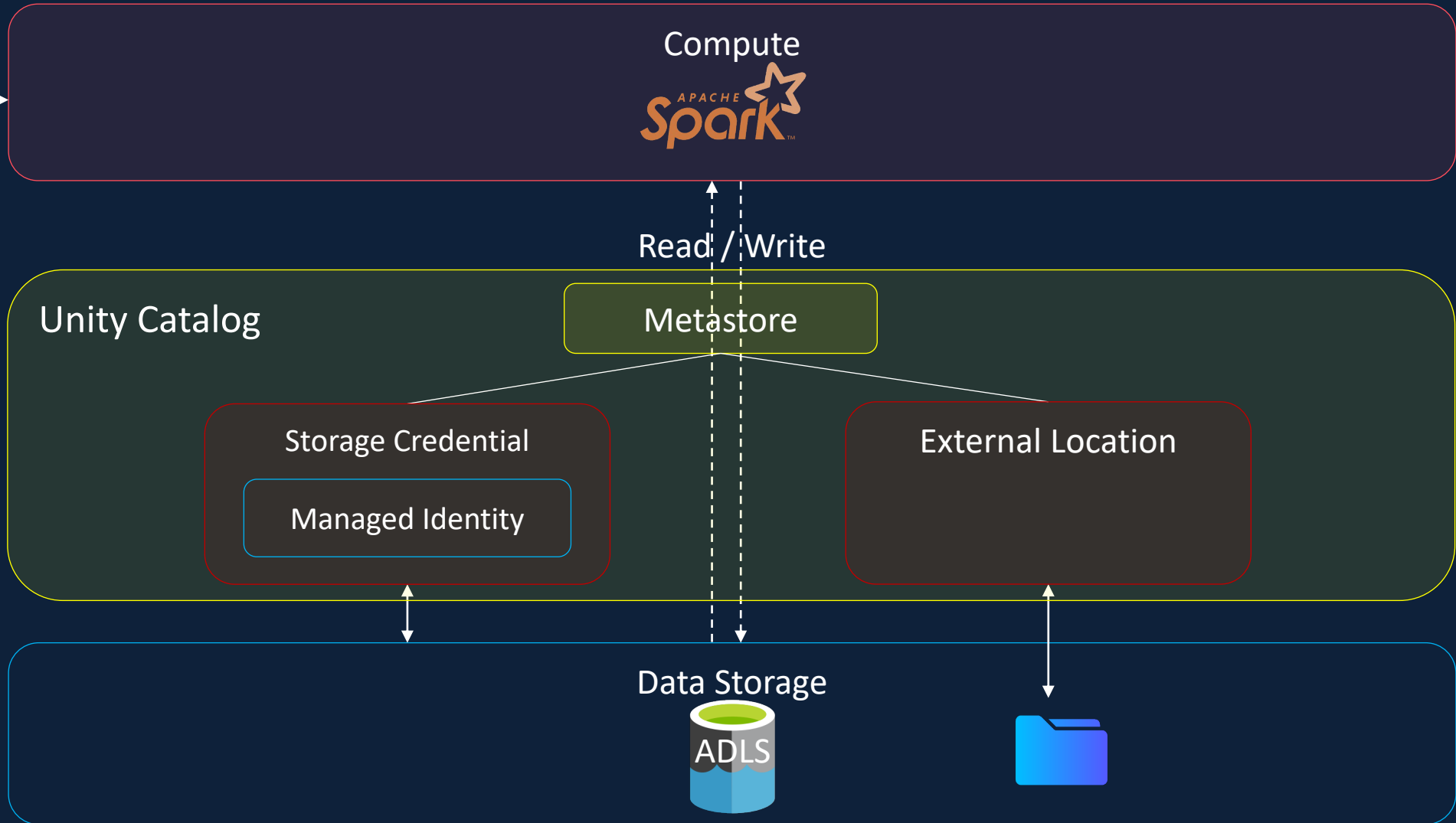
Configure Access to Cloud Storage



Configure Access to Cloud Storage



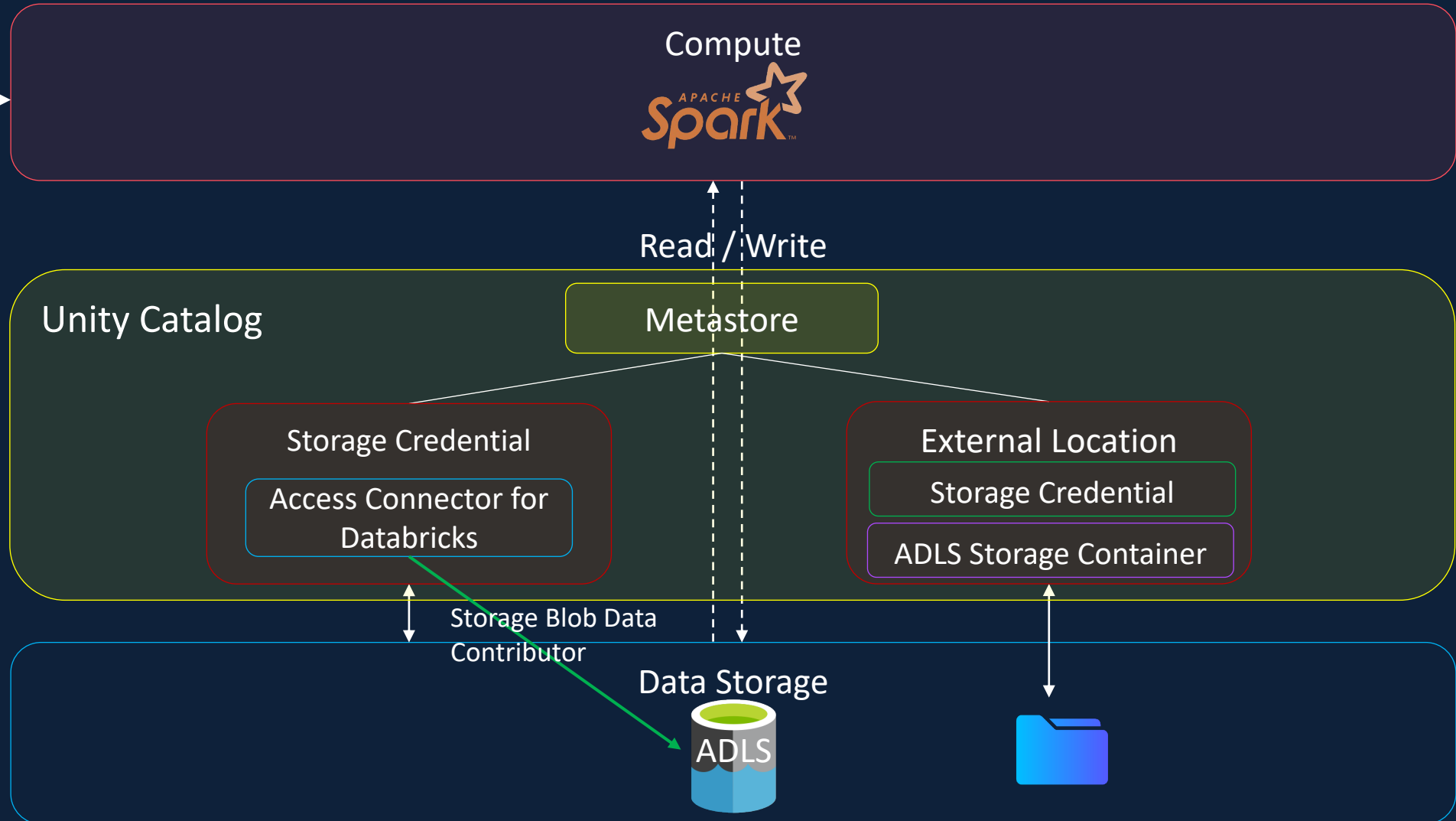
Users and Applications



Configure Access to Cloud Storage



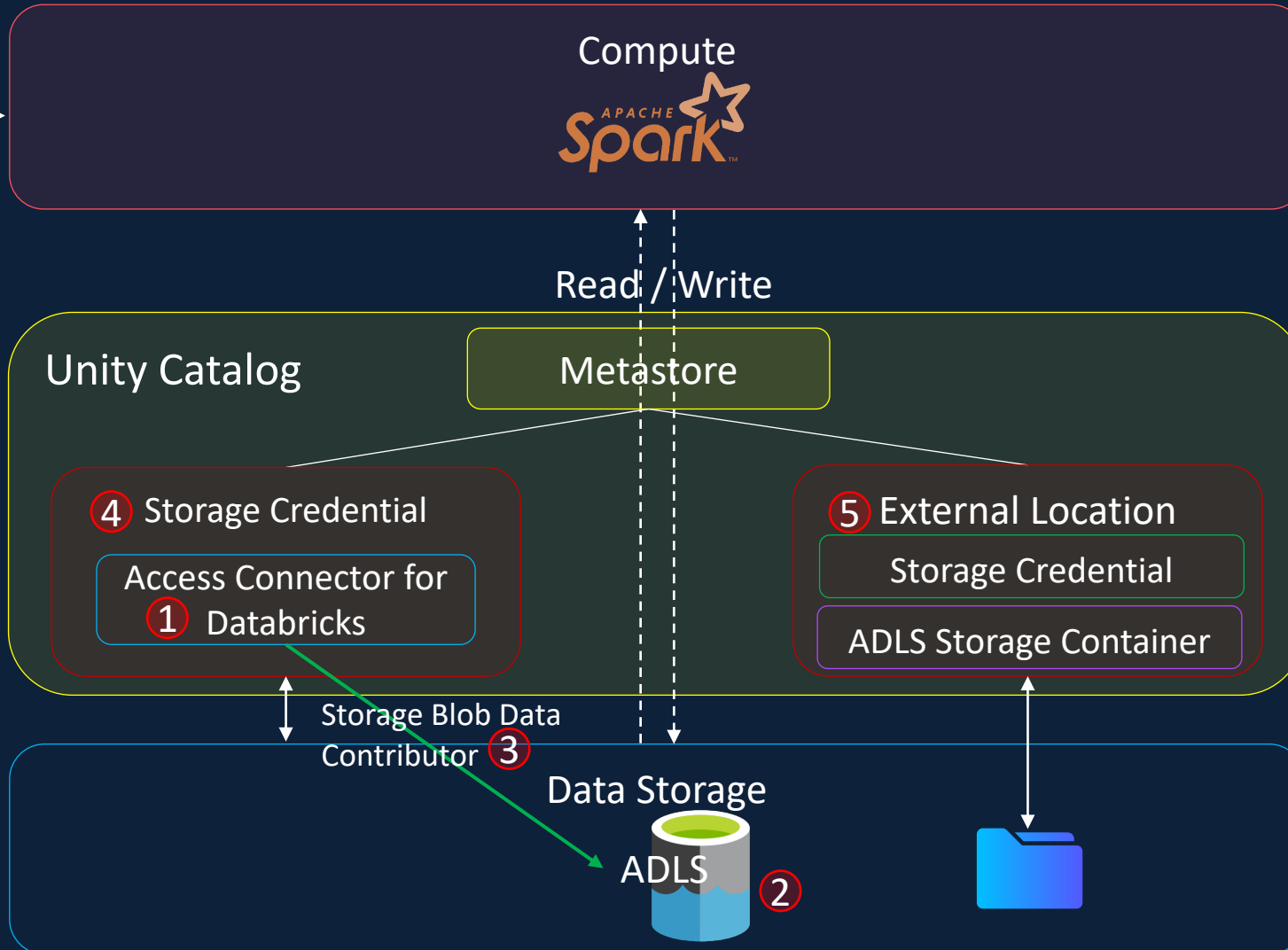
Users and Applications



Configure Access to Cloud Storage



Users and Applications



1. Create Access Connector

2. Create Azure Data Lake Storage

3. Assign Storage Blob Data Contributor role

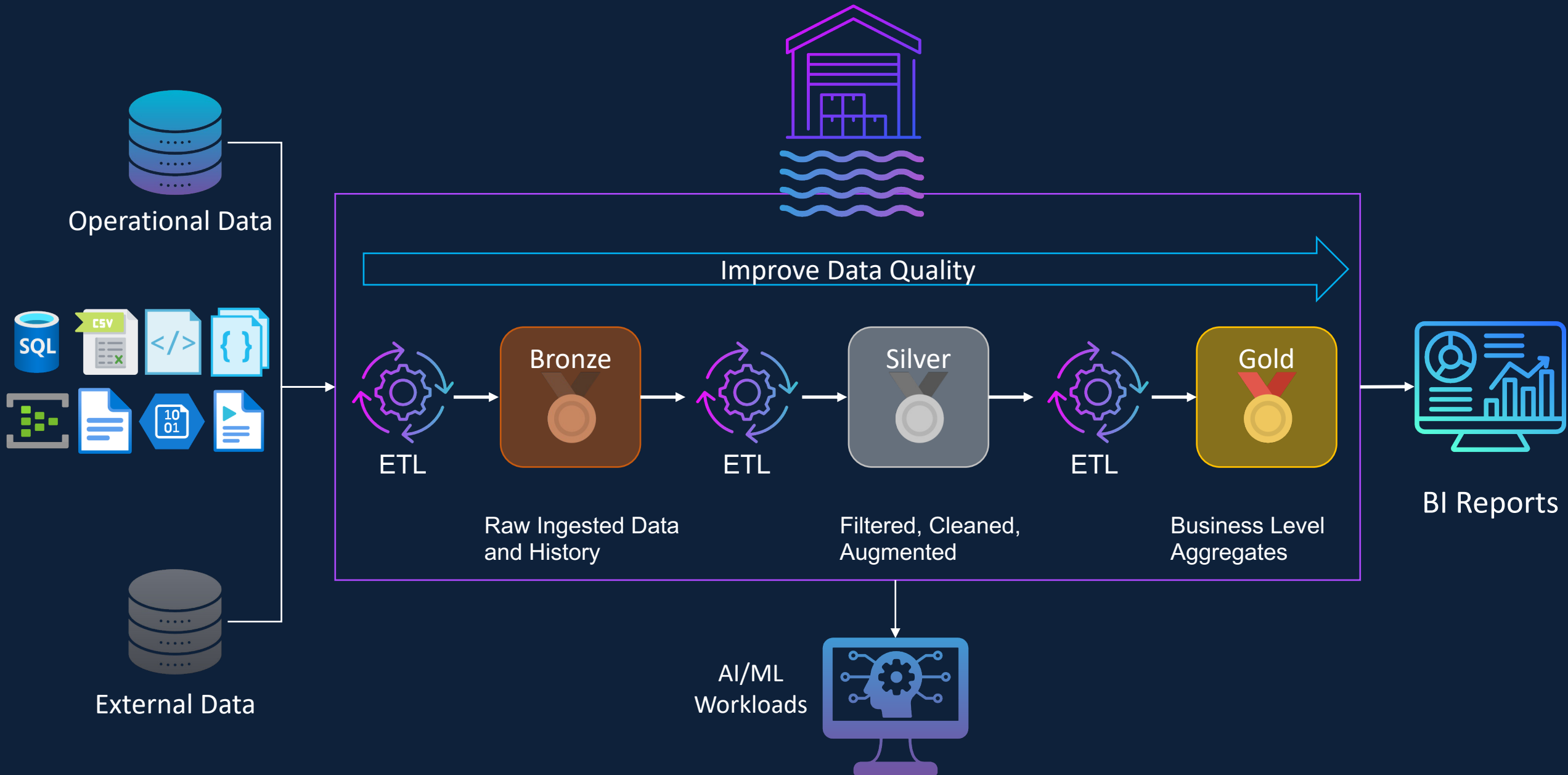
4. Create Storage Credential

5. Create External Location

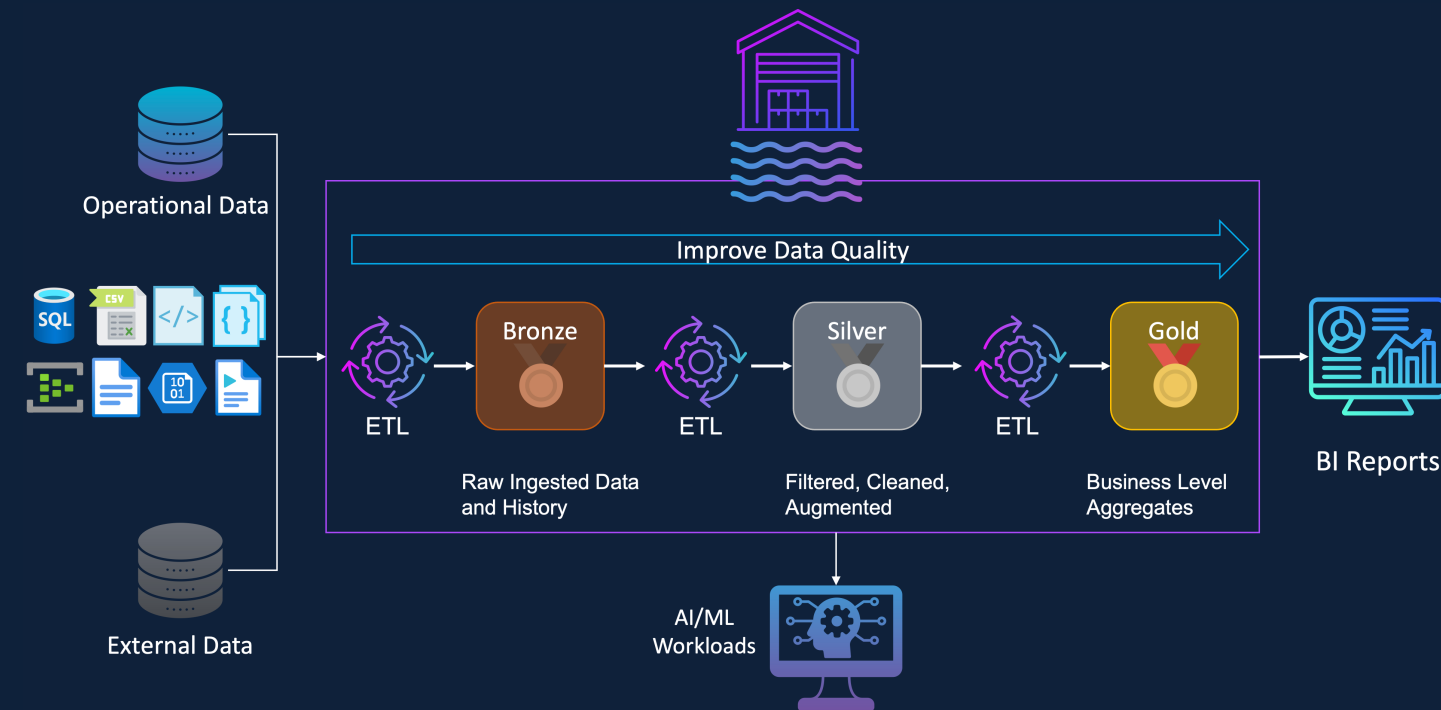
ETL With Apache Spark

Introduction

ETL With Apache Spark



ETL With Apache Spark



Query Files / Folders

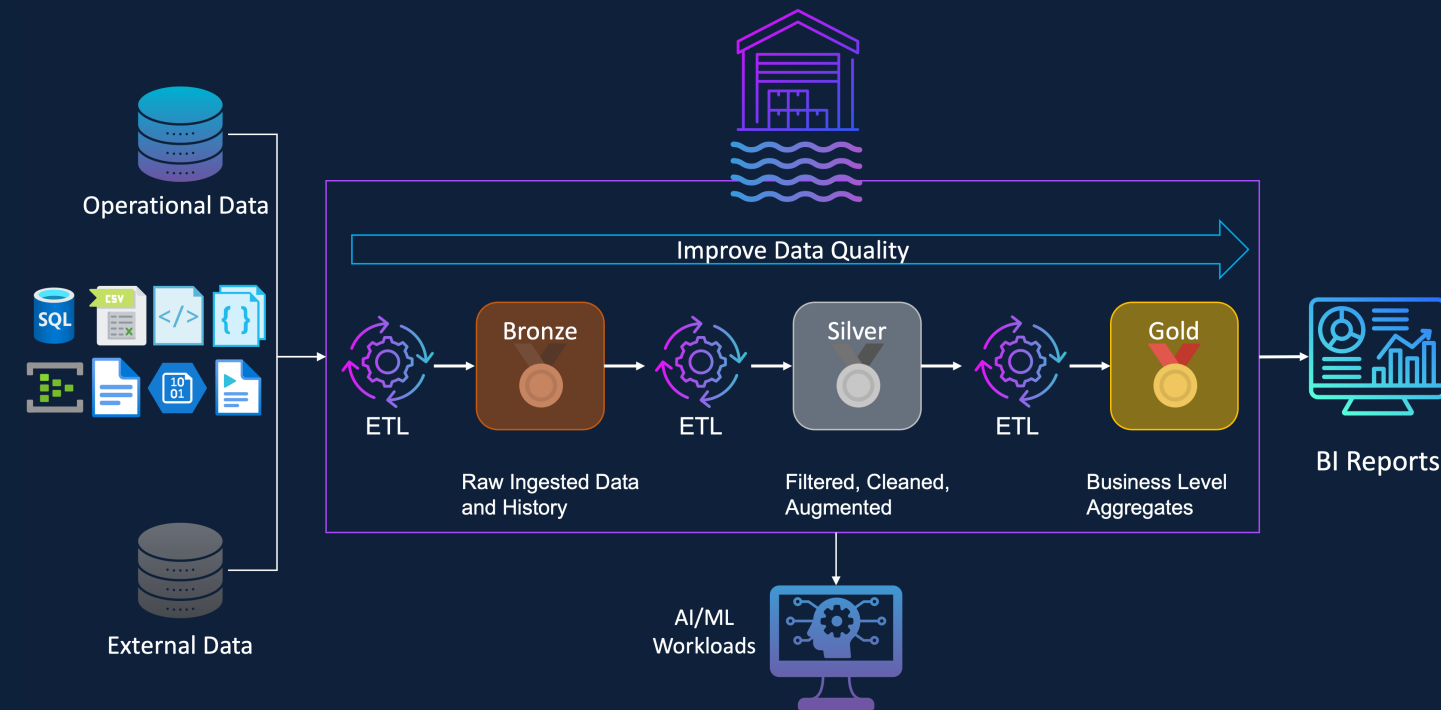
Query External Tables (Azure SQL, MySQL)

Validate and Clean the Data

Transform Data

Create Business Level Aggregates

ETL With Apache Spark



CSV, JSON and Binary Files

Access External Data via JDBC

Direct Query, External Tables, Views

Simple & Complex JSON Structures

Validate and Fix Data Issues

Simple Transformations

User Defined Functions

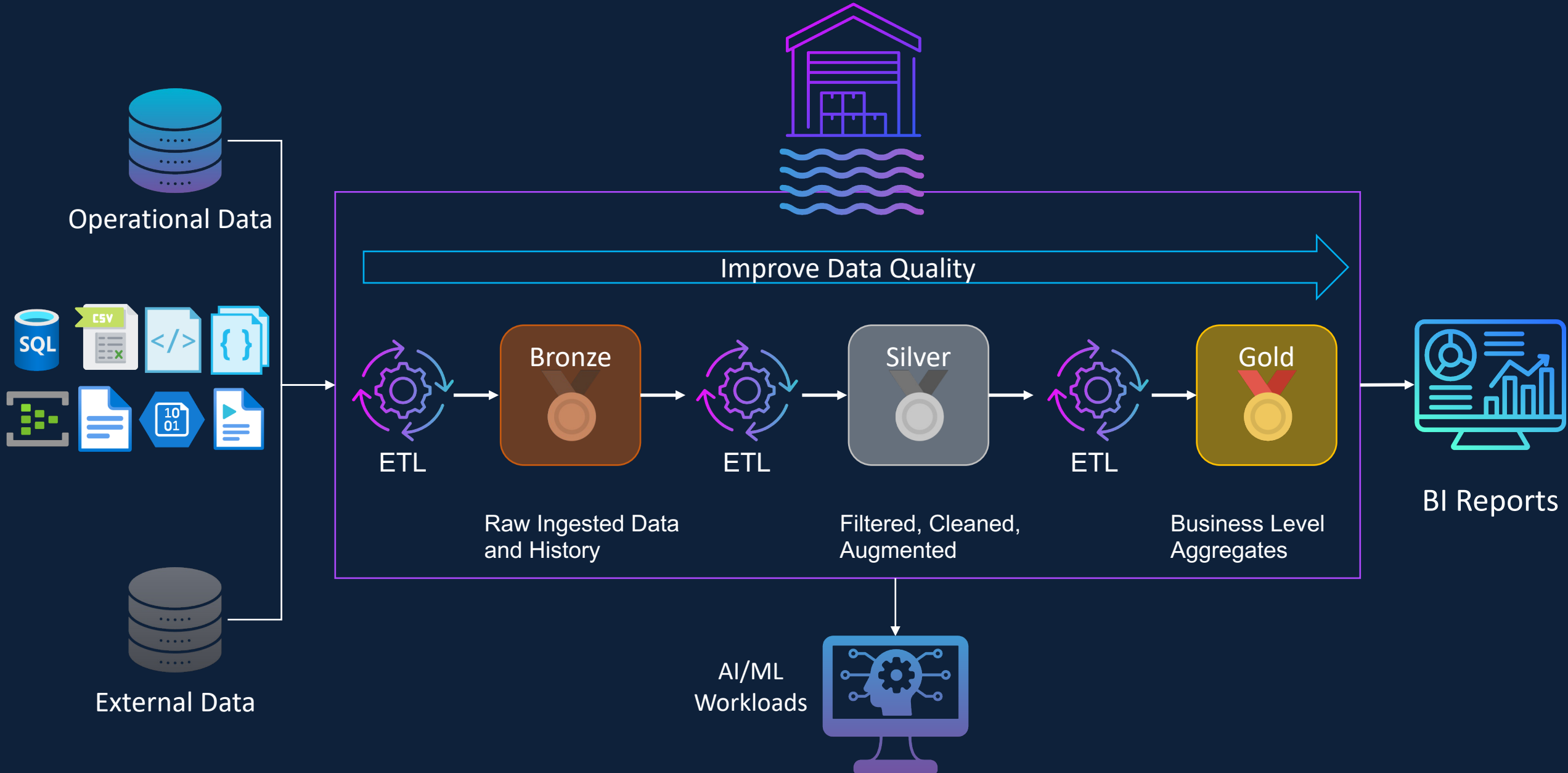
Create Tables

Spark SQL & Python

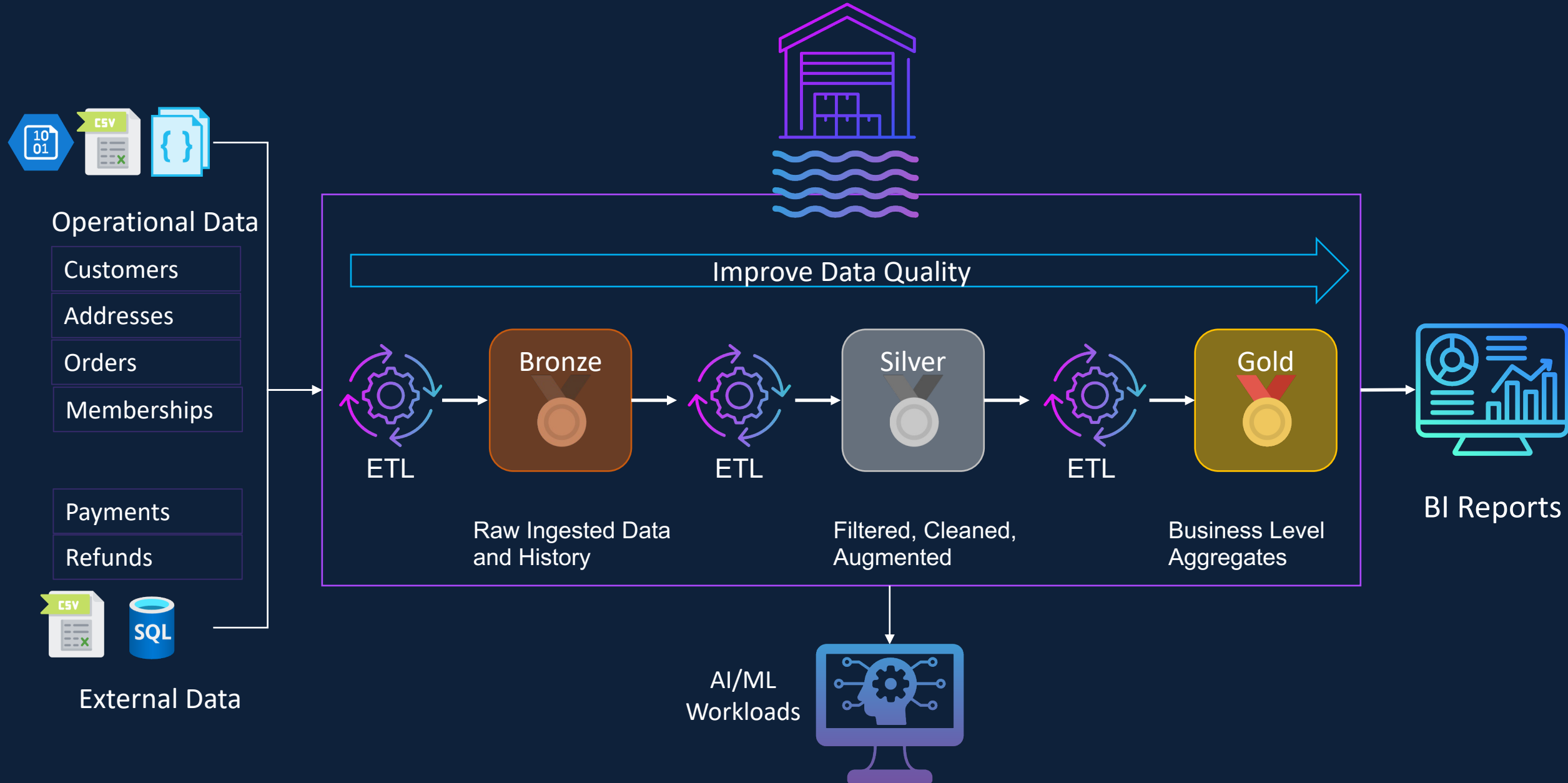
ETL With Apache Spark

Project Introduction

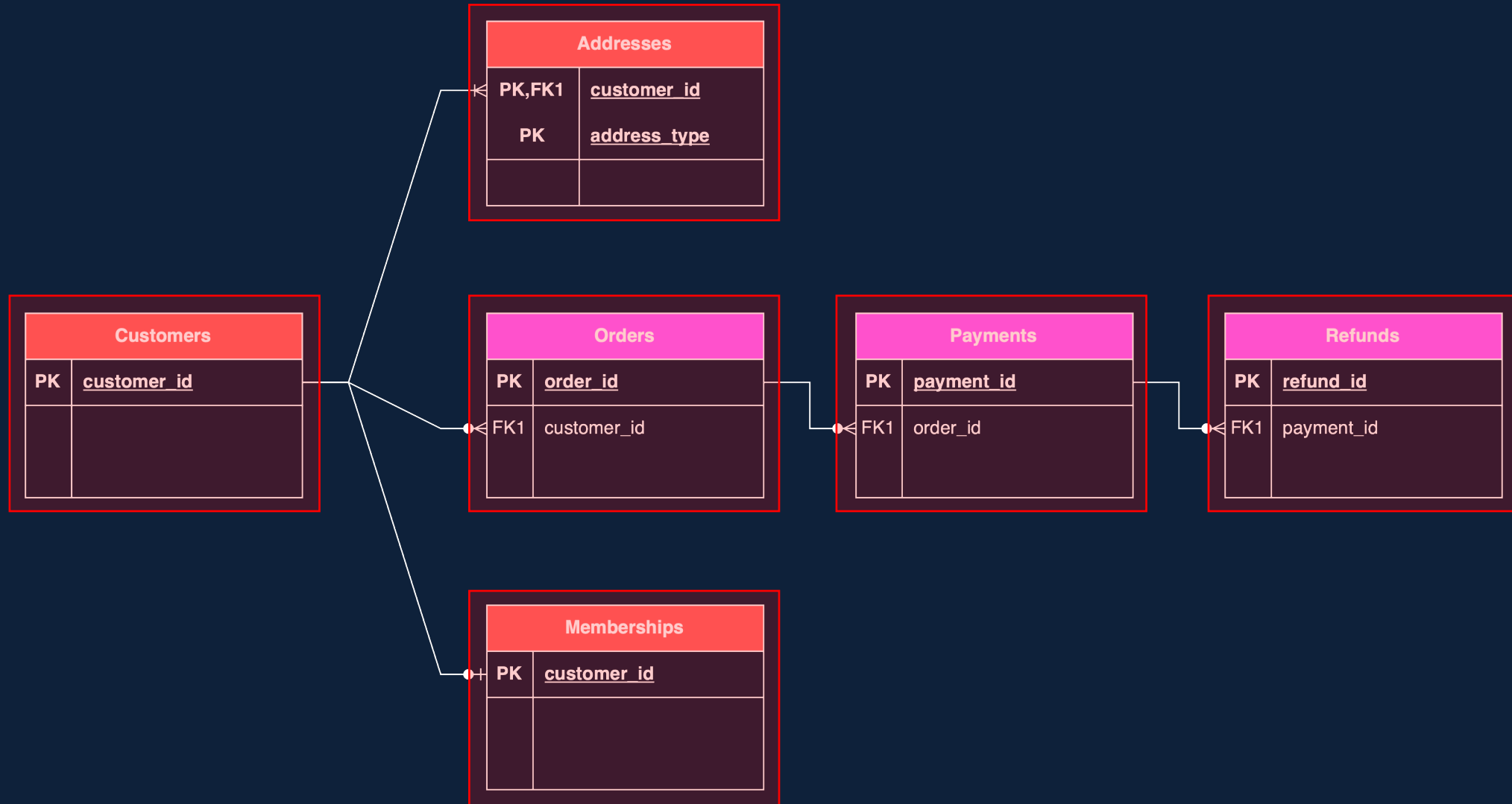
Data Lakehouse Project - GizmoBox



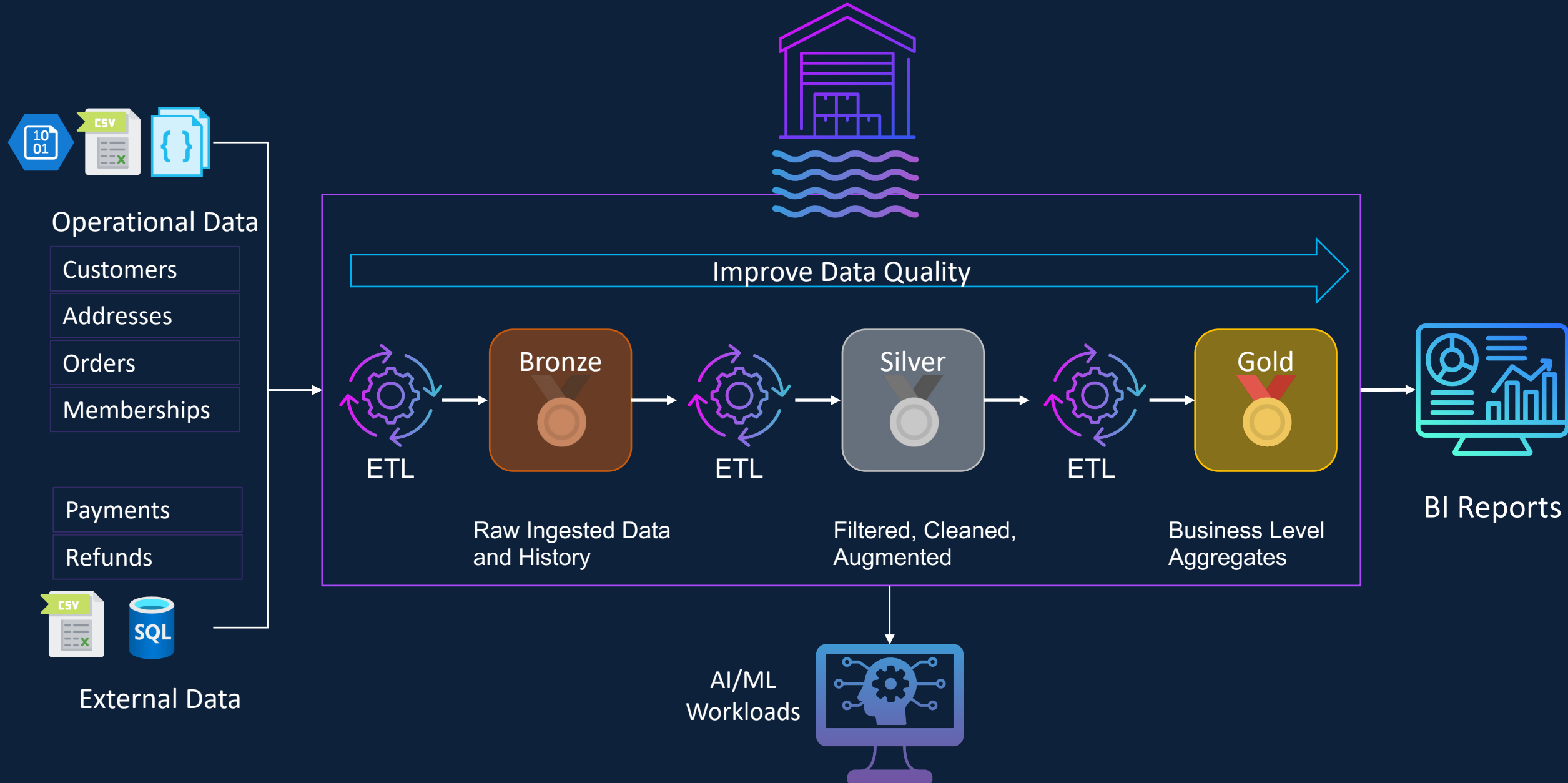
Data Lakehouse Project - GizmoBox



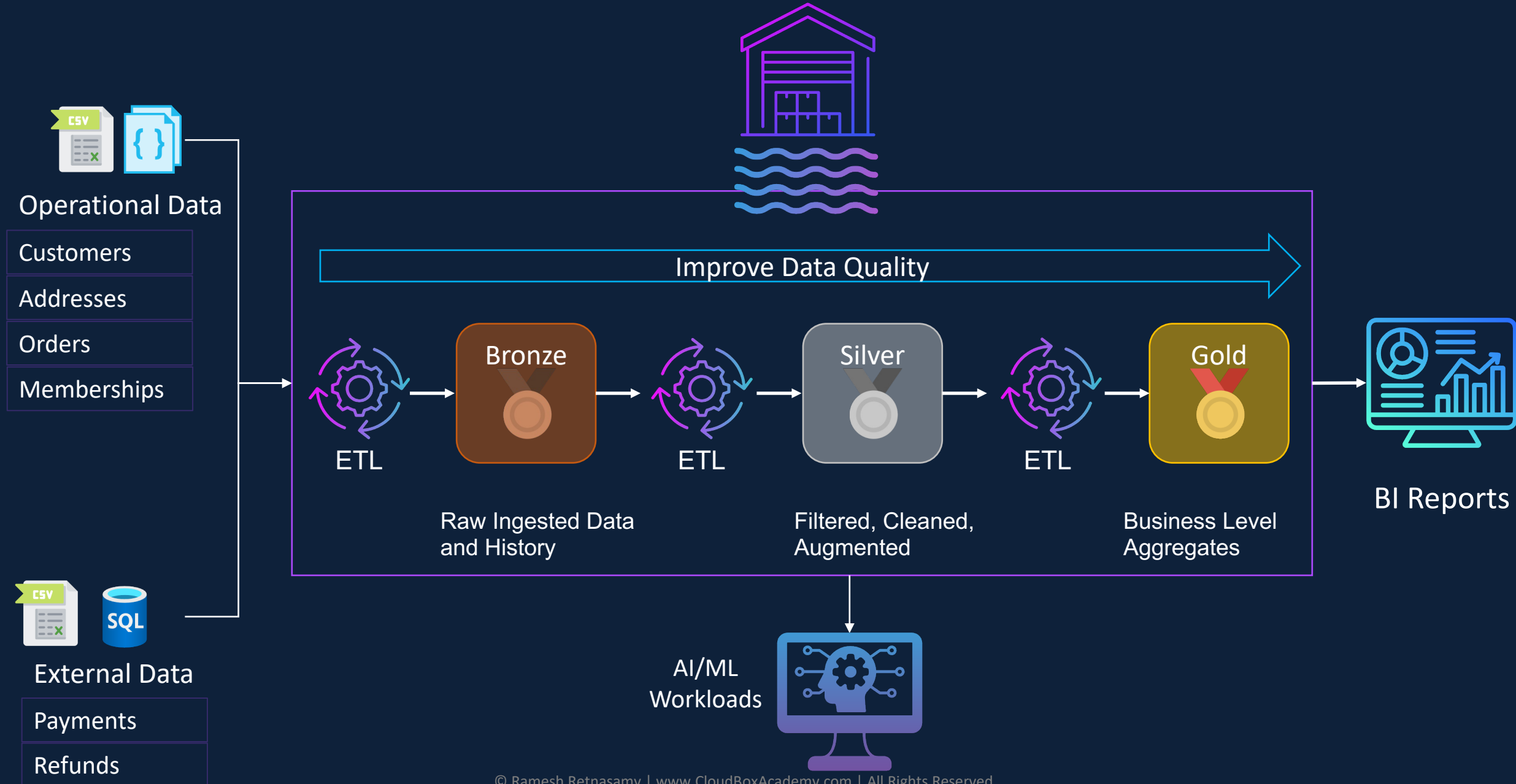
Data Lakehouse Project - GizmoBox



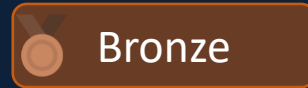
Data Lakehouse Project - GizmoBox



Data Architecture - GizmoBox



Data Architecture - GizmoBox



Operational Data

Customers

Addresses

Orders

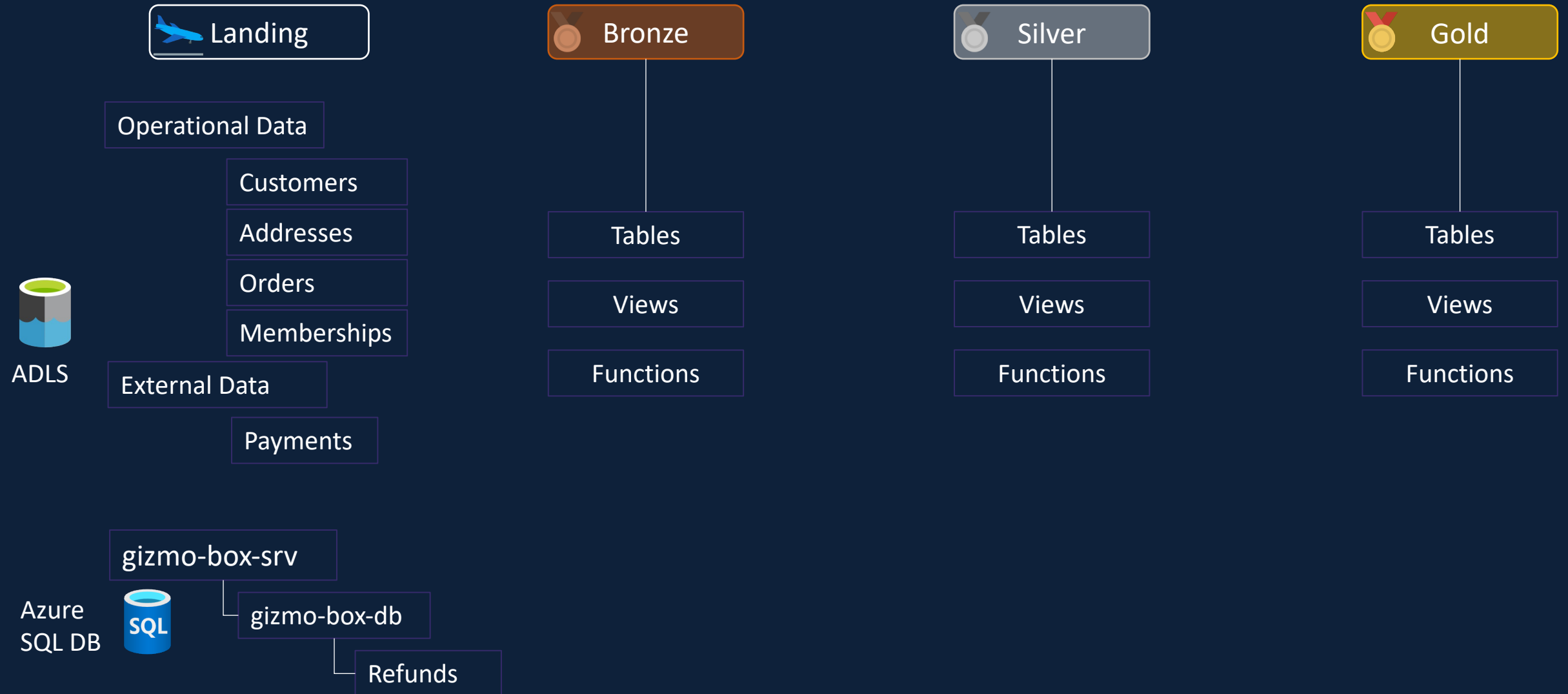
Memberships

External Data

Payments

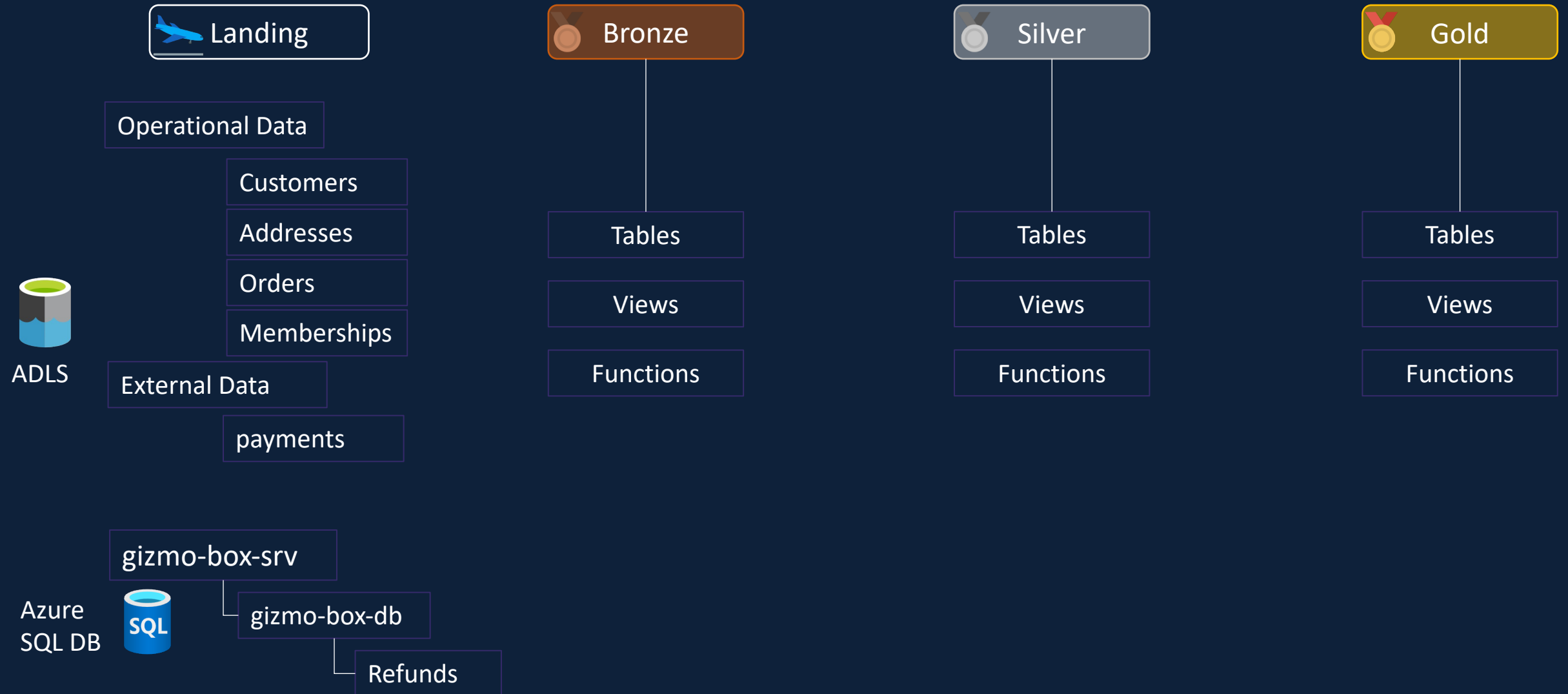
Refunds

Data Architecture - GizmoBox



Set-up Project Environment

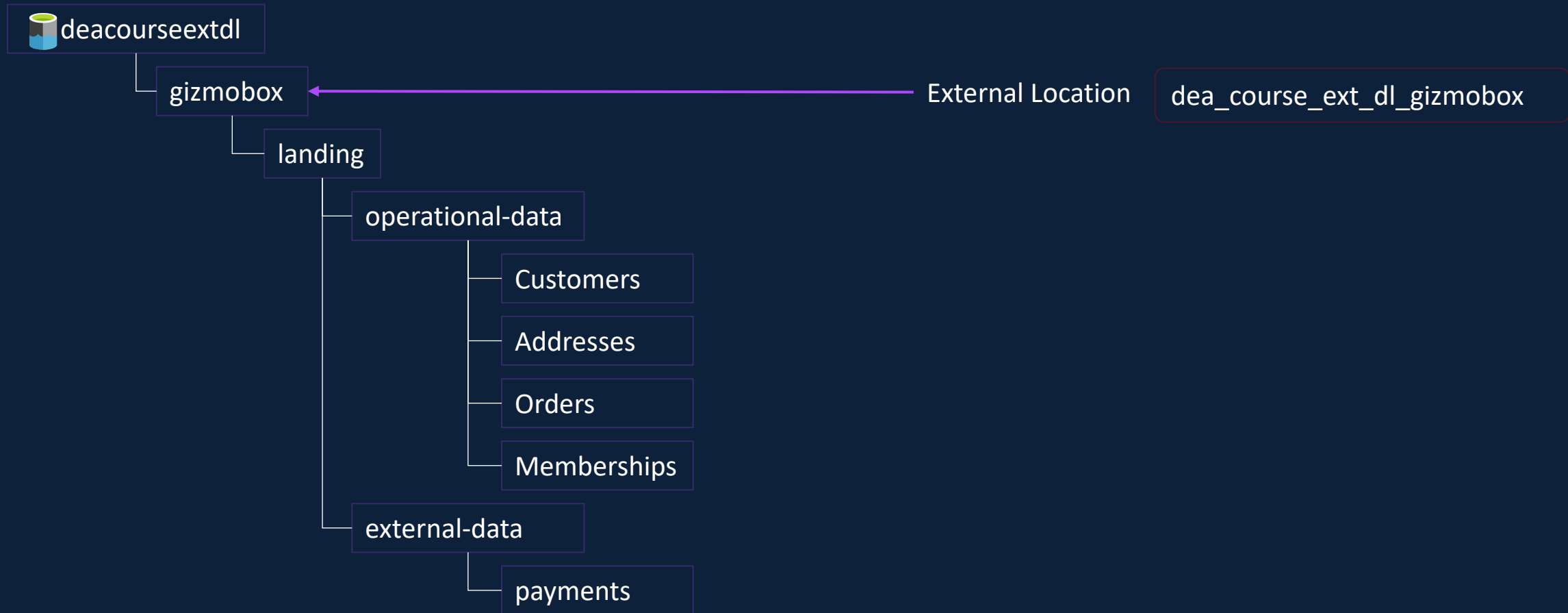
Data Architecture - GizmoBox



Data Architecture - GizmoBox

Azure Data Lake Storage Set-Up

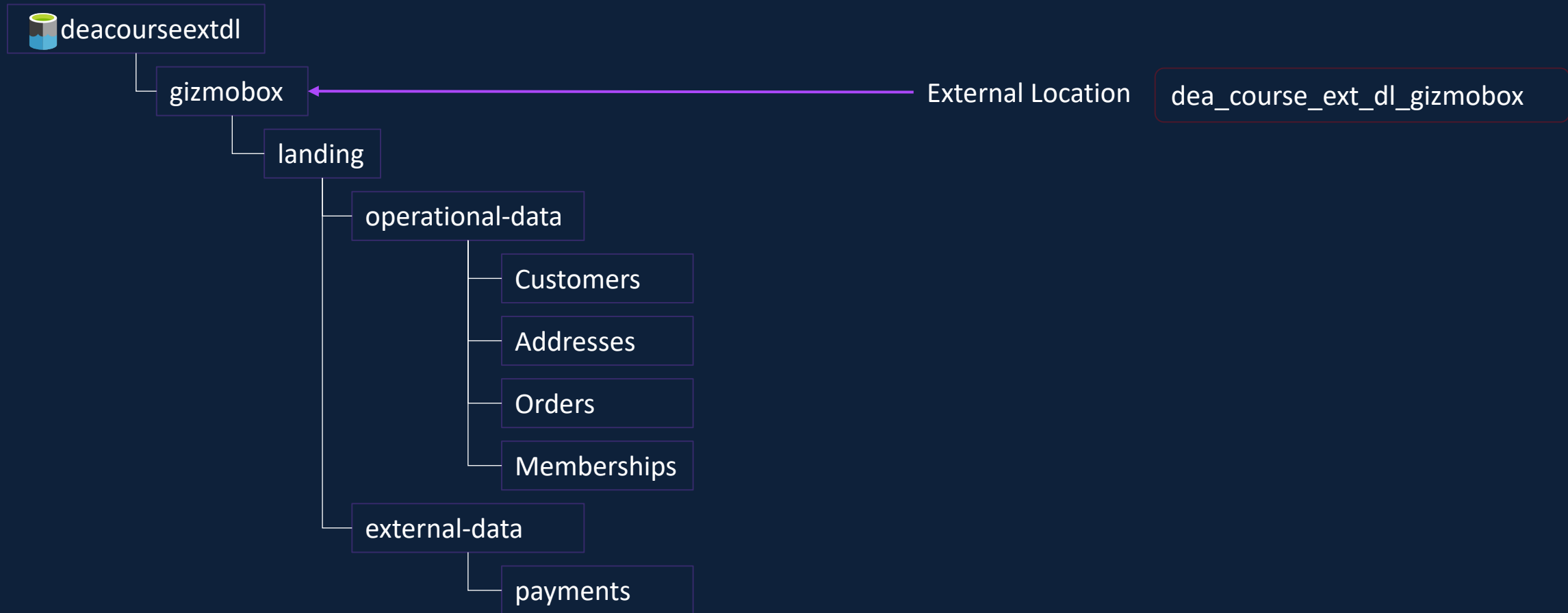
Unity Catalog Set-up



Data Architecture - GizmoBox

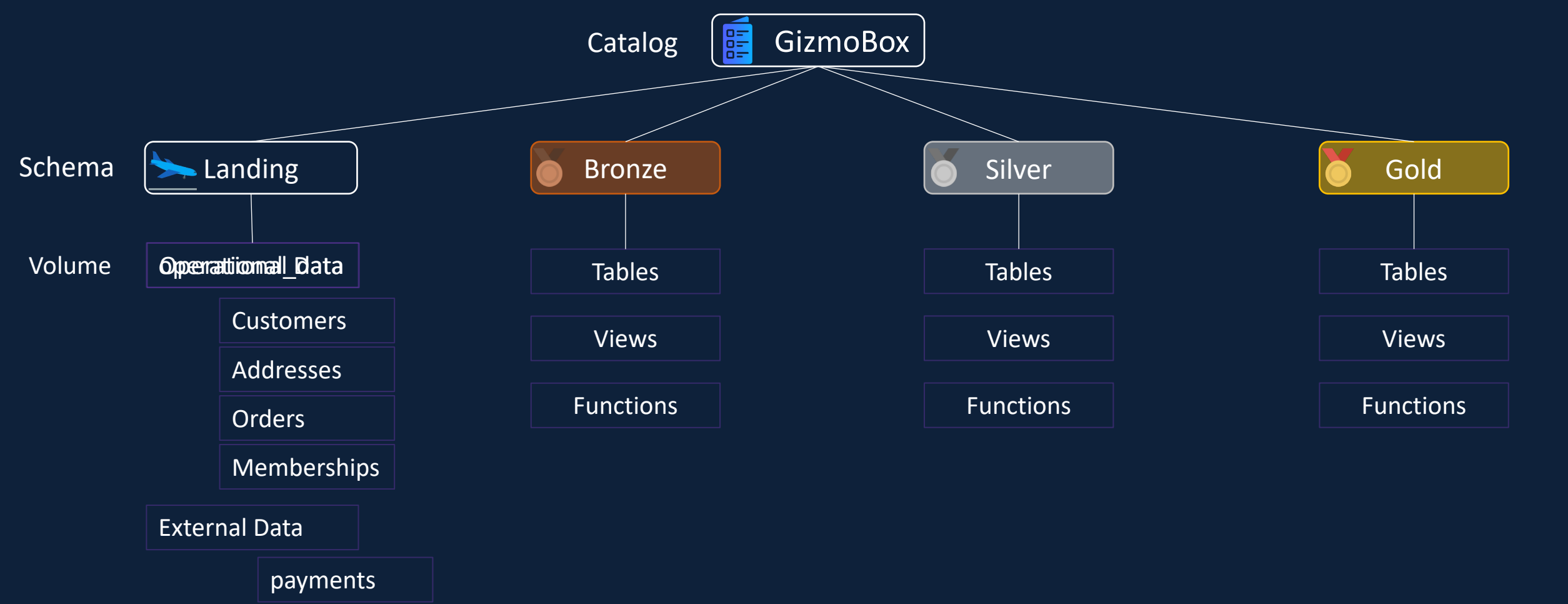
Azure Data Lake Storage Set-Up

Unity Catalog Set-up



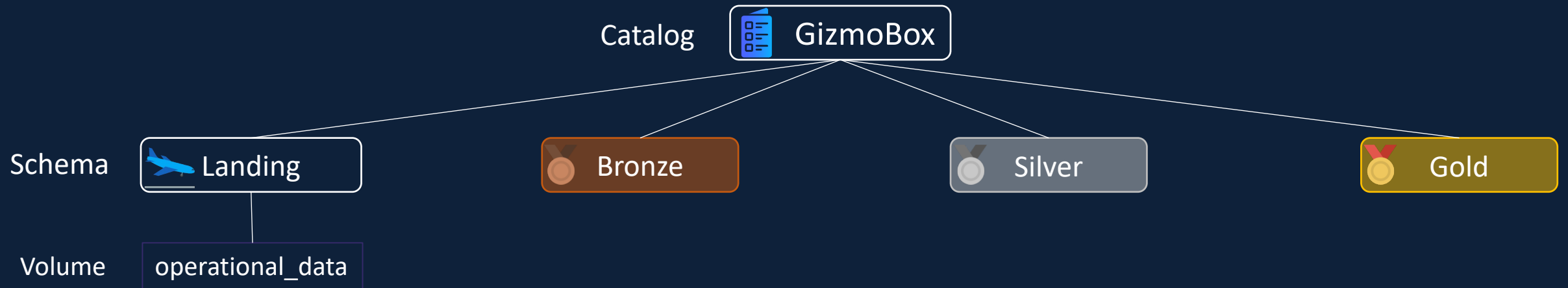
Data Architecture - GizmoBox

Unity Catalog



Data Architecture - GizmoBox

Unity Catalog

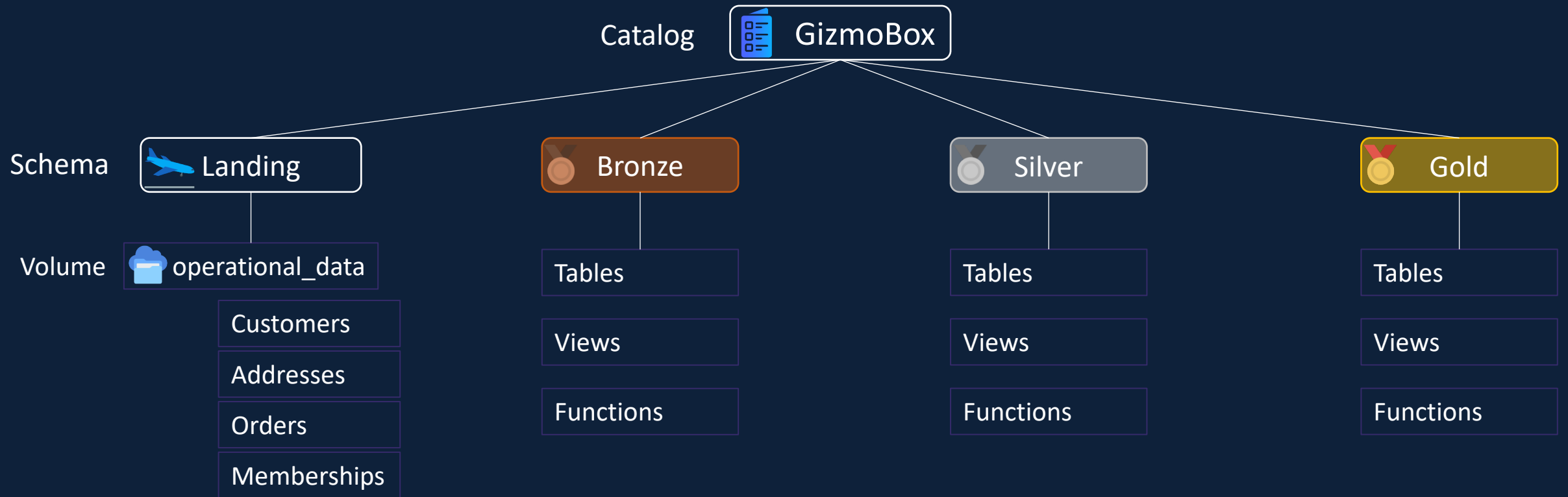


ETL With Apache Spark

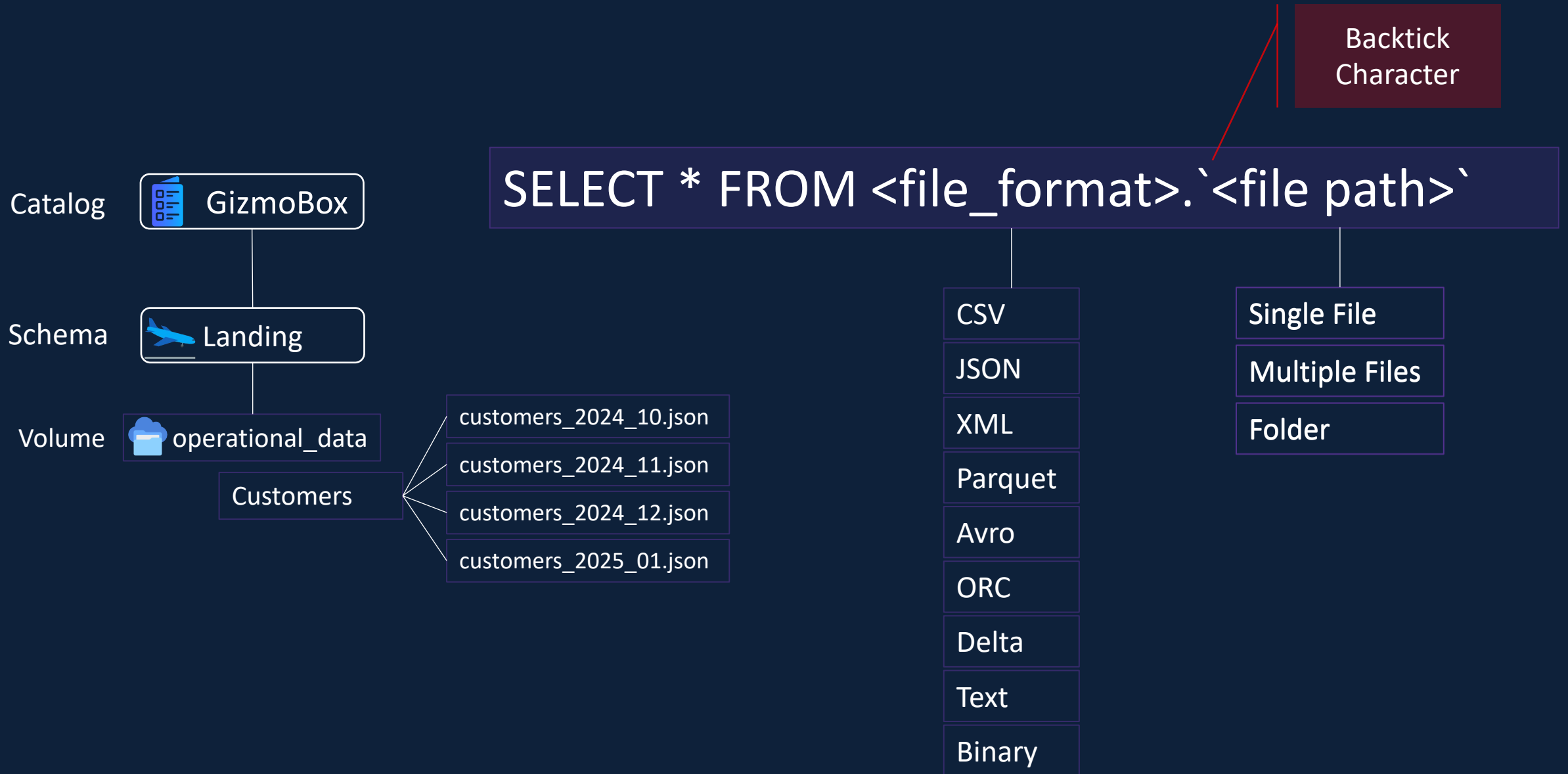
Querying Files

Querying Files

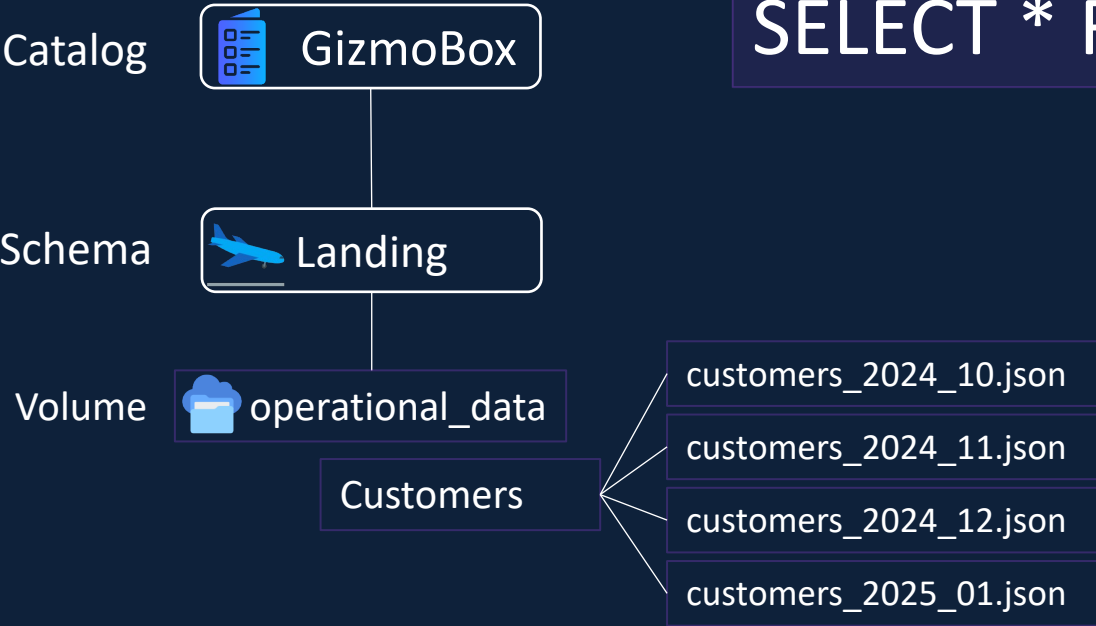
Unity Catalog



Querying Files



Querying Files - JSON



`SELECT * FROM <file_format>.`<file path>``

JSON

Single File

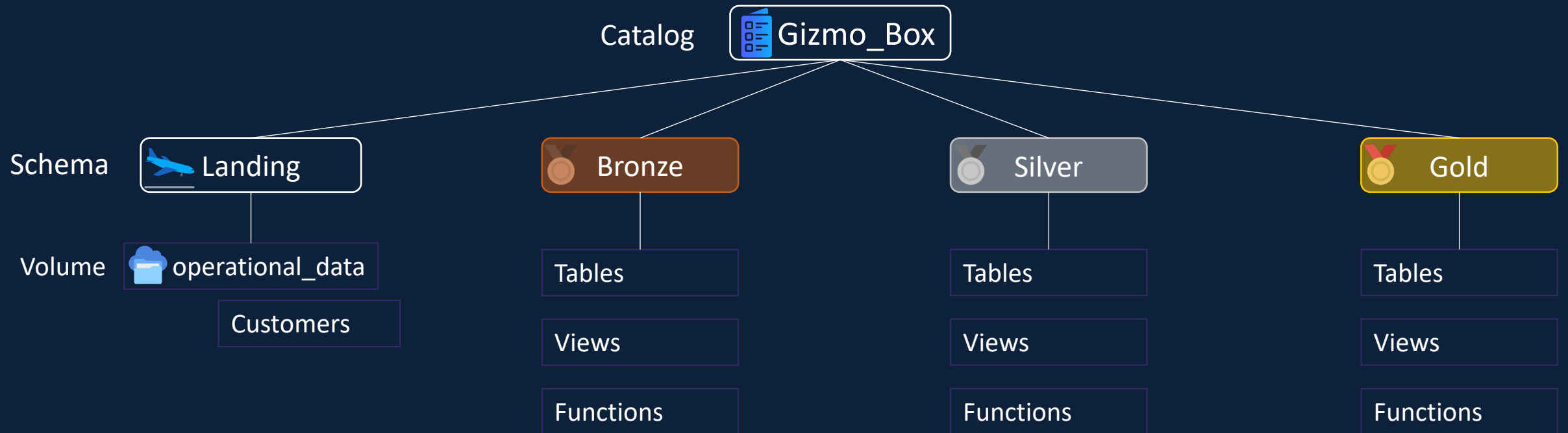
Multiple Files

Folder

Single File	<code>SELECT * FROM JSON.`/Volumes/gizmobox/landing/operational_data/customers/customers_2024_10.json`</code>
Multiple Files	<code>SELECT * FROM JSON.`/Volumes/gizmobox/landing/operational_data/customers/customers_2024_*.json`</code>
Folder	<code>SELECT * FROM JSON.`/Volumes/gizmobox/landing/operational_data/customers`</code>

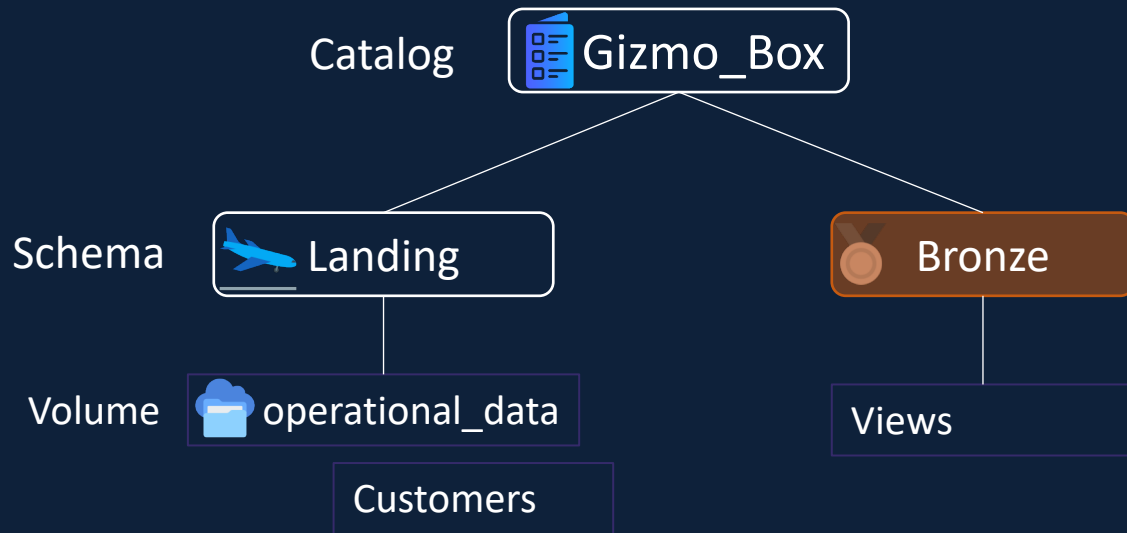
Create Views

Unity Catalog



Create Views

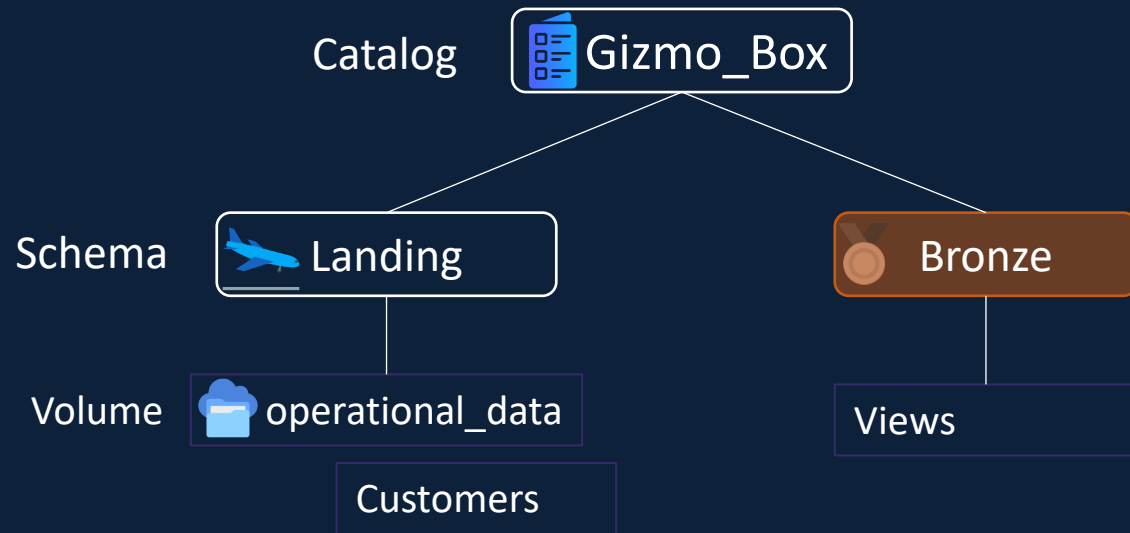
Unity Catalog



```
CREATE OR REPLACE VIEW v_customers
AS
SELECT *
FROM JSON.`/Volumes/gizmo_box/landing/operational_data/customers`
```

Create Views

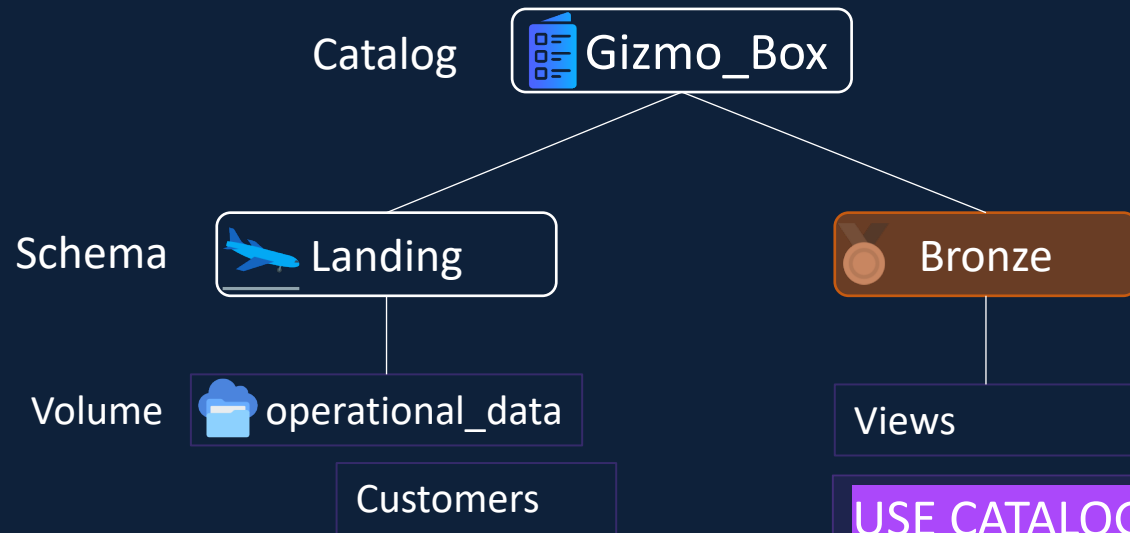
Unity Catalog



```
CREATE OR REPLACE VIEW gizmo_box.bronze.v_customers
AS
SELECT *
FROM JSON.`/Volumes/gizmo_box/landing/operational_data/customers`
```

Create Views

Unity Catalog



```
USE CATALOG gizmo_box;  
USE SCHEMA bronze;
```

```
CREATE OR REPLACE VIEW v_customers  
AS
```

```
SELECT *  
FROM JSON.`/Volumes/gizmo_box/landing/operational_data/customers`
```

Create Temporary Views

View

```
CREATE OR REPLACE VIEW tv_customers  
AS  
SELECT *  
FROM JSON.`/Volumes/gizmo_box/landing/operational_data/customers`
```

Temporary View

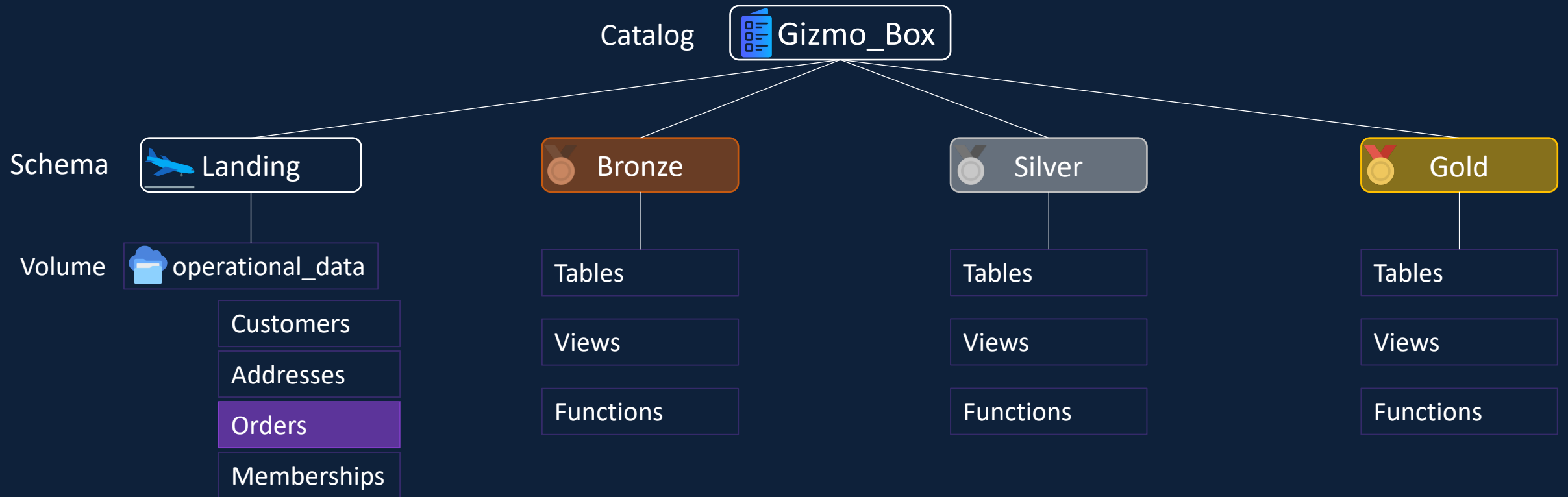
```
CREATE OR REPLACE TEMPORARY VIEW tv_customers  
AS  
SELECT *  
FROM JSON.`/Volumes/gizmo_box/landing/operational_data/customers`
```

Global Temporary View

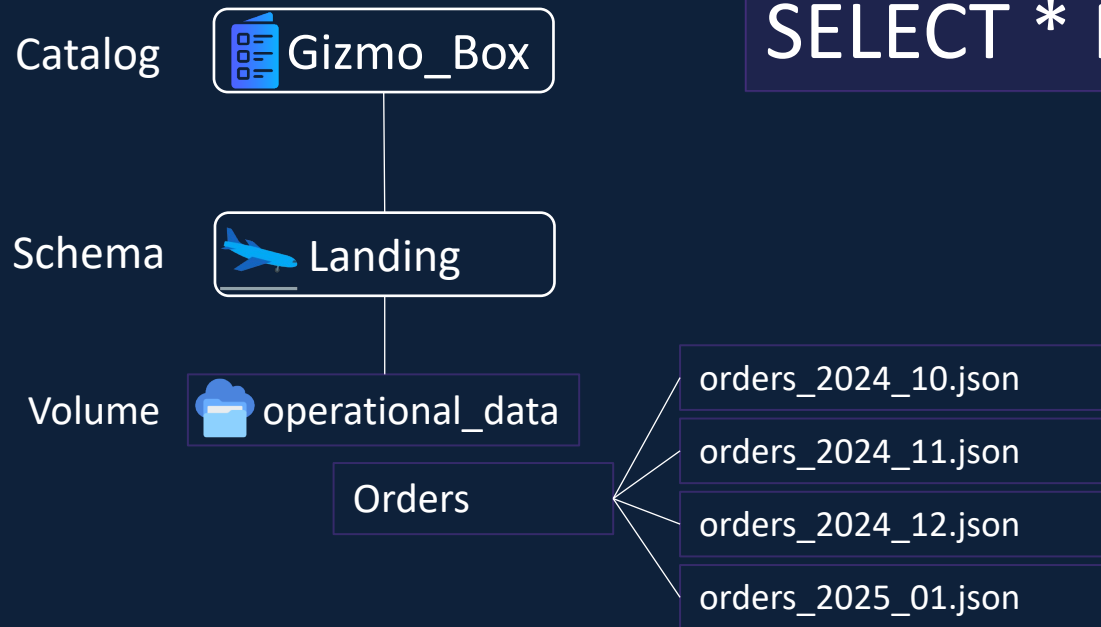
```
CREATE OR REPLACE GLOBAL TEMPORARY VIEW tv_customers  
AS  
SELECT *  
FROM JSON.`/Volumes/gizmo_box/landing/operational_data/customers`
```

Query Files

Unity Catalog



Query Files



```
SELECT * FROM <file_format>.`<file path>`
```

CSV

JSON

XML

Parquet

Avro

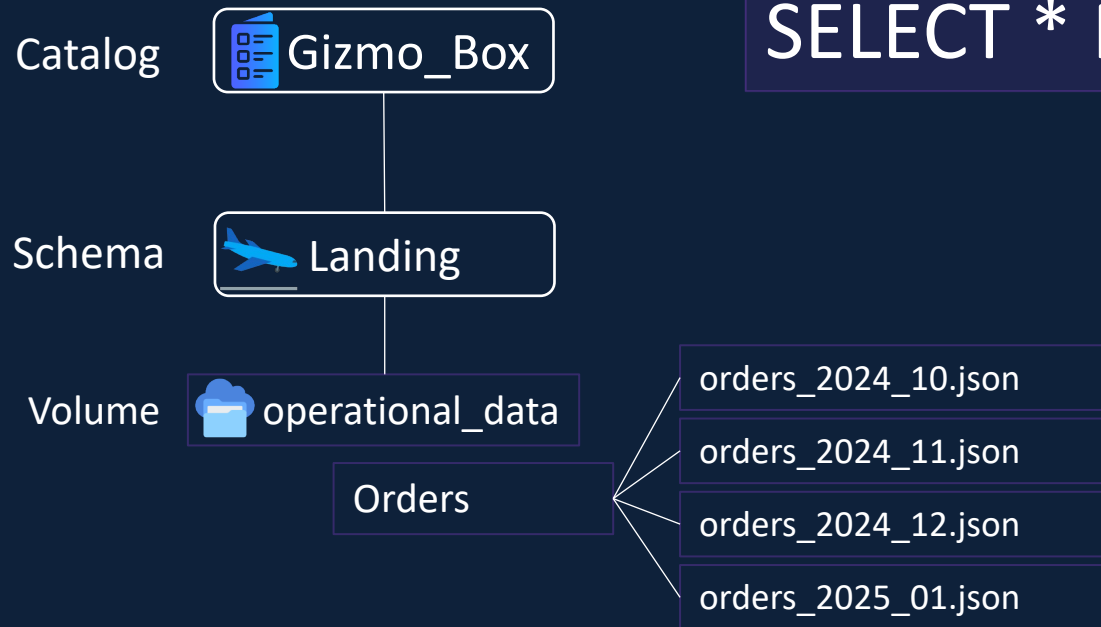
ORC

Delta

Text

Binary

Query Files – JSON/ Text



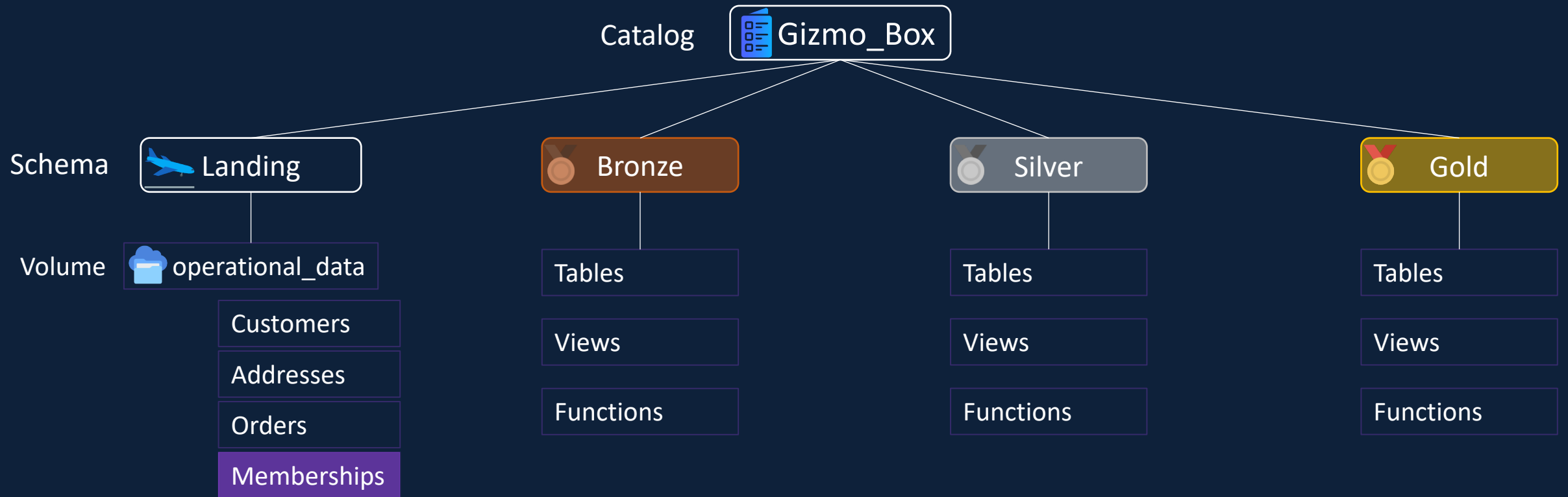
```
SELECT * FROM <file_format>.`<file path>`
```

JSON

Text

Query Image Files

Unity Catalog



Query Image Files

Catalog  Gizmo_Box

Schema  Landing

Volume  operational_data

Memberships

2024_10/customer_id.png

2024_11/customer_id.png

2024_12/customer_id.png

2025_01/customer_id.png

```
SELECT * FROM <file_format>.`<file path>`
```

CSV

JSON

XML

Parquet

Avro

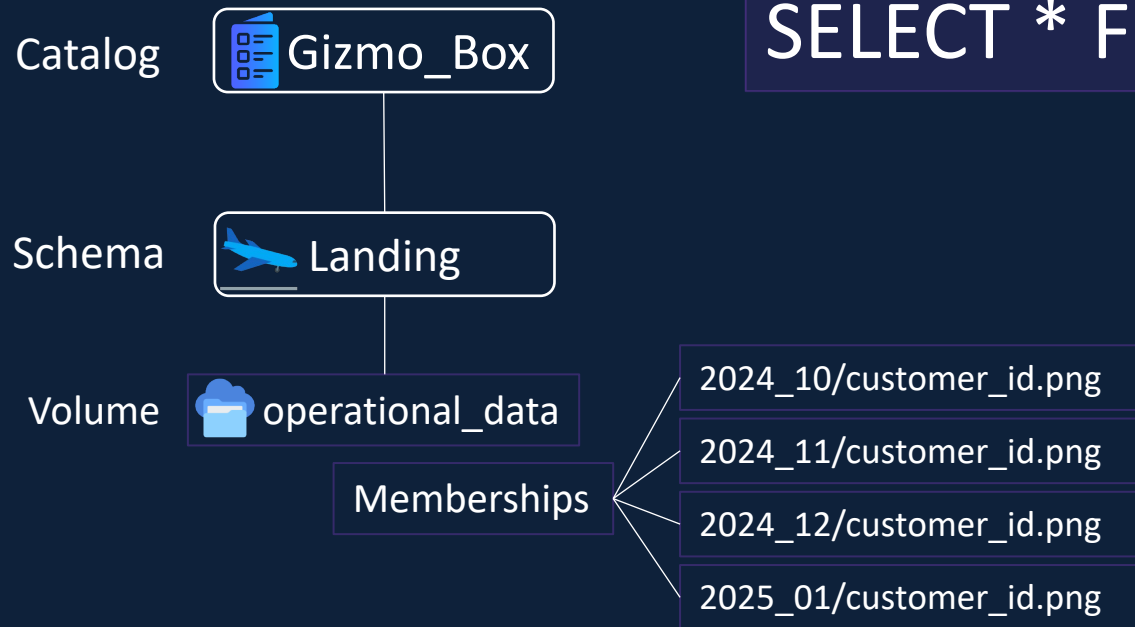
ORC

Delta

Text

binaryFile

Query Image Files

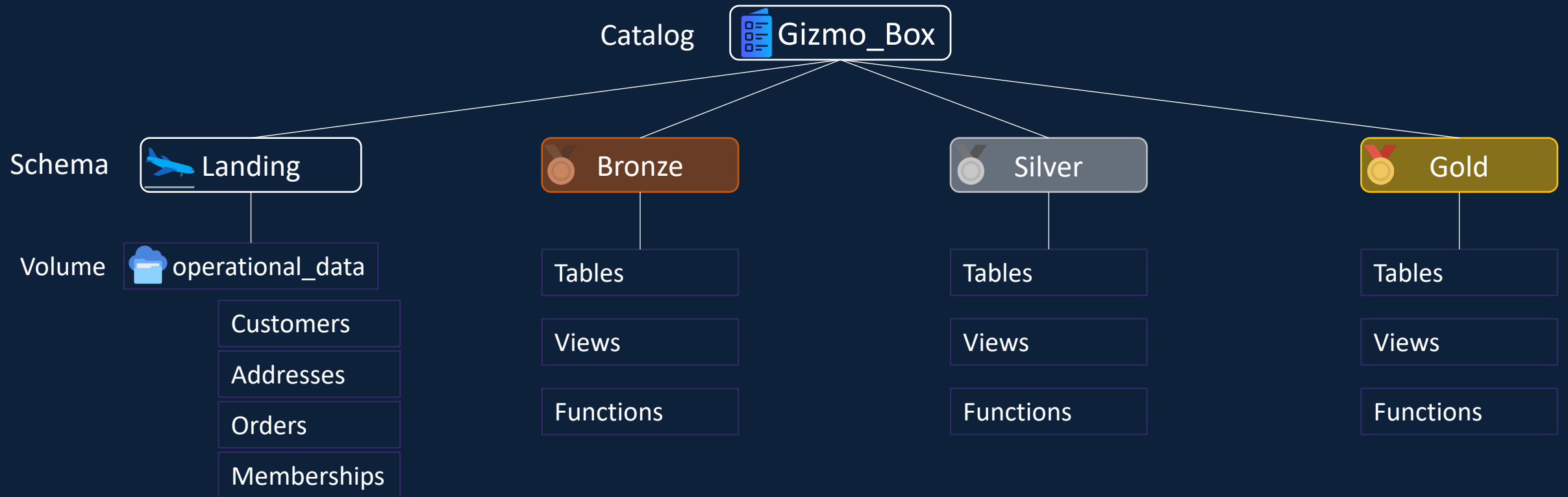


```
SELECT * FROM <file_format>.`<file path>`
```

binaryFile

Query Files – CSV

Unity Catalog



Query Files – CSV

Catalog  Gizmo_Box

Schema  Landing

Volume  operational_data

Addresses

addresses_2024_10.tsv

addresses_2024_11.tsv

addresses_2024_12.tsv

addresses_2025_01.tsv

```
SELECT * FROM <file_format>.`<file path>`
```

CSV

JSON

XML

Parquet

Avro

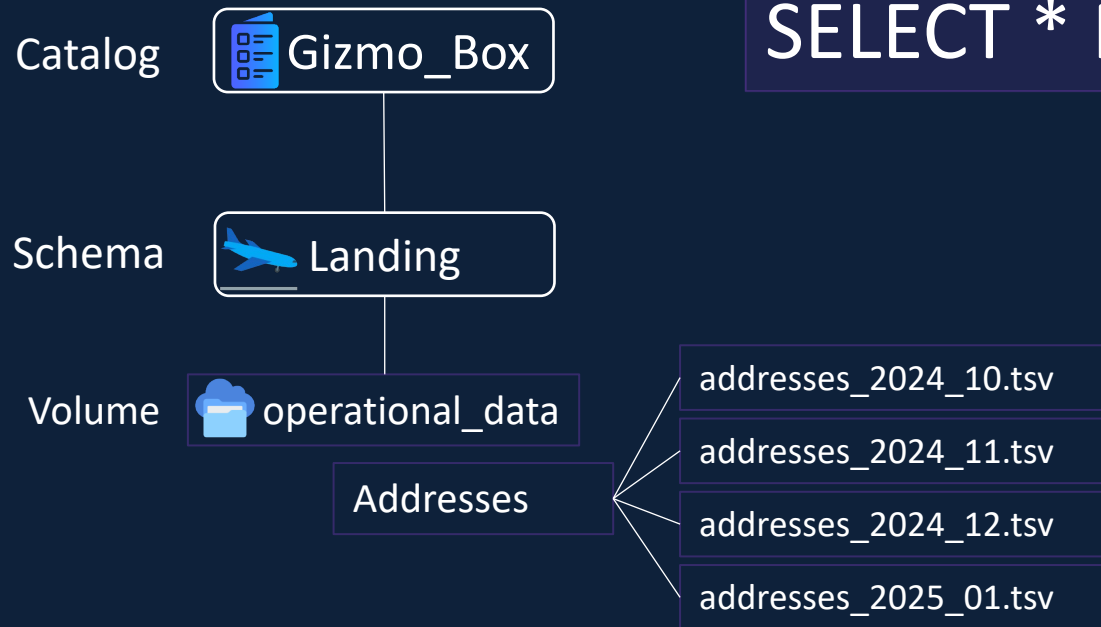
ORC

Delta

Text

Binary

Query Files – CSV



```
SELECT * FROM <file_format>.`<file path>`
```

CSV

Query Files – CSV

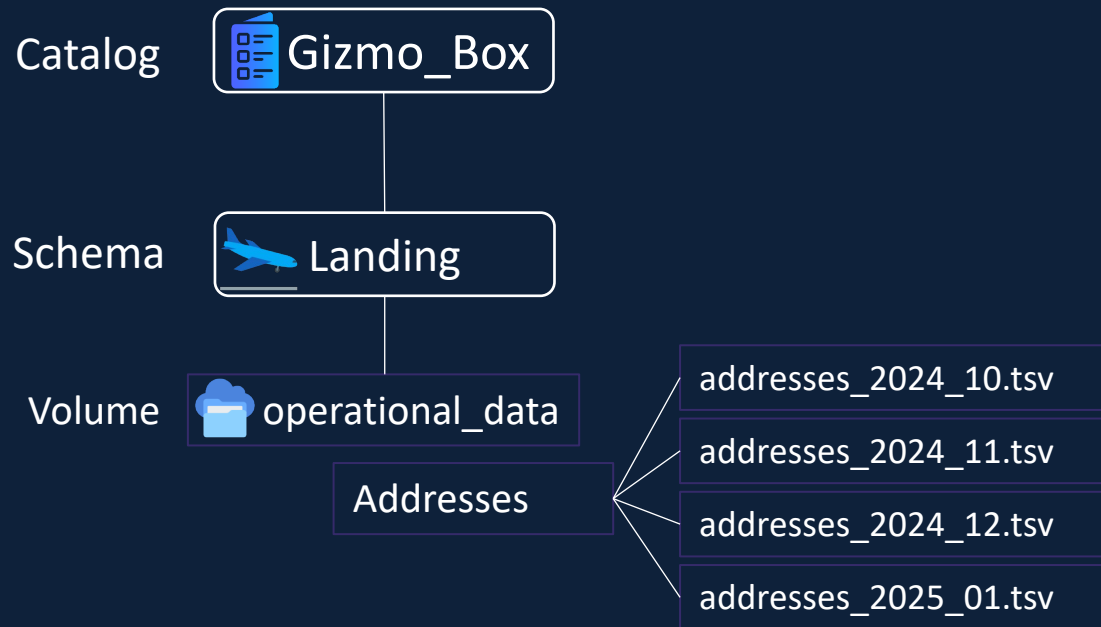
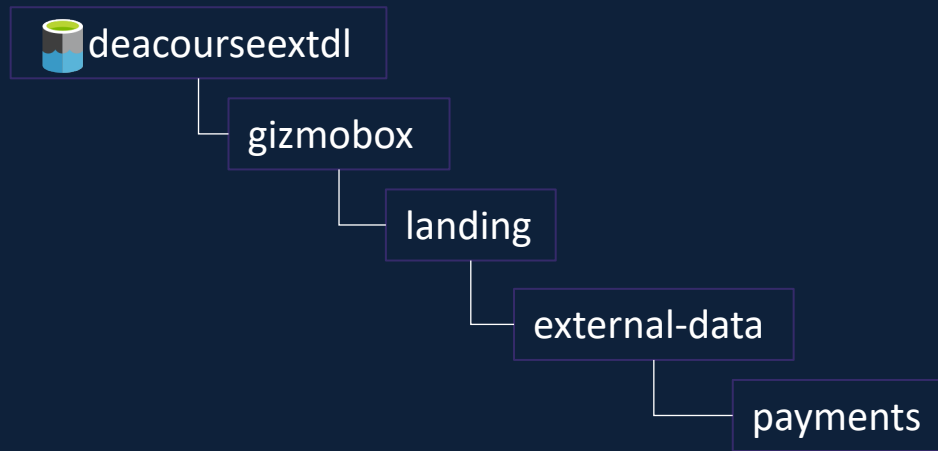


Table Valued Functions (e.g. `read_files`)

External Tables

Create External Tables

Azure Data Lake Storage



Create External Tables

Azure Data Lake Storage

Databricks Unity Catalog

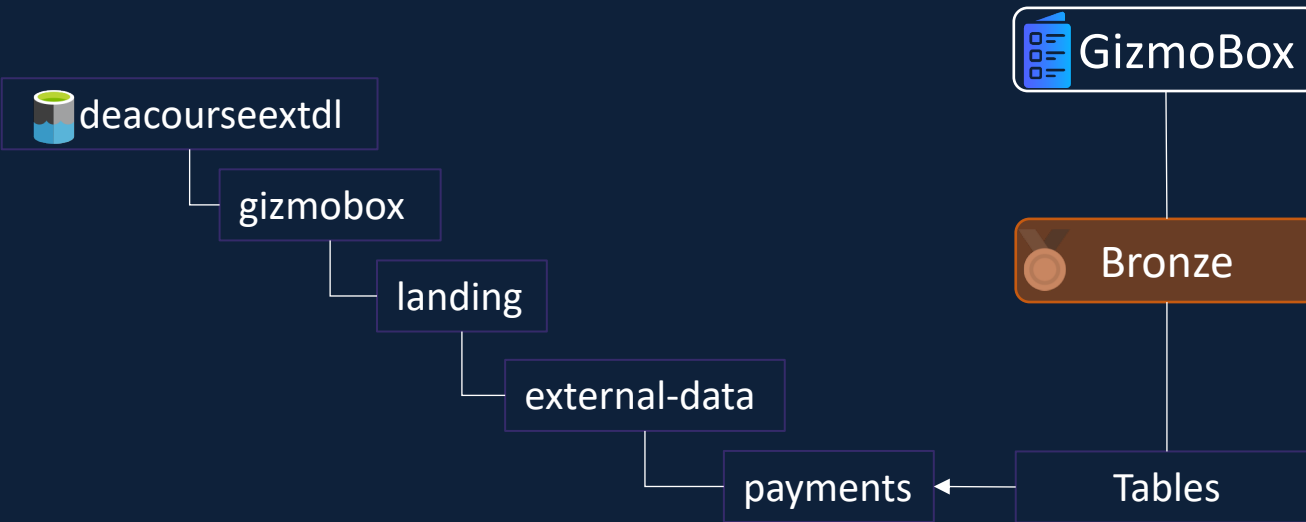


```
CREATE TABLE <catalog_name.schema_name.table_name>
(
  List of column_names column_types
)
USING <file format>
OPTIONS (
  List of key = value
)
LOCATION "file_path";
```

Create External Tables

Azure Data Lake Storage

Databricks Unity Catalog



```
CREATE TABLE hive_metastore.bronze.payments
```

```
(
```

```
  payment_id INTEGER,  
  customer_id INTEGER,  
  payment_date TIMESTAMP,  
  payment_status STRING,  
  payment_method STRING
```

```
)
```

```
USING CSV
```

```
OPTIONS (
```

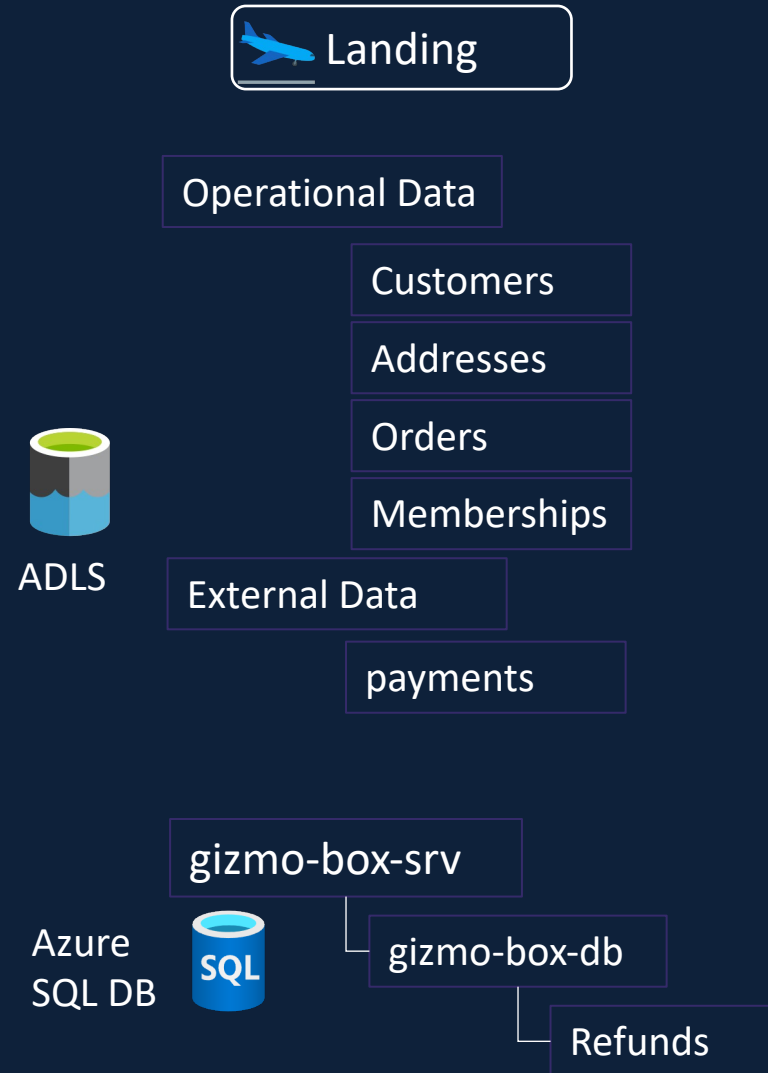
```
  header = "true",  
  delimiter = ","
```

```
)
```

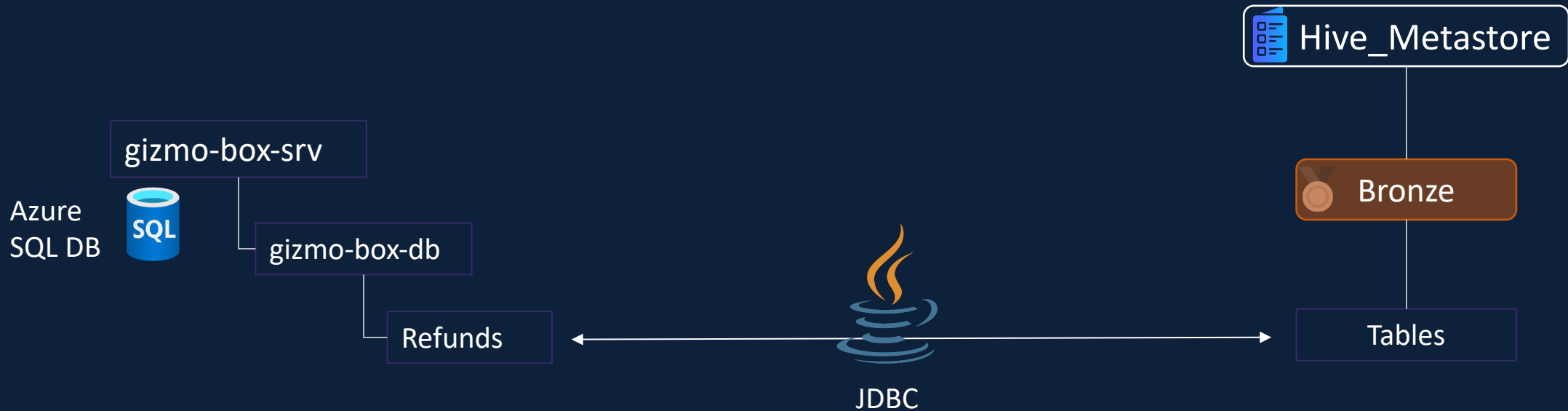
```
LOCATION
```

```
"abfss://gizmobox@deacourseextdl.dfs.core.windows.net/landing/  
external_data/payments";
```

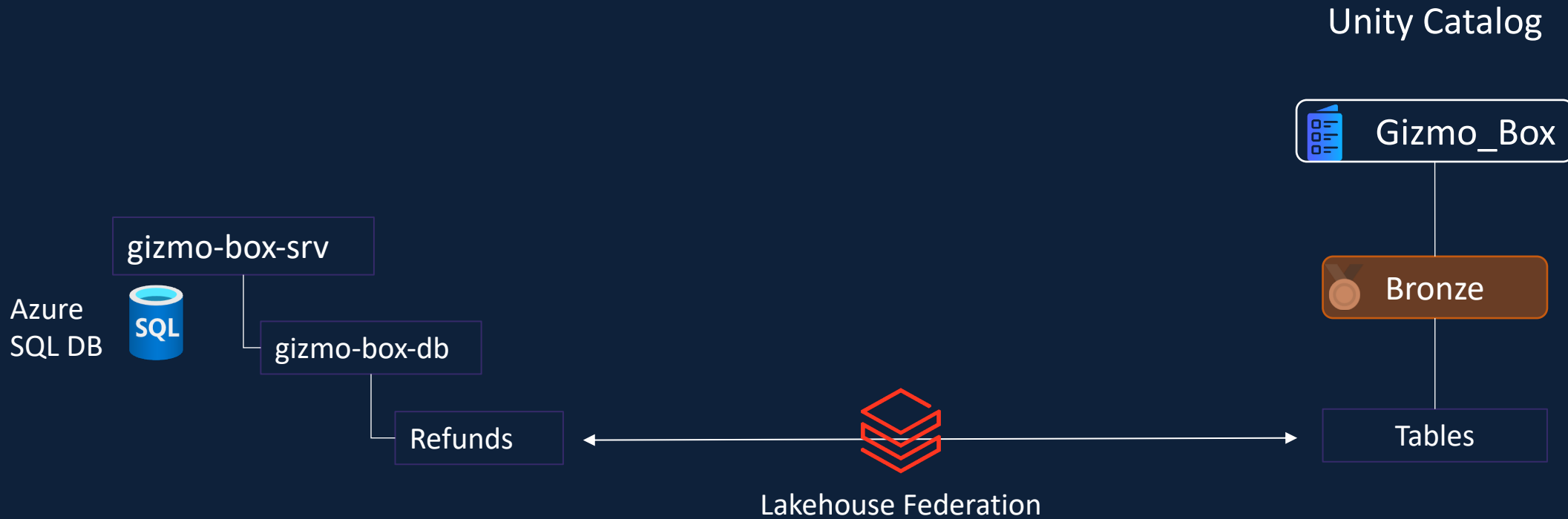
Create External Tables – Via JDBC



Create External Tables – Via JDBC



Create External Tables – Via JDBC





What is PySpark?

What is PySpark?



Python API for Apache Spark

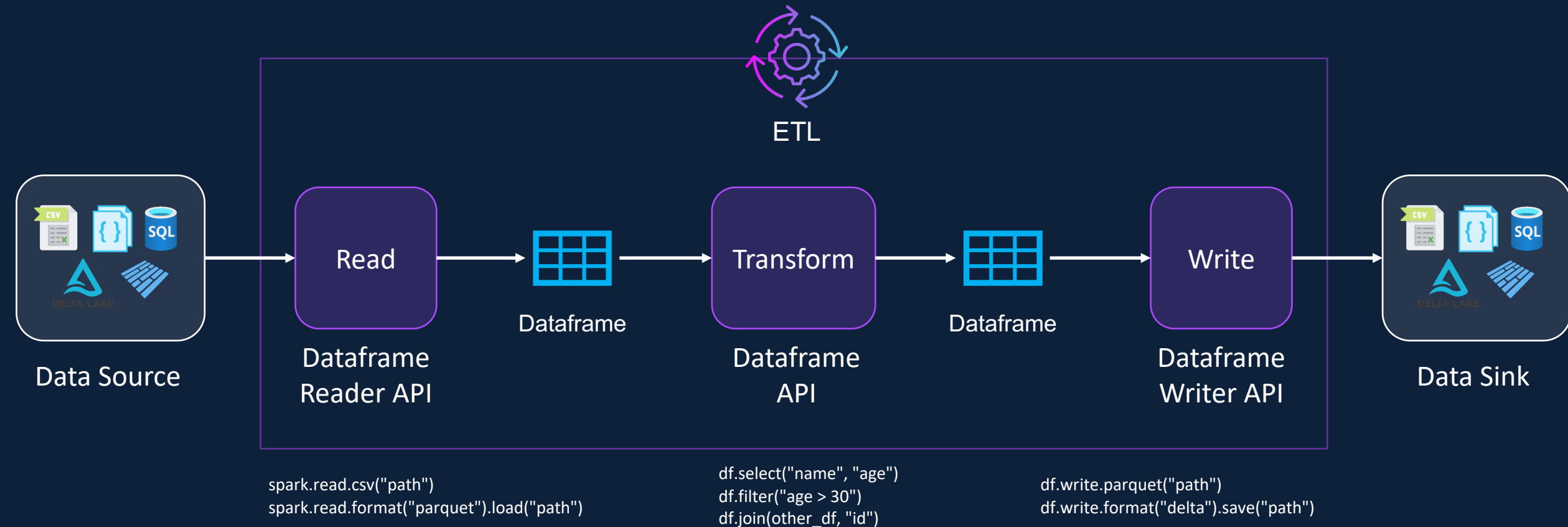
Offers greater flexibility than Spark SQL

Integrates with Python Libraries

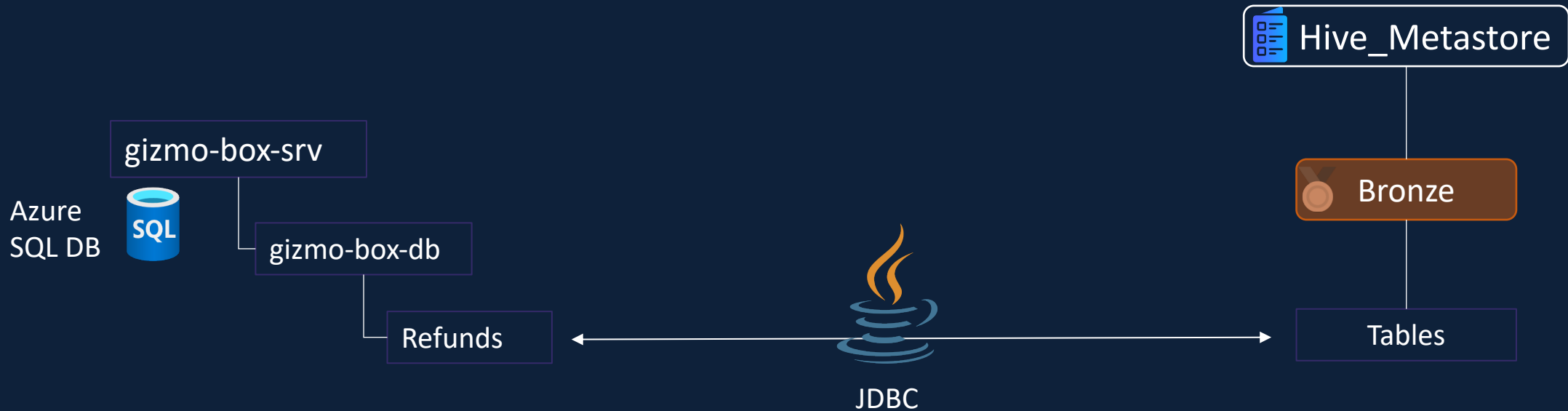


What are Dataframes?

What are Dataframes?



Create External Tables – Via JDBC



Spark Structured Streaming

Spark Structured Streaming

Batch Processing

Data is collected over a period of time, and then processed in bulk at scheduled intervals.

Insights are delayed

Not suitable for real time analytics

Spark Structured Streaming

Stream Processing

Data is processed continuously as it arrives, enabling real-time and near real-time analytics

Real time fraud detection

Live monitoring

Instant product recommendation

Spark Structured Streaming

Spark Structured Streaming

Scalable, Fault Tolerant Stream Processing Engine built on top of Apache Spark.

Uses Dataframe and Dataset APIs

Micro-Batch Processing

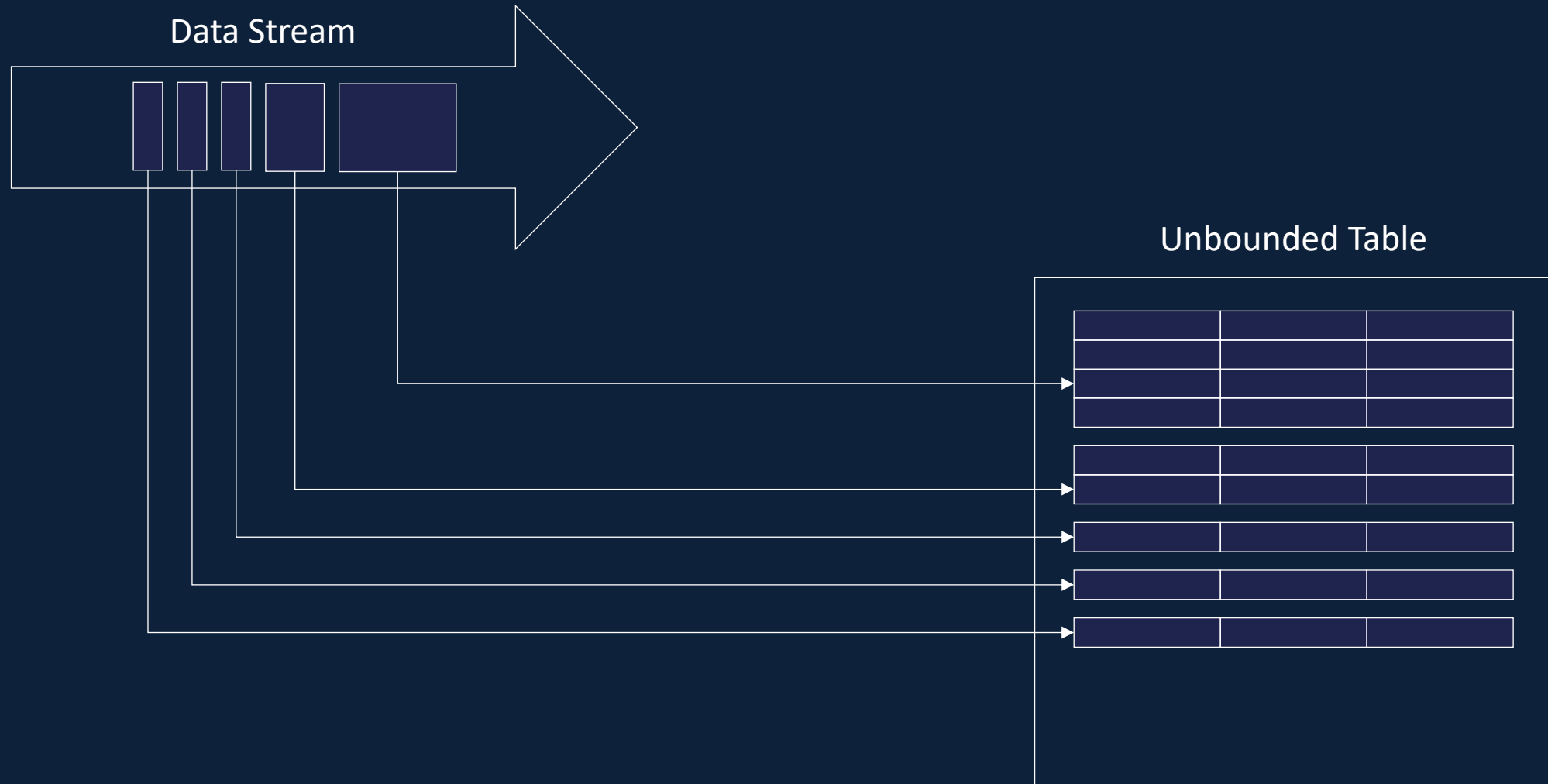
Exactly once guarantees

Built-in Fault tolerance

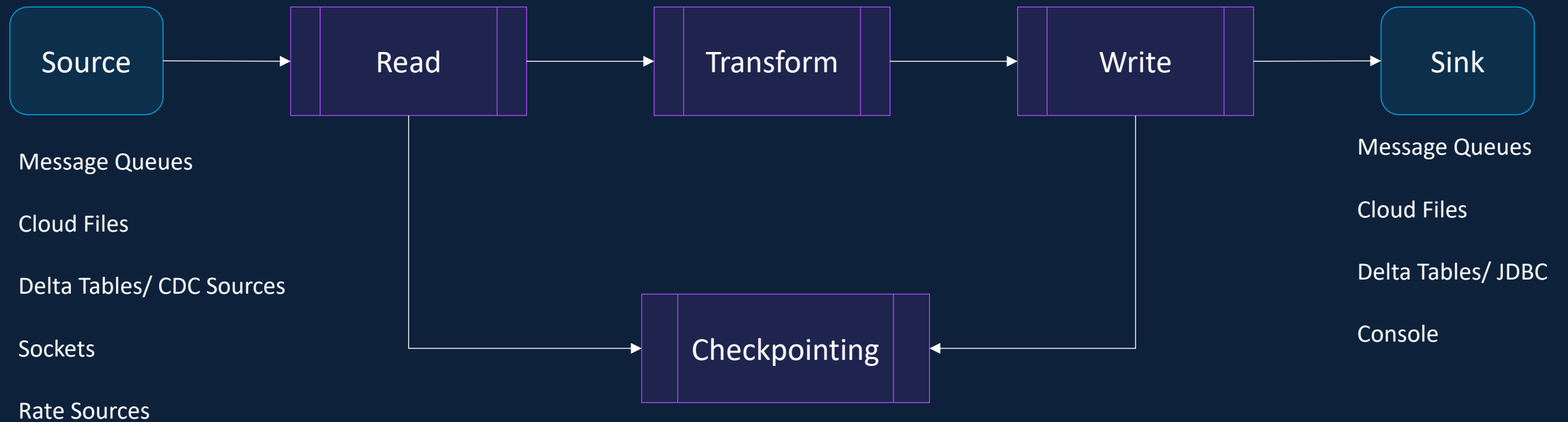
Scalable to large scale workloads

Supports various sources and sinks

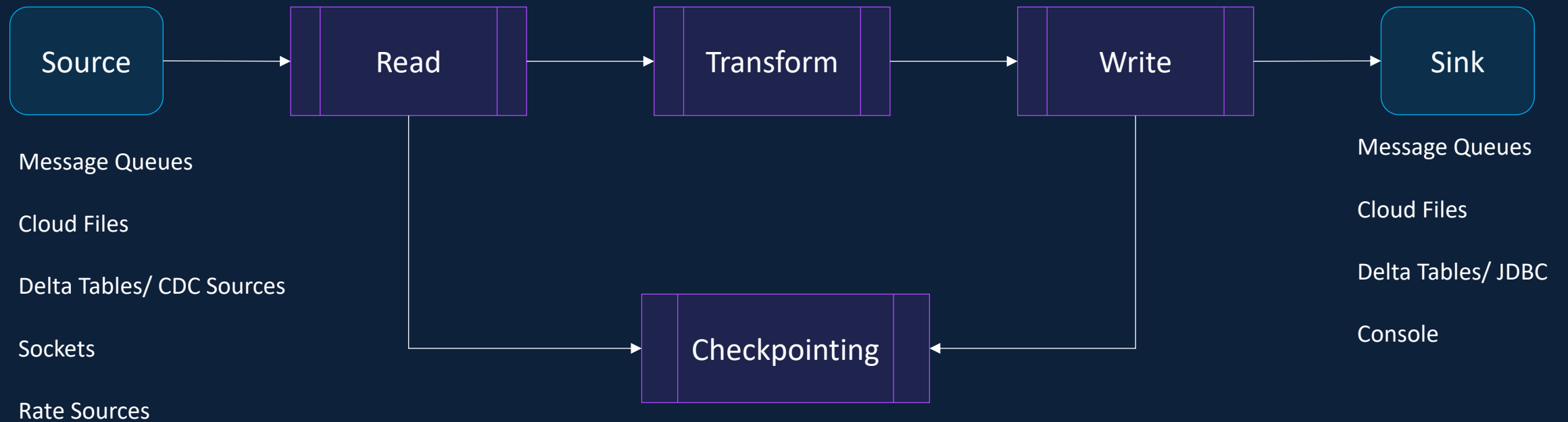
Spark Structured Streaming



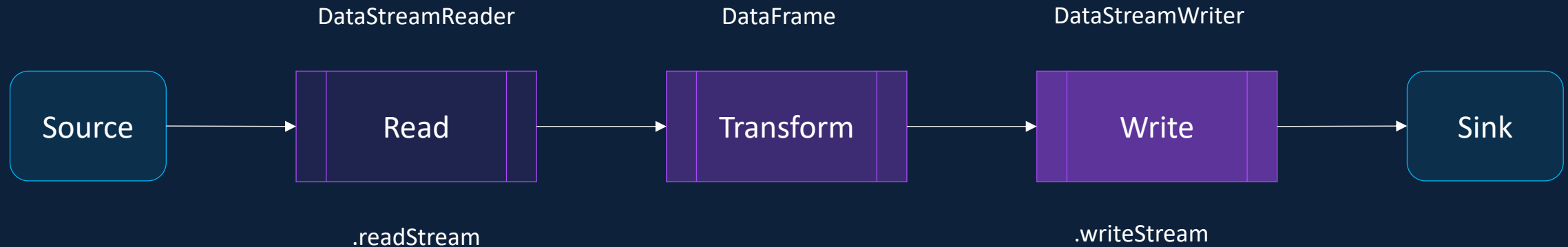
Spark Structured Streaming



Spark Structured Streaming



Spark Structured Streaming



```
customers_df = spark.readStream
    .format("json")
    .schema(customers_schema)
    .load("/Volumes/gizmobox/landing/operational_data/customers_stream")
```

```
customers_transformed_df = customers_df.withColumn("file_path", col("_metadata.file_path"))
    .withColumn("ingestion_date", current_timestamp())
```

```
customers_transformed_df.writeStream
    .format("delta")
    .option("checkpointLocation",
        "/Volumes/gizmobox/landing/operational_data/customers_stream/_checkpoint")
    .toTable("gizmobox.bronze.customers_streamed")
```

Spark Structured Streaming

Trigger

Output Mode

Spark Structured Streaming

Processing Modes

Micro-Batch Processing (Default)

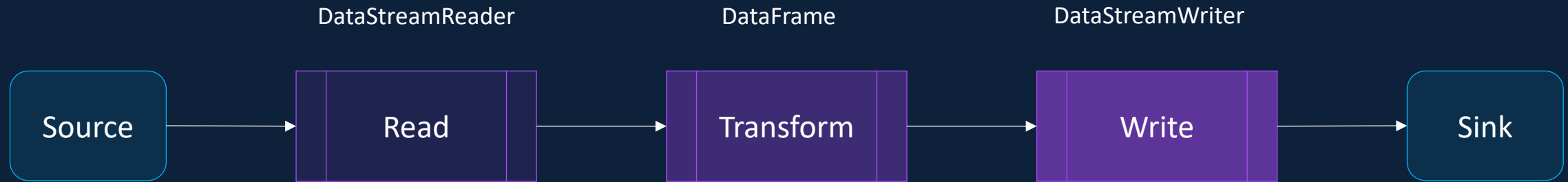
Continuous Processing (Experimental)

Spark Structured Streaming

Trigger

Trigger Type	Trigger Syntax	Description
Default (No Trigger)	No trigger specified	500 microseconds interval
Fixed Interval	<code>.trigger(processingTime = "2 minutes")</code>	Micro batch starts at the user specified interval
Triggered Once	<code>.trigger(once=True)</code>	Processes all data available as one micro-batch and stops [Deprecated]
Available Now	<code>.trigger(availableNow=True)</code>	Processes all data available as multiple micro-batch and stops
Continuous	<code>.trigger(continuous="2 seconds")</code>	Processes data continuously, but checkpoints at interval specified [Experimental]

Spark Structured Streaming



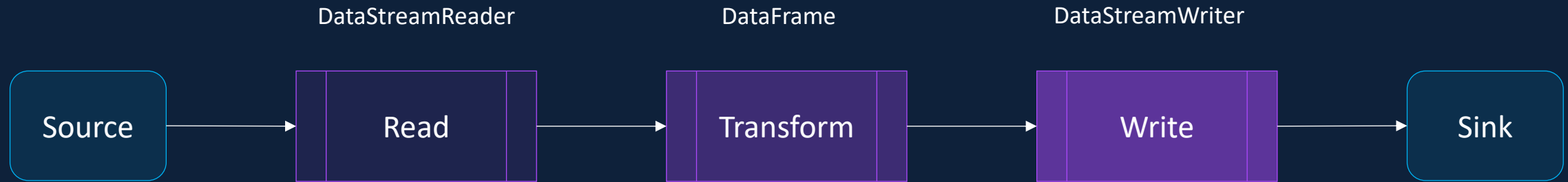
```
customers_transformed_df.writeStream
    .format("delta")
    .trigger(processingTime = "2 minutes")
    .option("checkpointLocation",
            "/Volumes/gizmobox/landing/operational_data/customers_stream/_checkpoint")
    .toTable("gizmobox.bronze.customers_streamed")
```

Spark Structured Streaming

outputMode

Output Mode	Description
Append (Default)	Writes the new rows arrived since the last micro-batch
Complete	Writes the entire result to the sink every time
Update	Writes only the rows that have changed since the last micro-batch

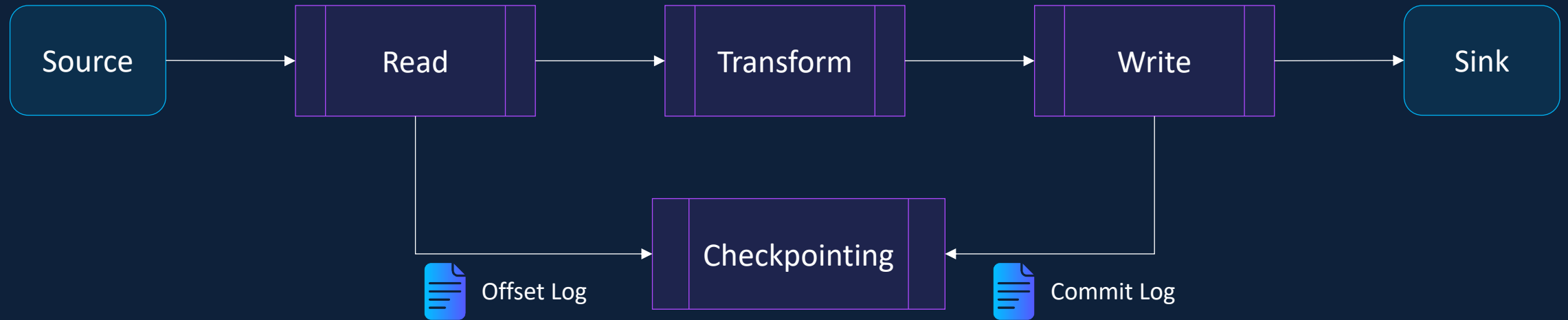
Spark Structured Streaming



```
customers_transformed_df.writeStream
    .format("delta")
    .outputMode("append")
    .trigger(processingTime = "2 minutes")
    .option("checkpointLocation",
            "/Volumes/gizmobox/landing/operational_data/customers_stream/_checkpoint")
    .toTable("gizmobox.bronze.customers_streamed")
```

Spark Structured Streaming

Checkpointing



Checkpointing is a fault-tolerance mechanism that allows a query to recover from failures and resume processing from where it left off without data loss or duplication.



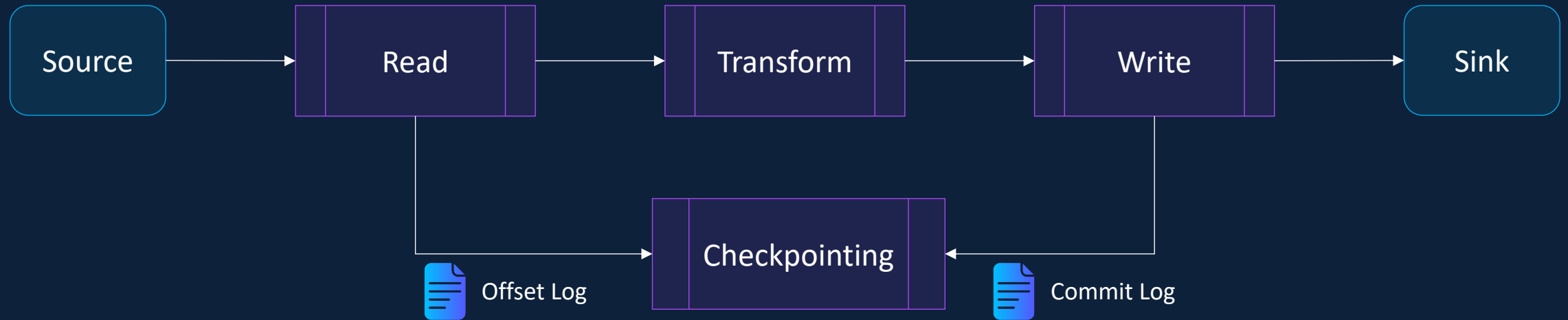
CheckPoint

Stores Metadata about streaming query, execution plan

Tracks processed offsets and committed results

Spark Structured Streaming

Checkpointing

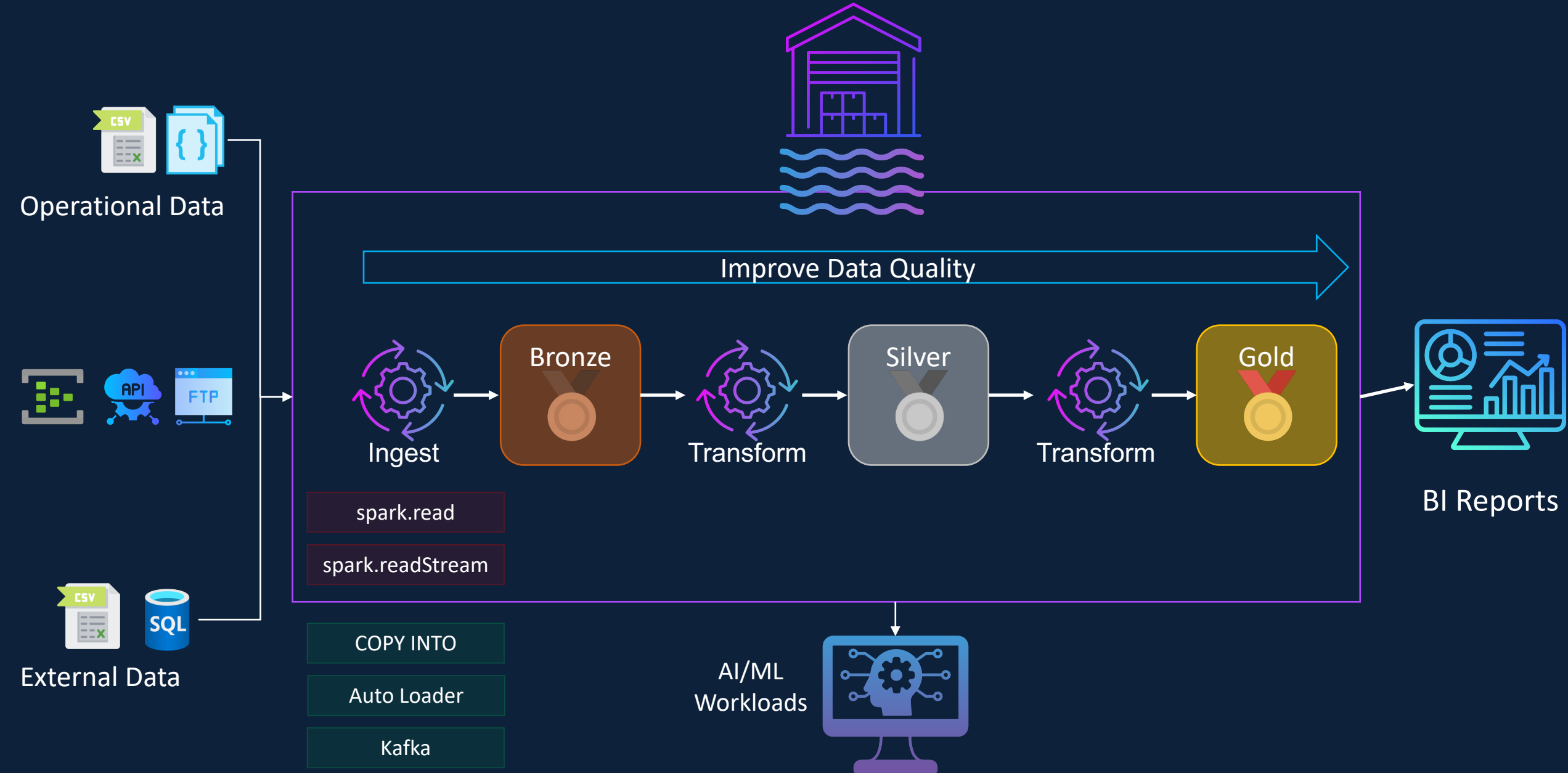


Write-Ahead Logs (WAL) and Checkpointing helps provide Fault Tolerance

Idempotent Sinks enables exactly once guarantees

Data Ingestion in Databricks

Data Ingestion in Databricks



Data Ingestion in Databricks

Managed Connectors



SaaS
Connectors



Database
Connectors



Lakeflow Connect
(Managed Connectors)



Authentication



Incremental
Loading



Schema
Evolution



Error
Handling



Fully Managed

Databricks manages the
entire ingestion process

Standard Connectors



Auto Loader



Kafka



SFTP (*Only with Spark)

and more ...



Lakeflow Spark Declarative Pipelines



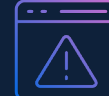
Scheduling



Data Quality



Monitoring



Error Handling



**Less Control
More Automation**

Databricks manages
orchestration,
monitoring, retries.



Spark Structured Streaming



Full Control



Custom Logic



Checkpoint



More Control

We manage the entire
ingestion process,
execution, checkpoint

Auto Loader

Auto Loader - Introduction

Auto Loader is a scalable and efficient way to ingest data from cloud storage

Can be used within Spark Structured Streaming as well as within Lakeflow Declarative Pipelines.

Incrementally and efficiently processes new data files as they arrive in the cloud storage

Supports Azure Data Lake Storage, Amazon S3, Google Cloud Storage, Databricks File System

Supports JSON, CSV, XML, PARQUET, AVRO, ORC, TEXT, and BINARYFILE file formats

Why Auto Loader?

Traditional File Stream Source

Inefficient File Listing

Scalability Issues

Schema Evolution Problems

Why Auto Loader?

Auto Loader

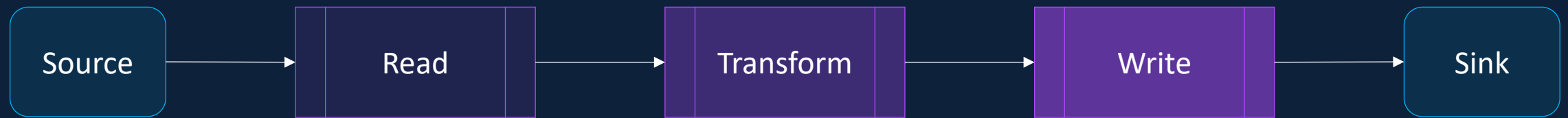
Efficient File Detection using Cloud Services

Scalability Improvements

Schema Evolution & Resiliency

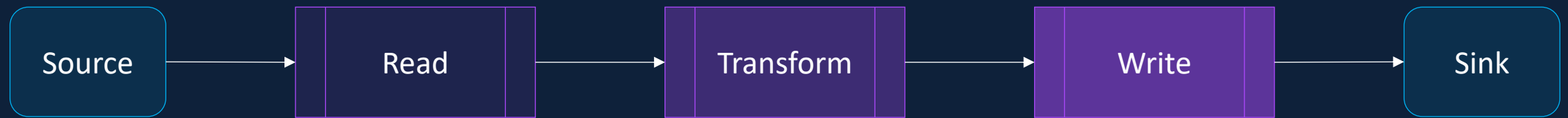
Recommended in Lakeflow Declarative Pipelines

How to use Auto Loader?



```
customers_df = spark.readStream  
    .format("json")  
    .schema(customers_schema)  
    .load("/Volumes/gizmobox/landing/operational_data/customers_stream")
```

How to use Auto Loader?



```
customers_df = spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .schema(customers_schema)
    .load("/Volumes/gizmobox/landing/operational_data/customers_stream")
```

How to use Auto Loader?

```
customers_df = spark.readStream  
    .format("cloudFiles")  
    .option("cloudFiles.format", "json")  
    .option("cloudFiles.useNotifications", "true")  
    .schema(customers_schema)  
    .load("/Volumes/gizmobox/landing/operational_data/customers_stream")
```

How to use Auto Loader?

```
customers_df = spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.useNotifications", "true")
    .option("cloudFiles.schemaLocation",
            "/Volumes/gizmobox/landing/operational_data/customers_stream/_schema")
    .option("cloudFiles.inferColumnTypes", "true")
    .load("/Volumes/gizmobox/landing/operational_data/customers_stream")
```

How to use Auto Loader?

```
customers_df = spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.useNotifications", "true")
    .option("cloudFiles.schemaLocation",
            "/Volumes/gizmobox/landing/operational_data/customers_stream/_schema")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("/Volumes/gizmobox/landing/operational_data/customers_stream")
```

addNewColumns – Automatically adds new columns to the schema

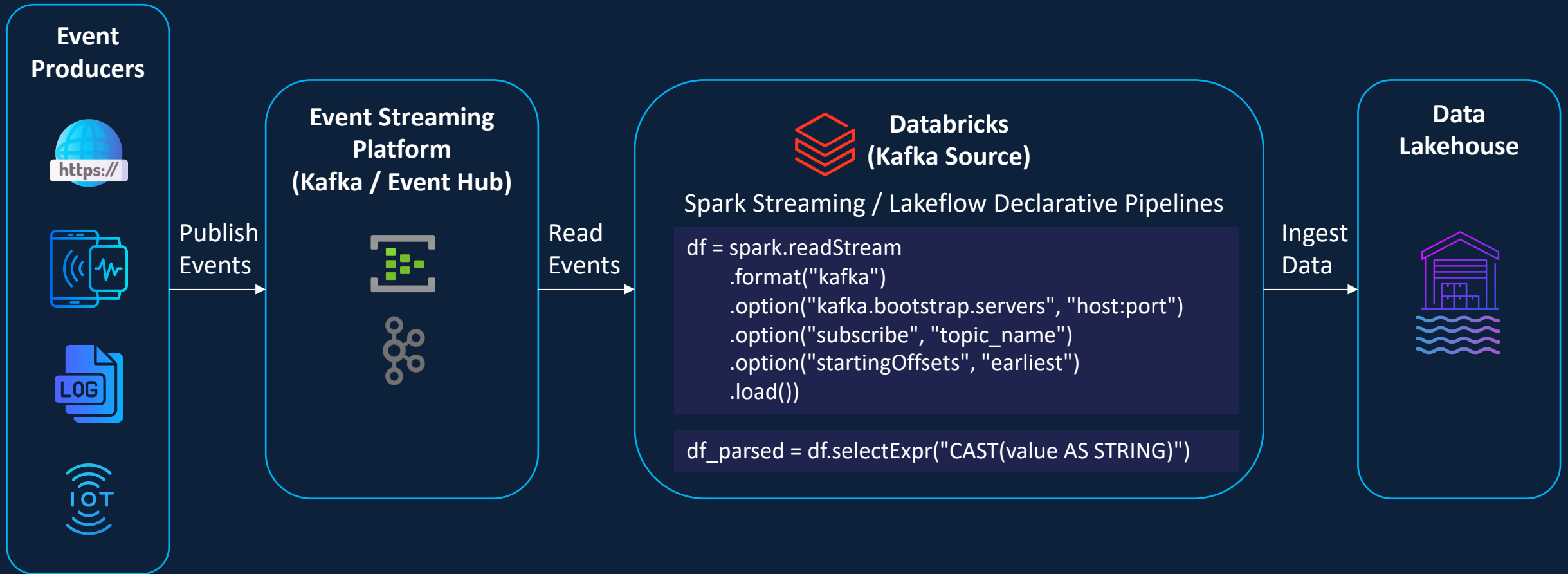
failOnNewColumns – Fail if the schema changes

rescue – Stores unexpected data in `_rescued_data`

none – Ignores the schema change completely



Ingesting Streaming Event Data



Lakeflow Connect (Managed SaaS Connectors)

Data Ingestion in Databricks

Managed Connectors



SaaS
Connectors



Database
Connectors



Lakeflow Connect (Managed Connectors)



Authentication



Incremental
Loading



Schema
Evolution



Error
Handling



Fully Managed

Databricks manages the entire ingestion process

Standard Connectors



Auto Loader



Kafka



SFTP

(* Only with Spark)



Lakeflow Spark Declarative Pipelines



Scheduling



Data Quality



Monitoring



Error Handling



**Less Control
More Automation**

Databricks manages orchestration, monitoring, retries etc.



Spark Structured Streaming



Full Control



Custom Logic



Checkpoint

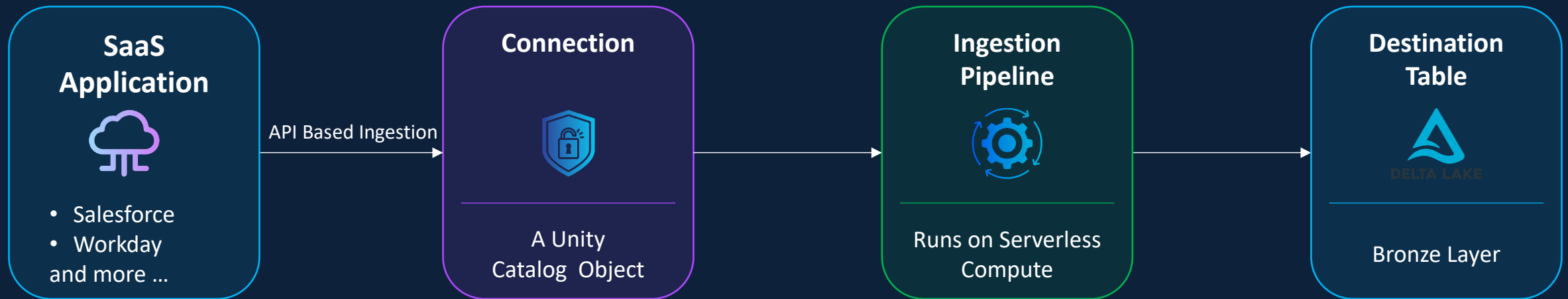


More Control

We manage the entire ingestion process, execution, checkpoint etc

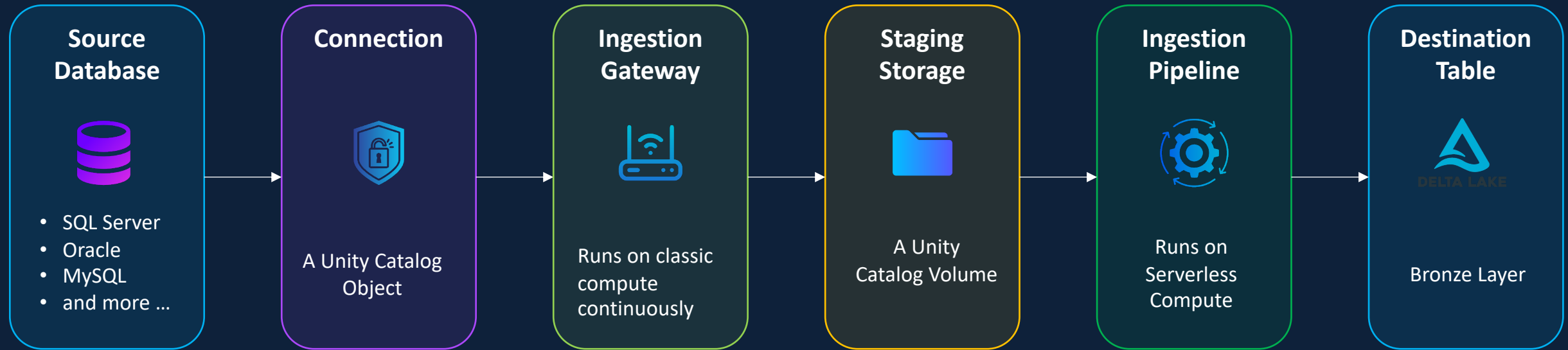
Lakeflow Connect – SaaS Connector Architecture

Managed solution for ingesting data from SaaS applications into the Data Lakehouse

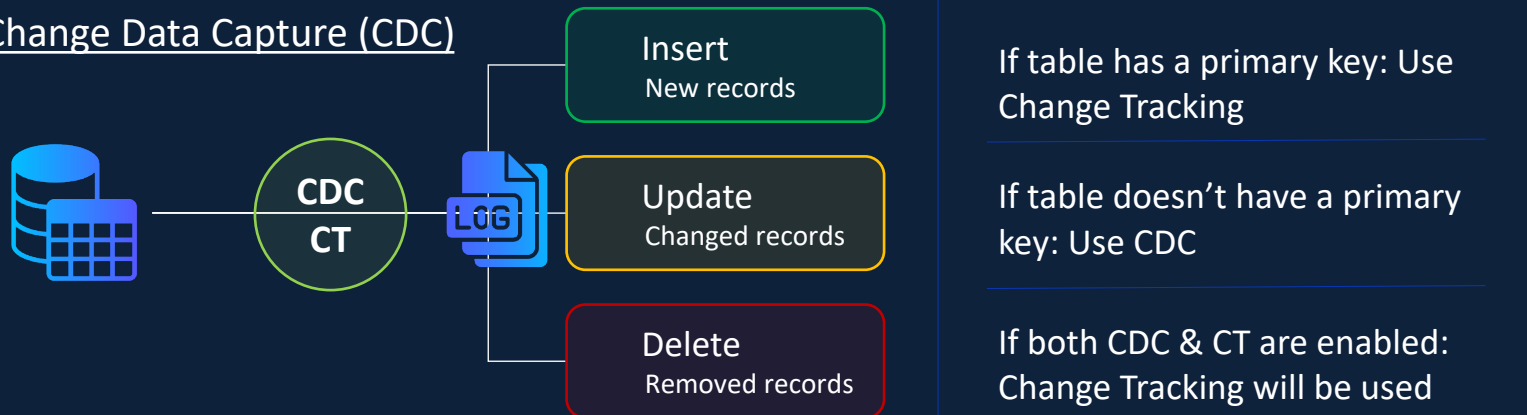


Lakeflow Connect (Managed Database Connectors)

Lakeflow Connect – Database Connector Architecture



Change Data Capture (CDC)





What is Delta Lake?

What is Delta Lake?



Delta Lake is the optimized storage layer that provides the foundation for tables in a Lakehouse on Databricks. Delta Lake is open source software that extends Parquet data files with a file-based transaction log for ACID transactions and scalable metadata handling.

It allows you to use a single copy of data for both batch and streaming operations and providing incremental processing at scale

- *Databricks*

What is Delta Lake?



Delta Lake is the **optimized storage layer** that provides the foundation for tables in a Lakehouse on Databricks. Delta Lake is open source software that extends Parquet data files with a file-based transaction log for **ACID transactions** and **scalable metadata handling**.

It allows you to use a single copy of data for **both batch and streaming operations** and providing **incremental processing at scale** *- Databricks*

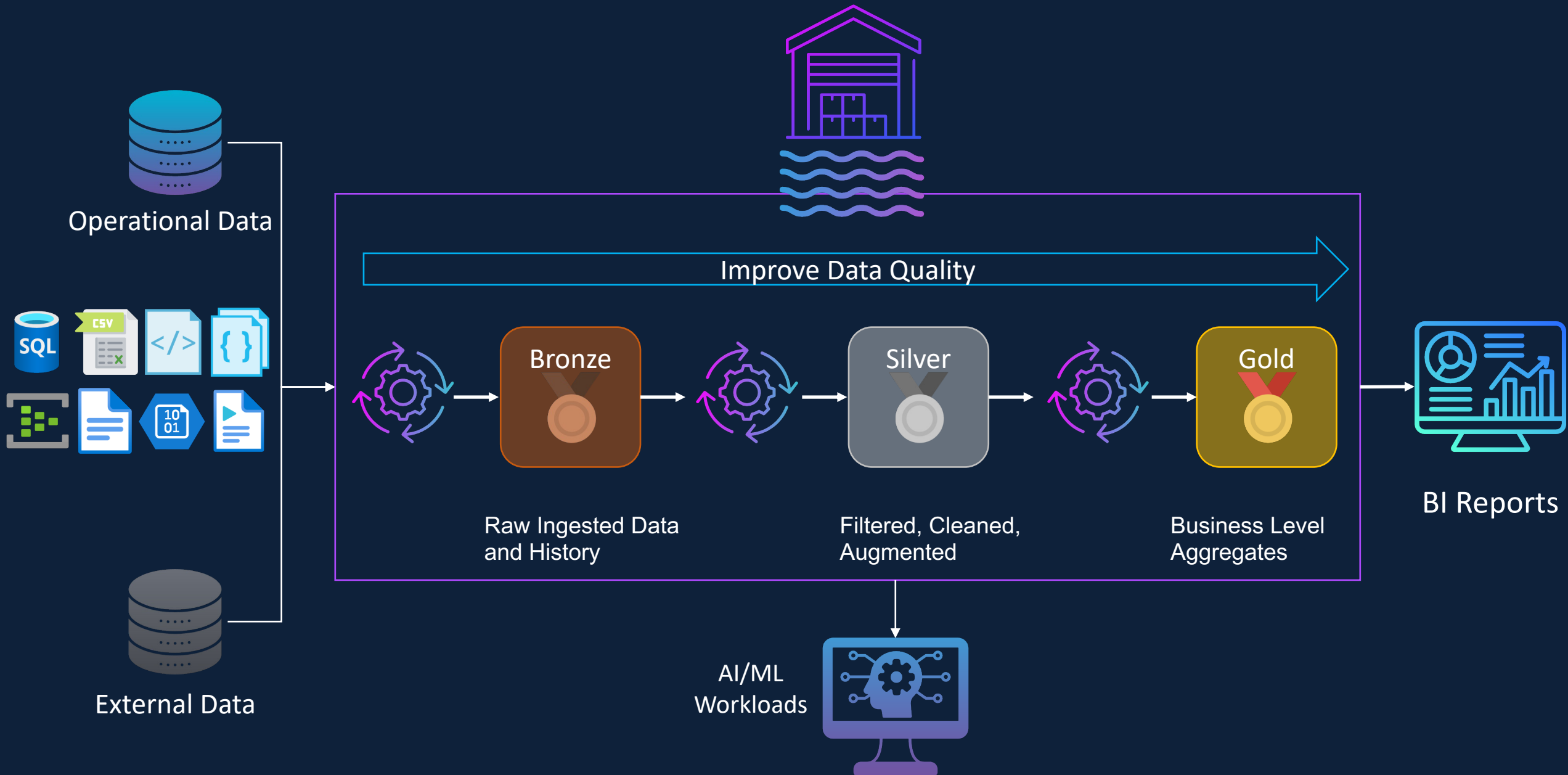
What is Delta Lake?



Delta Lake is the optimized storage layer that provides the foundation for tables in a Lakehouse on Databricks. Delta Lake is open source software that extends Parquet data files with a file-based transaction log for ACID transactions and scalable metadata handling.

It allows you to use a single copy of data for both batch and streaming operations and providing incremental processing at scale *- Databricks*

What is Delta Lake?



Delta Lake – Key Characteristics



ACID Transactions

Atomicity

Consistency

Isolation

Durability

Delta Lake – Key Characteristics



ACID Transactions

Scalable Metadata

Time Travel

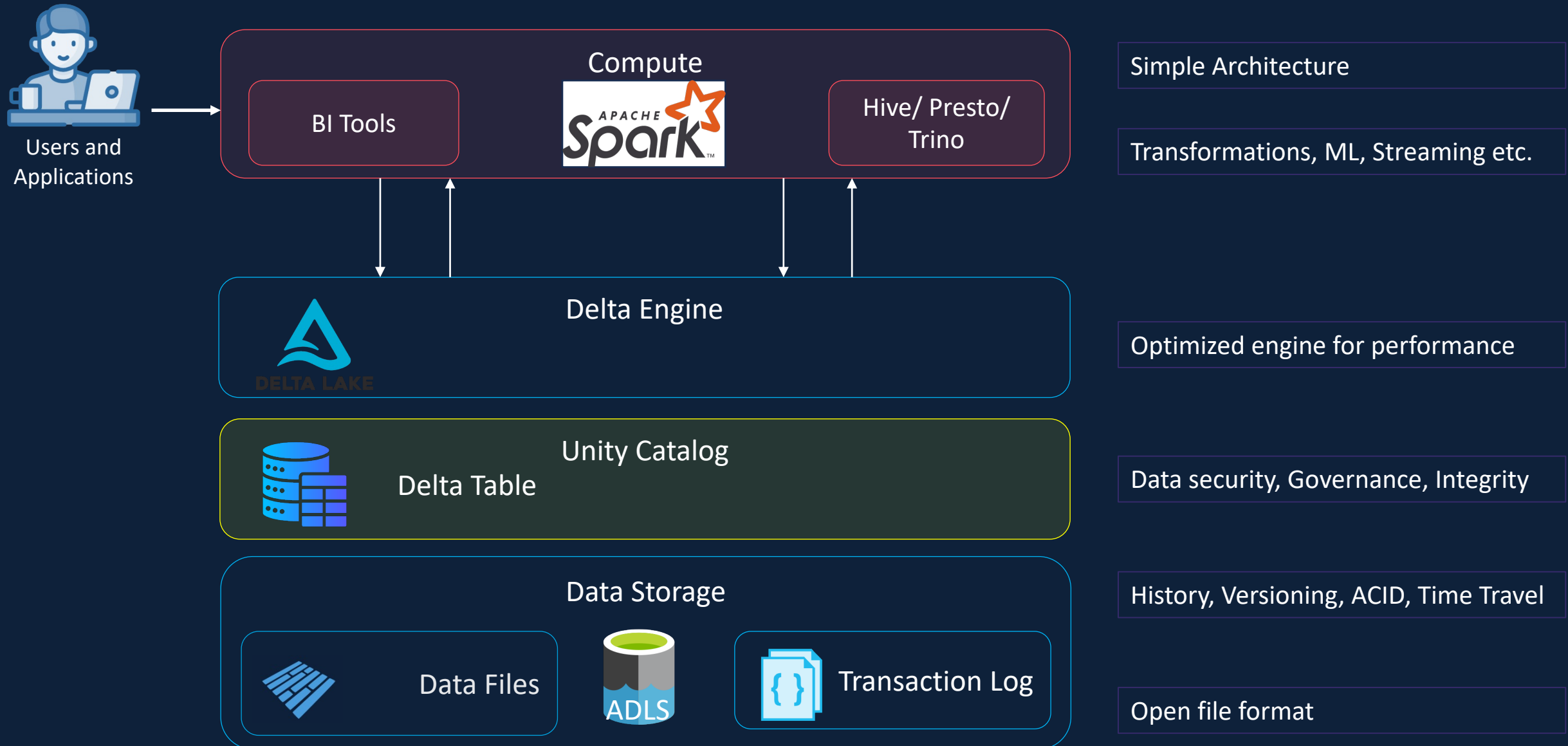
Simple Solution Architecture

Support for DML Operations

Better Performance

Open Source

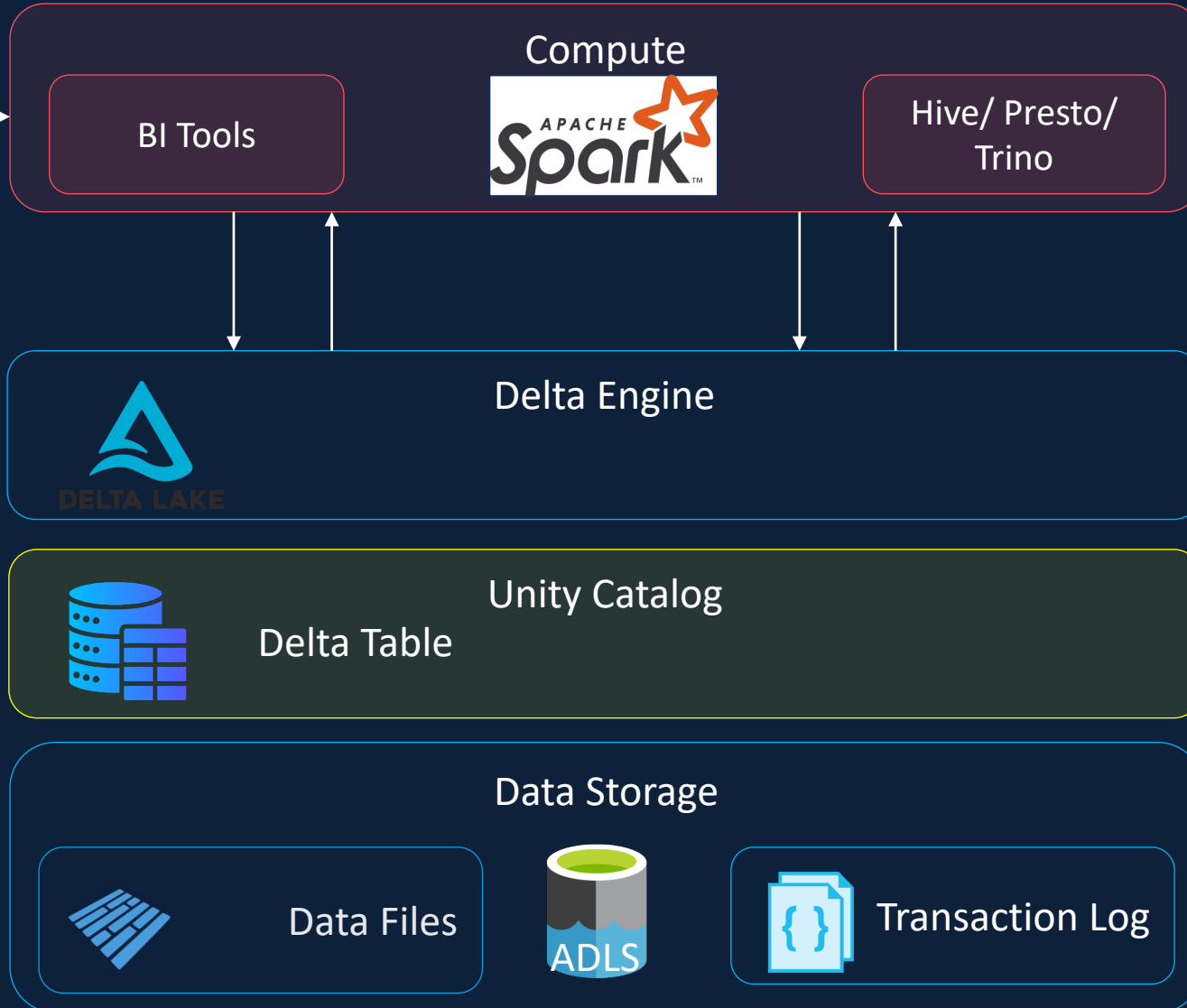
Delta Lake – Architecture



Delta Transaction Log



Users and Applications



Simple Architecture

Transformations, ML, Streaming etc.

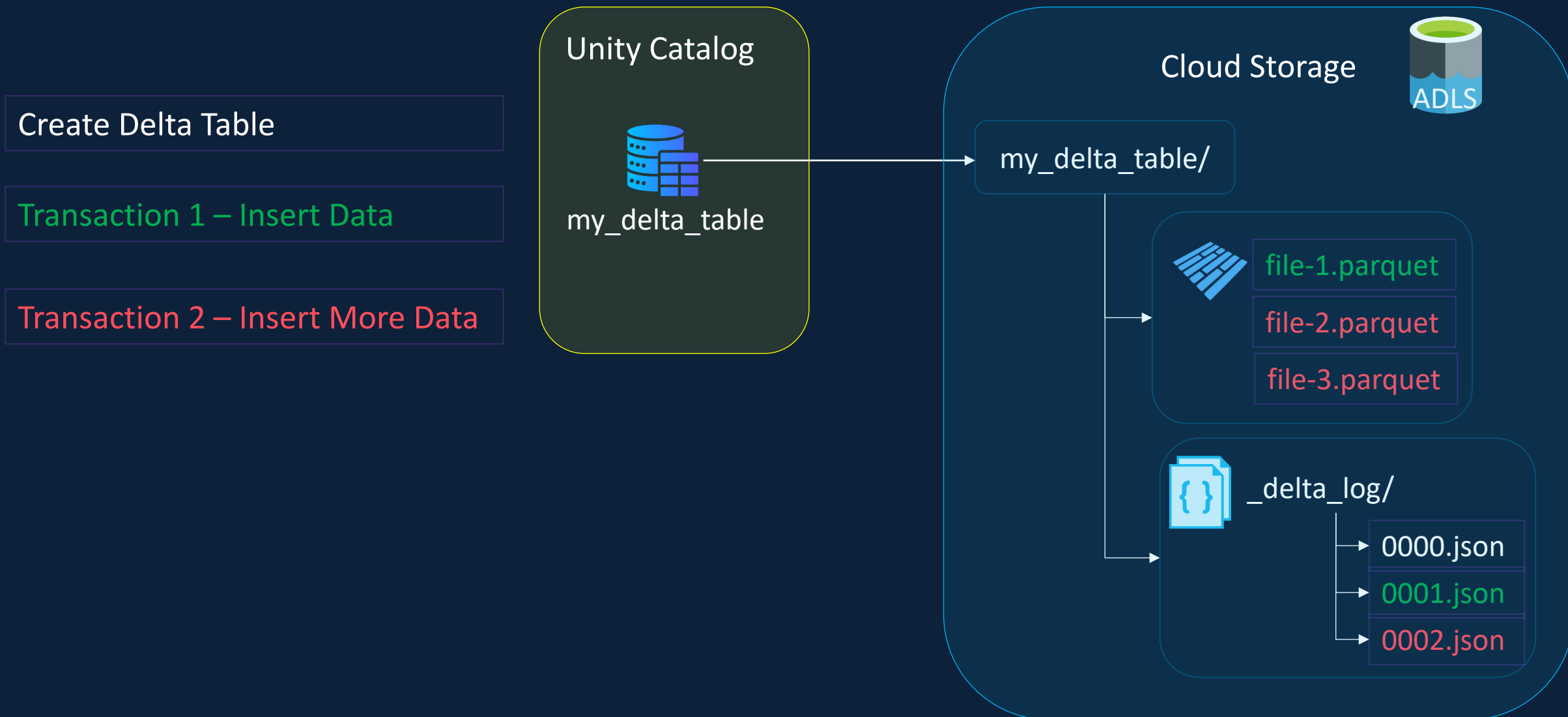
Optimized engine for performance

Data security, Governance, Integrity

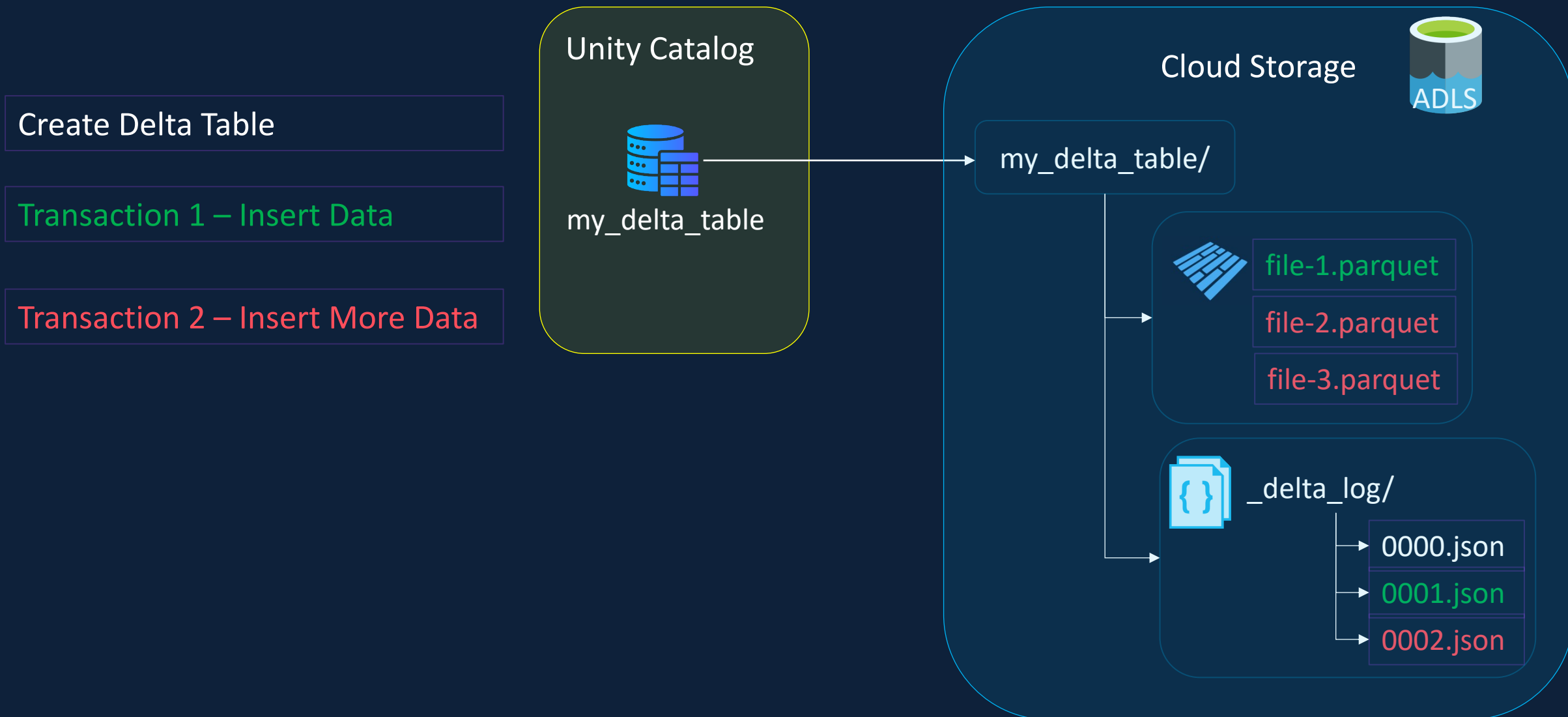
History, Versioning, ACID, Time Travel

Open file format

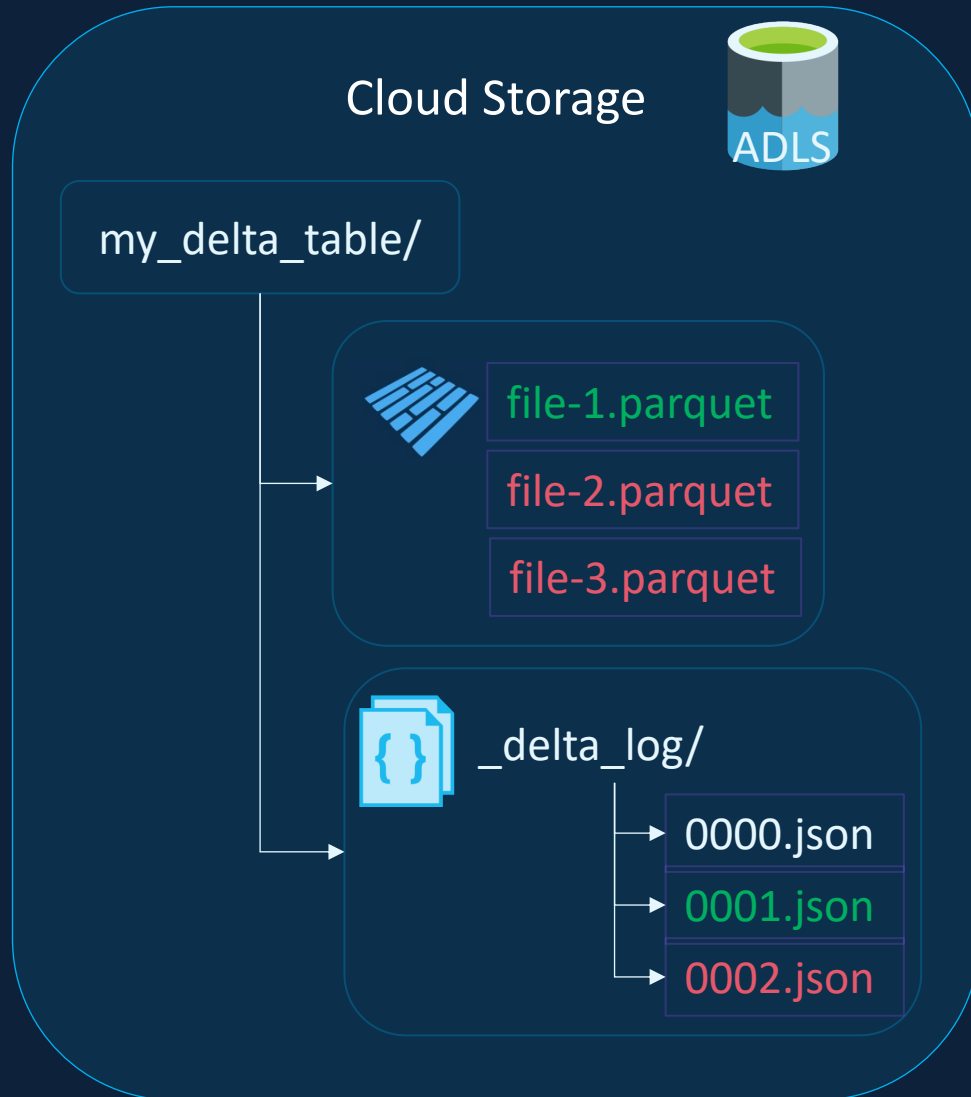
Delta Transaction Log



Delta Lake – ACID Transactions



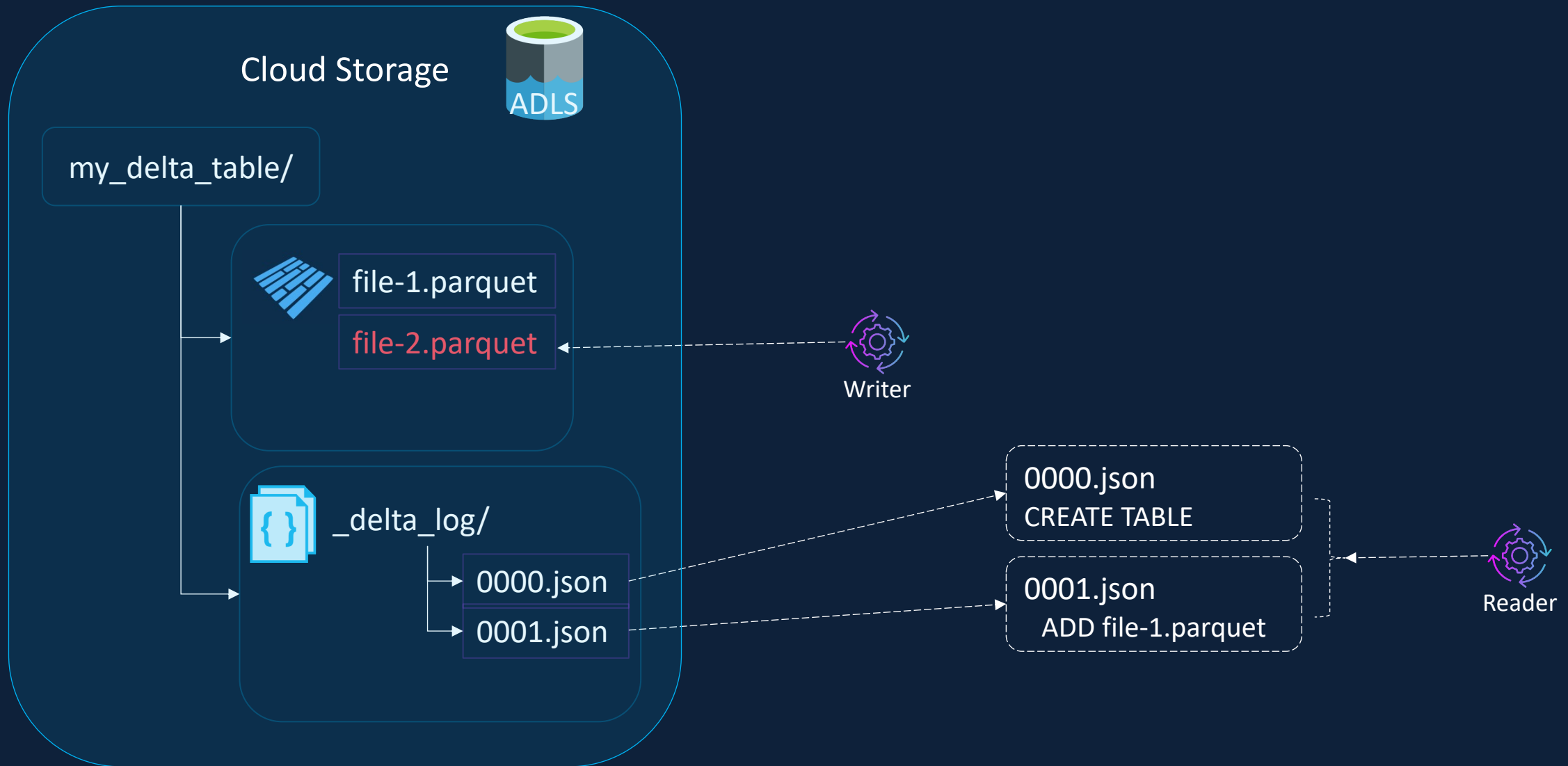
Delta Lake – ACID Transactions



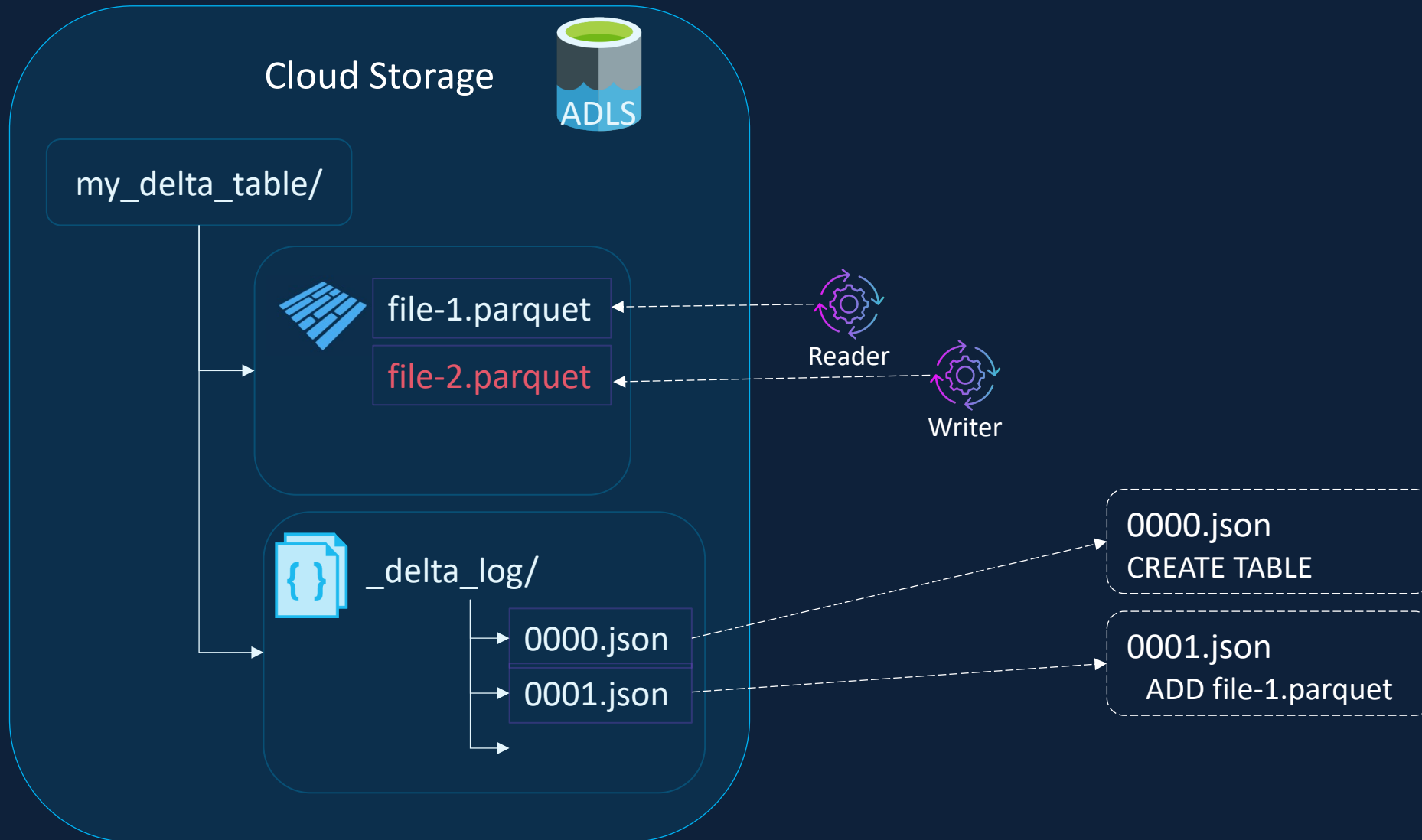
1. Transaction Logs are written at the end of the transaction

2. Readers will always read the transaction logs first to identify the list of data files to read

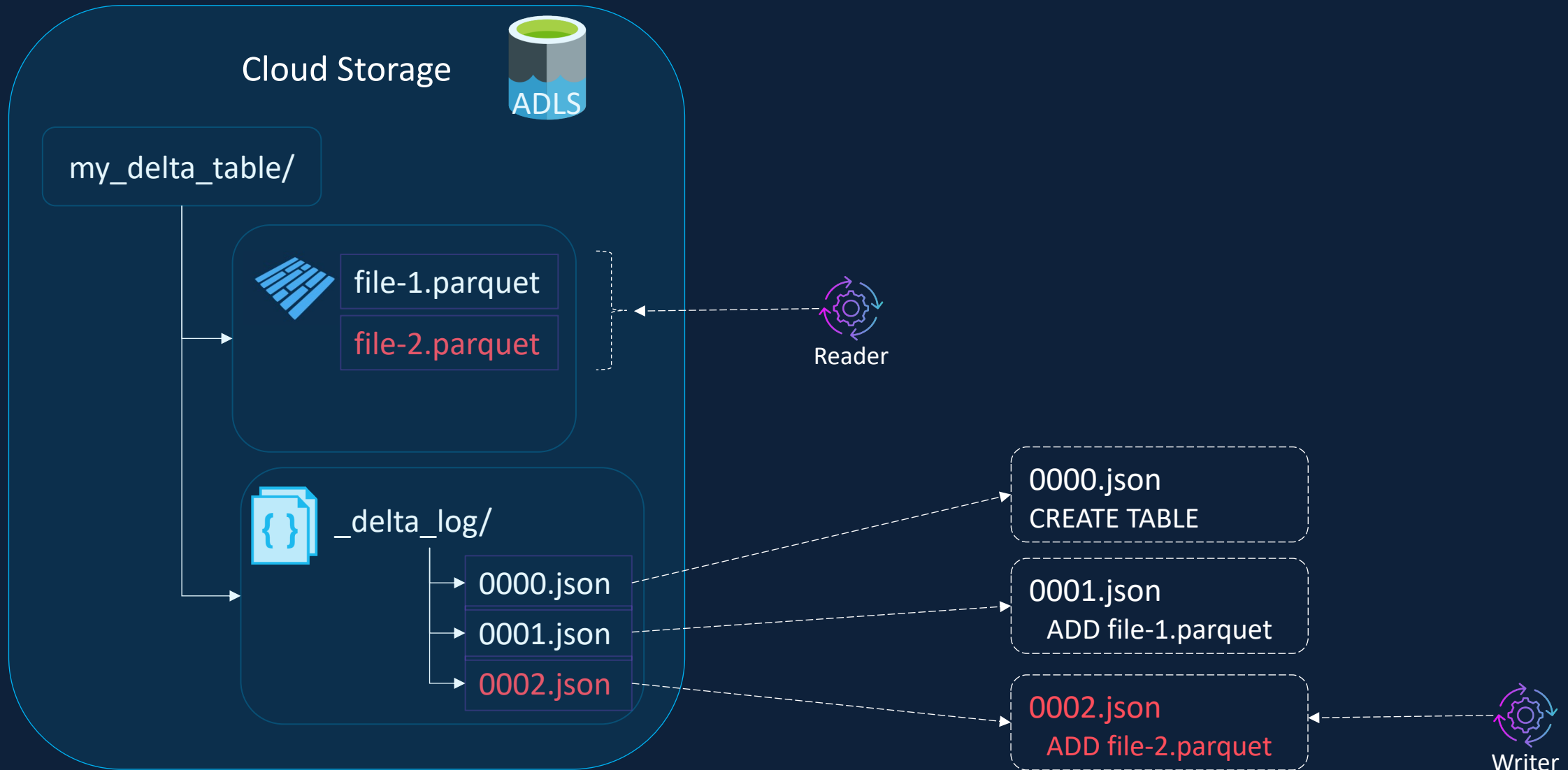
Delta Lake – ACID Transactions



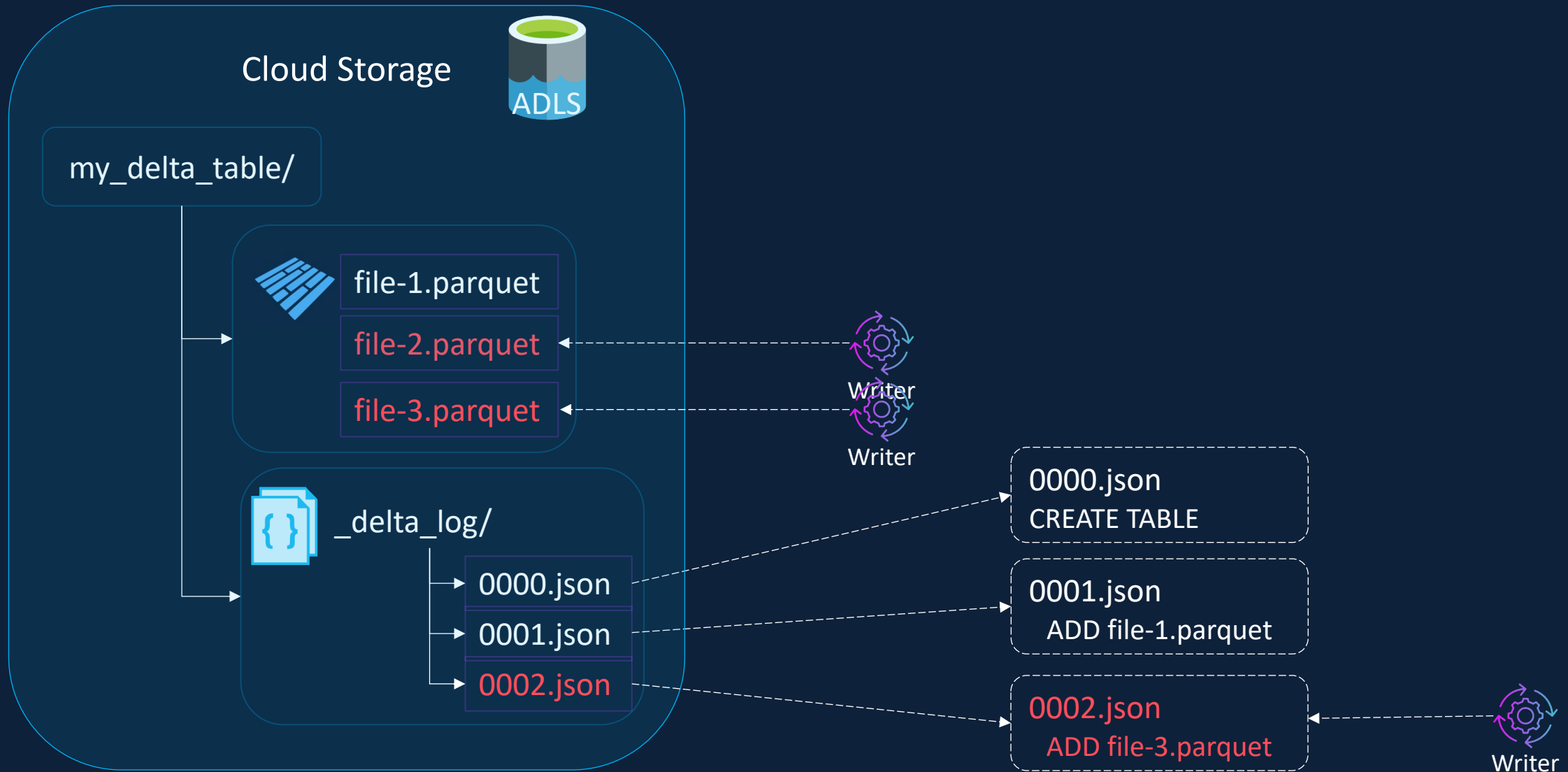
Delta Lake – ACID Transactions



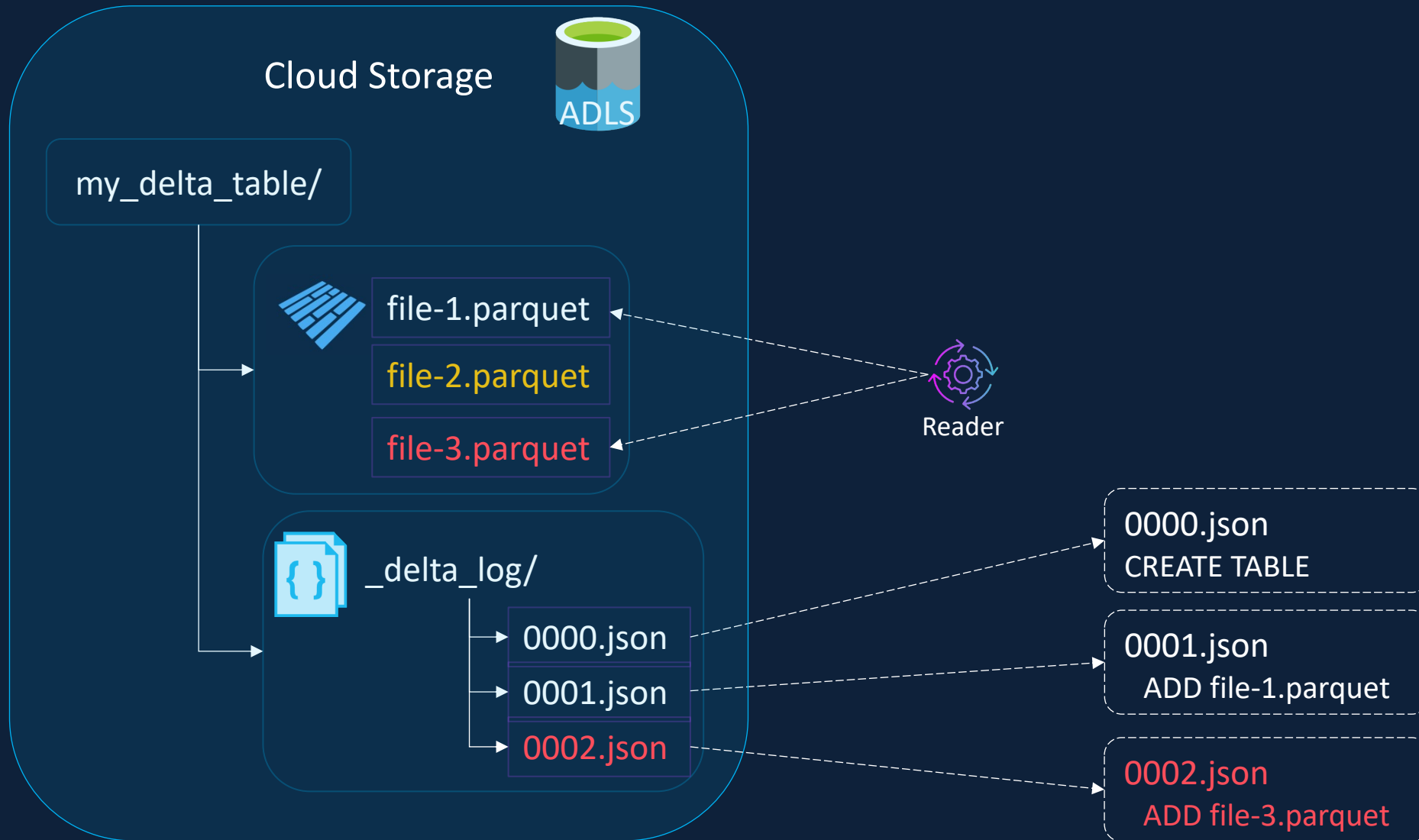
Delta Lake – ACID Transactions



Delta Lake – ACID Transactions



Delta Lake – ACID Transactions



Delta Lake – CREATE TABLE

```
{ { [CREATE OR] REPLACE TABLE |  
  CREATE [EXTERNAL] TABLE [ IF NOT EXISTS ] }  
table_name  
[ table_specification ]  
[ USING data_source ]  
[ table_clauses ]  
[ AS query ] }
```

table_specification
({ column_identifier column_type [column_properties] } [, ...]
[, table_constraint] [...])

column_properties
{ NOT NULL |
 COLLATE collation_name |
 GENERATED ALWAYS AS (expr) |
 GENERATED { ALWAYS | BY DEFAULT } AS IDENTITY [([START WITH start] [INCREMENT BY step])] |
 DEFAULT default_expression |
 COMMENT column_comment |
 column_constraint |
 MASK clause } [...]

table_clauses
{ OPTIONS clause |
 PARTITIONED BY clause |
 CLUSTER BY clause |
 clustered_by_clause |
 LOCATION path [WITH (CREDENTIAL credential_name)] |
 COMMENT table_comment |
 TBLPROPERTIES clause |
 WITH { ROW FILTER clause } } [...]

Delta Lake – CREATE TABLE

```
{ { [CREATE OR] REPLACE TABLE |  
  CREATE [EXTERNAL] TABLE [ IF NOT EXISTS ] }  
  table_name  
  [ table_specification ]  
  [ USING data_source ]  
  [ table_clauses ]  
  [ AS query ] }
```

```
table_specification  
  ( { column_identifier column_type [ column_properties ] } [, ...]  
    [ , table_constraint ] [...] )
```

```
column_properties  
  { NOT NULL |  
    COLLATE collation_name |  
    GENERATED ALWAYS AS ( expr ) |  
    GENERATED { ALWAYS | BY DEFAULT } AS IDENTITY [ ( [ START  
WITH start ] [ INCREMENT BY step ] ) ) |  
    DEFAULT default_expression |  
    COMMENT column_comment |  
    column_constraint |  
    MASK clause } [ ... ]
```

```
table_clauses  
  { OPTIONS clause |  
    PARTITIONED BY clause |  
    CLUSTER BY clause |  
    clustered_by_clause |  
    LOCATION path [ WITH ( CREDENTIAL credential_name ) ] |  
    COMMENT table_comment |  
    TBLPROPERTIES clause |  
    WITH { ROW FILTER clause } } [...]
```

Delta Lake – CREATE TABLE

```
{ { [CREATE OR] REPLACE TABLE |  
  CREATE [EXTERNAL] TABLE [ IF NOT EXISTS ] }  
  table_name  
  [ table_specification ]  
  [ USING data_source ]  
  [ table_clauses ]  
  [ AS query ] }
```

```
table_specification  
( { column_identifier column_type [ column_properties ] } [, ...]  
  [ , table_constraint ] [...] )
```

```
column_properties  
{ NOT NULL |  
  COLLATE collation_name |  
  GENERATED ALWAYS AS ( expr ) |  
  GENERATED { ALWAYS | BY DEFAULT } AS IDENTITY [ ( [ START  
WITH start ] [ INCREMENT BY step ] ) ] |  
  DEFAULT default_expression |  
  COMMENT column_comment |  
  column_constraint |  
  MASK clause } [ ... ]
```

```
table_clauses  
{ OPTIONS clause |  
  PARTITIONED BY clause |  
  CLUSTER BY clause |  
  clustered_by_clause |  
  LOCATION path [ WITH ( CREDENTIAL credential_name ) ] |  
  COMMENT table_comment |  
  TBLPROPERTIES clause |  
  WITH { ROW FILTER clause } } [...]
```

Delta Lake – CREATE TABLE

```
{ { [CREATE OR] REPLACE TABLE |  
  CREATE [EXTERNAL] TABLE [ IF NOT EXISTS ] }  
  table_name  
  [ table_specification ]  
  [ USING data_source ]  
  [ table_clauses ]  
  [ AS query ] }
```

```
table_specification  
( { column_identifier column_type [ column_properties ] } [, ...]  
  [ , table_constraint ] [...] )
```

```
column_properties  
{ NOT NULL |  
  COLLATE collation_name |  
  GENERATED ALWAYS AS ( expr ) |  
  GENERATED { ALWAYS | BY DEFAULT } AS IDENTITY [ ( [ START  
WITH start ] [ INCREMENT BY step ] ) ) |  
  DEFAULT default_expression |  
  COMMENT column_comment |  
  column_constraint |  
  MASK clause } [ ... ]
```

```
table_clauses  
{ OPTIONS clause |  
  PARTITIONED BY clause |  
  CLUSTER BY clause |  
  clustered_by_clause |  
  LOCATION path [ WITH ( CREDENTIAL credential_name ) ] |  
  COMMENT table_comment |  
  TBLPROPERTIES clause |  
  WITH { ROW FILTER clause } } [...]
```

Delta Lake – CREATE TABLE

```
{ { [CREATE OR] REPLACE TABLE |  
  CREATE [EXTERNAL] TABLE [ IF NOT EXISTS ] }  
  table_name  
  [ table_specification ]  
  [ USING data_source ]  
  [ table_clauses ]  
  [ AS query ] }
```

```
table_specification  
( { column_identifier column_type [ column_properties ] } [, ...]  
  [ , table_constraint ] [...] )
```

```
column_properties  
{ NOT NULL |  
  COLLATE collation_name |  
  GENERATED ALWAYS AS ( expr ) |  
  GENERATED { ALWAYS | BY DEFAULT } AS IDENTITY [ ( [ START  
WITH start ] [ INCREMENT BY step ] ) ) |  
  DEFAULT default_expression |  
  COMMENT column_comment |  
  column_constraint |  
  MASK clause } [ ... ]
```

```
table_clauses  
{ OPTIONS clause |  
  PARTITIONED BY clause |  
  CLUSTER BY clause |  
  clustered_by_clause |  
  LOCATION path [ WITH ( CREDENTIAL credential_name ) ] |  
  COMMENT table_comment |  
  TBLPROPERTIES clause |  
  WITH { ROW FILTER clause } } [...]
```

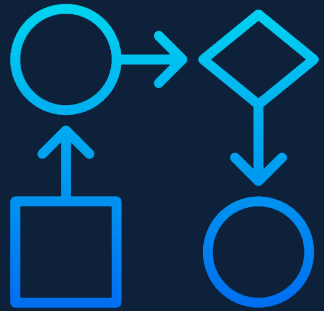
Delta Lake – CREATE TABLE

```
{ { [CREATE OR] REPLACE TABLE |  
  CREATE [EXTERNAL] TABLE [ IF NOT EXISTS ] }  
  table_name  
  [ table_specification ]  
  [ USING data_source ]  
  [ table_clauses ]  
  [ AS query ] }
```

table_specification
({ column_identifier column_type [column_properties] } [, ...]
 [, table_constraint] [...])

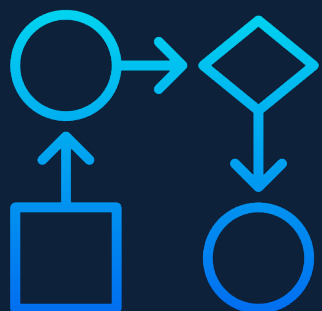
column_properties
{ NOT NULL |
 COLLATE collation_name |
 GENERATED ALWAYS AS (expr) |
 GENERATED { ALWAYS | BY DEFAULT } AS IDENTITY [([START
WITH start] [INCREMENT BY step])] |
 DEFAULT default_expression |
 COMMENT column_comment |
 column_constraint |
 MASK clause } [...]

table_clauses
{ OPTIONS clause |
 PARTITIONED BY clause |
 CLUSTER BY clause |
 clustered_by_clause |
 LOCATION path [WITH (CREDENTIAL credential_name)] |
 COMMENT table_comment |
 TBLPROPERTIES clause |
 WITH { ROW FILTER clause } } [...]



Databricks Workflows / Jobs

Databricks Workflows / Jobs

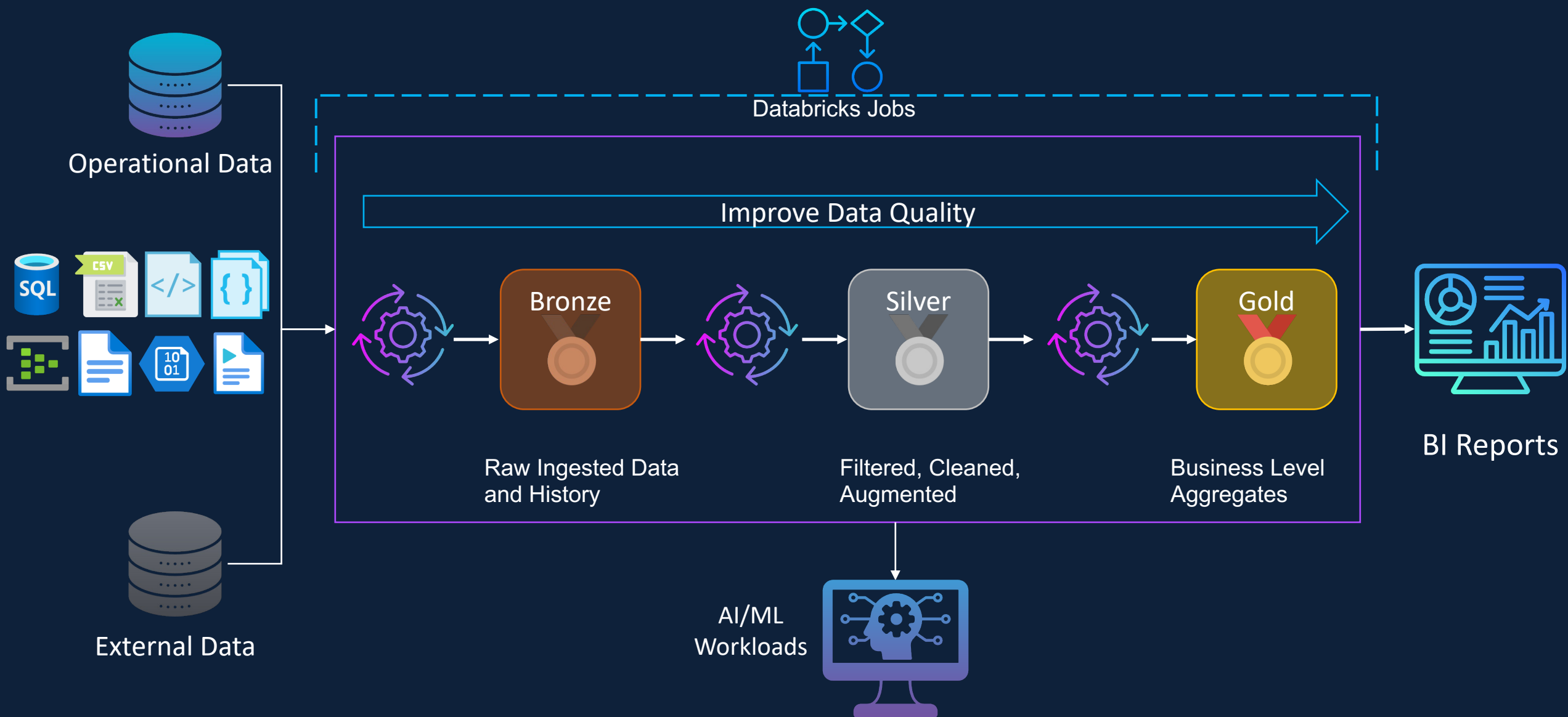


Databricks Workflows is a managed orchestration service, fully integrated with the Databricks Data Intelligence Platform.

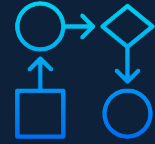
Workflows lets you easily define, manage and monitor multitask workflows for ETL, analytics and machine learning pipelines.

- *Databricks*

Databricks Workflows / Jobs



Databricks Workflows / Jobs



Databricks Jobs

Rerun / Retry

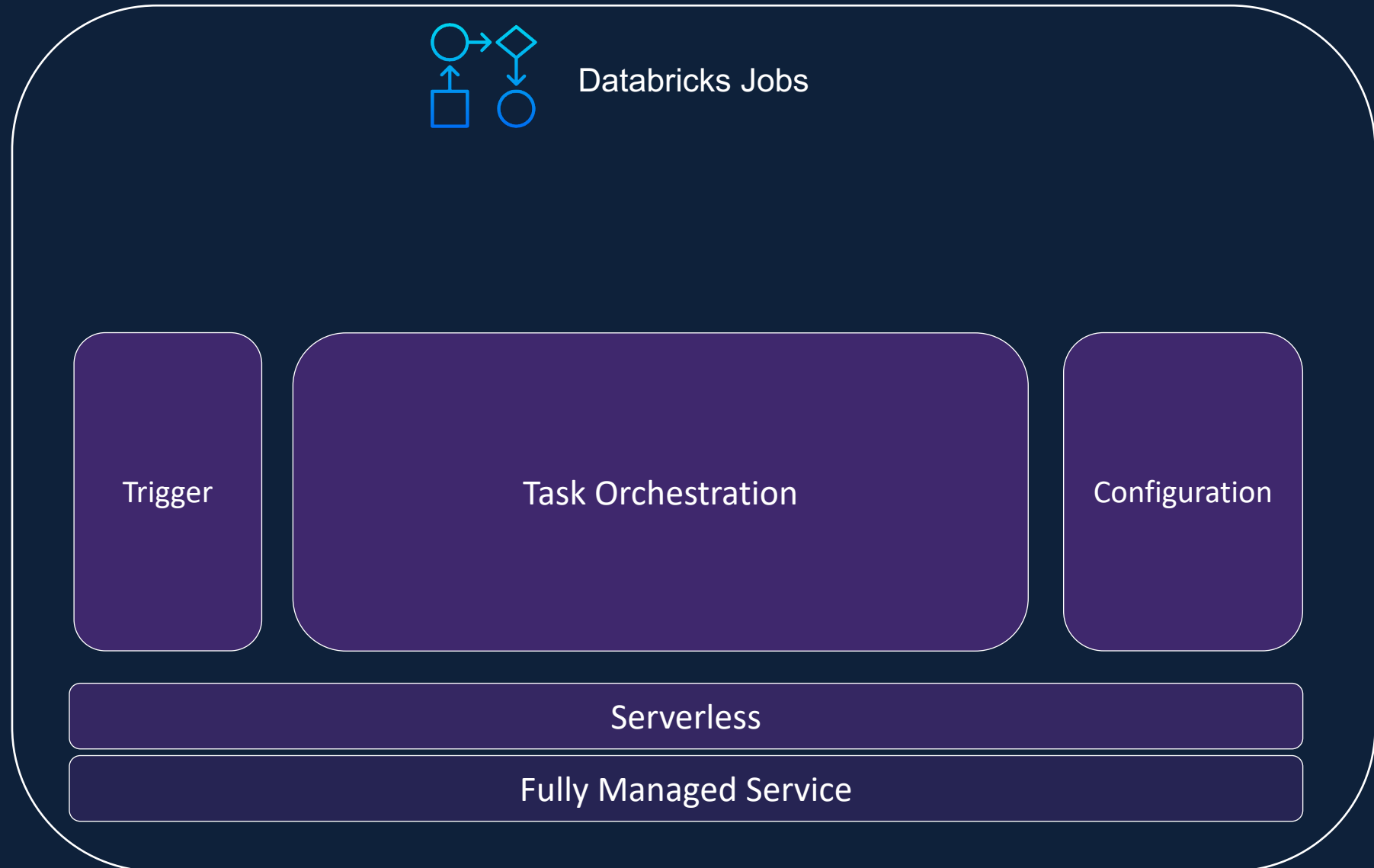
Monitoring

Job

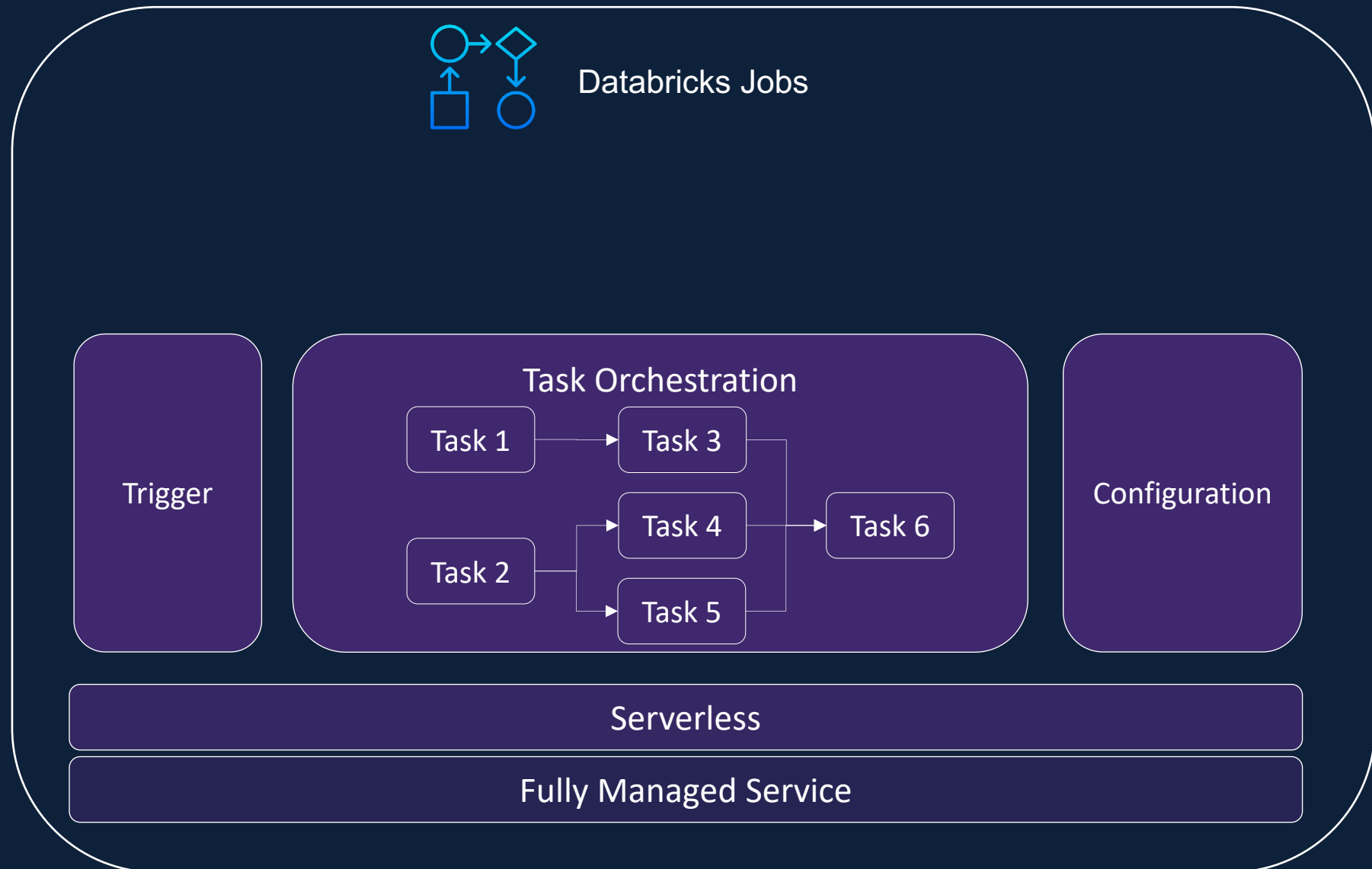
Serverless

Fully Managed Service

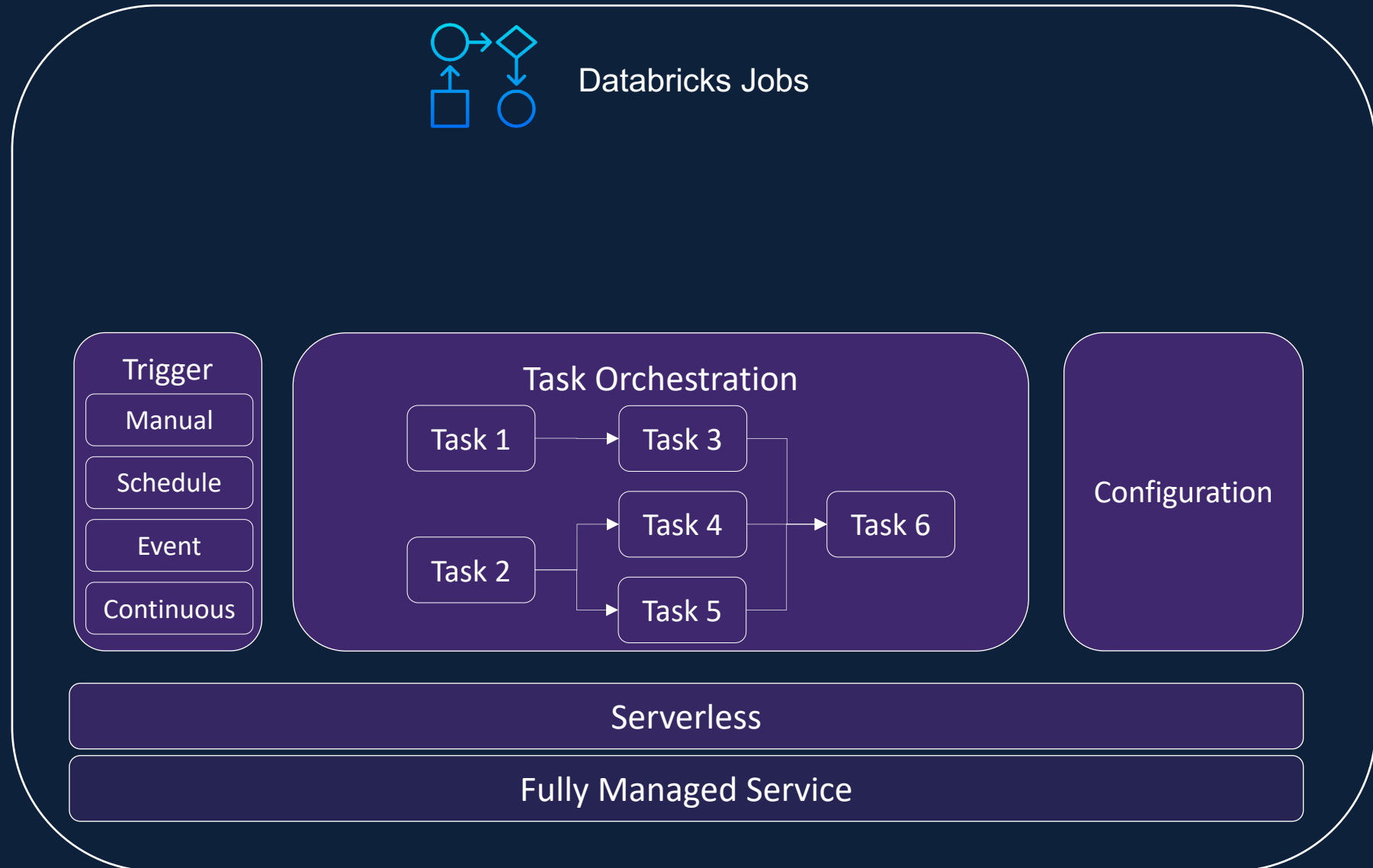
Databricks Workflows / Jobs



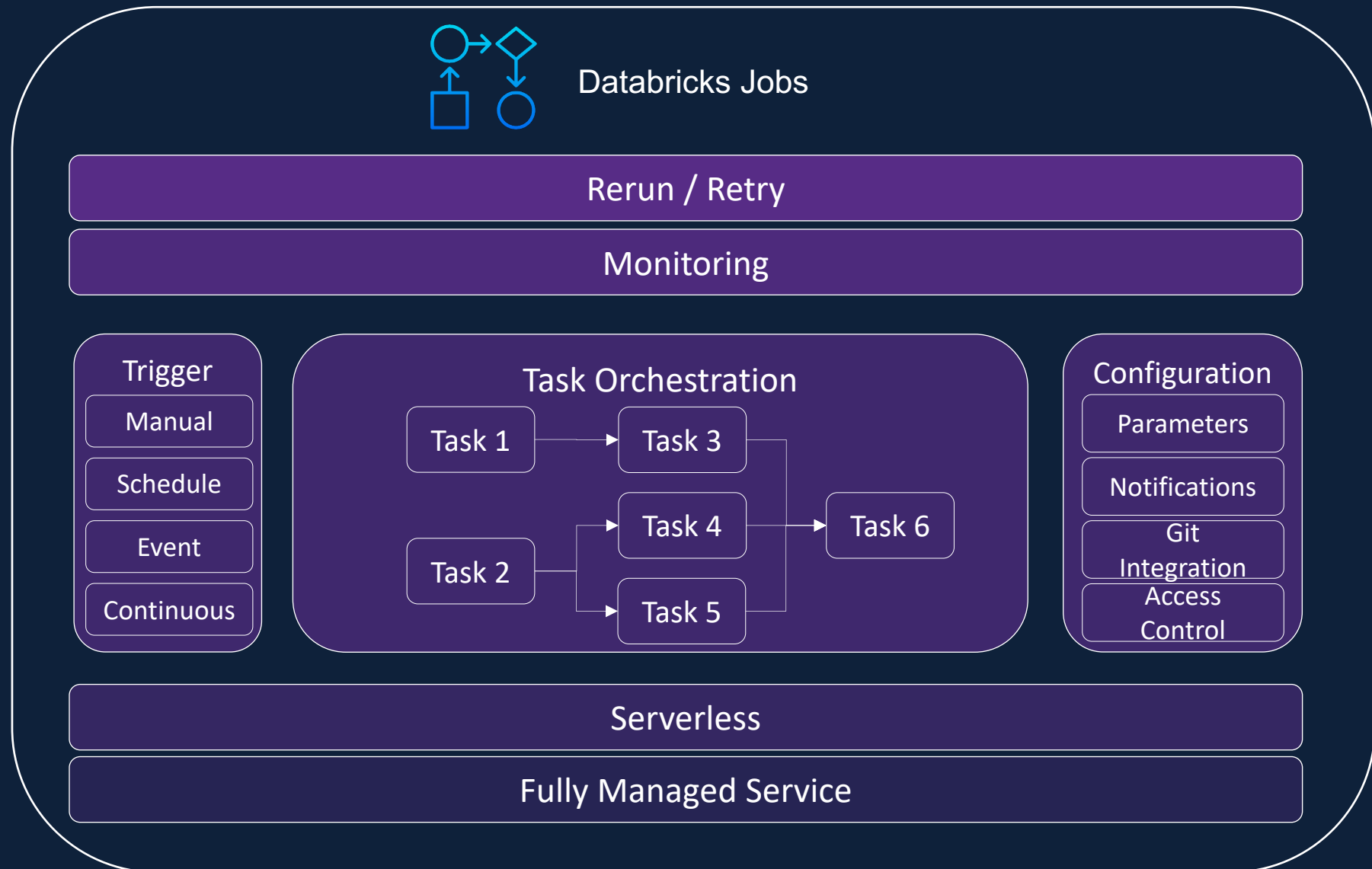
Databricks Workflows / Jobs



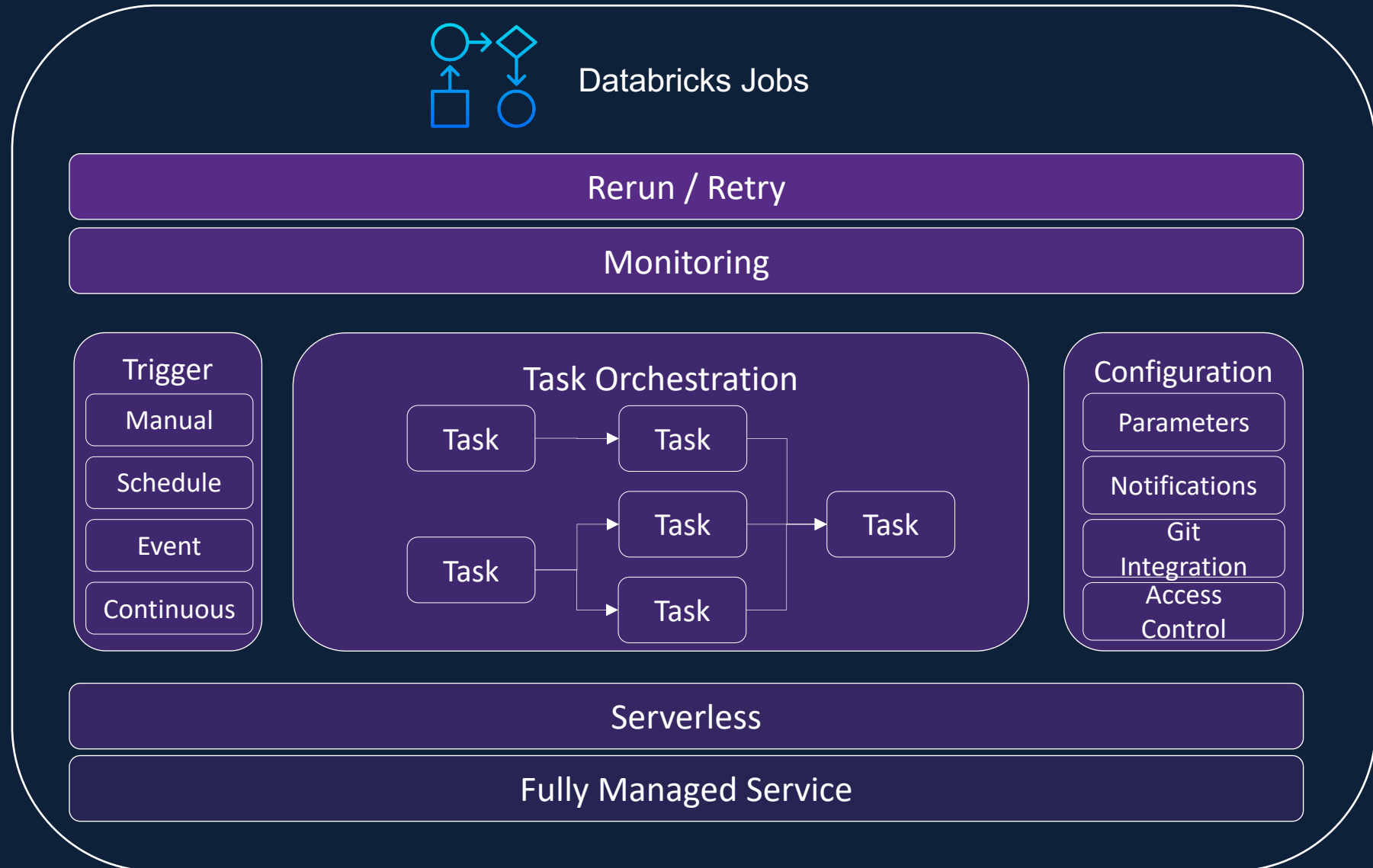
Databricks Workflows / Jobs



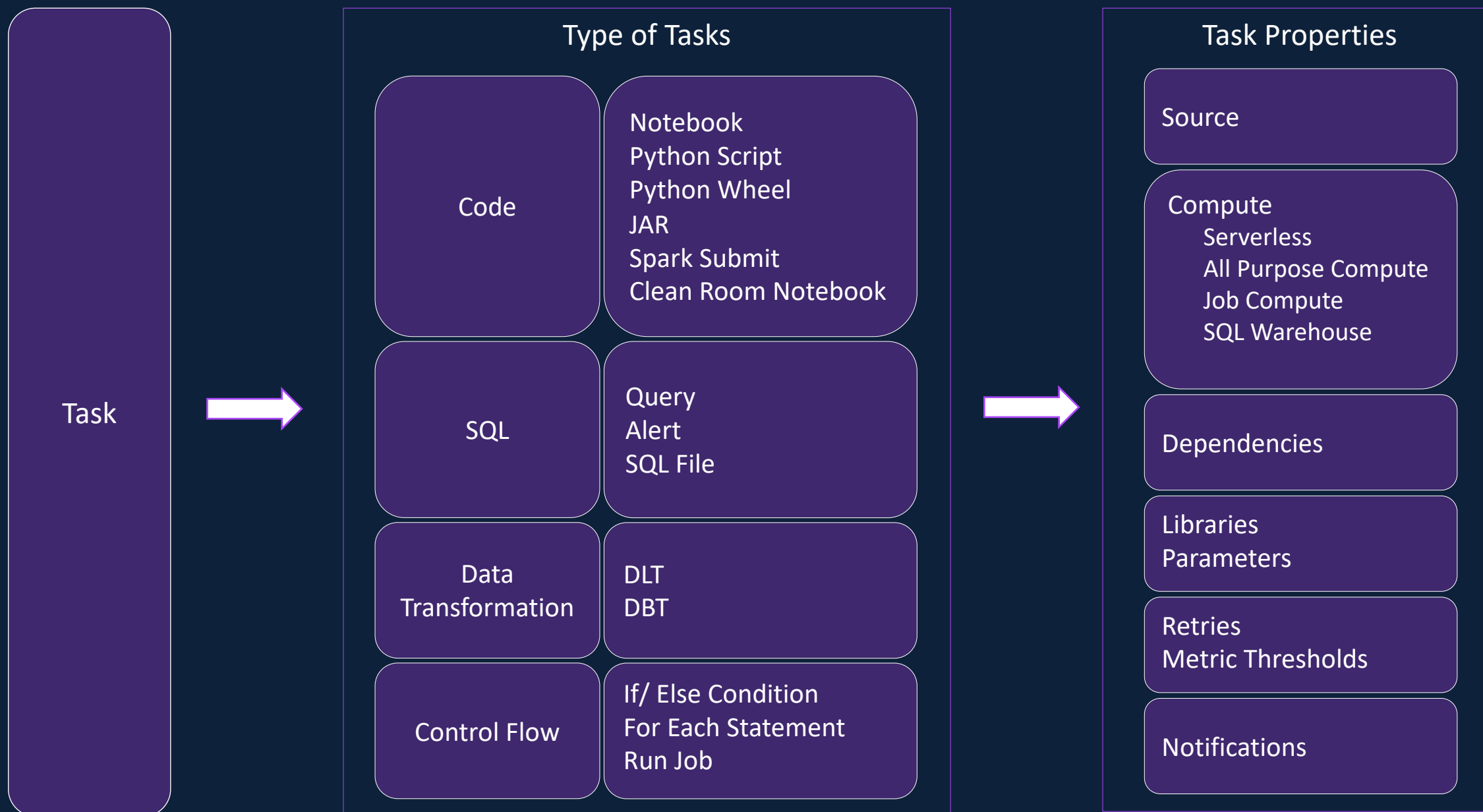
Databricks Workflows / Jobs



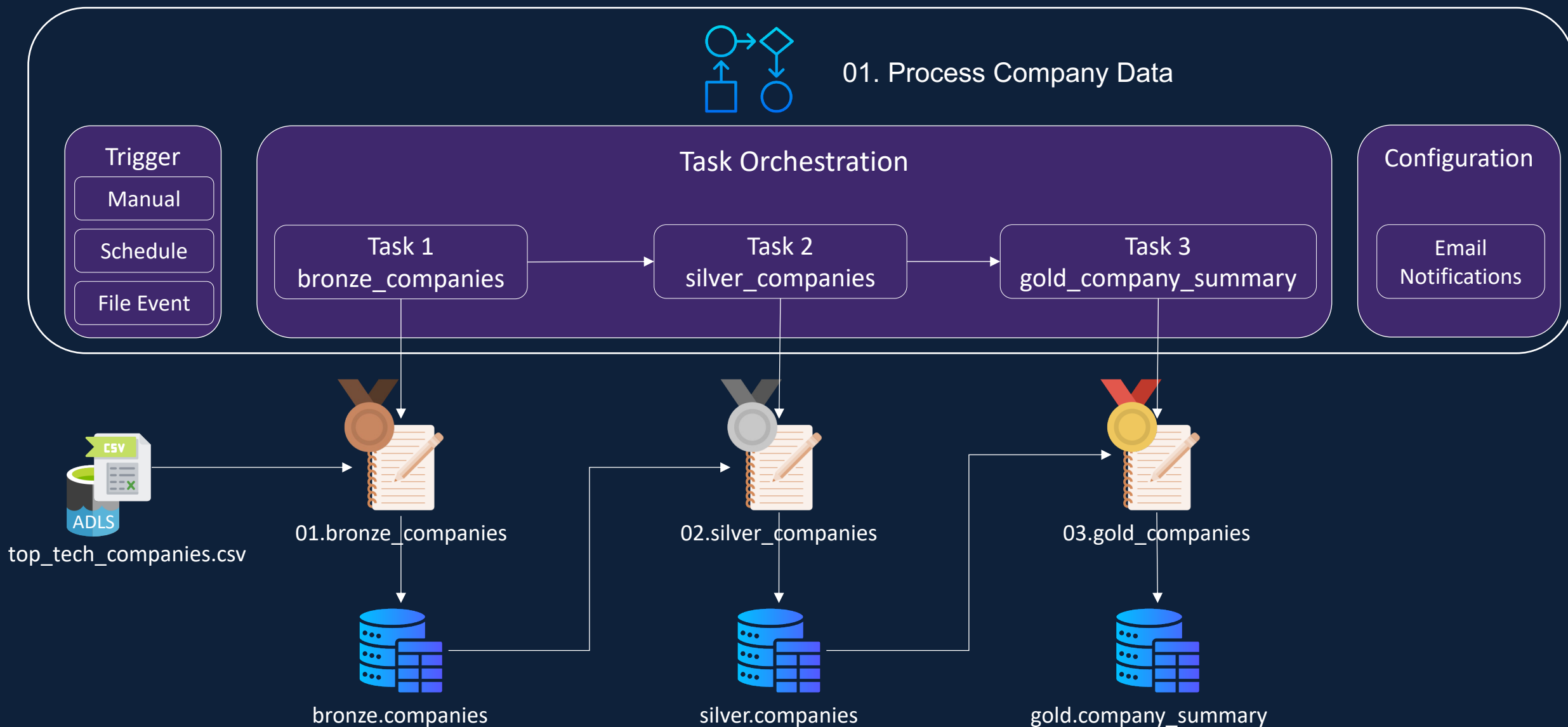
Databricks Workflows / Jobs



Databricks Workflows / Jobs



Databricks Workflows / Jobs





Lakeflow Spark Declarative Pipelines

Previously known as Delta Live Tables (DLT)



Lakeflow Spark Declarative Pipelines

Lakeflow Spark Declarative Pipelines is a declarative ETL framework for building reliable, maintainable, and testable data processing pipelines.

You define the transformations to perform on your data and the platform manages task orchestration, cluster management, monitoring, data quality, and error handling.

It handles both streaming and batch workloads with minimal manual intervention.

- Databricks



Lakeflow Spark Declarative Pipelines

Lakeflow Spark Declarative Pipelines is a **declarative ETL framework** for building reliable, maintainable, and testable data processing pipelines.

You define the transformations to perform on your data and the platform manages task orchestration, cluster management, monitoring, data quality, and error handling.

It handles both streaming and batch workloads with minimal manual intervention.

- Databricks



Lakeflow Spark Declarative Pipelines

Lakeflow Spark Declarative Pipelines is a **declarative ETL framework** for building reliable, maintainable, and testable data processing pipelines.

You define the transformations to perform on your data and the platform manages **task orchestration, cluster management, monitoring, data quality, and error handling**.

It handles both streaming and batch workloads with minimal manual intervention.

- Databricks



Lakeflow Spark Declarative Pipelines

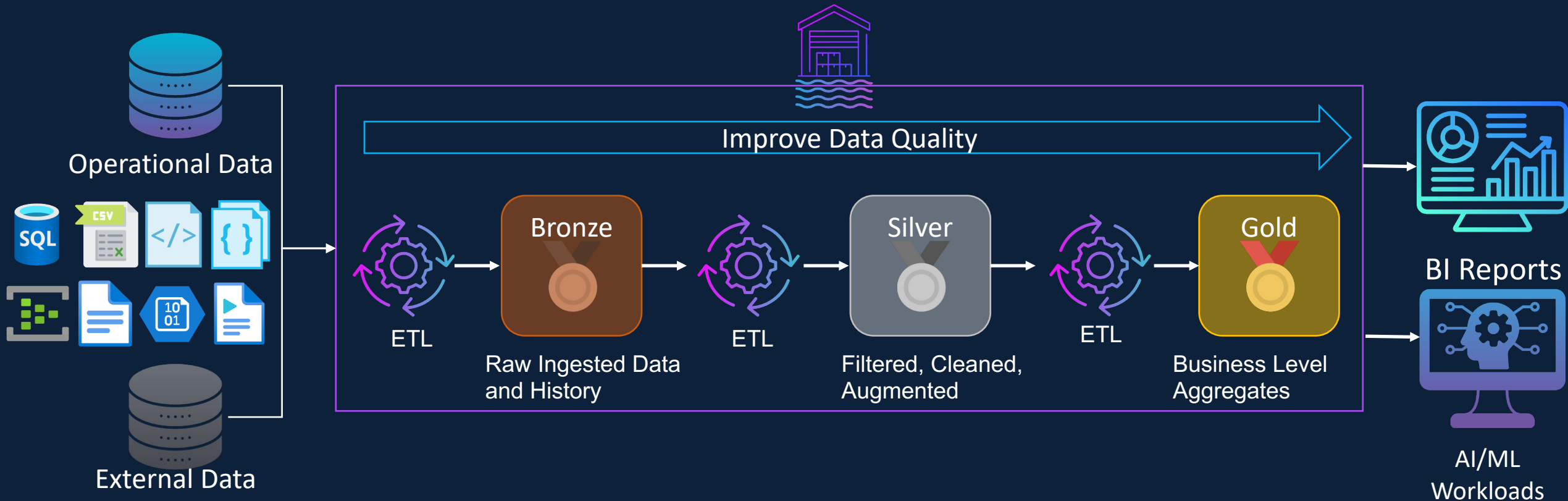
Lakeflow Spark Declarative Pipelines is a **declarative ETL framework** for building reliable, maintainable, and testable data processing pipelines.

You define the transformations to perform on your data and the platform manages **task orchestration, cluster management, monitoring, data quality, and error handling**.

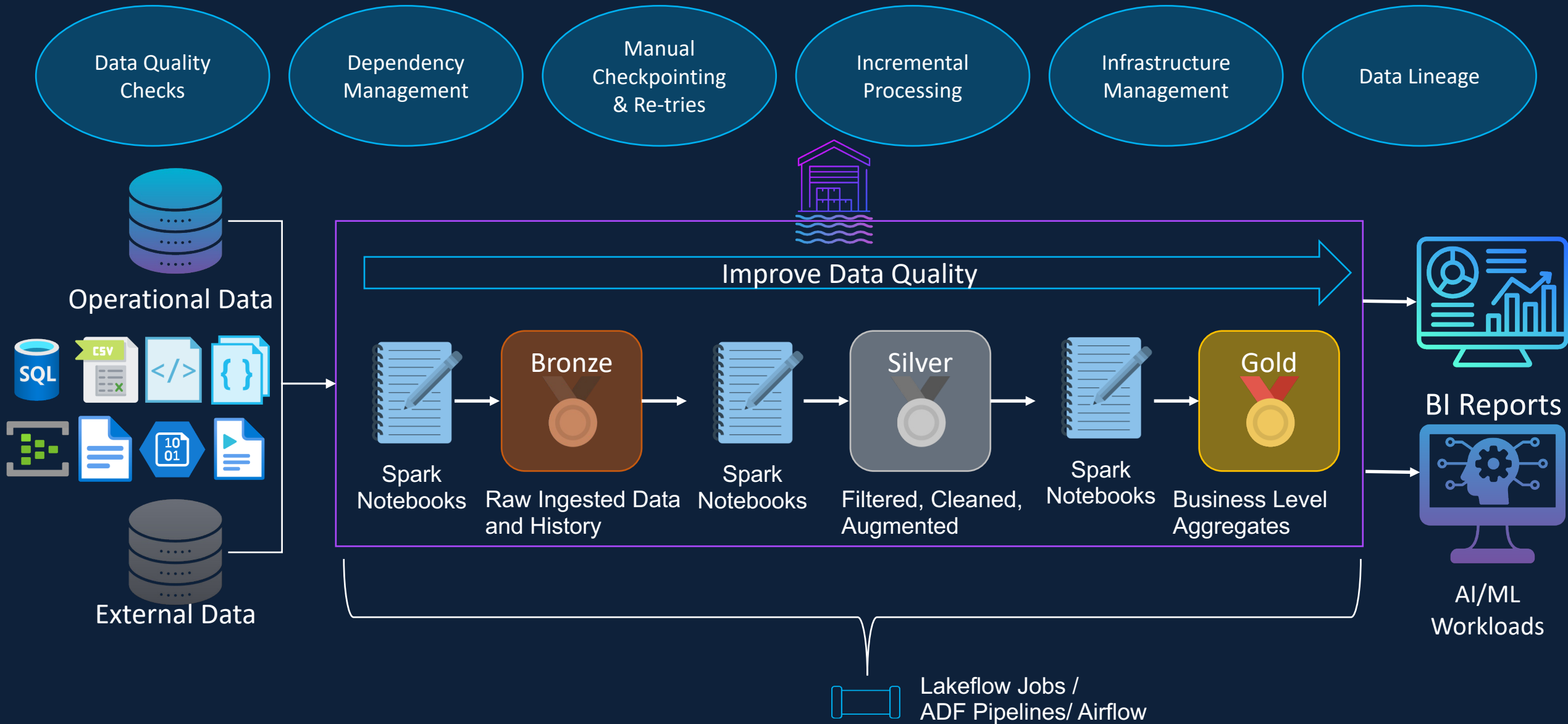
It handles both **streaming and batch workloads** with minimal manual intervention.

- Databricks

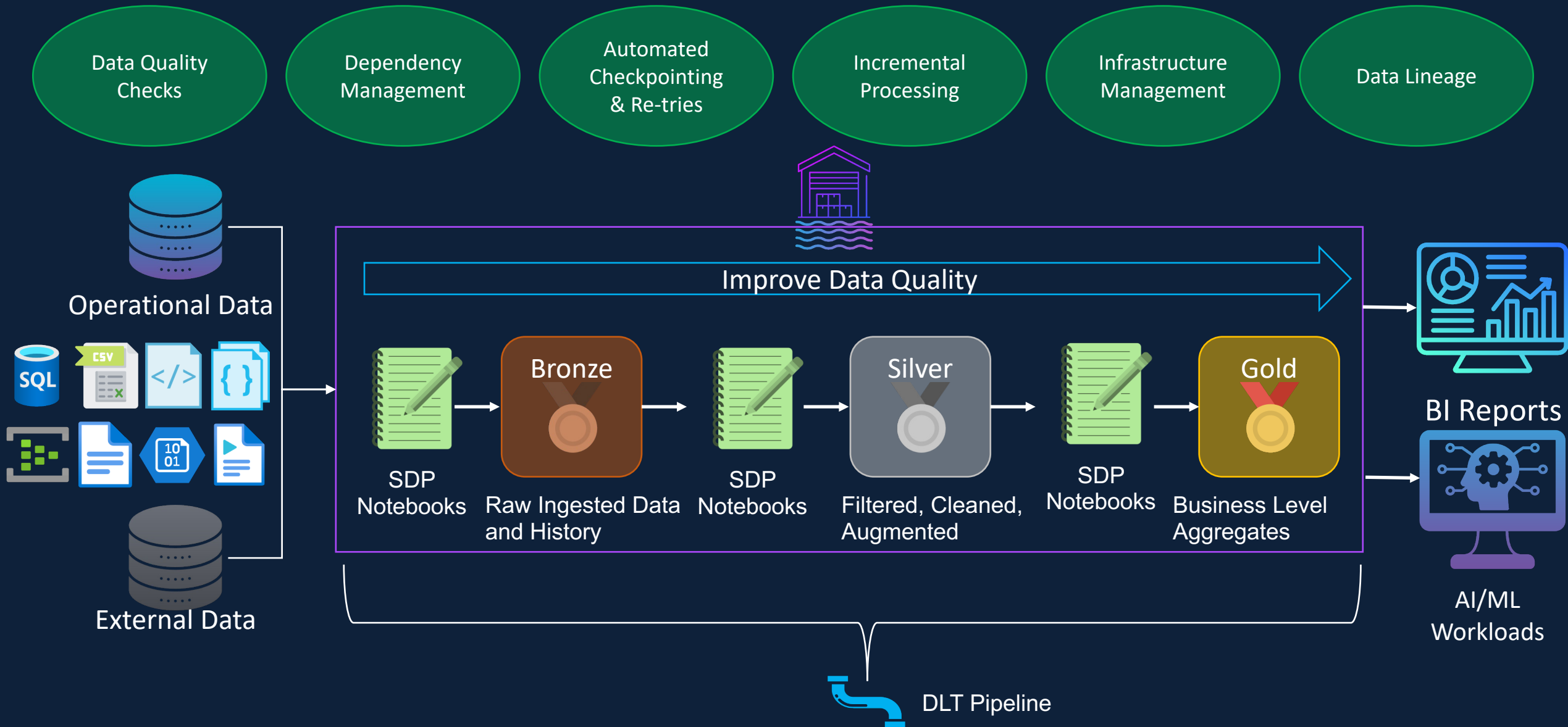
Lakeflow Spark Declarative Pipelines - Overview



Lakeflow Spark Declarative Pipelines - Overview



Lakeflow Spark Declarative Pipelines - Overview



Lakeflow Spark Declarative Pipelines / Lakeflow Jobs

Lakeflow Spark Declarative Pipelines

Performs task orchestration with a pipeline
For managing a data flow

Lakeflow Jobs

For managing a workflow
Scheduling Tasks (including Lakeflow Declarative Pipelines)
Trigger Tasks based on Events (e.g. File Arrival)
Orchestrate ETL/ BI/ ML Tasks

Lakeflow Spark Declarative Pipelines Architecture

Lakeflow Spark Declarative Pipelines - Architecture

Lakeflow Spark Declarative Pipelines (SDP)

User Interface

SDP Notebooks

SQL

Python



Data Transformations

Data Quality Expectations

SDP Pipelines



Hive Metastore

Unity Catalog

Lakeflow Spark Declarative Pipelines - Architecture

Lakeflow Spark Declarative Pipelines (SDP)

User Interface

SDP Notebooks

SQL

Python

Data Transformations

Data Quality Expectations

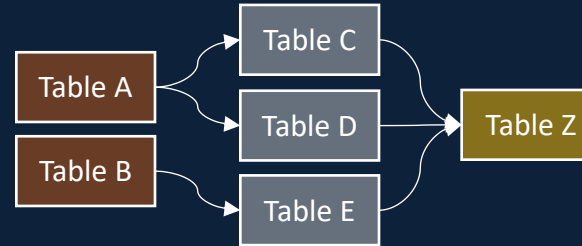
SDP Pipelines



Hive Metastore

Unity Catalog

DAG



Infrastructure



Metadata

Pipeline Status/ Health

Data Quality Checks

Data Lineage

Data Quality Checks

Dependency Management

Automated Checkpointing & Re-tries

Incremental Processing

Infrastructure Management

Data Lineage

Lakeflow Spark Declarative Pipelines - Programming

Lakeflow Spark Declarative Pipelines(SDP)

User Interface

SDP Notebooks

SQL

Python



Data Transformations

Data Quality Expectations

SDP Pipelines



Hive Metastore

Unity Catalog

Lakeflow Spark Declarative Pipelines - Programming

Lakeflow Spark Declarative Pipelines is a declarative ETL framework for building reliable, maintainable, and testable data processing pipelines.

Lakeflow Spark Declarative Pipelines - Programming

Imperative

How to do a task?

User provides step-by-step instructions

```
users_above_18 = []  
for user in users:  
    if user.age > 18:  
        users_above_18.append(user)
```

Declarative

What should be done?

System decides how to do it

```
SELECT * FROM users WHERE age > 18;
```

Lakeflow Spark Declarative Pipelines - Programming

Declarative

What should be done?

System decides how to do it

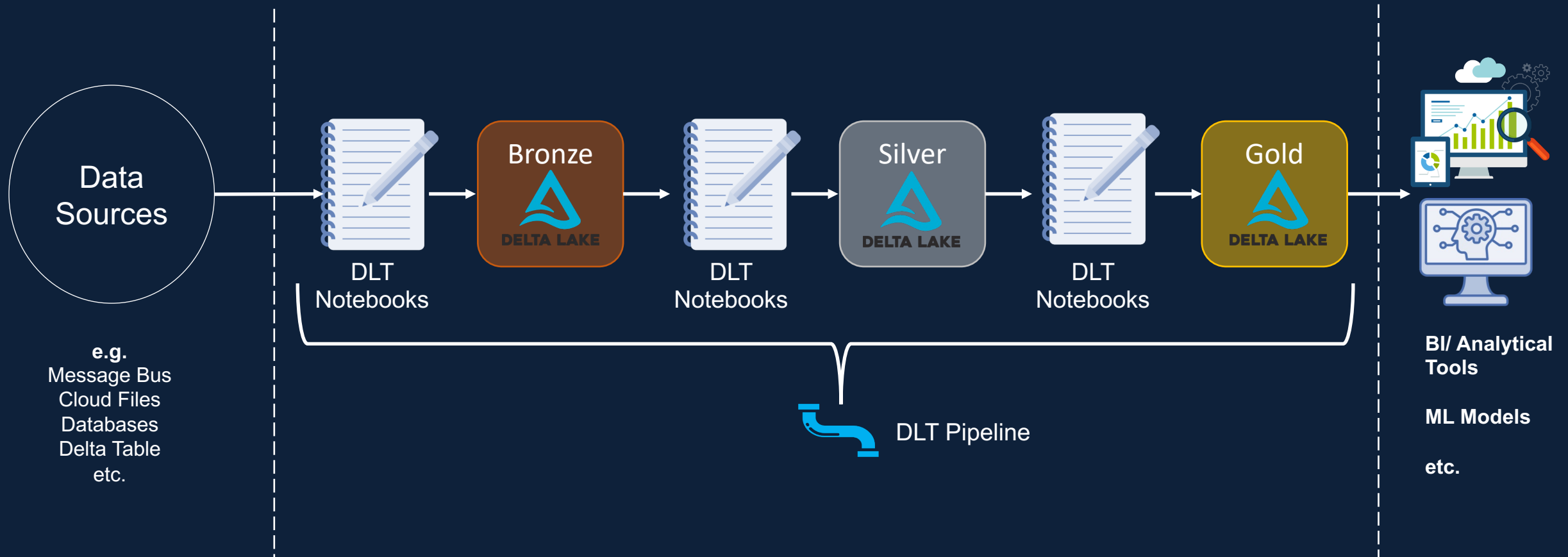
```
SELECT * FROM users WHERE age > 18;
```

No need to handle checkpoints, fault tolerance, dependencies etc

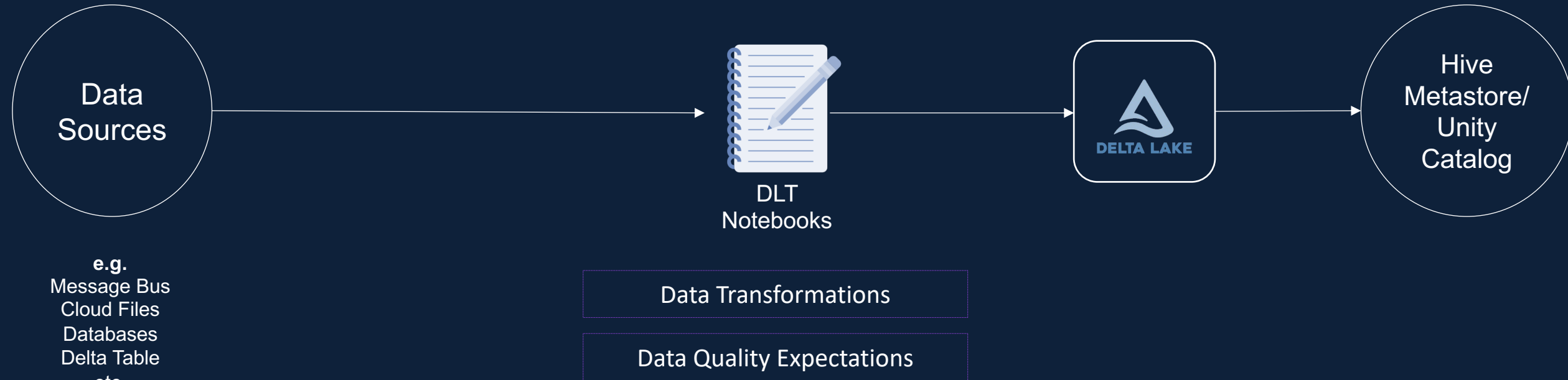
No need to write complex logic such as CDC, Schema Evolution etc.

Better Optimization

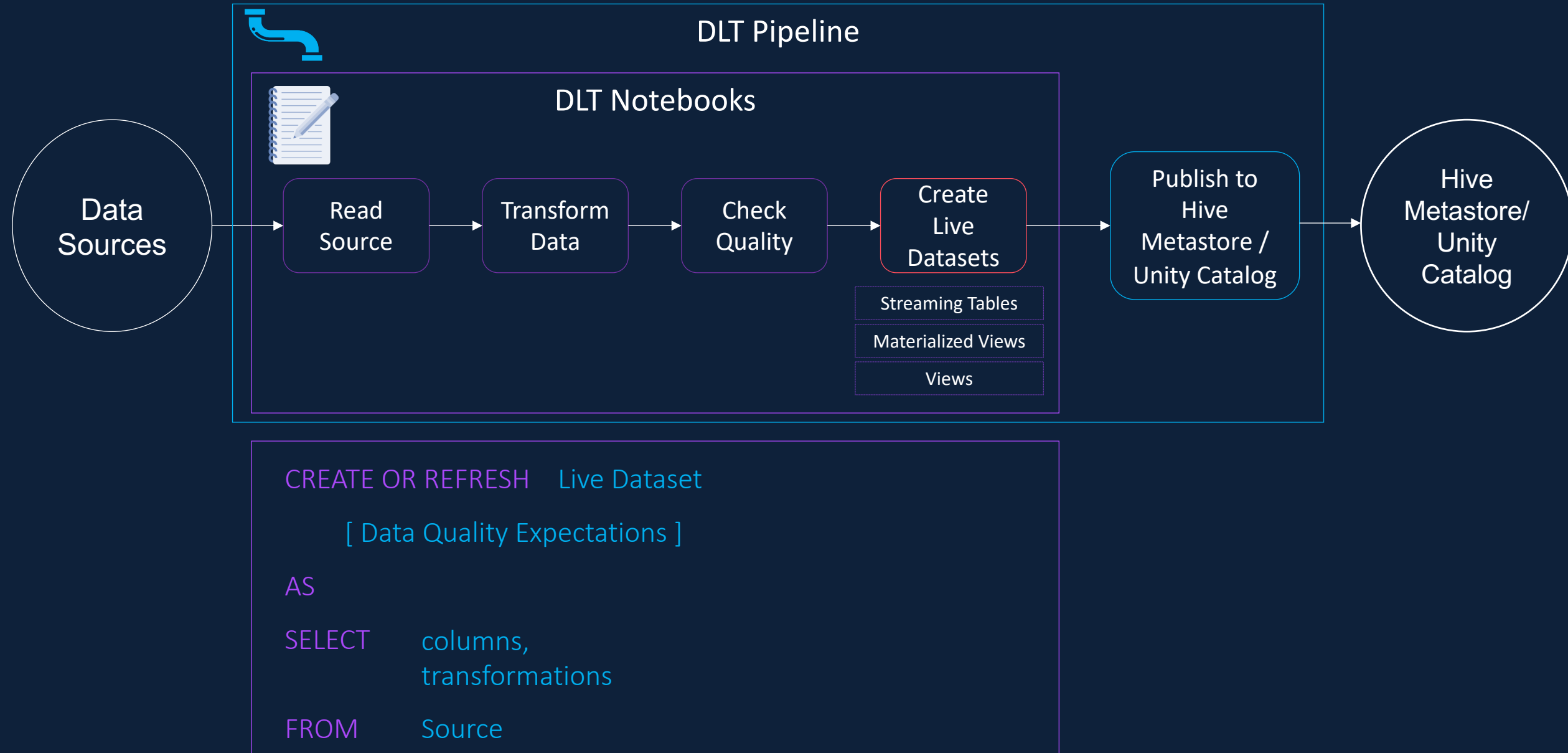
Lakeflow Spark Declarative Pipelines - Programming



Lakeflow Spark Declarative Pipelines - Programming



Lakeflow Spark Declarative Pipelines - Programming



Lakeflow Spark Declarative Pipelines - Programming



Streaming Tables

Delta Table to which streams write data

Reads from Eventhub, Kafka, Cloud Files, Delta Tables etc.

Offers exactly once guarantees

Incremental Data Ingestion Workloads

Low latency transformations

Massive amount of data

DML operations allowed



Materialized Views

Delta Table created from the result of a query

Full Refresh Data Ingestion Workloads

Build aggregate tables for reporting

Improve latency of BI reports

Data Transformation Workloads

No DML operations allowed - Change the query instead.



Views

No physical storage of the data

Scope is limited to the pipeline

Cannot be published to Hive Metastore/Unity Catalog

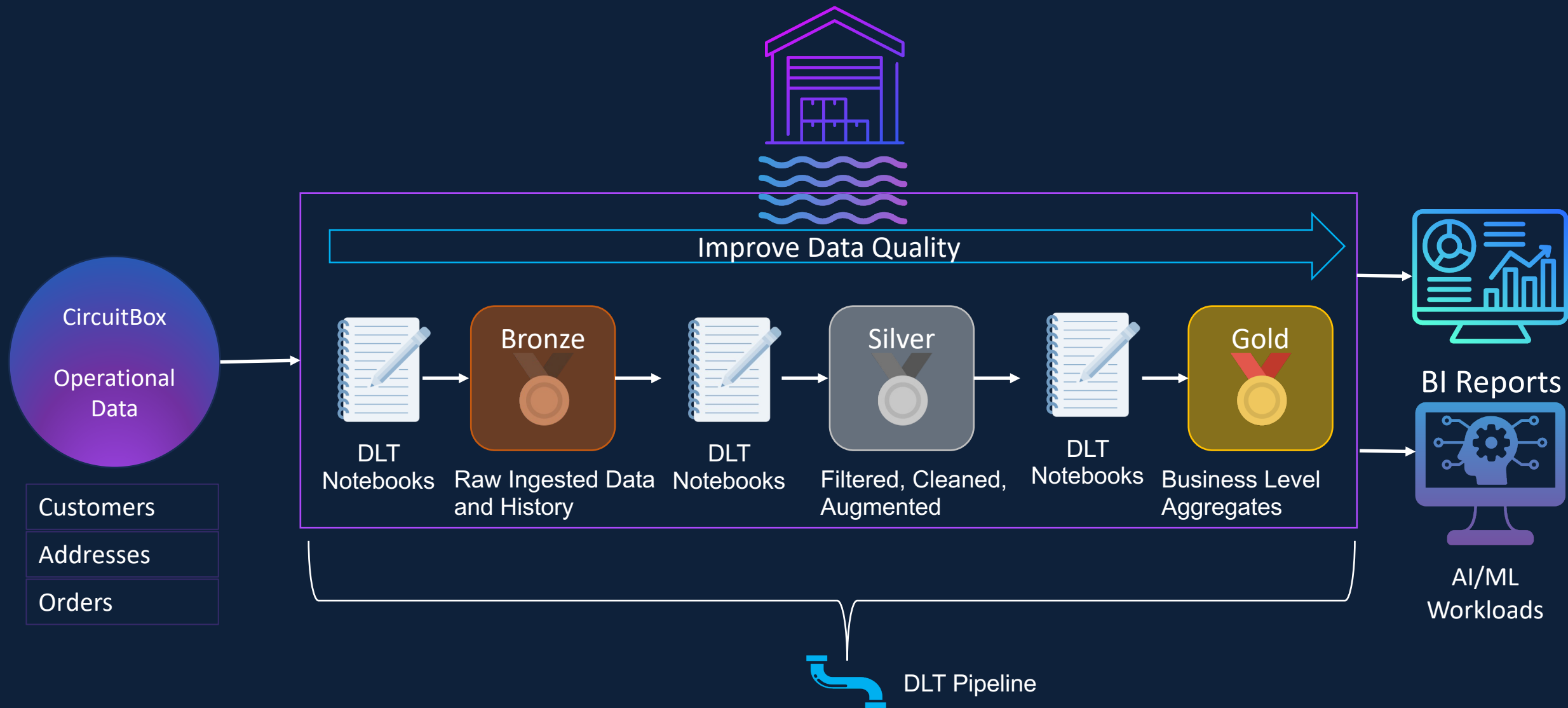
Store intermediate results to reduce complexity

To enforce data quality constraints

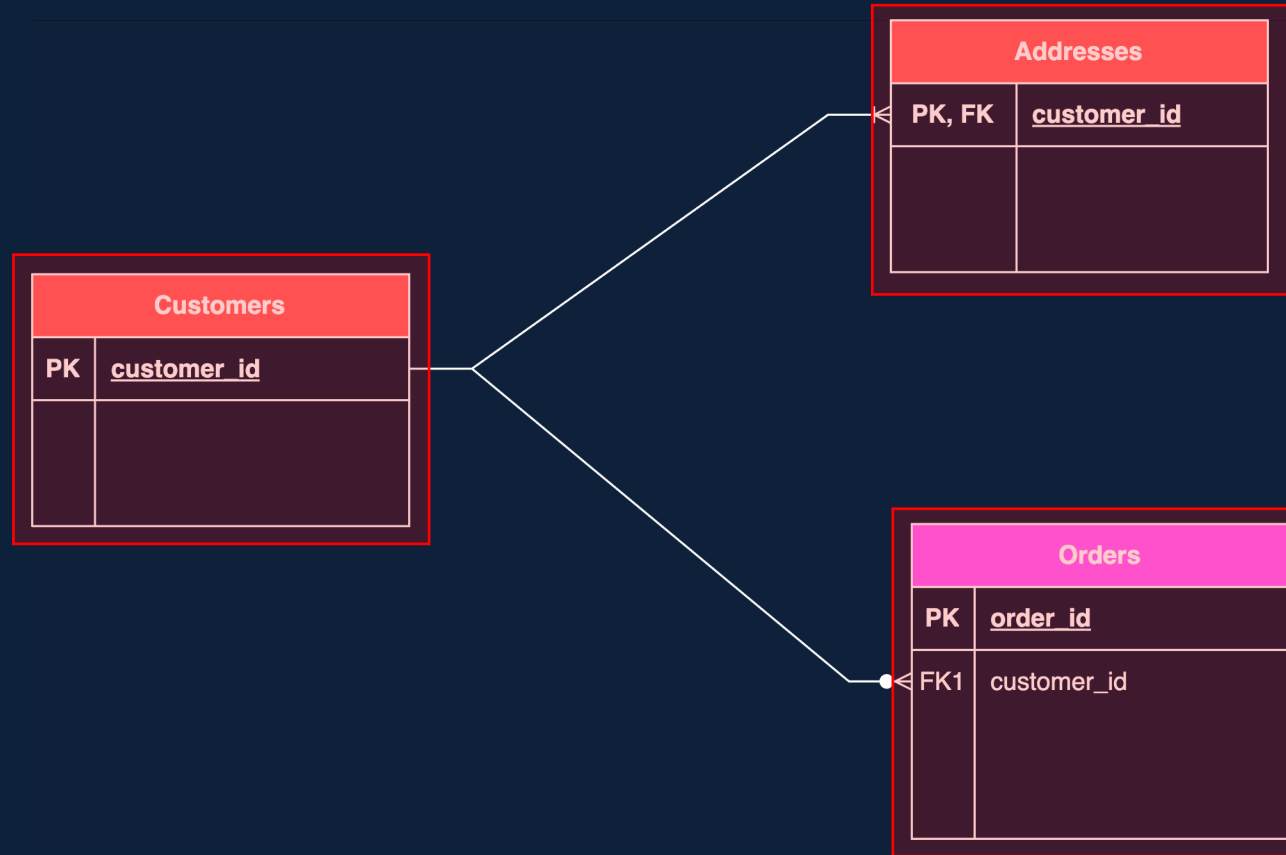
Delta Live Tables (DLT) Project - CircuitBox

Introduction

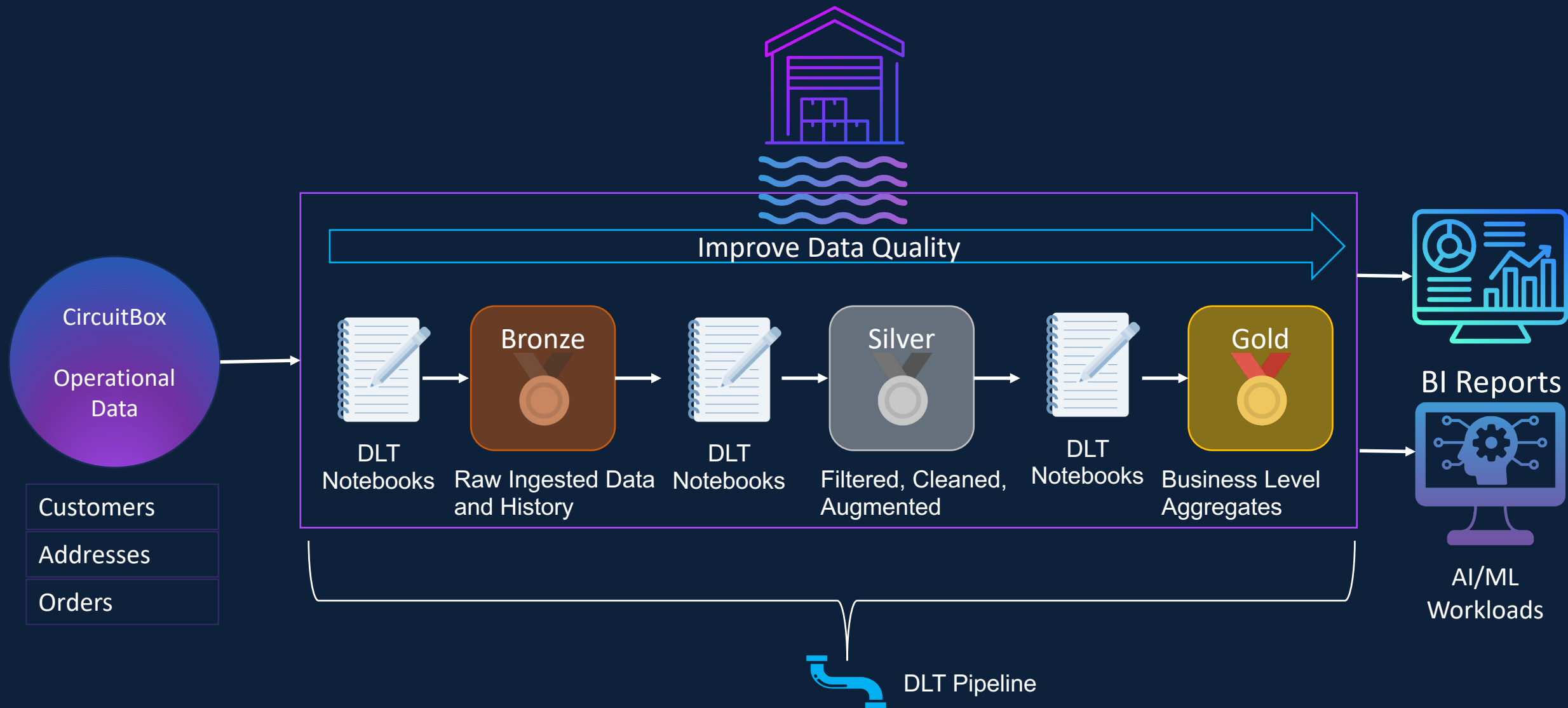
Delta Live Tables (DLT) Project - CircuitBox



Delta Live Tables (DLT) Project - CircuitBox



Delta Live Tables (DLT) Project - CircuitBox



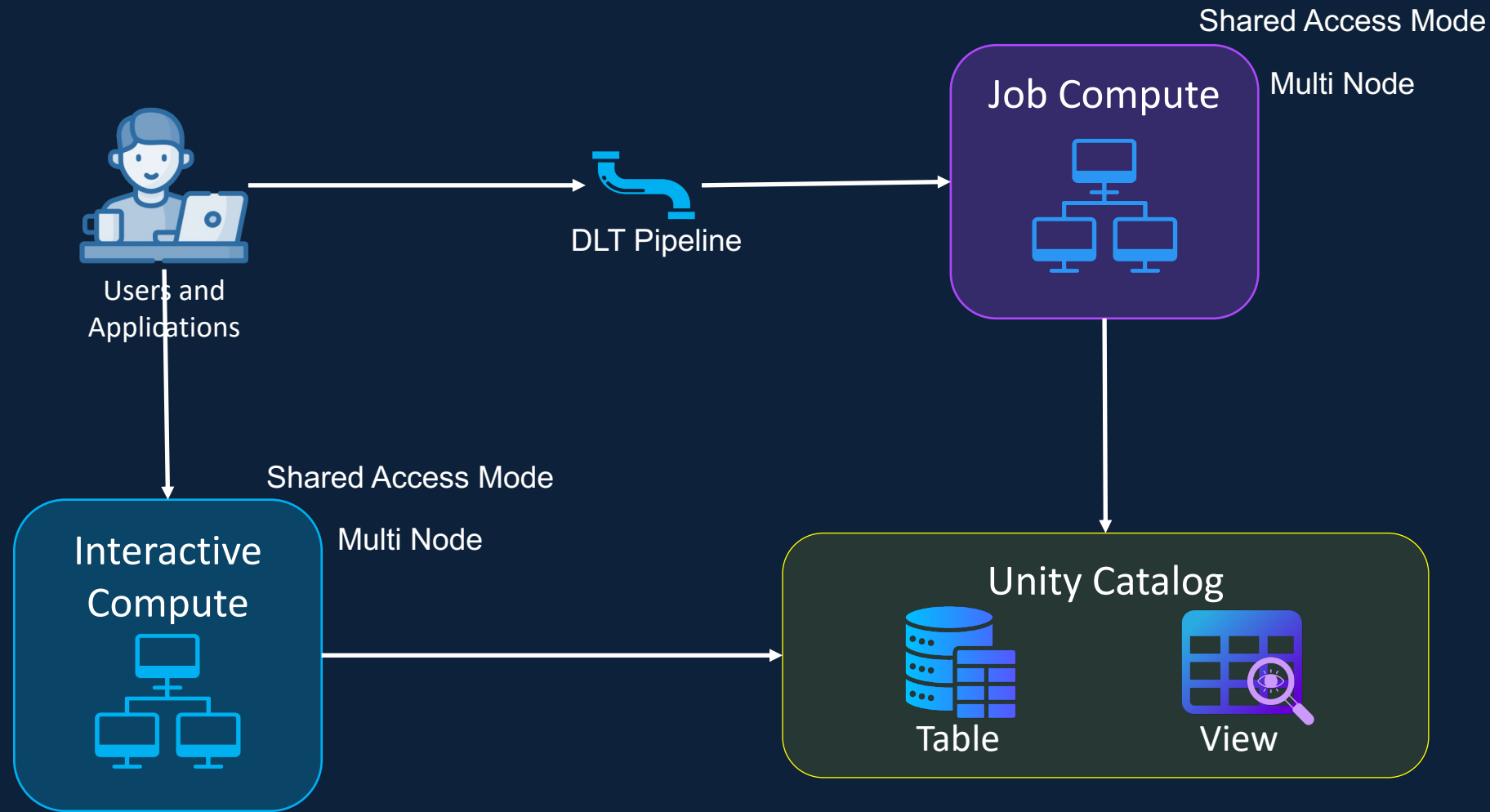
Delta Live Tables (DLT) Project - CircuitBox



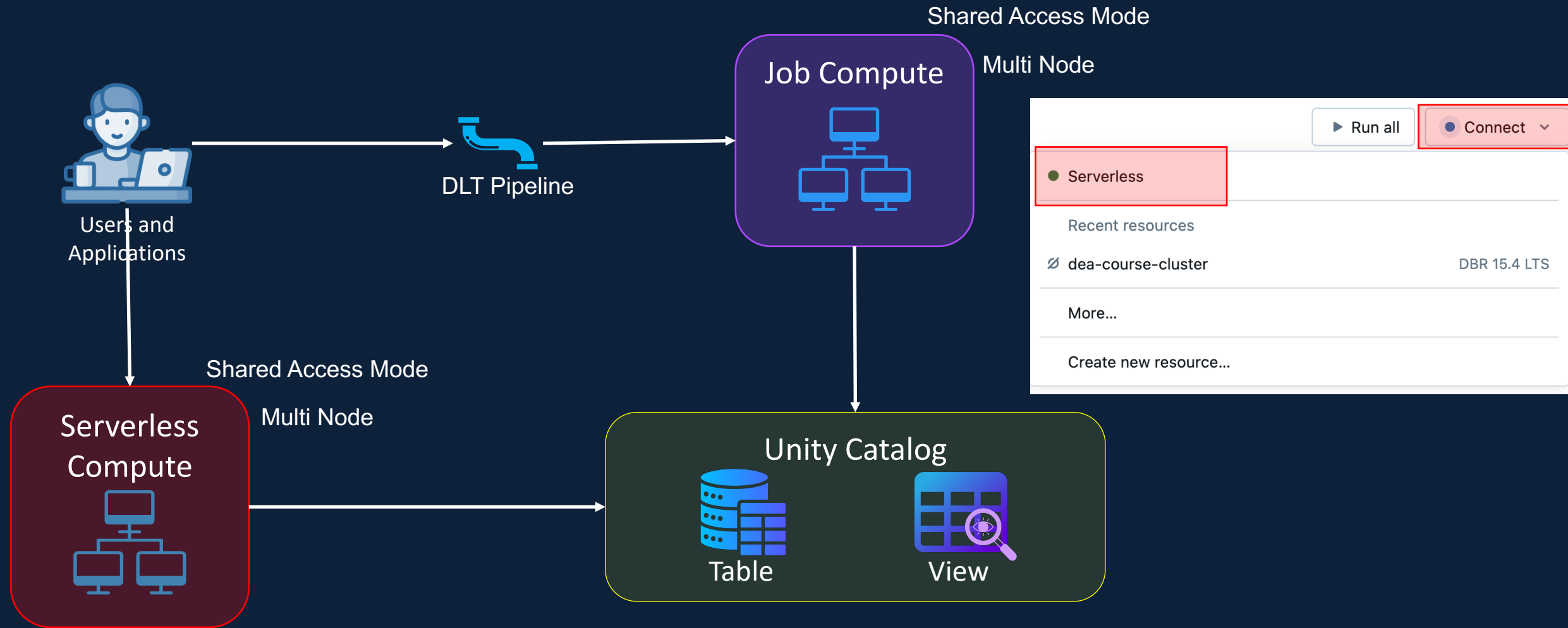
Compute/ Cluster Configuration

DLT Project

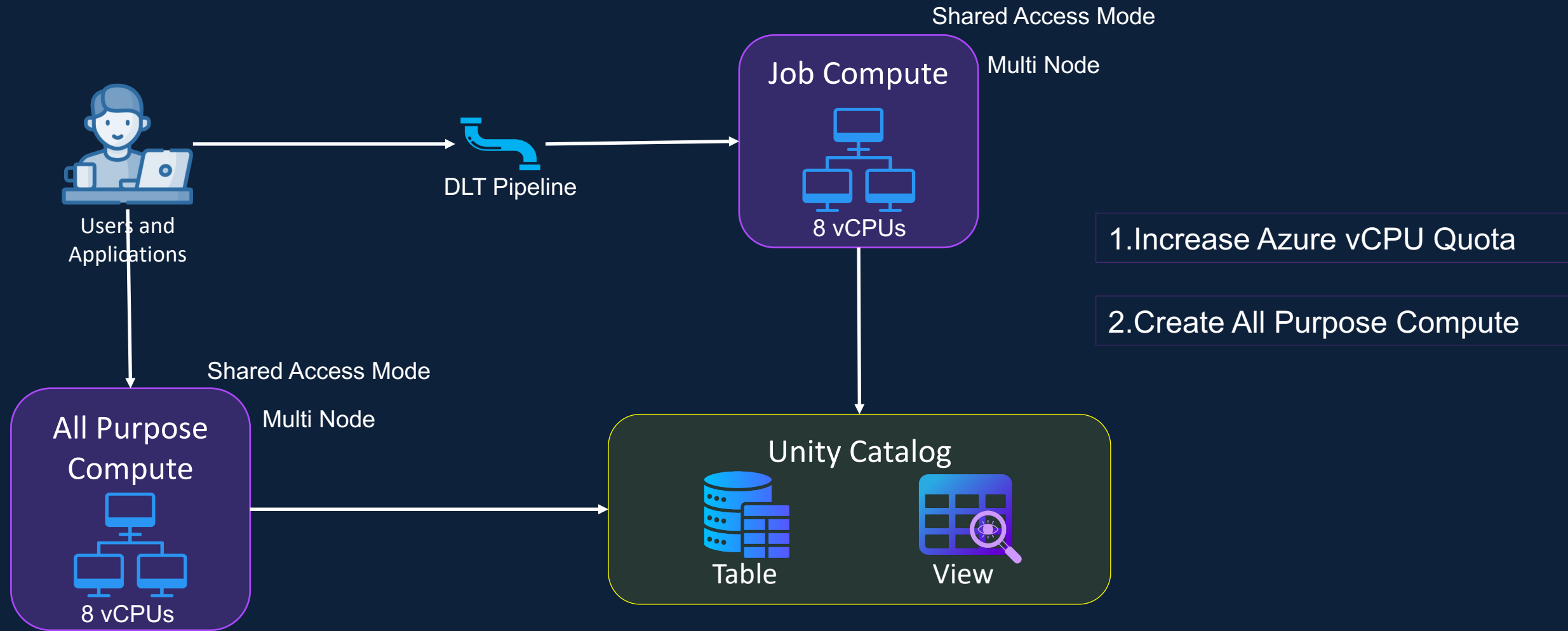
Compute/ Cluster Configuration



Compute/ Cluster Configuration

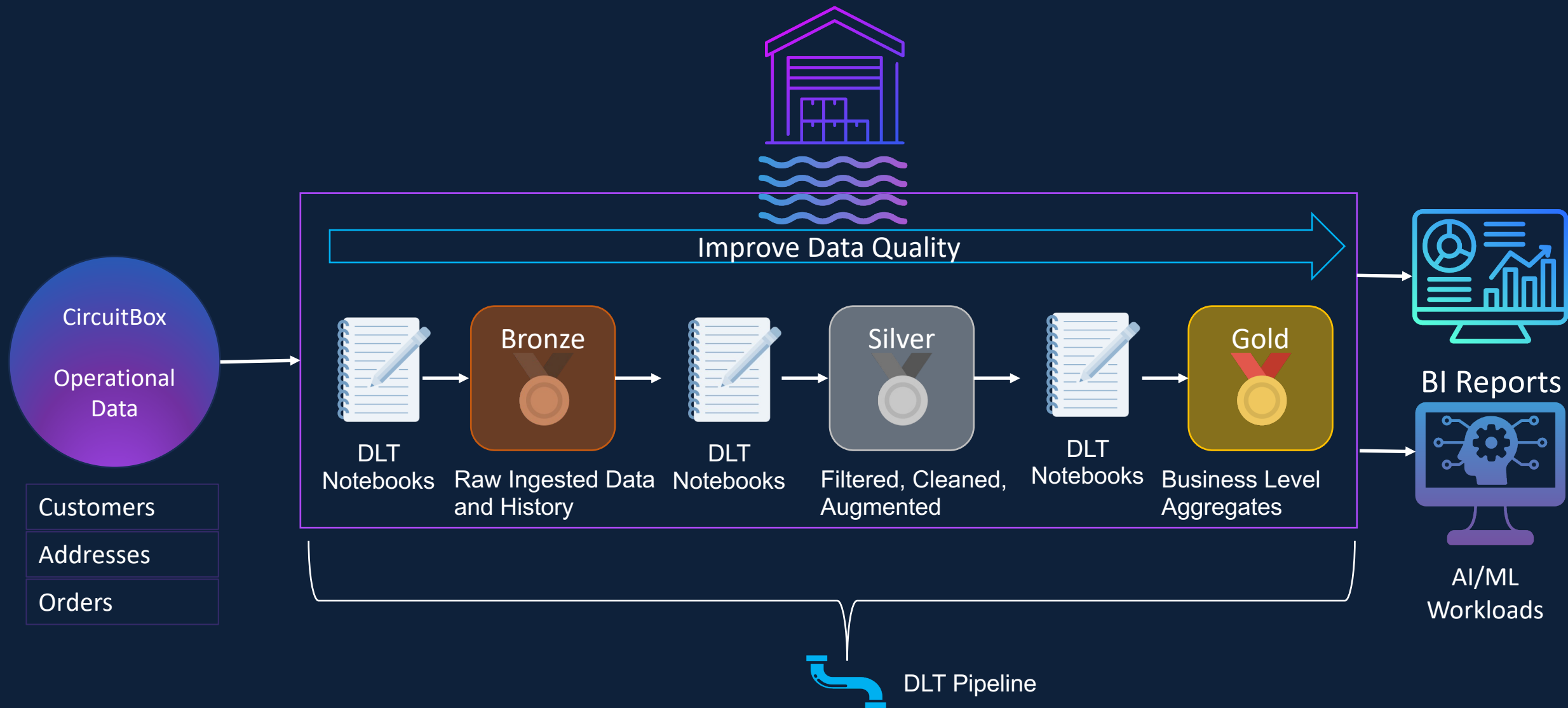


Compute/ Cluster Configuration

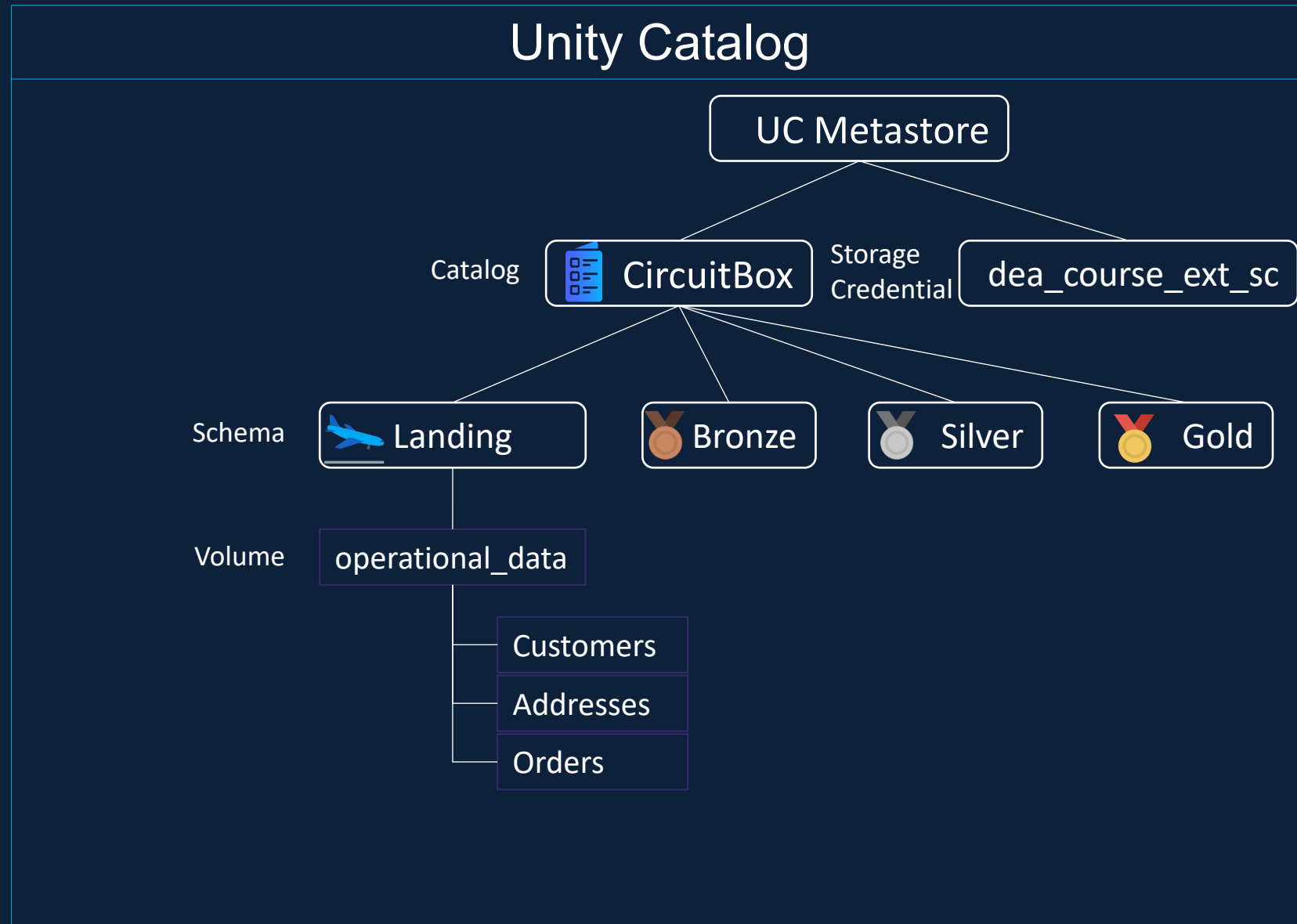


Set-up Project Environment

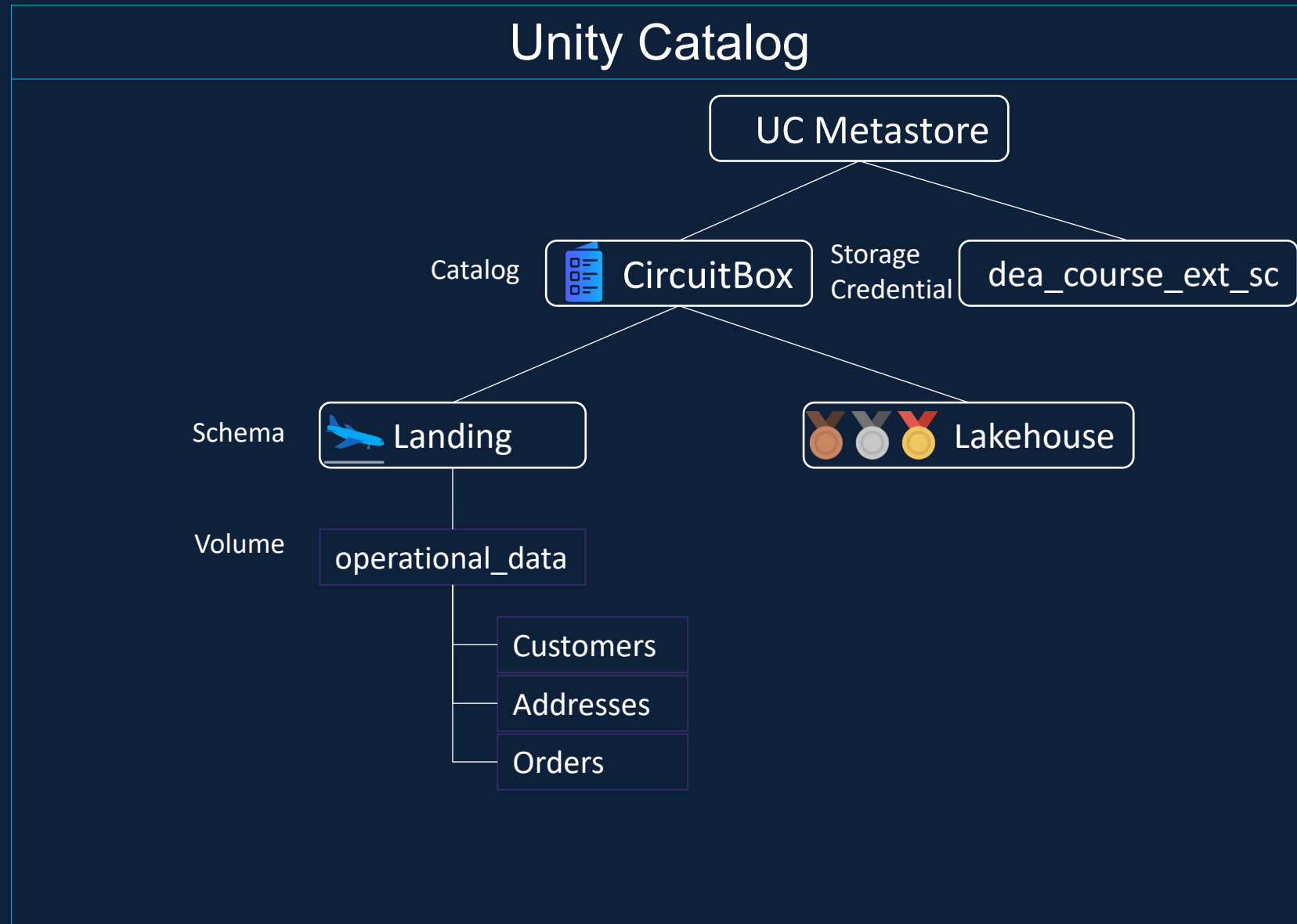
Delta Live Tables (DLT) Project - CircuitBox



Data Architecture - CircuitBox

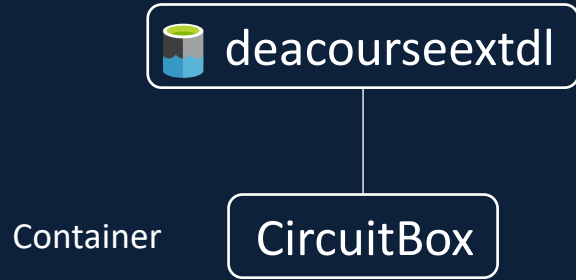


Data Architecture - CircuitBox

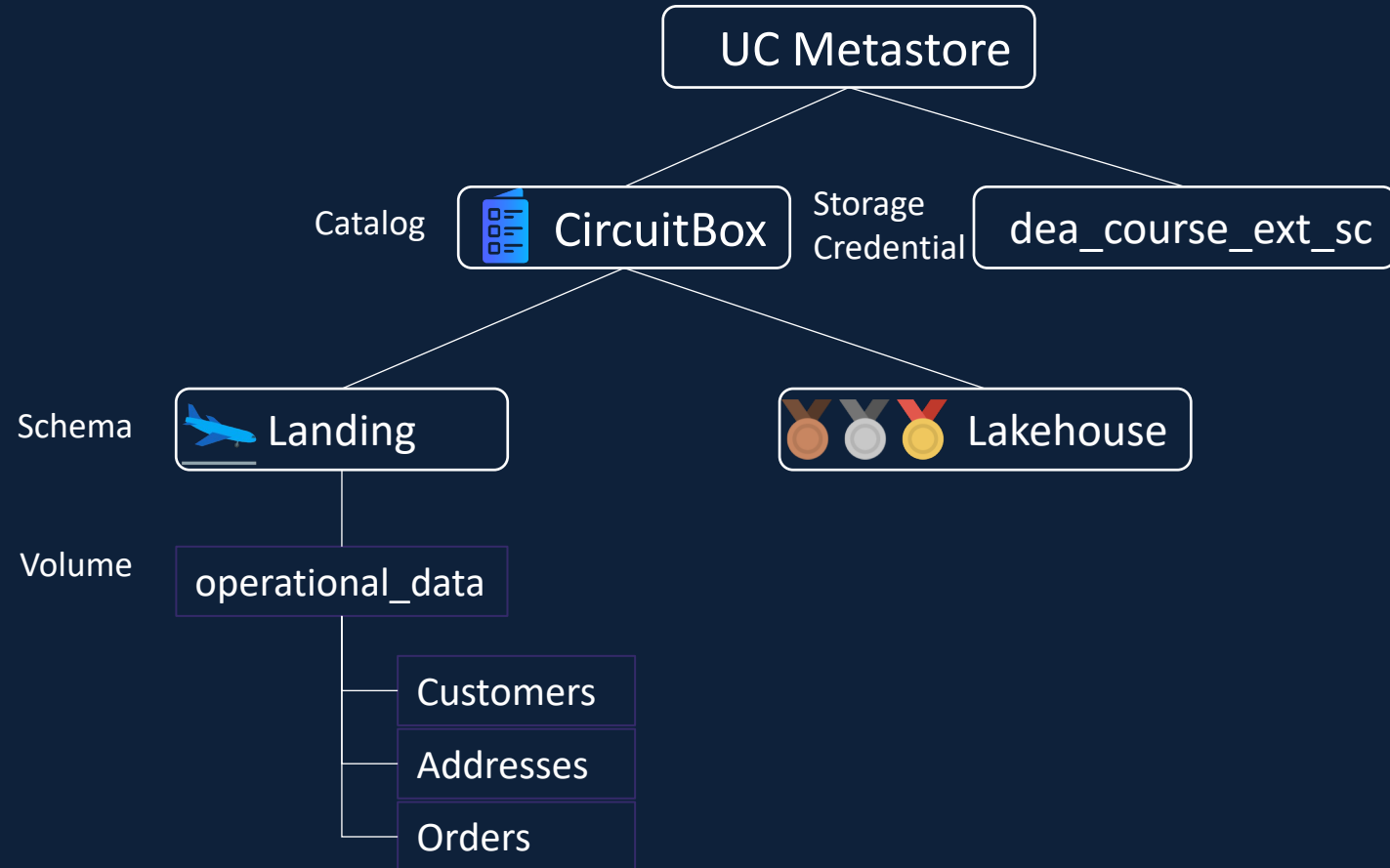


Data Architecture - CircuitBox

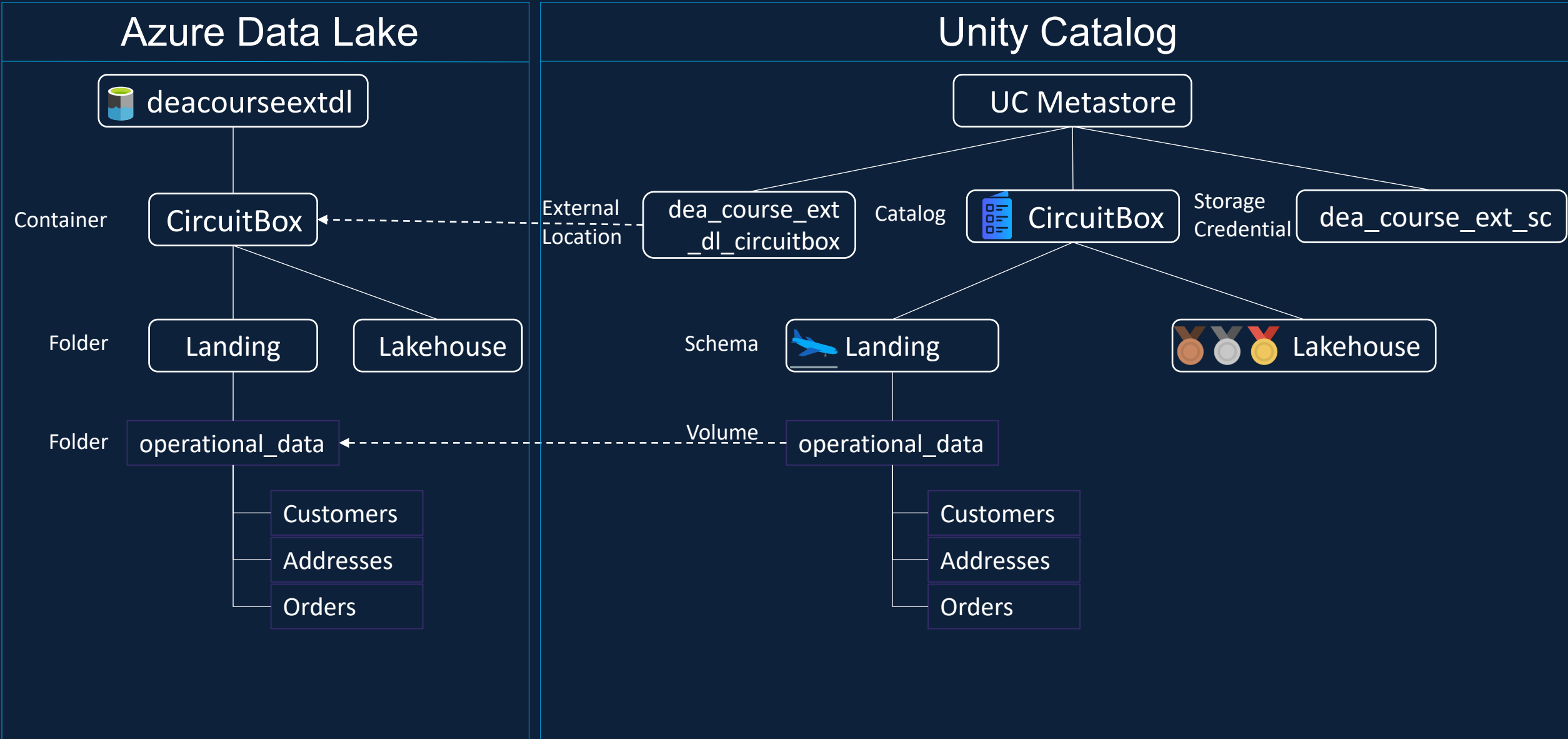
Azure Data Lake



Unity Catalog

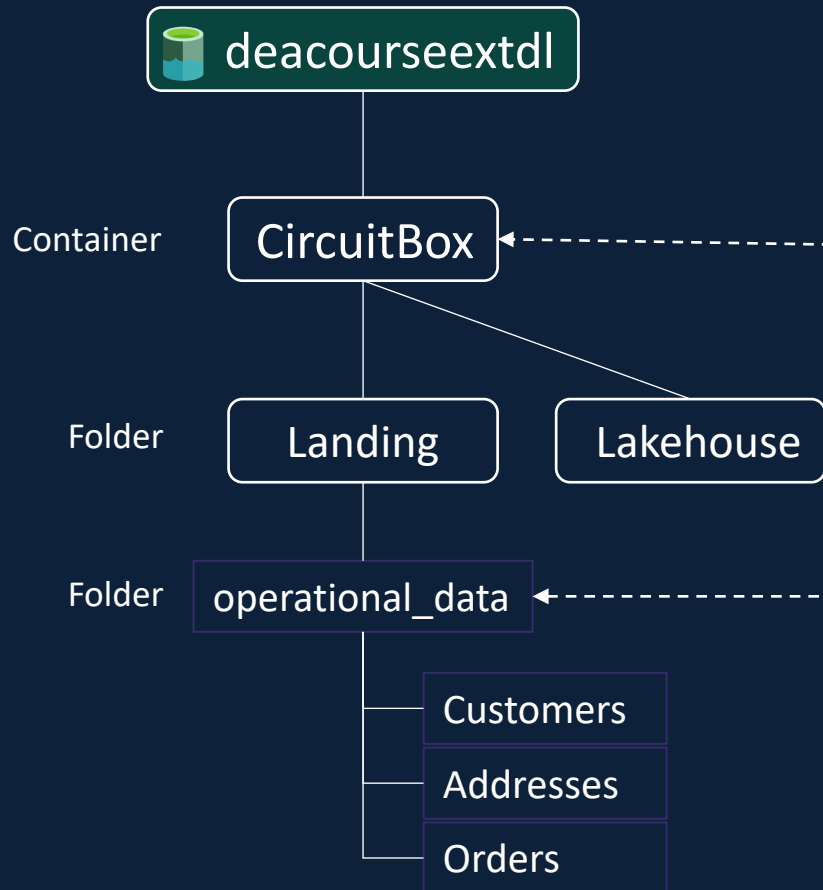


Data Architecture - CircuitBox

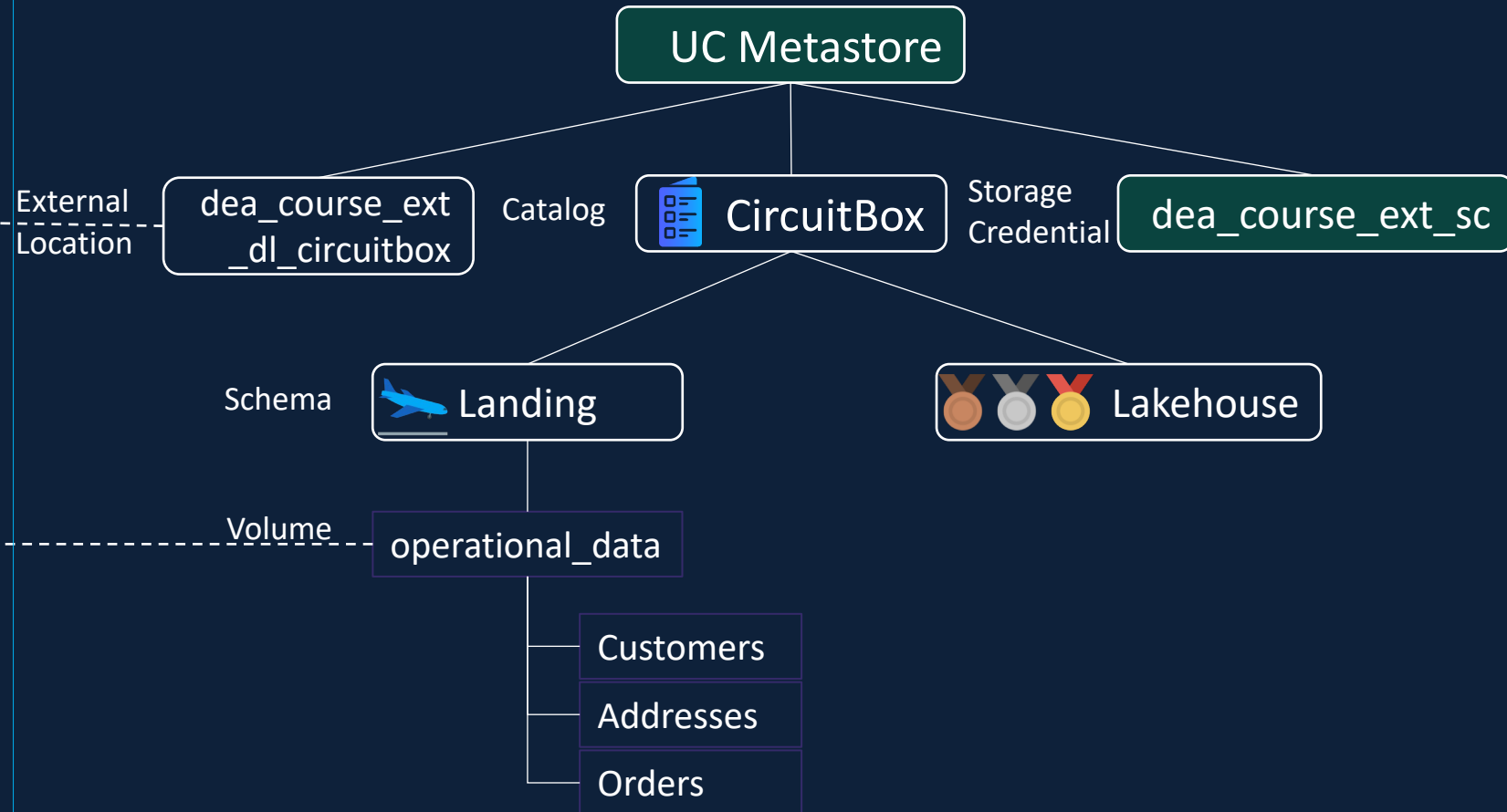


Data Architecture - CircuitBox

Azure Data Lake



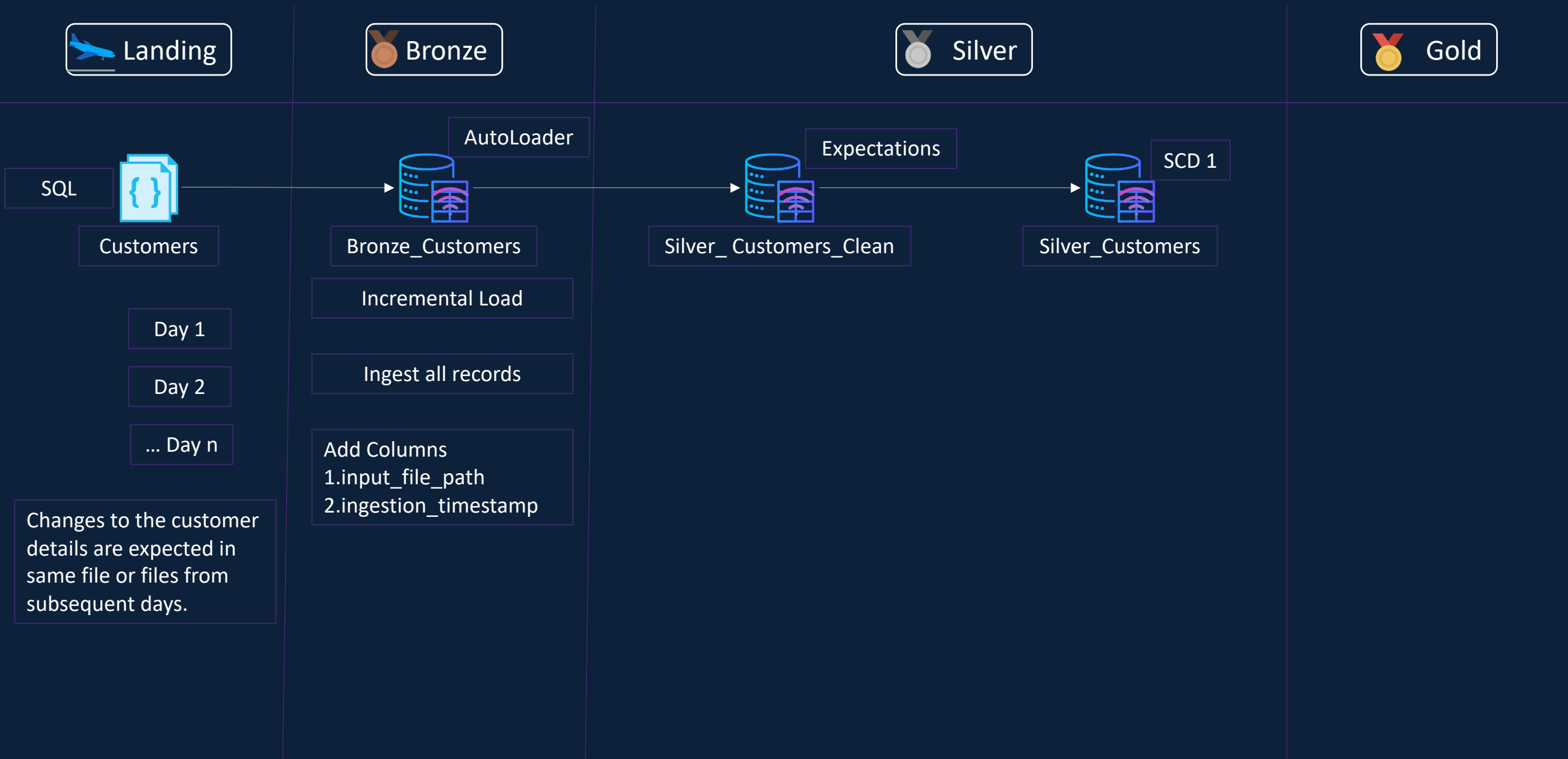
Unity Catalog



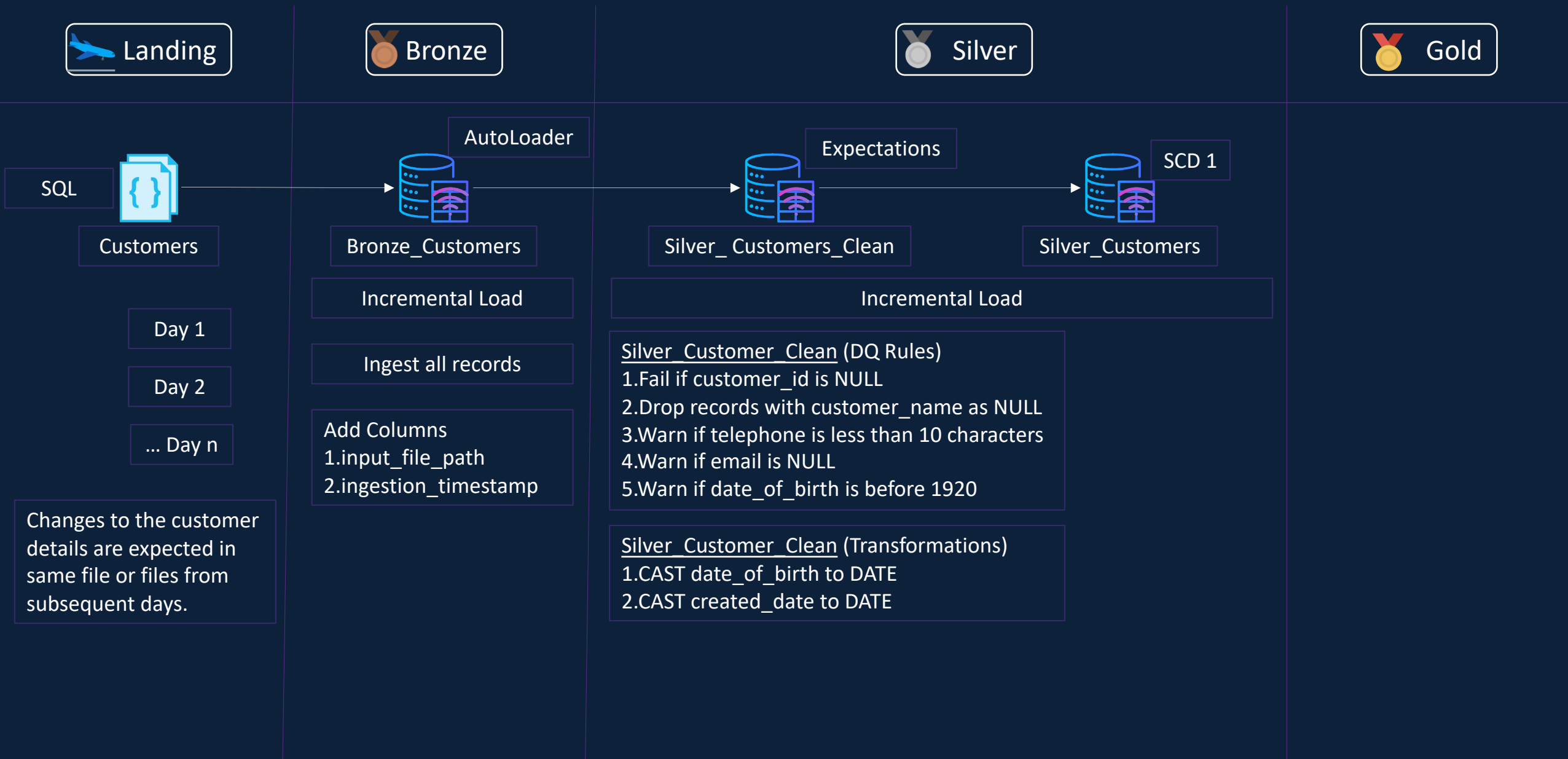
Delta Live Tables (DLT) Project - CircuitBox



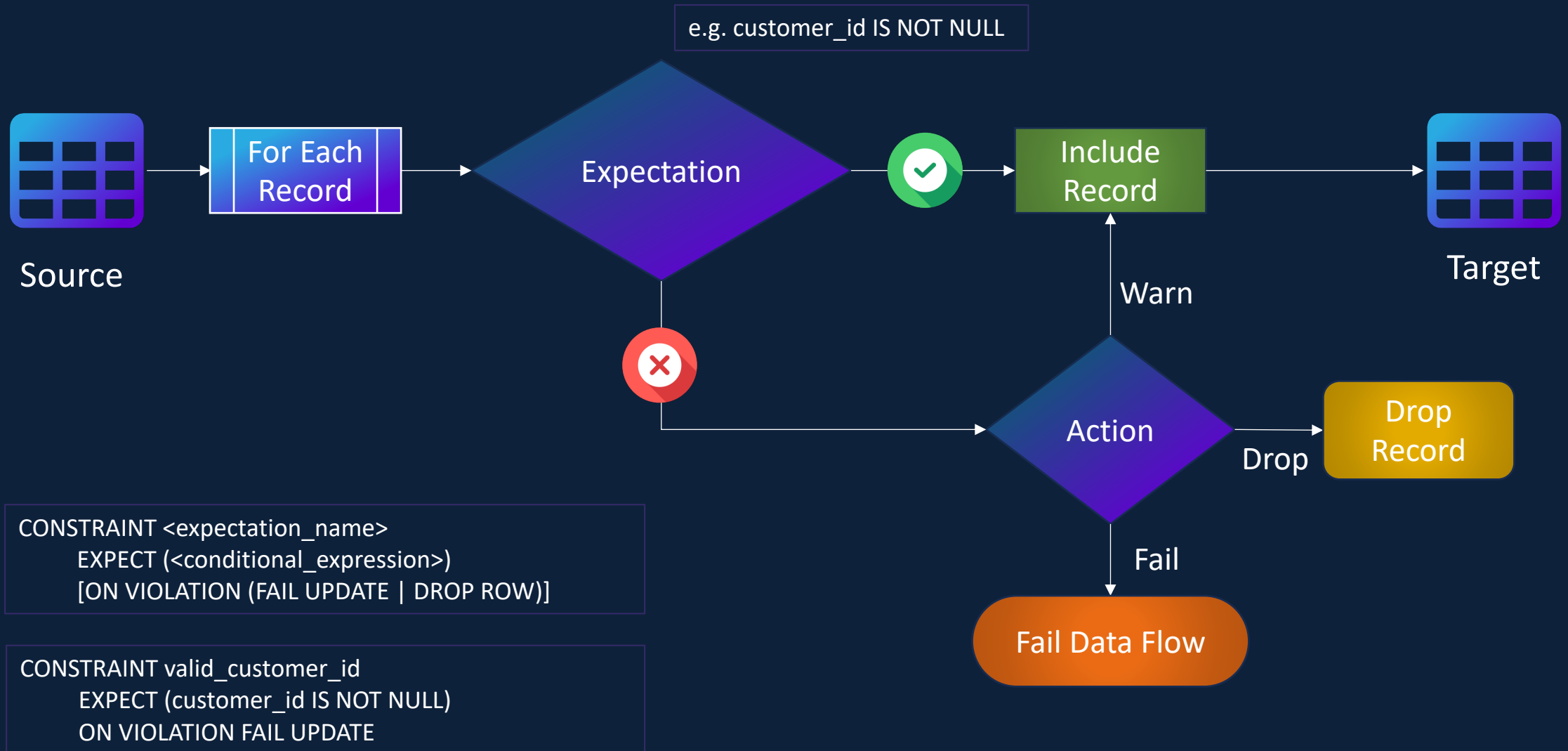
CircuitBox – Process Customers Data



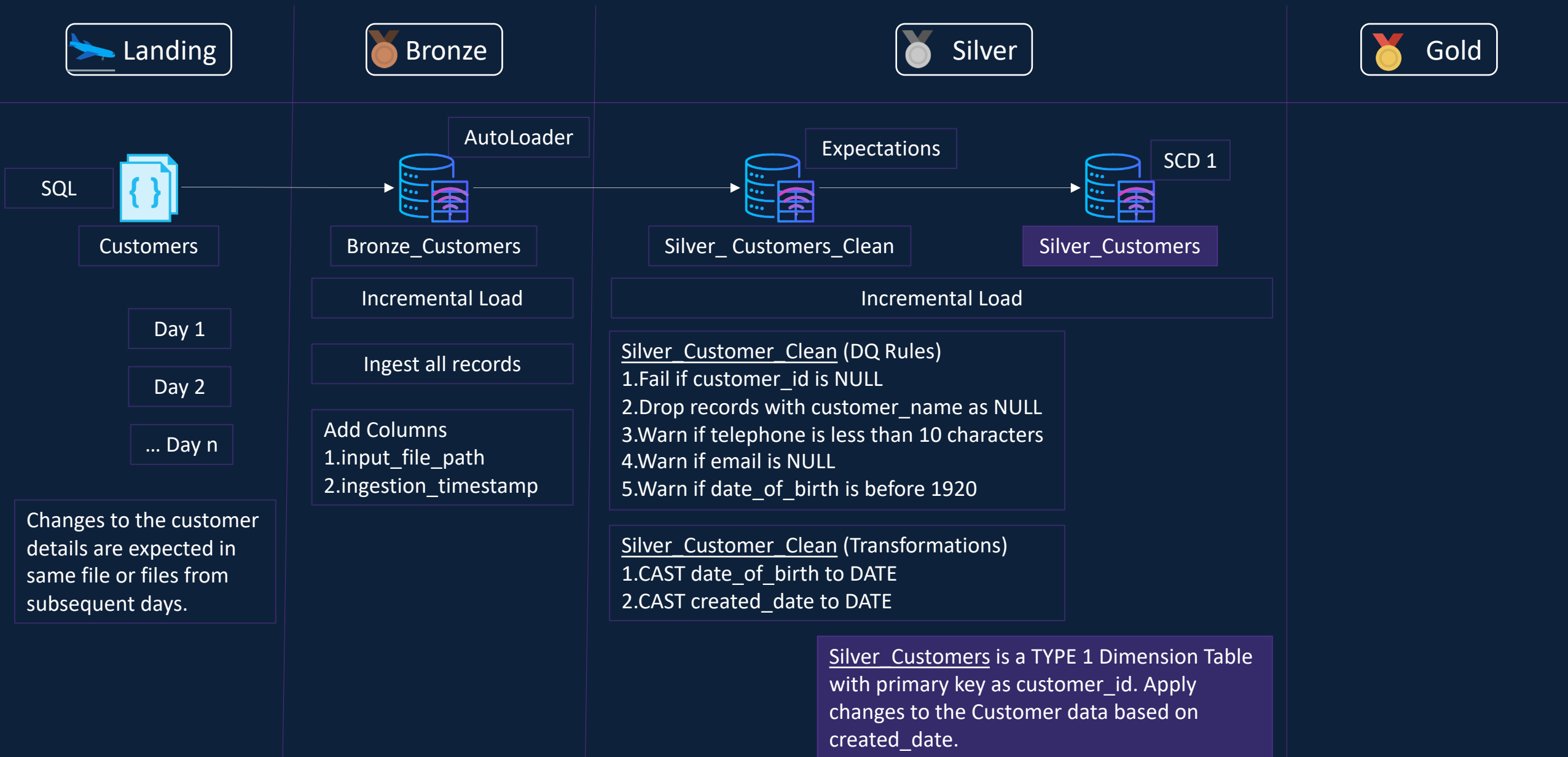
CircuitBox – Process Customers Data



Delta Live Tables (DLT) - Expectations

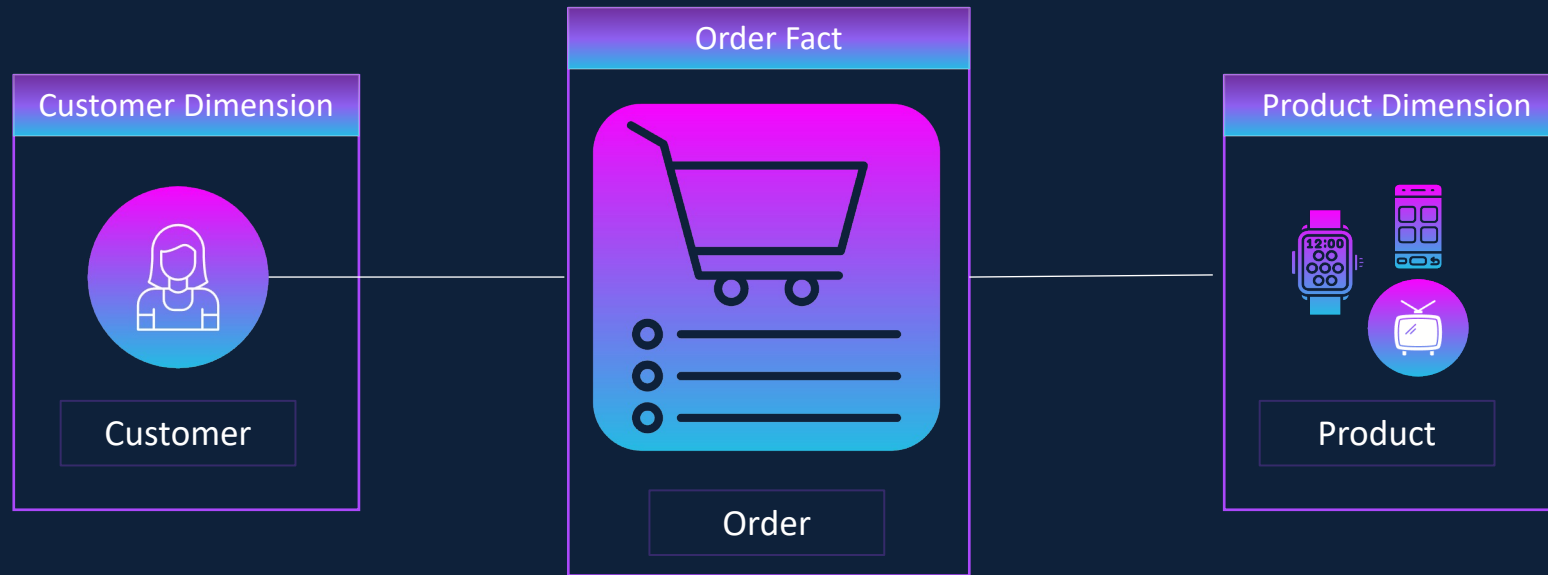


CircuitBox – Process Customers Data



Slowly Changing Dimensions

Slowly Changing Dimensions

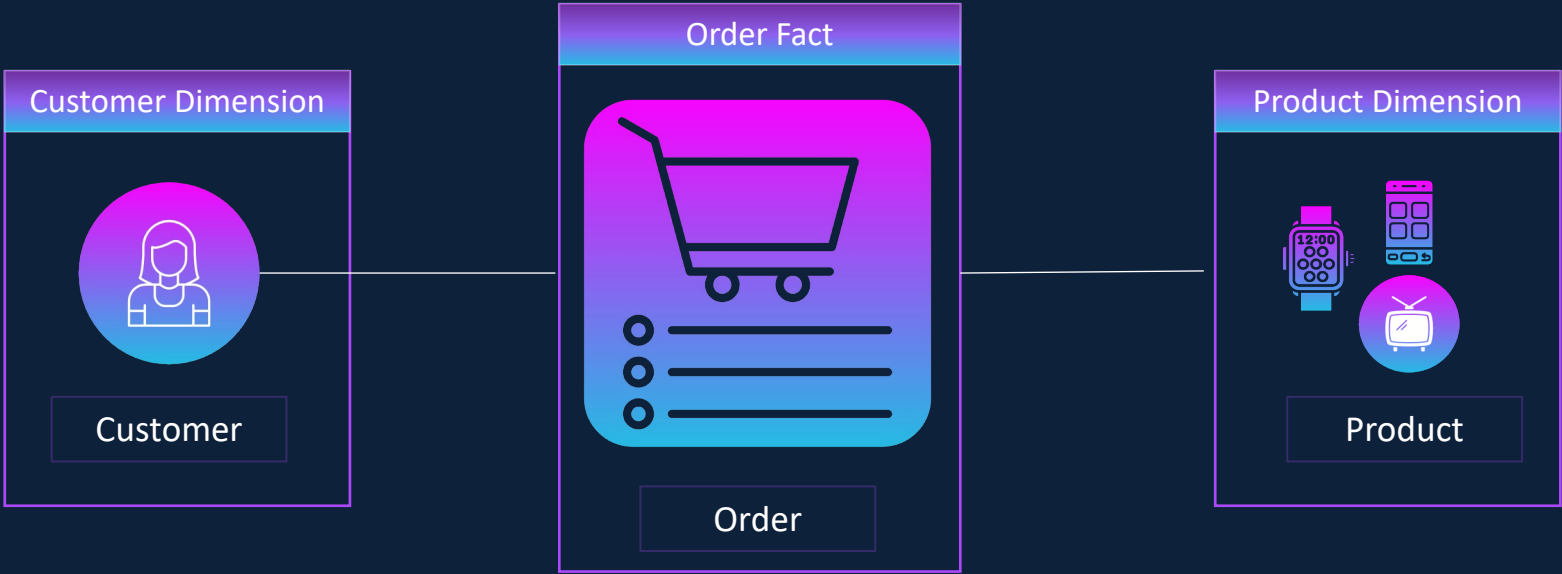


SCD Type 1 – Overwrite the data (No History Maintained)

Before	customer_id	Customer_name	telephone
	1001	Diana Miller	+1 212 555 1234

After	customer_id	Customer_name	telephone
	1001	Diana Miller	+65 6123 4567

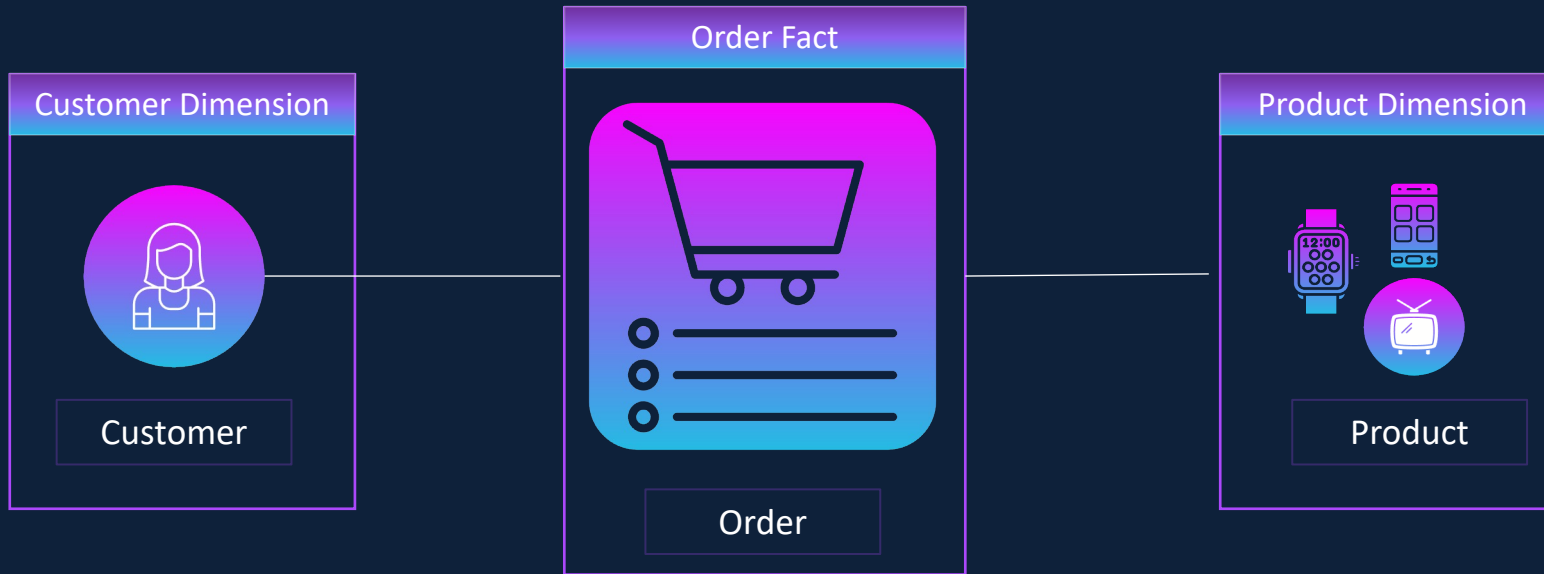
Slowly Changing Dimensions



SCD Type 2 – Full History Maintained

Before	customer_id	Customer_name	telephone	start_date	end_date
	1001	Diana Miller	+1 212 555 1234	2021-01-01	
After	customer_id	Customer_name	telephone	start_date	end_date
	1001	Diana Miller	+1 212 555 1234	2021-01-01	2025-01-26
	1001	Diana Miller	+65 6123 4567	2025-01-26	

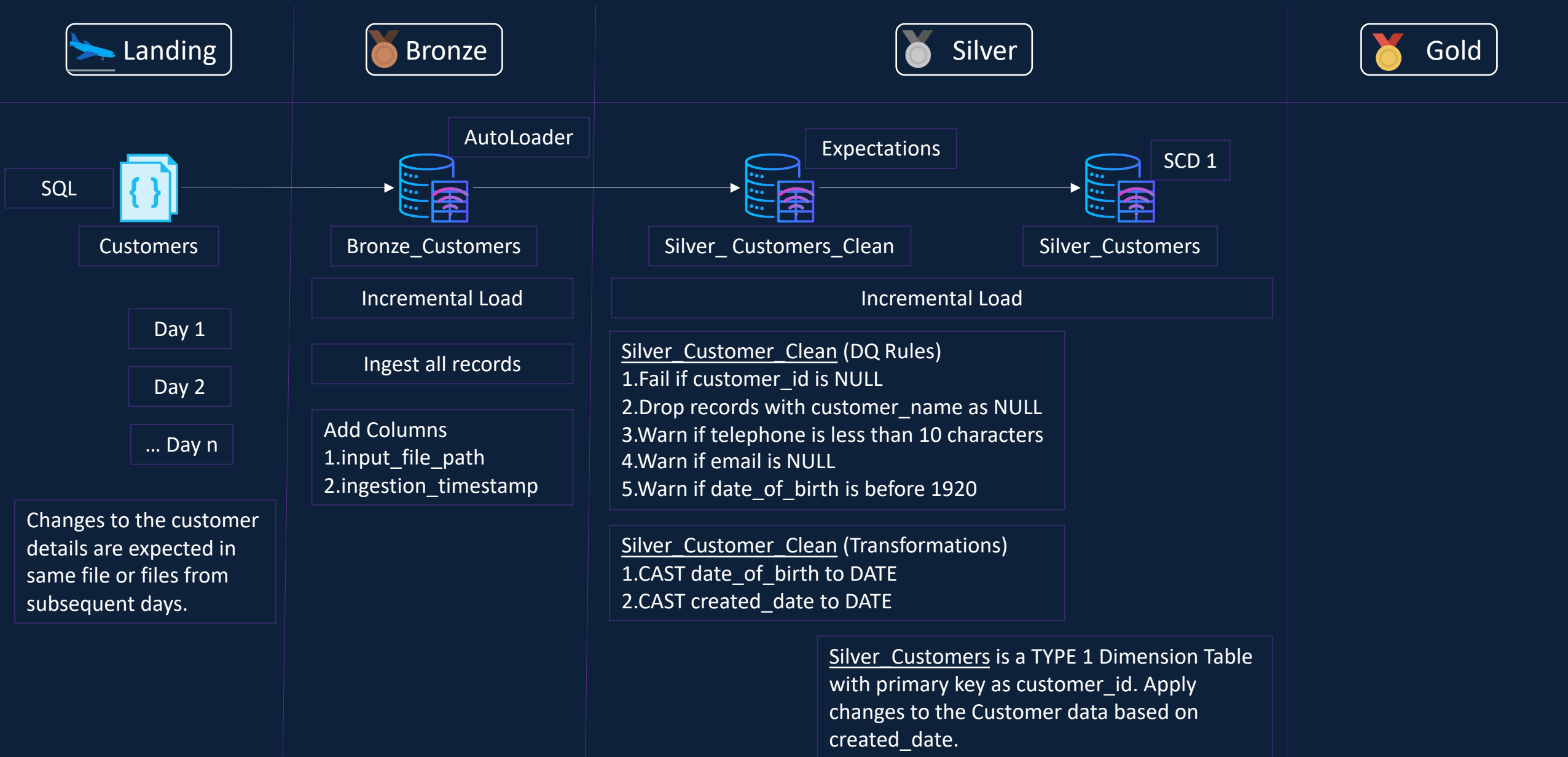
Slowly Changing Dimensions



SCD Type 3 – Only the most recent change is tracked

Before	customer_id	Customer_name	telephone	prev_telephone
	1001	Diana Miller	+1 212 555 1234	
After	customer_id	Customer_name	telephone	prev_telephone
	1001	Diana Miller	+65 6123 4567	+1 212 555 1234

CircuitBox – Process Customers Data



APPLY CHANGES API

```
CREATE OR REFRESH STREAMING TABLE <target_table>;
```

```
APPLY CHANGES INTO <target_table>  
  FROM <source_table>  
  KEYS <columns>  
  SEQUENCE BY <columns>  
  STORED AS <SCD TYPE 1 | SCD TYPE 2>
```



```
CREATE OR REFRESH STREAMING TABLE silver_customers;
```

```
APPLY CHANGES INTO LIVE.silver_customers  
  FROM STREAM(LIVE.silver_customers_clean)  
  KEYS (customer_id)  
  SEQUENCE BY created_date  
  STORED AS SCD TYPE 1;
```

APPLY CHANGES API

APPLY CHANGES INTO LIVE.table_name

FROM source

KEYS (keys)

[IGNORE NULL UPDATES]

[APPLY AS DELETE WHEN condition]

[APPLY AS TRUNCATE WHEN condition]

SEQUENCE BY orderByColumn

[COLUMNS {columnList | * EXCEPT (exceptColumnList)}]

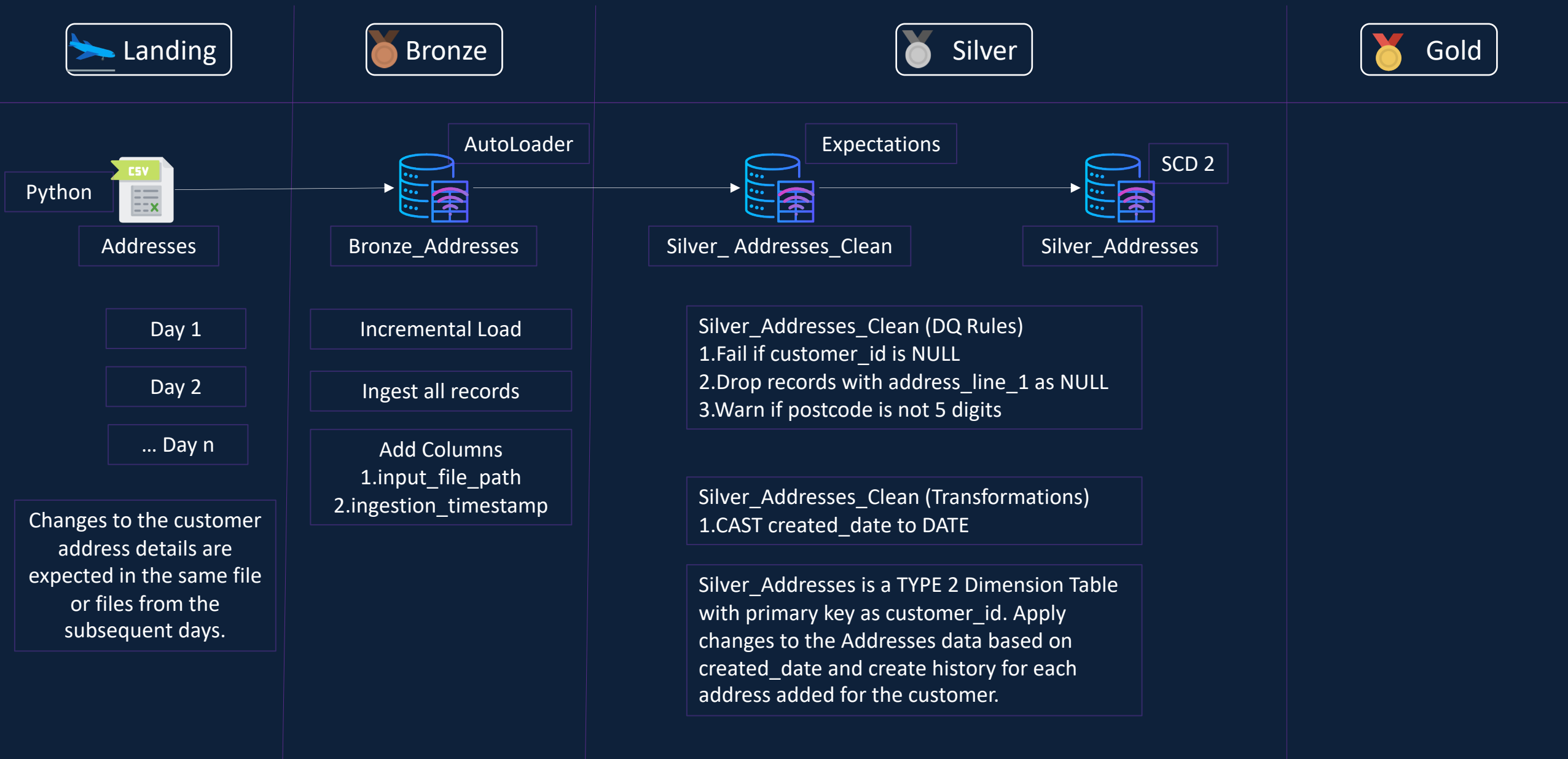
[STORED AS {SCD TYPE 1 | SCD TYPE 2}]

[TRACK HISTORY ON {columnList | * EXCEPT (exceptColumnList)}]

Delta Live Tables (DLT) Project - CircuitBox



CircuitBox – Process Addresses Data



DLT Programming using Python

PySpark

Read Source Data

```
input_df = spark.read....
```

```
input_df = spark.readStream...
```

Apply Transformations

```
transformed_df = input_df.transform()...
```

Write Transformed Data

```
transformed_df.write...
```

Delta Live Tables (DLT)

```
Import dlt
```

```
@dlt.table ()
```

```
def <function_name>():
```

```
    Read Source Data
```

```
    Apply Transformations
```

```
    return <dataframe>
```

DLT Programming using Python

Delta Live Tables (DLT)

```
Import dlt
```

```
@dlt.table ()
```

```
def <function_name>():  
    Read Source Data  
    Apply Transformations  
    return <dataframe>
```

Example

```
Import dlt
```

```
@dlt.table ()
```

```
def bronze_addresses():  
    return(  
        spark.readStream  
            .format("cloudFiles")  
            .option("cloudFiles.format", "csv")  
            .option("cloudFiles.inferColumnTypes", "true")  
            .load("/<volume_path>/addresses/")  
    )
```

DLT Programming using Python

Delta Live Tables (DLT)

```
Import dlt
```

```
@dlt.table ()
```

```
def <function_name>():  
    Read Source Data  
    Apply Transformations  
    return <dataframe>
```

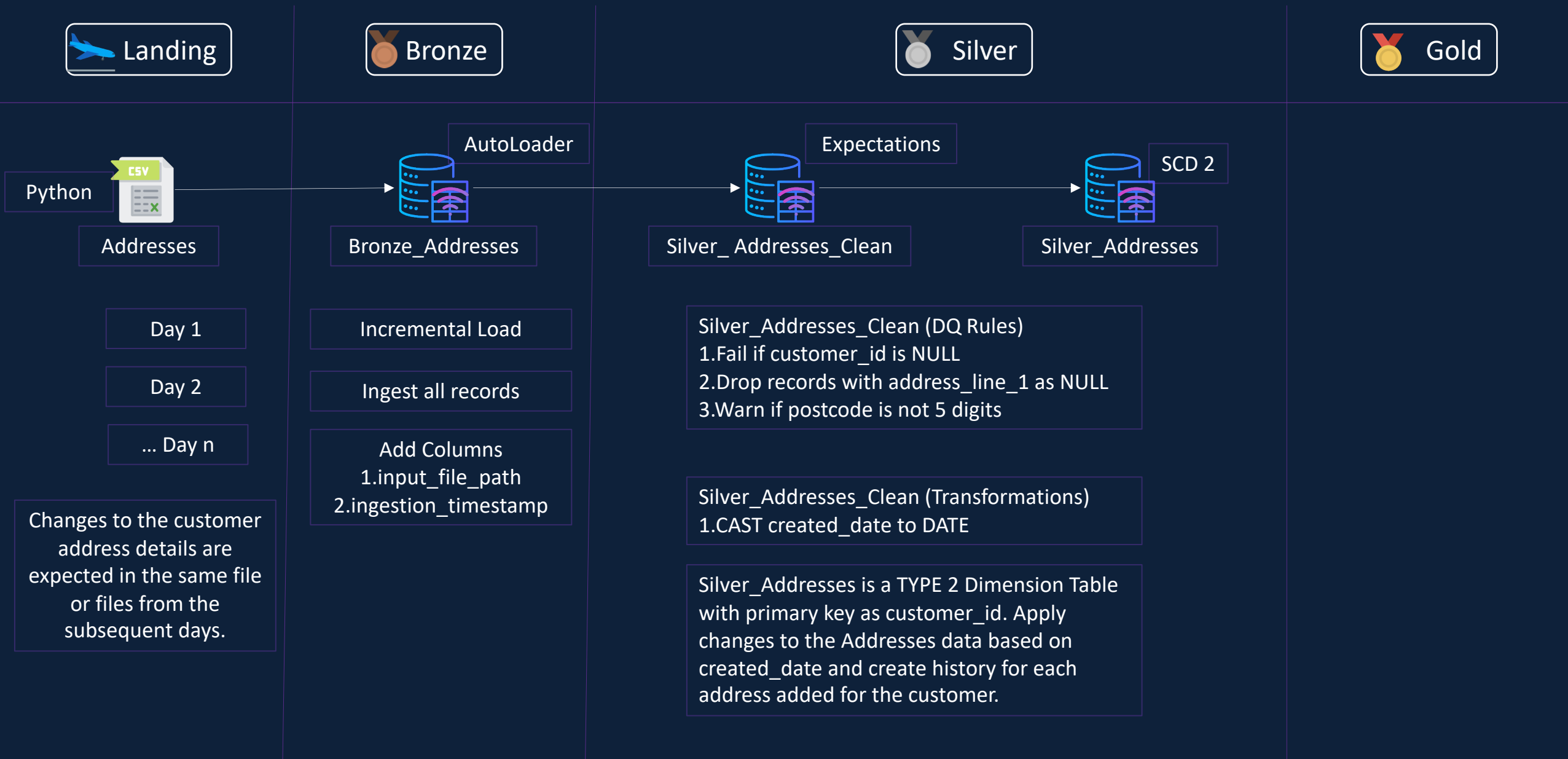
Example

```
Import dlt
```

```
@dlt.table (  
    name = 'bronze_addresses',  
    comment = 'raw addresses data',  
    table_properties = {'quality' : 'bronze'}  
)
```

```
def bronze_addresses():  
    return(  
        spark.readStream  
            .format("cloudFiles")  
            .option("cloudFiles.format", "csv")  
            .option("cloudFiles.inferColumnTypes", "true")  
            .load("/<volume_path>/addresses/")  
    )
```

CircuitBox – Process Addresses Data



DLT Programming using Python



Stream Read

```
dlt.read_stream()
```

```
spark.readStream.table()
```

Static Read

```
dlt.read ()
```

```
spark.read.table()
```

DLT Programming using Python



Stream Read

```
spark.readStream.table()
```

Static Read

```
spark.read.table()
```

Example

```
Import dlt
```

```
@dlt.table ()
```

```
def silver_addresses_clean():  
    return(  
        spark.readStream.table("LIVE.bronze_addresses")  
    )
```

DLT Programming using Python

Warning

`@dlt.expect`

```
@dlt.expect("valid_postcode",  
            "length(postcode) = 5")
```

Drop

`@dlt.expect_or_drop`

```
@dlt.expect_or_drop("valid_address_line_1",  
                    "address_line_1 IS NOT NULL")
```

Fail

`@dlt.expect_or_fail`

```
@dlt.expect_or_fail("valid_customer_id",  
                    "customer_id IS NOT NULL")
```

DLT Programming using Python

Warning

```
@dlt.expect_all
```

Drop

```
@dlt.expect_all_or_drop
```

Fail

```
@dlt.expect_all_or_fail
```

```
Valid_records = {"valid_customer_id" : "customer_id IS NOT NULL",  
                 "valid_name" : " name IS NOT NULL",  
                 "valid_postcode" : "postcode IS NOT NULL"}
```

```
@dlt.expect_all_or_drop(valid_records)
```

DLT Programming using Python

Warning

```
@dlt.expect
```

```
@dlt.expect_all
```

Drop

```
@dlt.expect_or_drop
```

```
@dlt.expect_all_or_drop
```

Fail

```
@dlt.expect_or_fail
```

```
@dlt.expect_all_or_fail
```

Example

```
import dlt
```

```
@dlt.table ()
```

```
def silver_addresses_clean():  
    return(  
        spark.readStream.table("LIVE.bronze_addresses")  
    )
```

DLT Programming using Python

Warning

```
@dlt.expect
```

```
@dlt.expect_all
```

Drop

```
@dlt.expect_or_drop
```

```
@dlt.expect_all_or_drop
```

Fail

```
@dlt.expect_or_fail
```

```
@dlt.expect_all_or_fail
```

Example

```
import dlt
```

```
@dlt.table ()
```

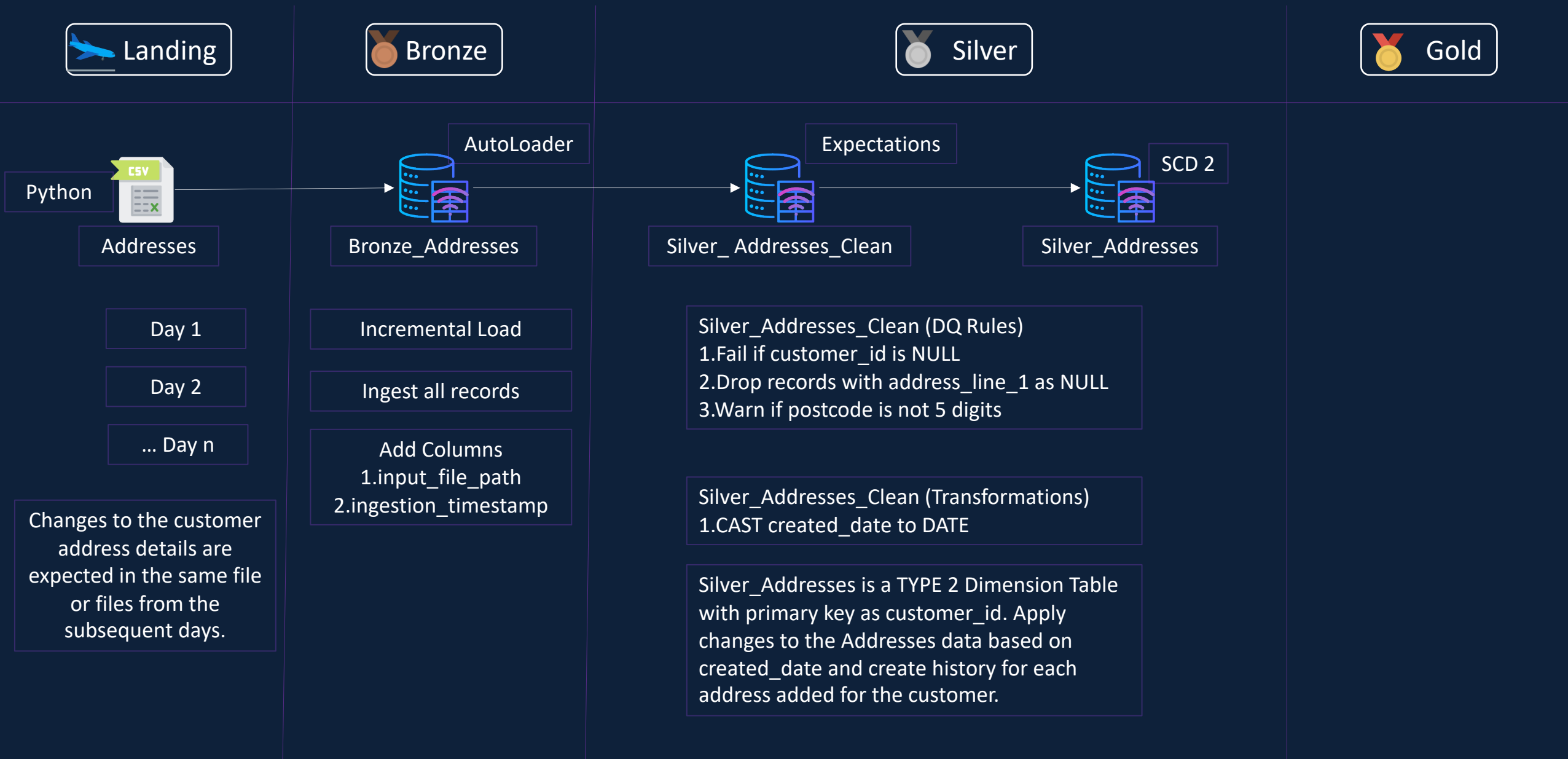
```
@dlt.expect_or_fail("customer_id", "customer_id IS NOT NULL")
```

```
@dlt.expect_or_drop("address_line_1", "address_line_1 IS NOT NULL")
```

```
@dlt.expect("postcode", "length(postcode) = 5")
```

```
def silver_addresses_clean():  
    return(  
        spark.readStream.table("LIVE.bronze_addresses")  
    )
```

CircuitBox – Process Addresses Data



DLT Programming using Python

Create Streaming Table

```
create_streaming_table(  
    name = "<table-name>",  
    comment = "<comment>",  
    spark_conf={"<key>" : "<value>", "<key>" : "<value>"},  
    table_properties={"<key>" : "<value>", "<key>" : "<value>"},  
    partition_cols=["<partition-column>", "<partition-column>"],  
    cluster_by = ["<clustering-column>", "<clustering-column>"],  
    path="<storage-location-path>",  
    schema="schema-definition",  
    expect_all = {"<key>" : "<value>", "<key>" : "<value>"},  
    expect_all_or_drop = {"<key>" : "<value>", "<key>" : "<value>"},  
    expect_all_or_fail = {"<key>" : "<value>", "<key>" : "<value>"},  
    row_filter = "row-filter-clause"  
)
```

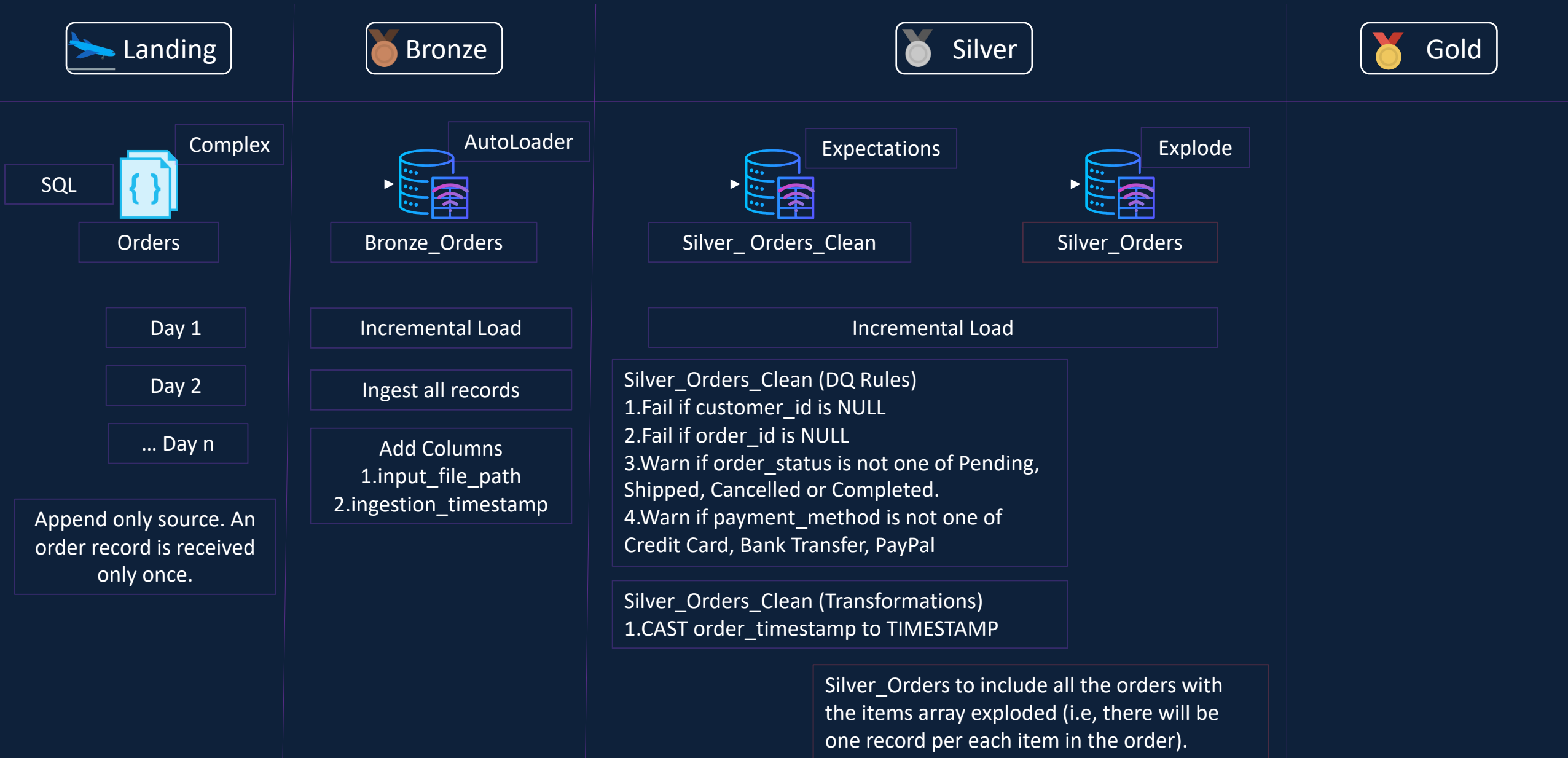
Apply Changes API

```
apply_changes(  
    target = "<target-table>",  
    source = "<data-source>",  
    keys = ["key1", "key2", "keyN"],  
    sequence_by = "<sequence-column>",  
    ignore_null_updates = False,  
    apply_as_deletes = None,  
    apply_as_truncates = None,  
    column_list = None,  
    except_column_list = None,  
    stored_as_scd_type = <type>,  
    track_history_column_list = None,  
    track_history_except_column_list = None  
)
```


Delta Live Tables (DLT) Project - CircuitBox



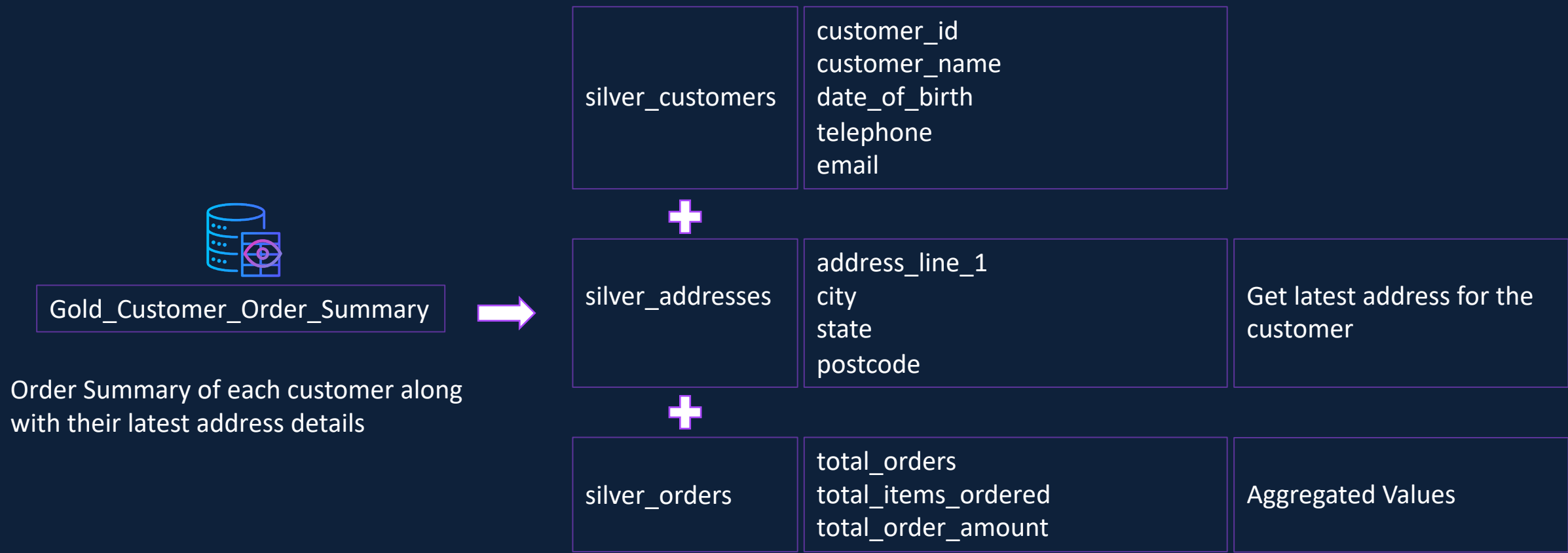
CircuitBox – Process Orders Data



Delta Live Tables (DLT) Project - CircuitBox



CircuitBox – Create Customer Order Summary



Delta Live Tables - Programming



Streaming Tables

Delta Table to which streams write data

Reads from Eventhub, Kafka, Cloud Files, Delta Tables etc.

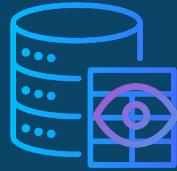
Offers exactly once guarantees

Incremental Data Ingestion Workloads

Low latency transformations

Massive amount of data

DML operations allowed



Materialized Views

Delta Table created from the result of a query

Full Refresh Data Ingestion Workloads

Build aggregate tables for reporting

Improve latency of BI reports

Data Transformation Workloads

No DML operations allowed - Change the query instead.



Views

No physical storage of the data

Scope is limited to the pipeline

Cannot be published to Hive Metastore/Unity Catalog

Store intermediate results to reduce complexity

To enforce data quality constraints

Delta Live Tables – Legacy Terminology

New Terminology



Legacy Terminology

Materialized View

CREATE OR REFRESH MATERIALIZED VIEW

LIVE Table

CREATE OR REFRESH LIVE TABLE

Streaming Table

CREATE OR REFRESH STREAMING TABLE

Streaming LIVE Table

CREATE OR REFRESH STREAMING LIVE TABLE

Databricks SQL

Databricks SQL

Component of the Databricks Lakehouse Platform, that helps deliver data analytics and reporting on the data stored in the Data Lakehouse

Who is it for?

Enables Data Analysts to perform Ad-hoc Analysis, Data Visualizations and Dashboarding

Enables Business Users to extract insights from the data via Dashboards and AI/BI Genie

Helps Data Scientists to explore data quickly before writing advanced analytics in Databricks notebooks

Data Engineers should be aware of configuring clusters, alerting, scheduling, performance tuning etc.

Databricks SQL



Users and
Applications

User Interface



SQL Editor



Dashboards



Genie



Alert

SQL Warehouse

Serverless



Provisioned



Data Lakehouse

Unity Catalog



Volume



Table



View



Function

Data Storage



ADLS



AWS S3



GCS



DELTA LAKE

Data Governance

Data Governance

What is Data Governance

Framework of policies, process and technologies

Ensures data accuracy, security, and compliance

Defines how data is managed, accessed, and protected

Data Governance

Why do we need Data Governance

Prevents security risks, compliance violations, and poor decisions

Avoids data inconsistencies and unauthorized access

Improves operational efficiency and risk management

Data Governance

Key Benefits of Data Governance

Ensures data quality

Strengthens security

Ensures compliance

Optimizes decision-making

Reduces costs & risks

Data Governance

Data Governance in Databricks

Unity Catalog

Delta Lake

Delta Sharing

Cluster & Workspace Logging

Unity Catalog – Data Governance Solution



Unity Catalog – Data Governance Solution



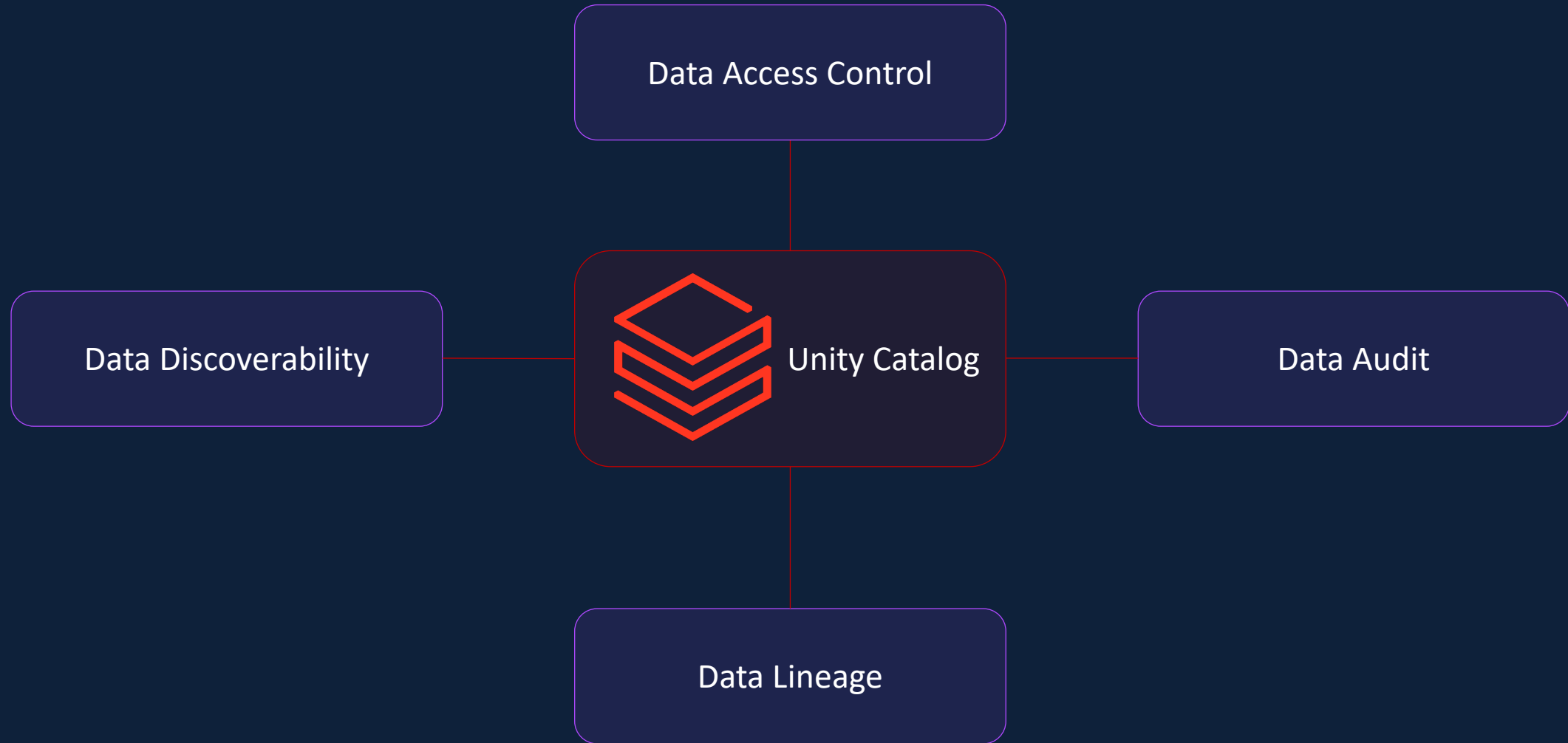
Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

Unity Catalog – Data Governance Solution

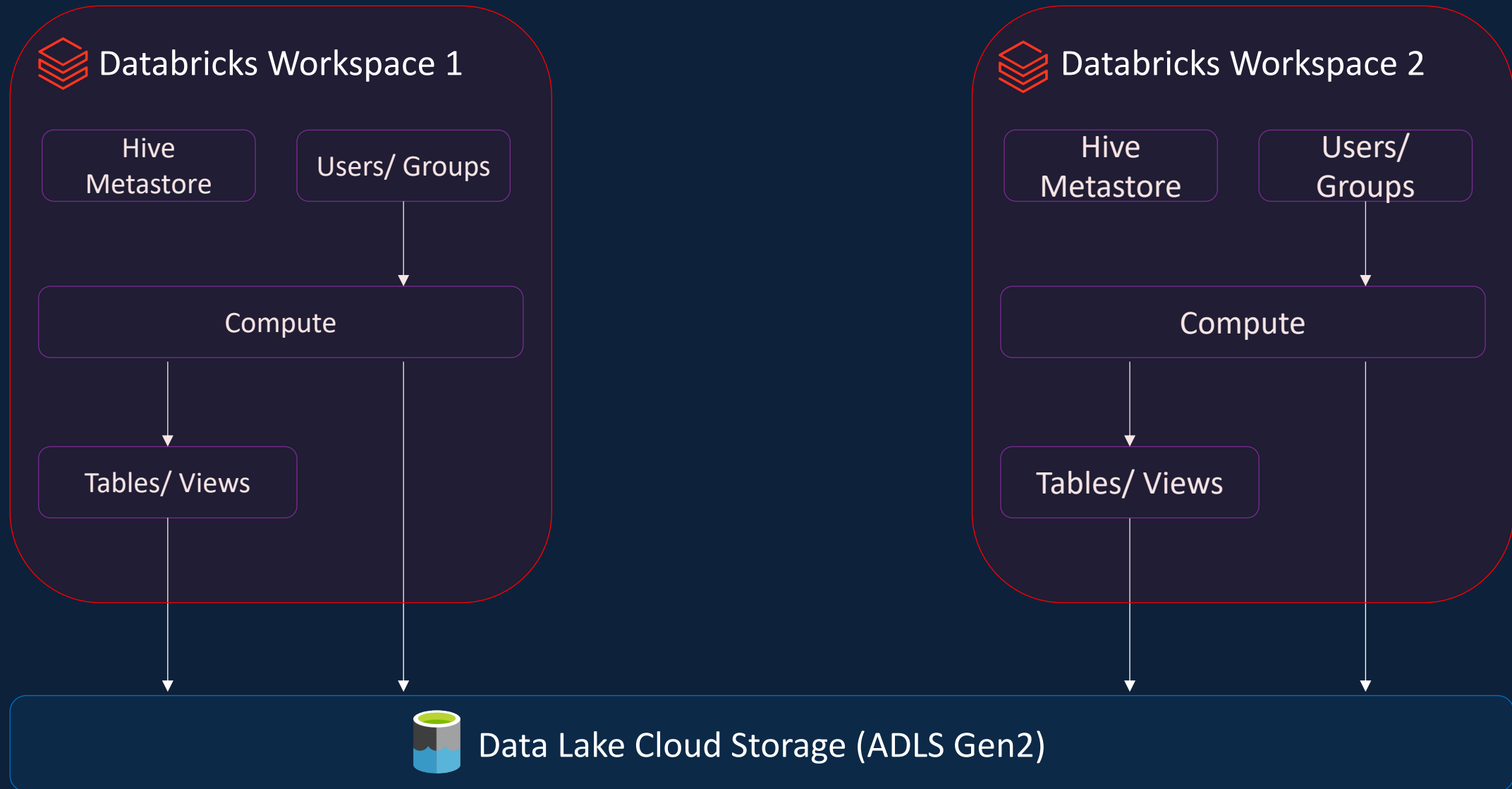


Unity Catalog is a Databricks offered **unified solution** for implementing **data governance** in the Data Lakehouse

Unity Catalog – Data Governance Solution



Unity Catalog – Data Governance Solution



Unity Catalog – Data Governance Solution



Unity Catalog – Data Governance Solution

Without Unity Catalog



Databricks
Workspace 1

User Management

Metastore

Compute



Databricks
Workspace 2

User Management

Metastore

Compute

Databricks Account

With Unity Catalog

Databricks Unity Catalog

User Management

Metastore



Databricks
Workspace 1

Compute



Databricks
Workspace 2

Compute

Databricks Account

Unity Catalog – Data Governance Solution

Without Unity Catalog



Databricks
Workspace 1

User Management

Metastore

Compute



Databricks
Workspace 2

User Management

Metastore

Compute

Databricks Account

With Unity Catalog

Databricks Unity Catalog

User Management

Metastore



Databricks
Workspace 1

Compute

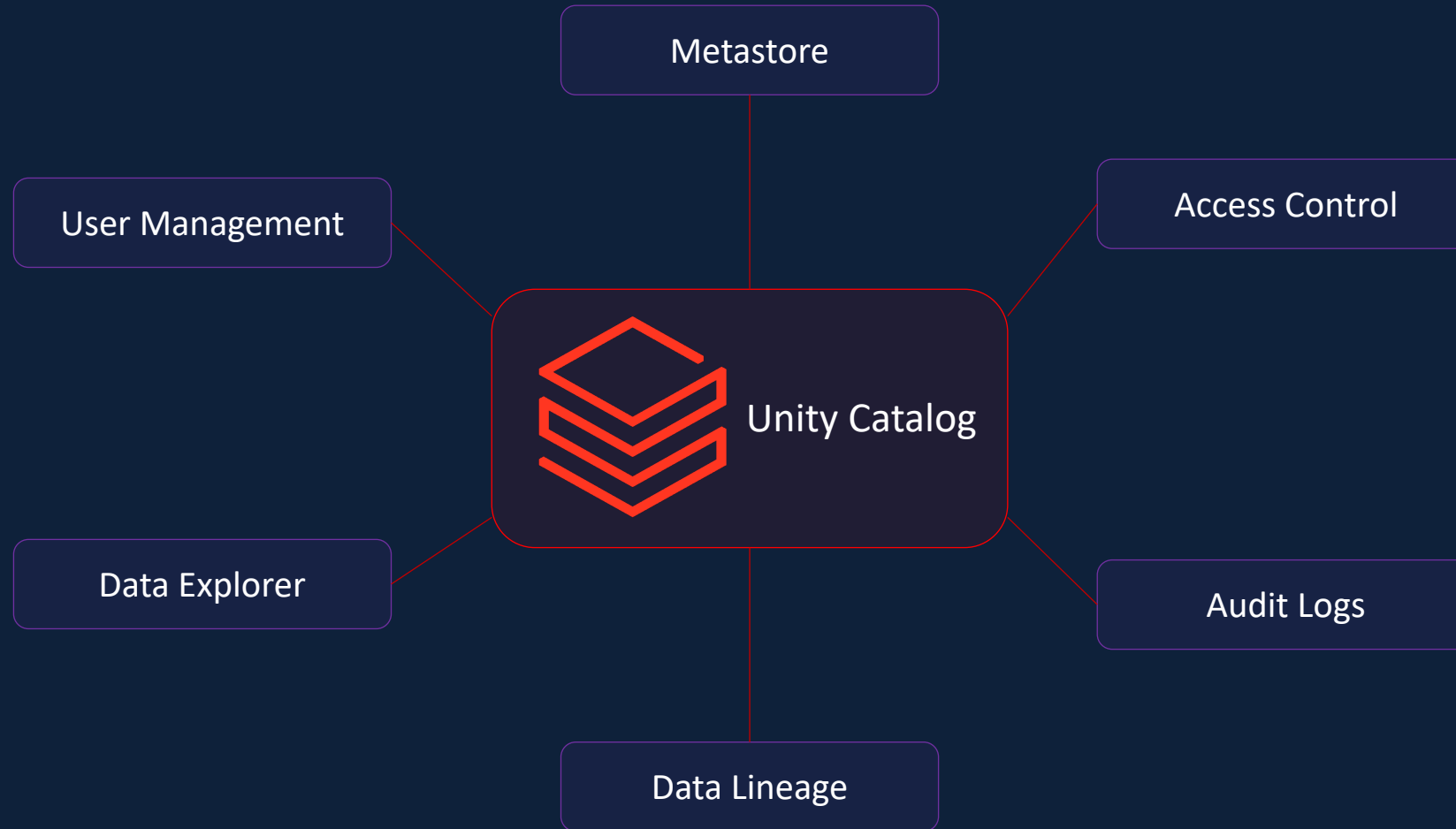


Databricks
Workspace 2

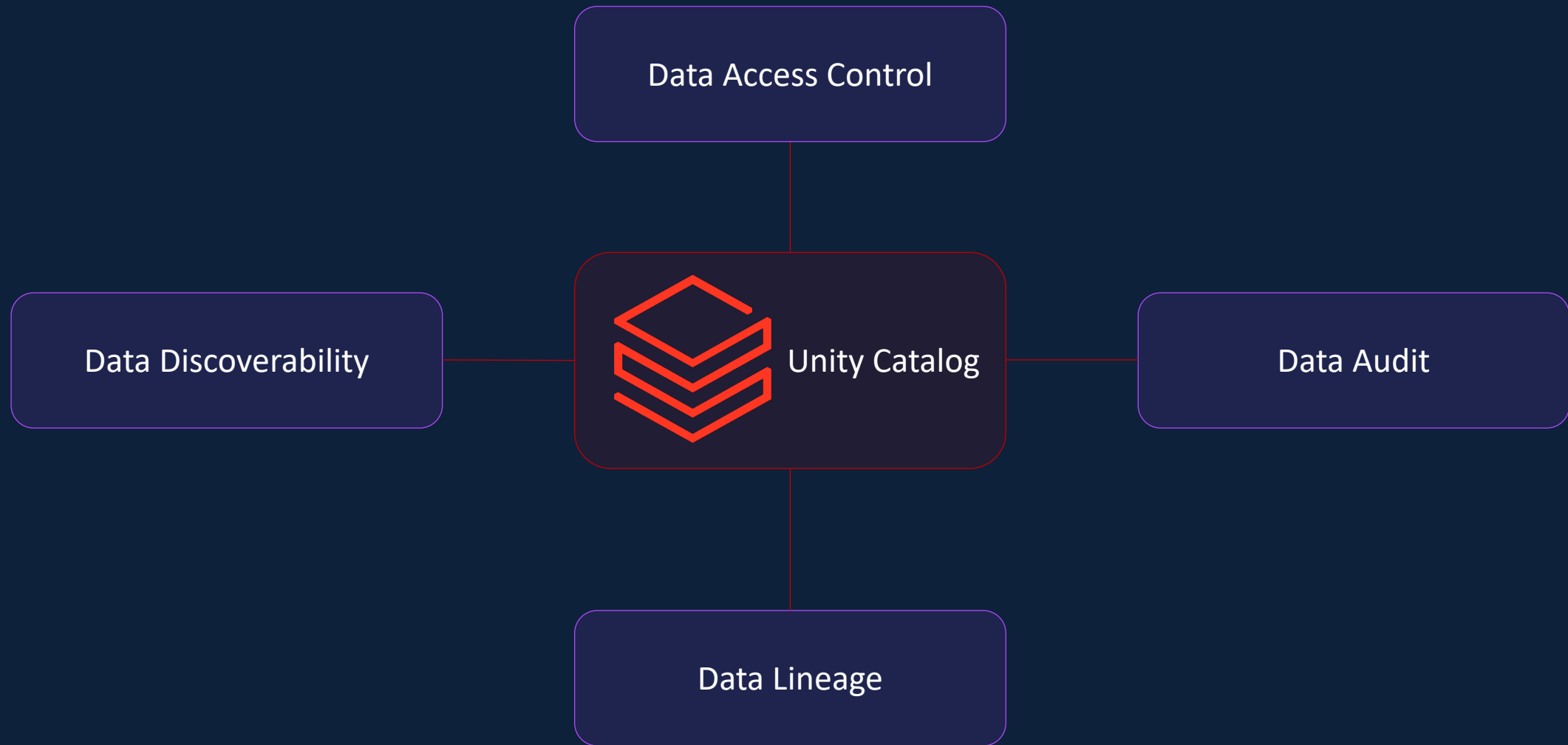
Compute

Databricks Account

Unity Catalog – Data Governance Solution



Unity Catalog – Data Governance Solution



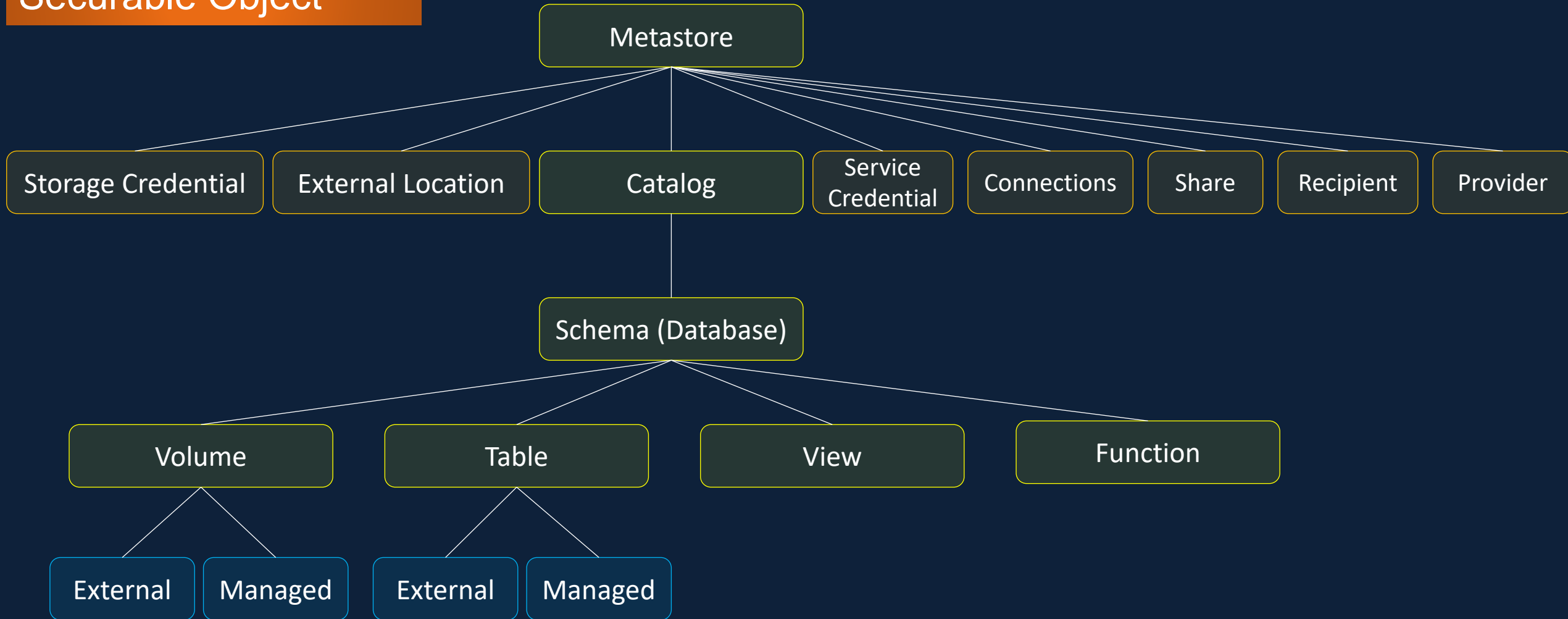
Data Governance - Demo

Unity Catalog - Security Model



Unity Catalog - Security Model

Securable Object

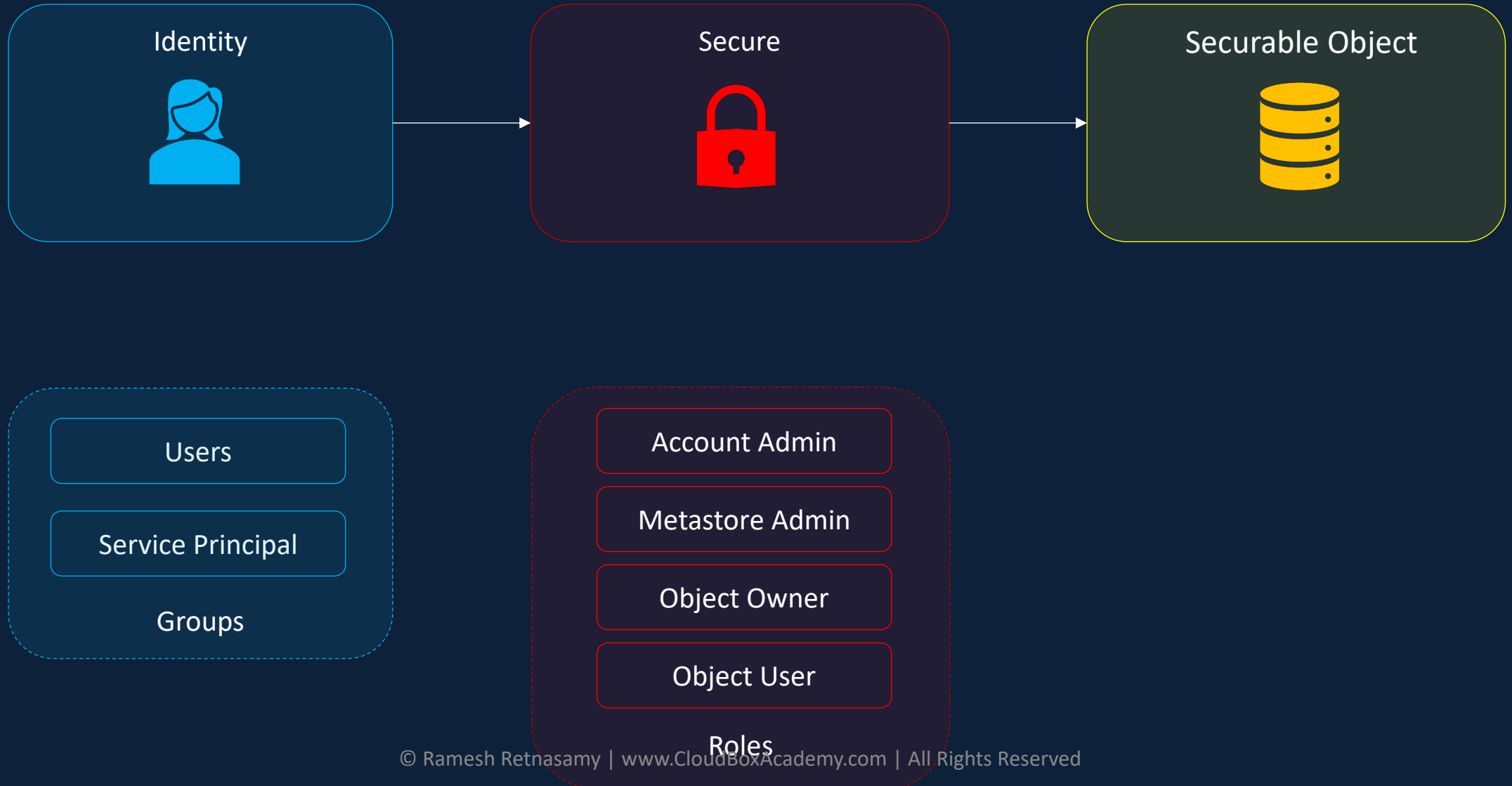


Unity Catalog - Security Model



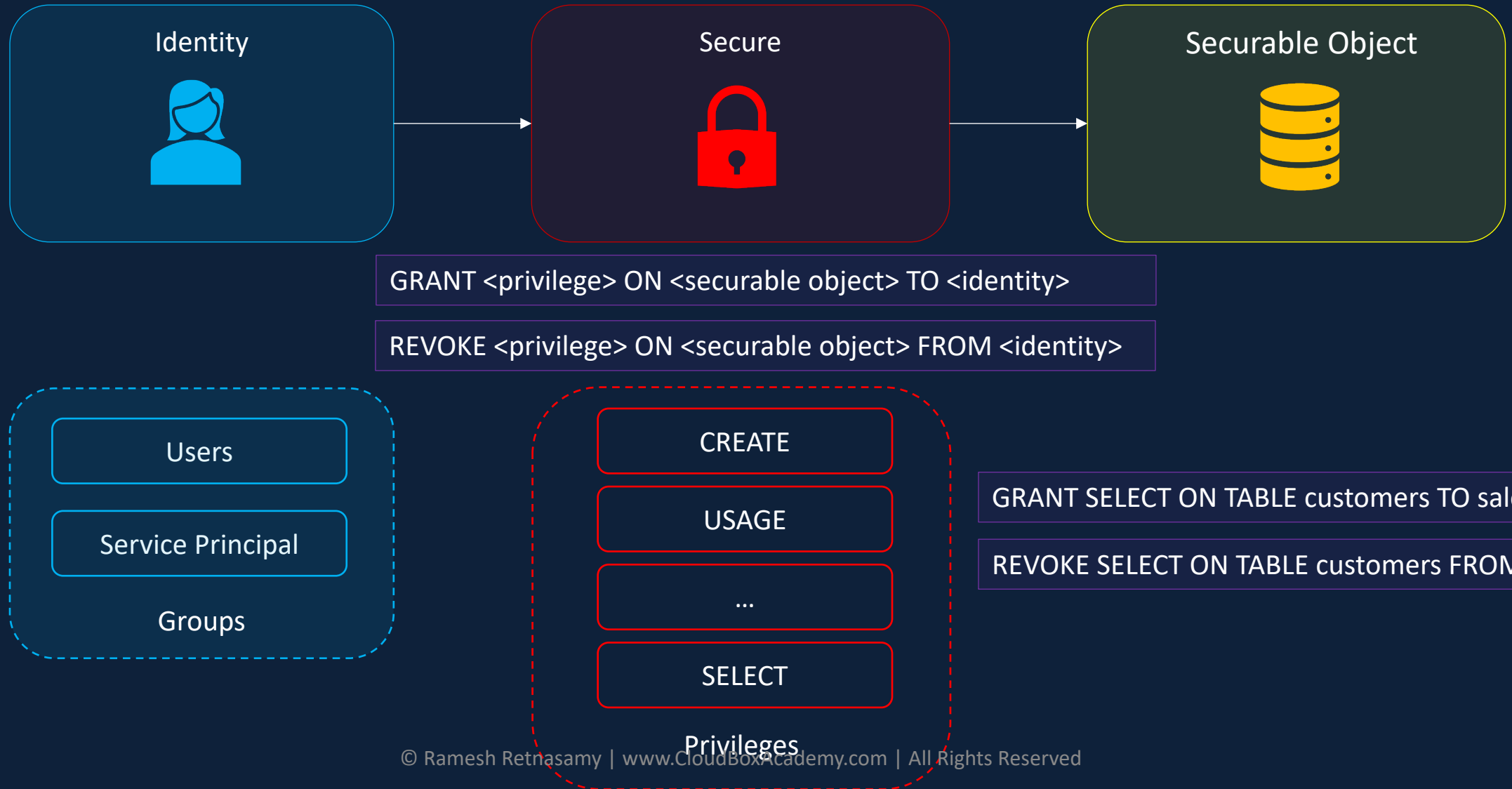
Unity Catalog - Security Model

Role Based Access



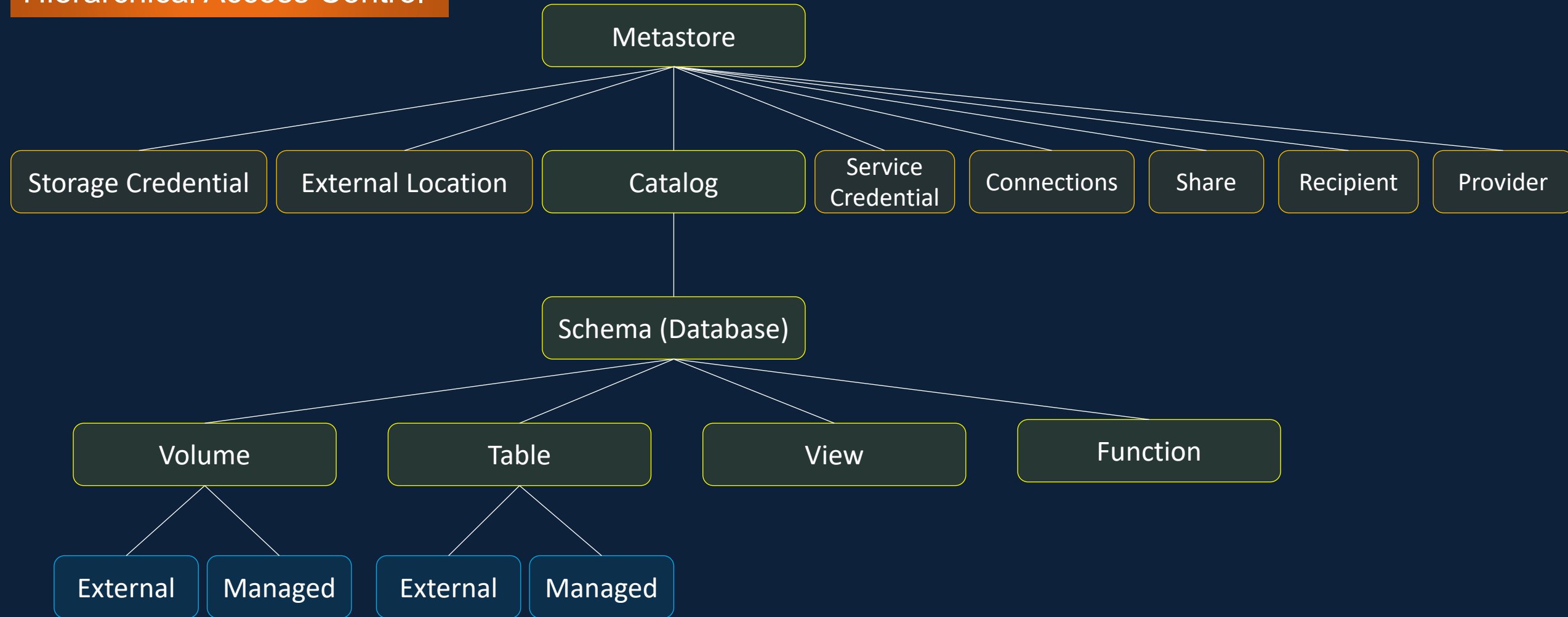
Unity Catalog - Security Model

Access Control List



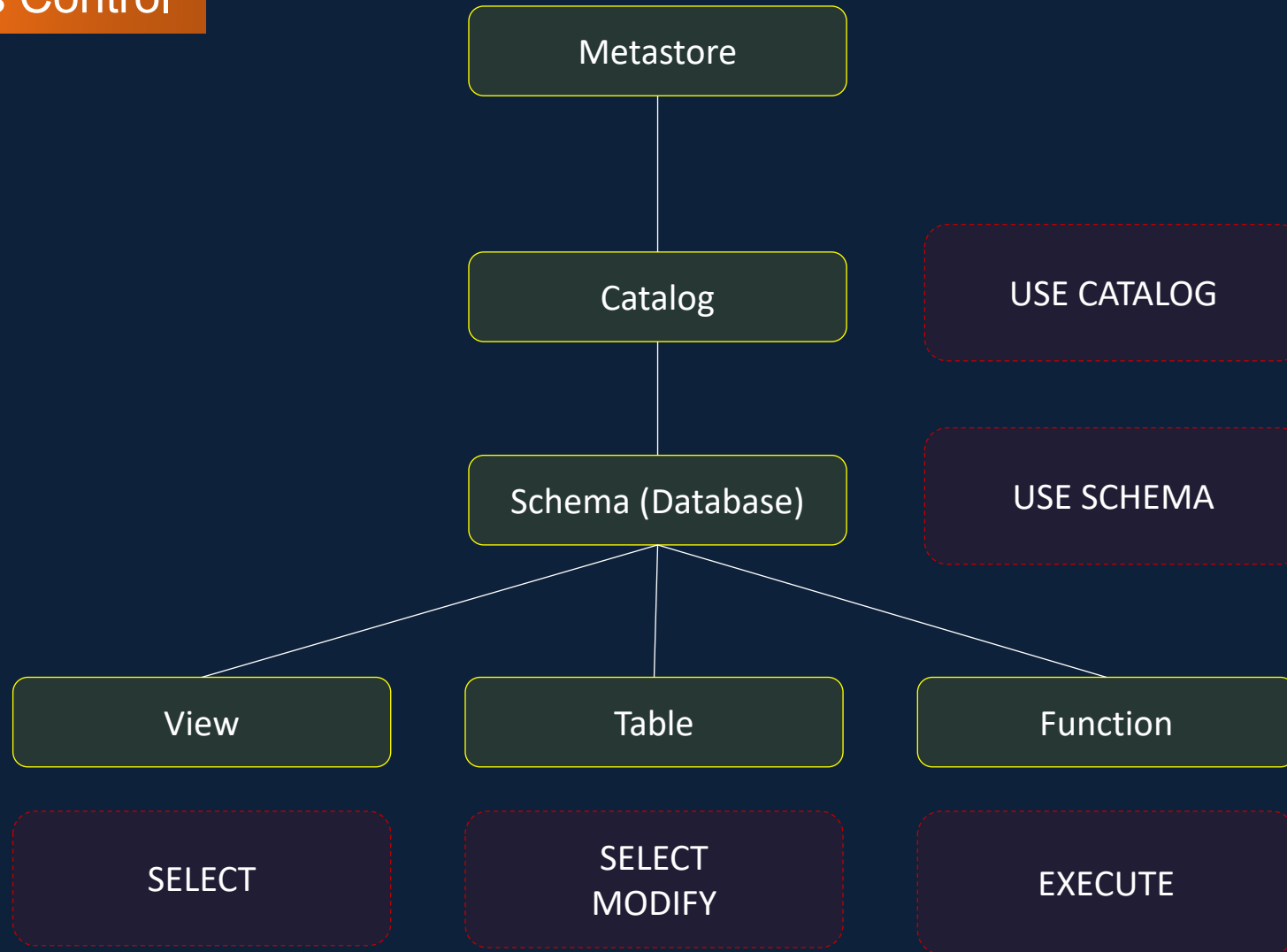
Unity Catalog - Security Model

Hierarchical Access Control



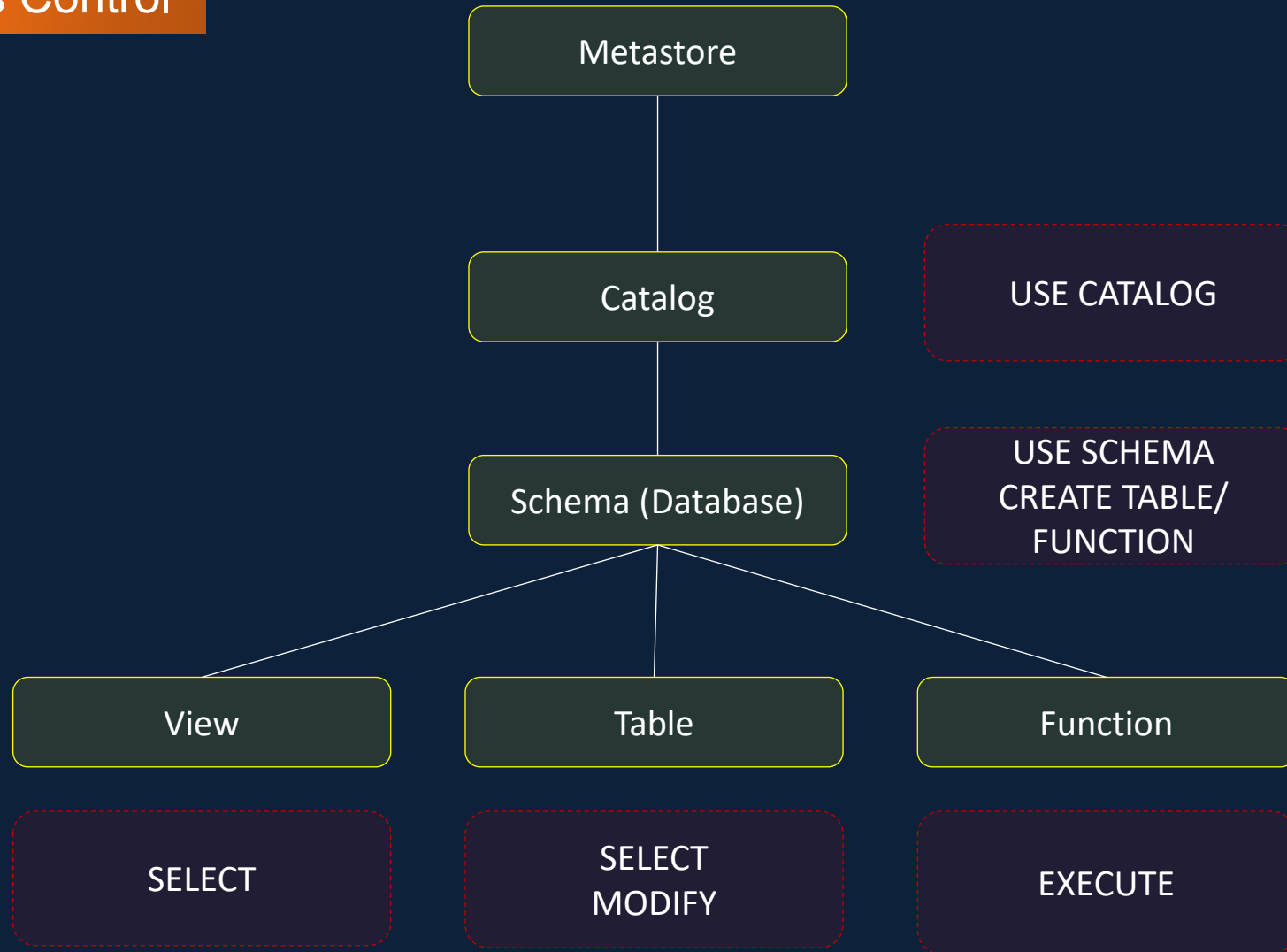
Unity Catalog - Security Model

Hierarchical Access Control



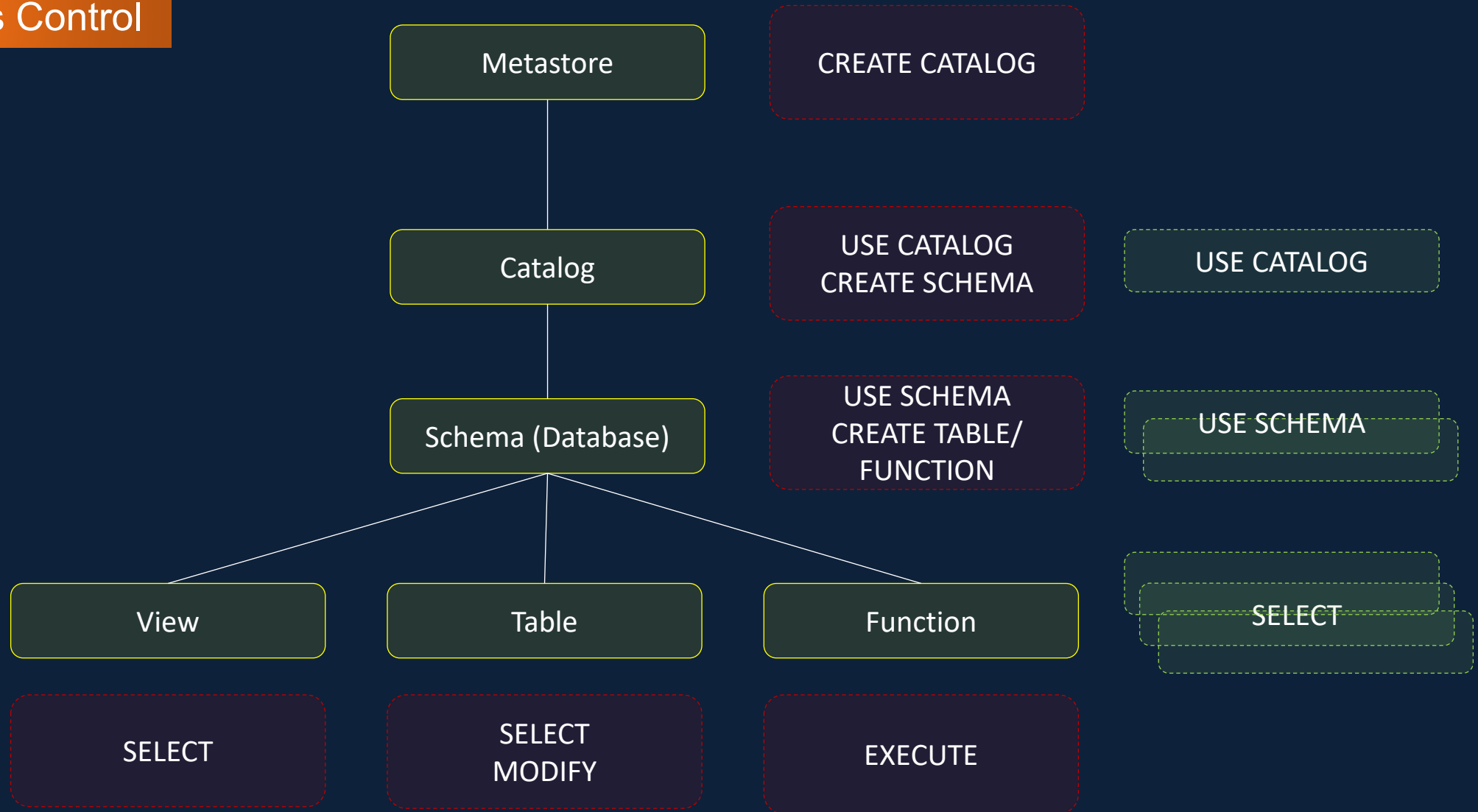
Unity Catalog - Security Model

Hierarchical Access Control



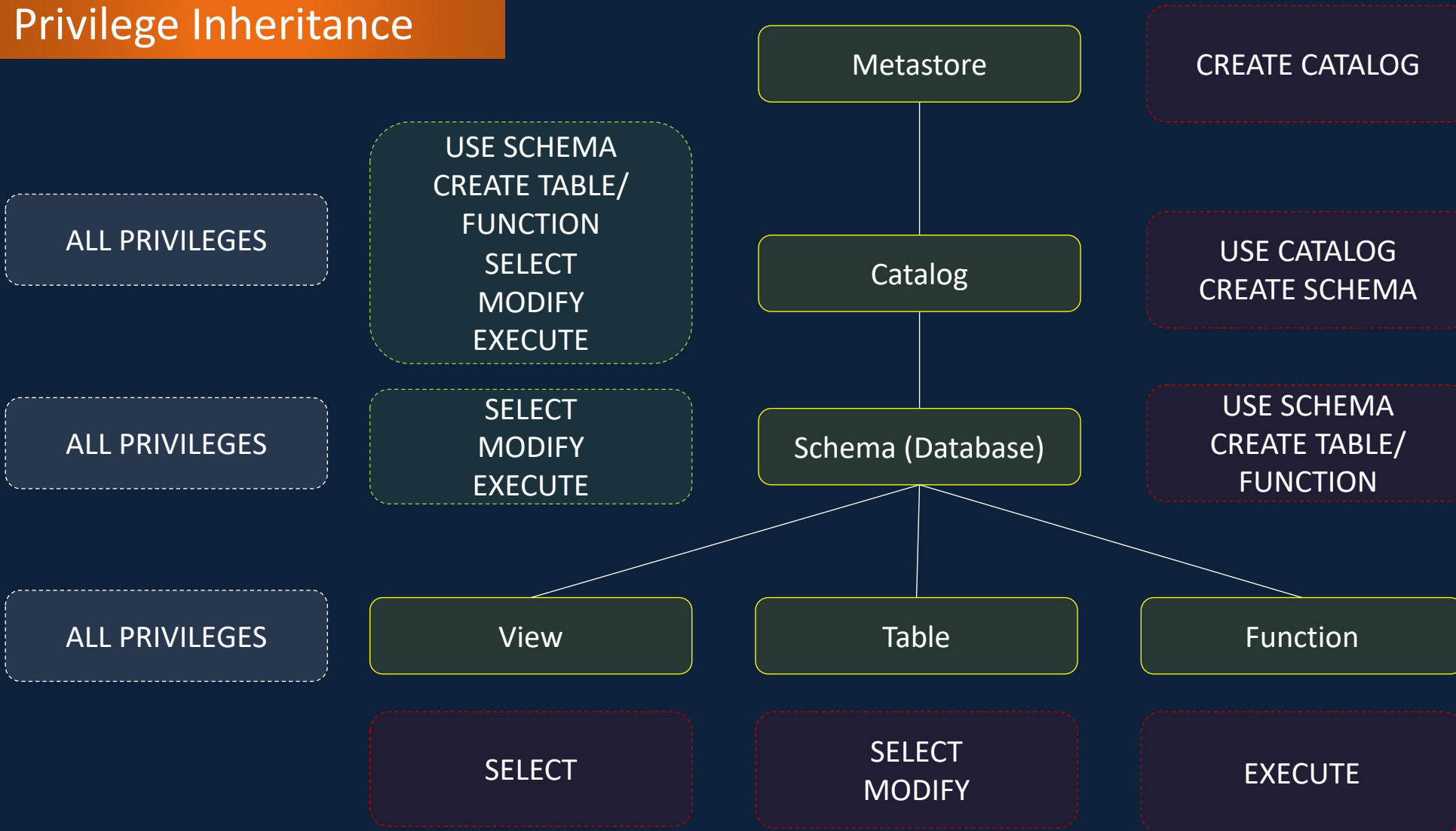
Unity Catalog - Security Model

Hierarchical Access Control



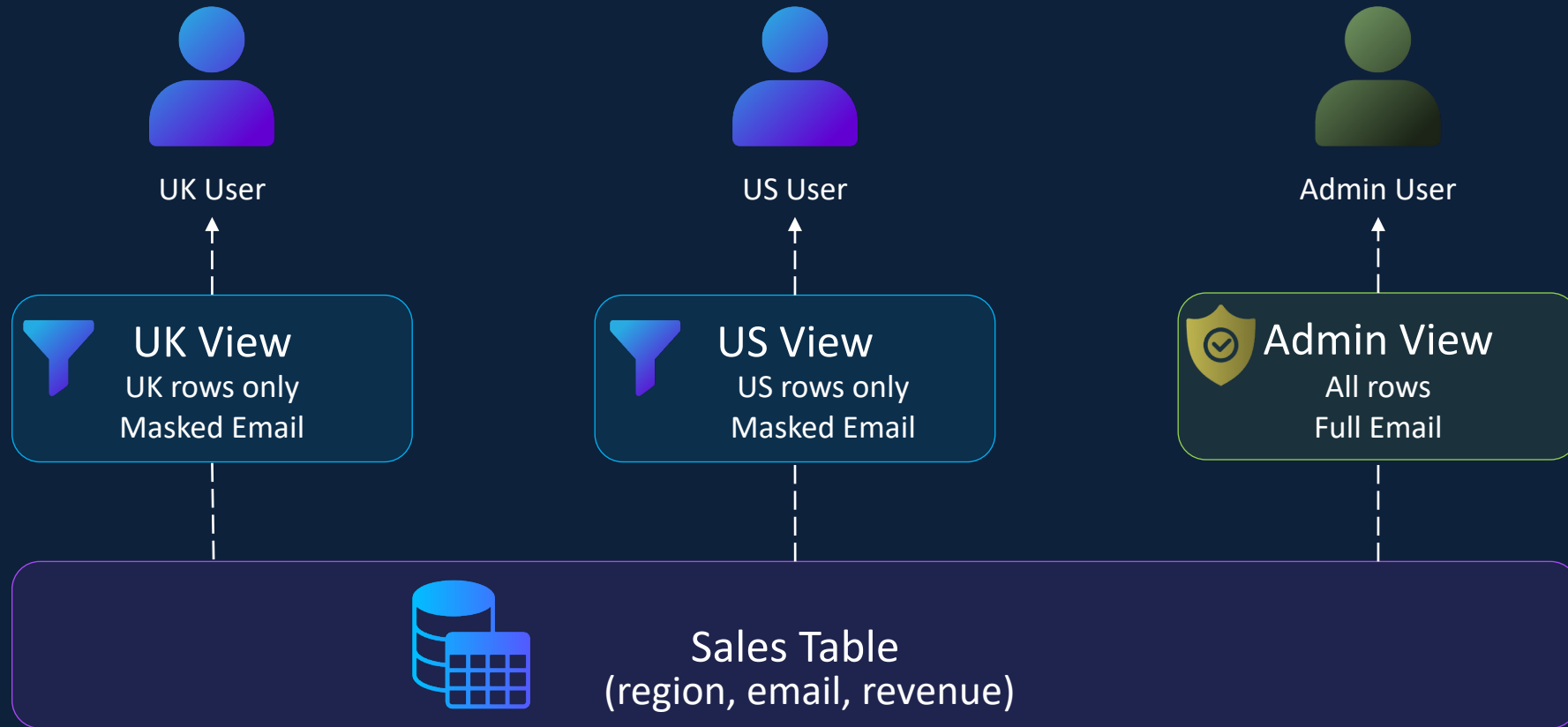
Unity Catalog - Security Model

Privilege Inheritance



Data Level Security - Overview

Data Level Security - Overview



- **Same table**
- **Different users**
- **Different Visibility**

The Problem

Id	Region	Email	Revenue
1	UK	John.doe@gmail.com	10000
2	US	Mike.smith@outlook.com	20000
3	US	Sara.jones@gmail.com	15000
4	UK	David.brown@icloud.com	11000
..	..		

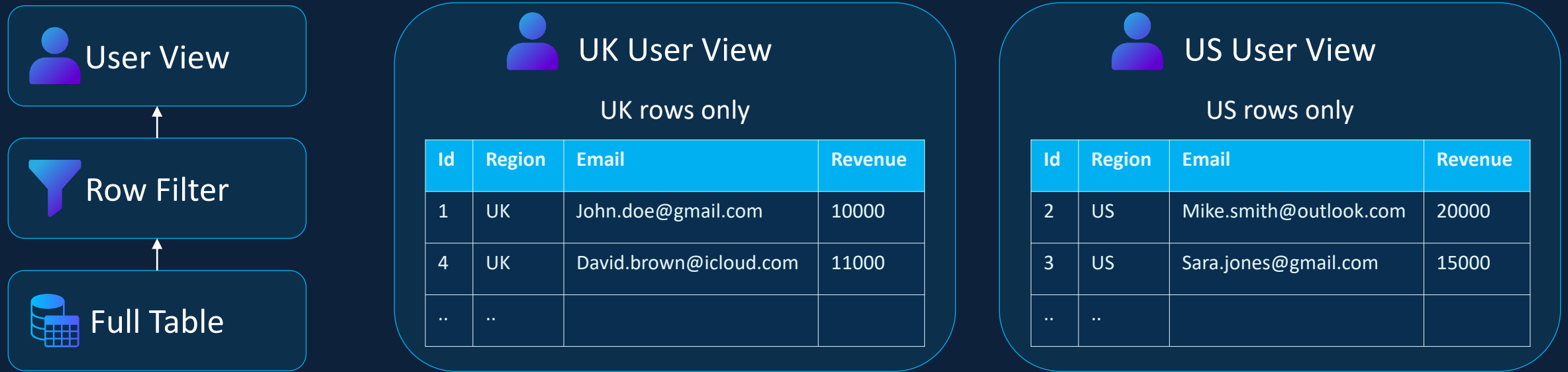
All users see
all rows

All users see
sensitive data

Table level access exposes more data than required

Row Level Security

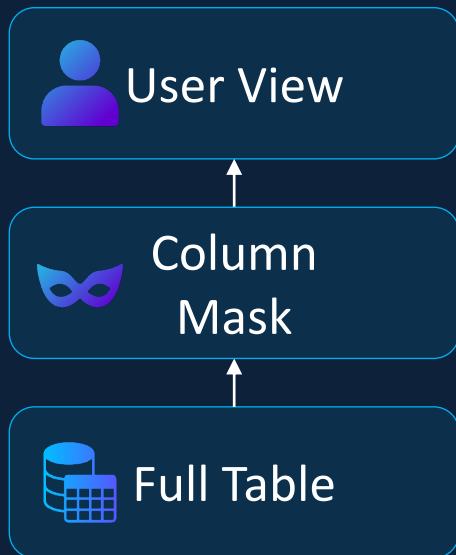
Controls which rows are visible



Only relevant rows are visible to the user based on the user or their group

Column Level Masking

Controls what column values are visible



 **Admin User**
Full Email visible

Id	Region	Email	Revenue
1	UK	John.doe@gmail.com	10000
2	US	Mike.smith@outlook.com	20000
..	..		

 **Regular User**
Masked Email

Id	Region	Email	Revenue
1	UK	J***@gmail.com	10000
2	US	M***@outlook.com	20000
..	..		

Sensitive values are protected without removing the column

Data Level Security - Implementation

1

Dynamic Views

Control data visibility
using Logic in Views

2

Row Filters & Column Masks

Apply row-level filters
and column masks
written in UDFs directly
to tables

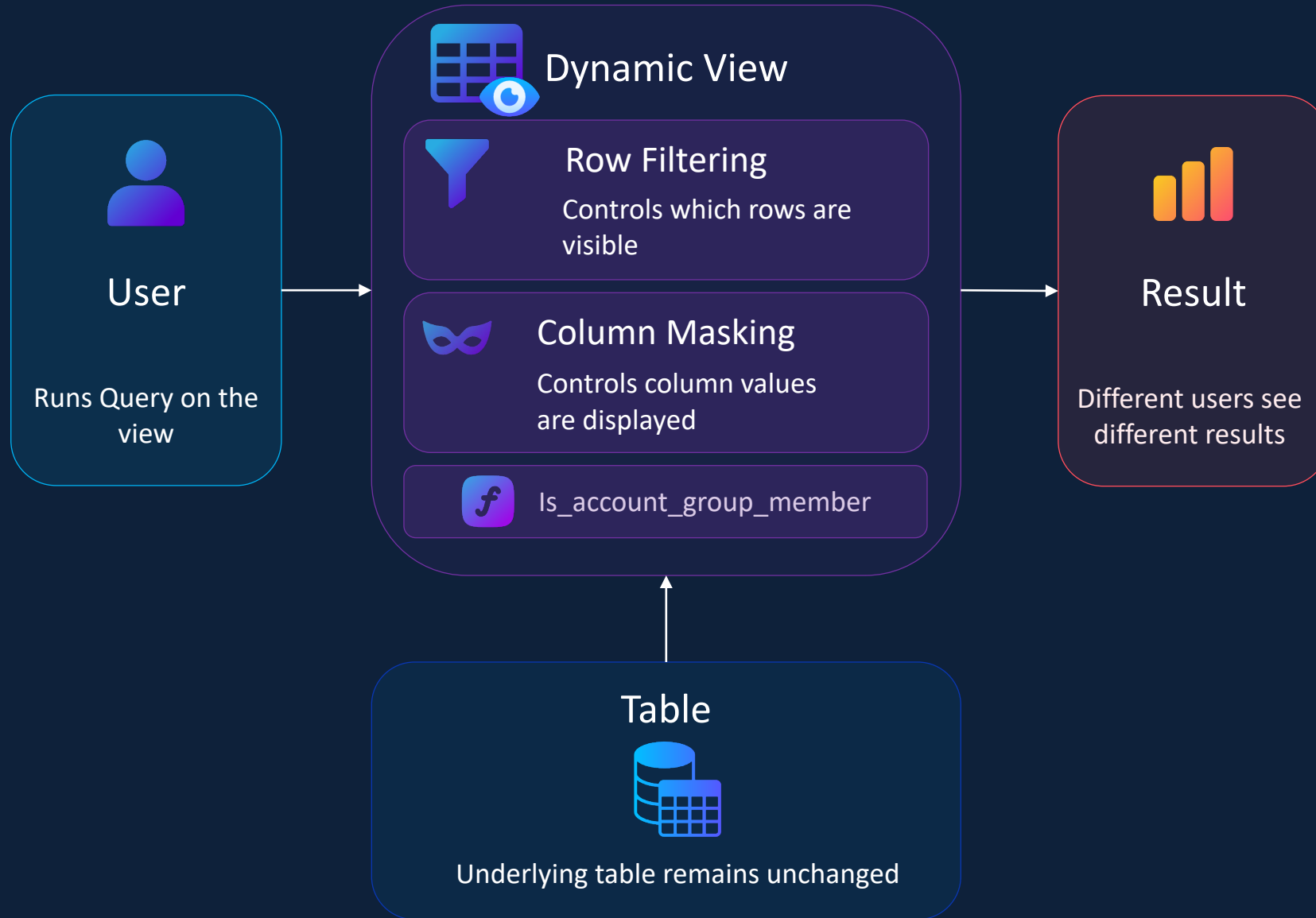
3

Attribute Based Access Control (ABAC)

Apply rules centrally
using attributes for a
scalable and flexible
model

Data Level Security using Dynamic Views

Dynamic Views - Overview



- A dynamic view sits on top of a table and applies logic dynamically
- The view evaluates the logic based on the user at query time
- The logic controls which rows are returned and how values are shown
- The same view can return different results for different users

Data Level Security using Row Filters & Column Masks

Row Filters & Column Masks (Policies)



Security rules are applied directly to the table using **row filters** and **column masks**. Any query on the table automatically enforces these rules.



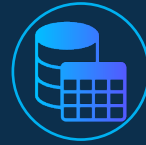
1. Row Filter

A row filter function defines which rows a user is allowed to see based on their group membership



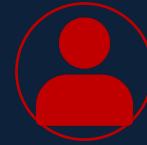
2. Column Mask

A column mask function defines how sensitive values are displayed based on the user or their group



3. Applied to Table

The row filter is attached to the table and column masks are attached to the columns. Users query the table as usual.



4. Automatic Enforcement

The security roles are evaluated at query time. Each user automatically sees only rows and values they are allowed to see.



Dynamic Views - View Level Logic
Useful when custom logic is required

Policies - Table Level Logic
Useful when centralized enforcement is required

Data Level Security using ABAC (Governed Tags)

Data Level Security using ABAC (Governed Tags)



ABAC (Attribute-Based Access Control) uses attributes of the user and the data to decide what data should be visible and how it should be presented.



1. Tag the Data

Data is tagged using attributes

Example: pii = email



2. Define Policy

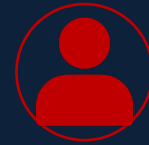
Policies are defined based on tags. They describe what should happen.

Example: Mask Columns where pii = email



3. Applied to UC Objects

The policy is attached to UC securable at the desired scope (e.g. catalog, schema, table)



4. Automatic Enforcement

The policy is applied at query time based on the user attributes (e.g. group) and data attributes (e.g. tags)



Centralized – Define rules once and apply across many objects

Consistent – Ensures uniform data security across the organisation

Scalable – Works seamlessly as data and teams grow



Understanding Spark Execution

Understanding Spark Execution

Single Machine

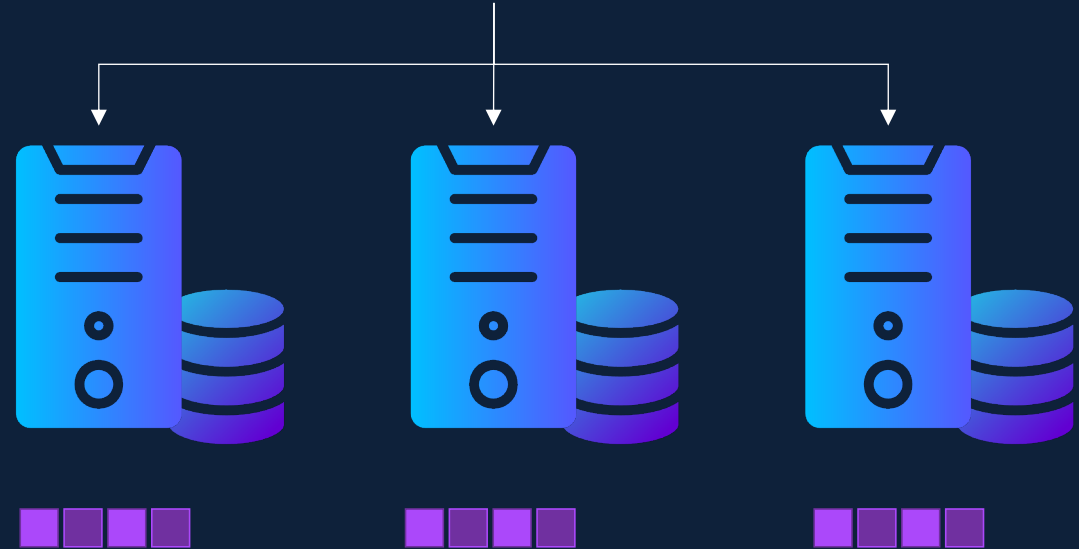


Huge dataset processed
on one machine



Slow

Distributed Processing



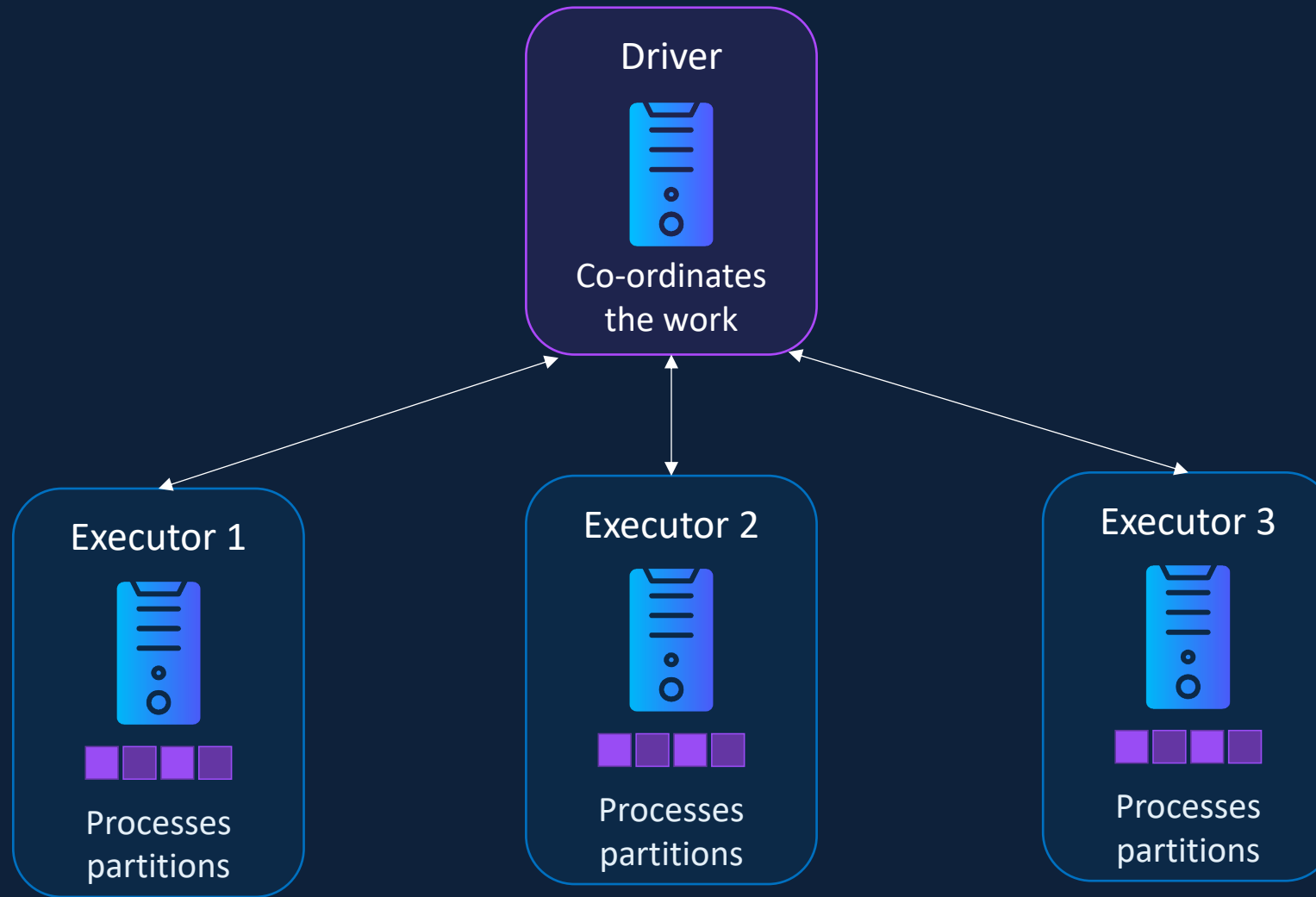
Dataset is split and processed in parallel across
multiple machines



Fast

Spark scales by distributing work across multiple machines

Understanding Spark Execution



Driver co-ordinates the work, while executors process the data in parallel

Understanding Spark Execution

Job

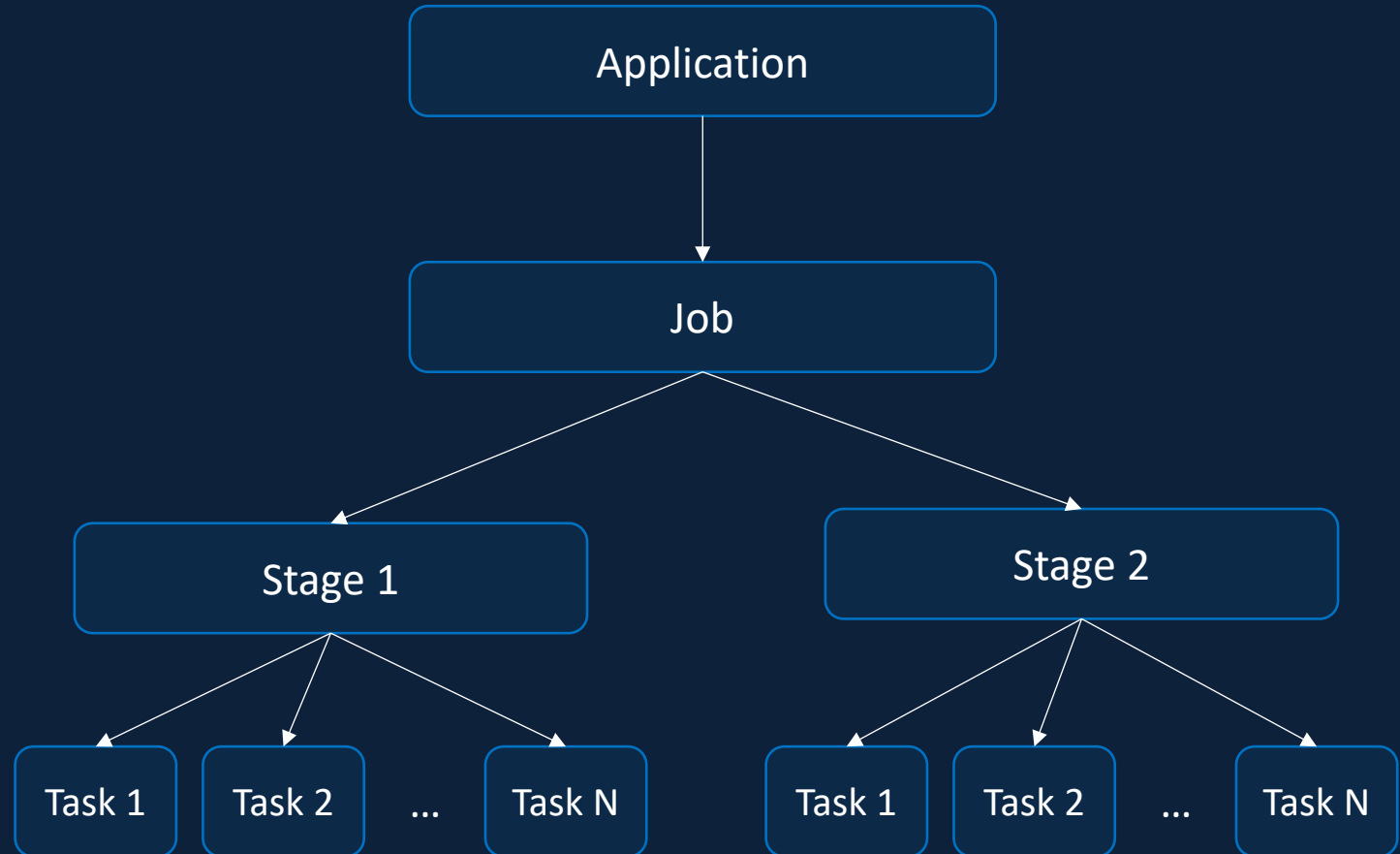
A complete unit of work triggered by an application/ action (ex. df.count)

Stage

A major phase of execution. Stages are created at shuffle boundaries

Task

The Smallest unit of work that processes partition. One task typically processes one partition



Common Spark Performance Bottlenecks



Excessive File Scanning

Small file problems

Shuffle operations

Data skew

Data Spilling



Common Spark Performance Bottlenecks

Common Spark Performance Bottlenecks



Excessive Data Scanning

Small file problems

Excessive Data Scanning

What is the problem?

Reading unnecessary data

Scanning irrelevant files or partitions

Increased processing time and compute cost

Spark UI Indicators (Example)

Duration:	2.1 h
Stages:	8
Completed Stages:	8
Tasks:	2700 total, 2700 succeeded
Input:	4.25 TB
Output:	12.3 GB
Shuffled Read:	1.12 TB
Shuffled Write:	620.3 GB

Large Input Size in Spark UI

How to reduce excessive data scanning?

Filter data early in the execution plan

Read only required columns

Partition data efficiently

Data skipping to avoid unnecessary file scans

Small File Problems

What is the problem?

Large numbers of tiny files

More file operations during processing

Increased metadata and scheduling overhead

Spark UI Indicators (Example)

Duration:	35 min
Stages:	12
Completed Stages:	12
Tasks:	50132 total, 50132 succeeded
Input:	48.6 GB
Output:	48.6 GB
Shuffled Read:	96.3 GB
Shuffled Write:	96.3 GB

Large number of tasks in Spark UI

How to reduce small file problems?

Periodically compact files (e.g. OPTIMIZE)

Avoid excessive partitioning

Write larger batches where appropriate

Predictive Optimization automates maintenance tasks

Common Spark Performance Bottlenecks



Excessive File Scanning

Small file problems



Common Spark Performance Bottlenecks

Common Spark Performance Bottlenecks



Shuffle operations

Data skew

Data Spilling

Shuffle Operations

What is the problem?

Data movement across executors

Common during joins and aggregations

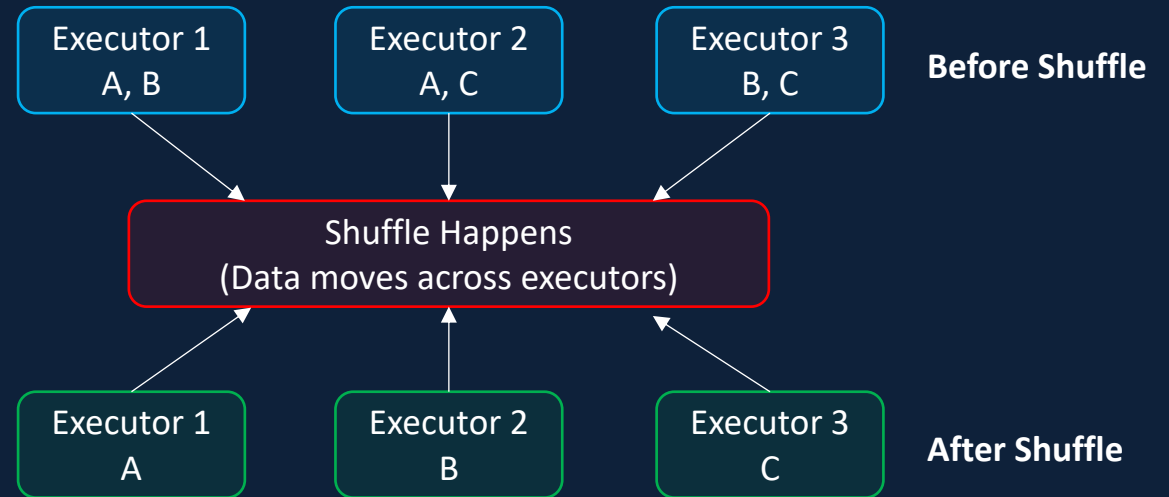
Increased network and disk activity

Spark UI Indicators (Example)

Stage Details Example	
Metric	Value
Shuffle Read	25.3 GB
Shuffle Write	18.7 GB
Input Size / Records	120 GB
Duration	2.4 min

High Shuffle Read / Write in Spark UI

How shuffle Happens



How to reduce shuffle?

Filter data early

Reduce unnecessary transformations

Use broadcast joins for small tables

Data Skew

What is the problem?

Some executors process much more data

Uneven workload distribution

Slow tasks delay the overall job

Spark UI Indicators (Example)



Long-running tasks in Spark UI

How Data Skew Happens

Balanced Workload

Executor 1
800 MB

Executor 2
750 MB

Executor 3
850 MB

Skewed Workload

Executor 1
1.4 GB

Executor 2
100 MB

Executor 3
100 MB

How to reduce data skew?

Repartition data more evenly

Avoid heavily skewed keys

Filter unnecessary data early

AQE can help optimize skew automatically

Data Spilling

What is the problem?

Happens when memory is insufficient

Spark temporarily writes data to disk

Disk operations are slower than memory operations

Spark UI Indicators (Example)

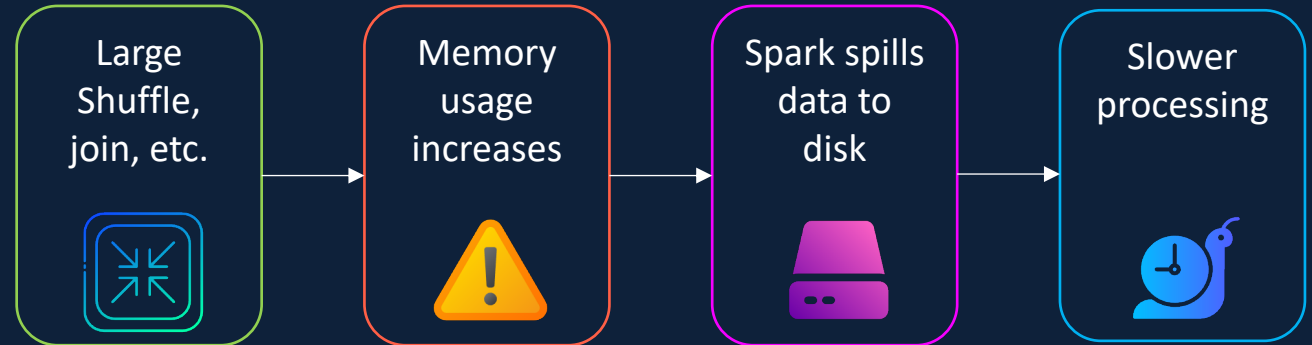
Stage Details for Stage 12 (Attempt 0)

Summary Metrics

Metric	Value
Memory Spilled	45.2 GB
Disk Spilled	312.6 GB
Shuffle Read	256.7 GB
Shuffle Write	278.8 GB

Spill metrics visible in Spark UI

How Data Spilling Happens



How to reduce data spilling?

Reduce large shuffle operations

Improve workload distribution

Reduce skew-related pressure

Common Spark Performance Bottlenecks



Shuffle operations

Data skew

Data Spilling



Delta Lake Optimization Concepts

Delta Lake Optimization Concepts

As Delta tables grow, their layout can become less efficient. Delta Lake provides features to maintain efficient storage layout and improve query performance.

How Tables Evolve Without Optimization



Growing Delta Table

Data is continuously added by batch and streaming workloads.



More Files + More Data

Small files accumulate and data gets spread across multiple files.



Layout Becomes Less Efficient

Queries scan more data, performance degrades, and storage costs increase.



Optimization Features Improve Performance and Maintenance

Delta Lake features help maintain efficient layout and consistent query performance over time.

Delta Lake Optimization Features



OPTIMIZE → File compaction

Combines many small files into fewer larger files.



Z-ORDER → Improved data skipping

Organizes data to speed up filtering and



VACUUM → Removes old unused files

Frees up storage by deleting obsolete files.



Liquid Clustering → Adaptive data layout

Automatically adapts layout as data and queries change.



Predictive Optimization → Automated maintenance

Databricks automatically runs maintenance in the background.

Delta Lake – Compaction

Process of combining many smaller files into few larger files to improve performance and optimize storage.



Small File Problem

Faster Reads

Efficient Storage

Delta Lake – Compaction



File Compaction (OPTIMIZE Command)

Z-Order Compaction (ZORDER BY Clause)

Liquid Clustering (Preview Feature)

Delta Lake – Vacuum

Used to remove old, unused files from the Delta Lake to free up storage. Permanently deletes files that are no longer referenced in the transaction log and older than retention threshold (default 7 days)



Reduces cloud storage costs

Improves query performance

Helps comply with privacy regulations

Liquid Clustering

Liquid Clustering

Liquid Clustering is an innovative data layout technique in Delta Lake that automatically manages how data is organized to improve performance and cost efficiency.

Motivation For Liquid Clustering

Hard to design right partitioning & z-ordering strategy

Leads to data skew and concurrency conflicts

Performance drops as data grows and workloads change

Re-partitioning required rewriting the entire data

Liquid Clustering

Data is divided into clusters based on the clustering keys

As new data arrives, databricks monitors and rebalances these clusters automatically.

No manual re-organization or z-ordering required

Changing the Clustering columns doesn't require re-writing entire data

Liquid Clustering - Benefits

Faster Queries

Automatic Optimization

Adaptive Data Layout

Lower maintenance cost

Simpler Operations

Liquid Clustering - Implementation

```
CREATE TABLE table1  
( col1 INT, col2 STRING )  
CLUSTER BY (col1);
```

```
ALTER TABLE table1  
CLUSTER BY (col1);
```




Predictive Optimization

Predictive Optimization

Automatically performs maintenance operations on Unity Catalog managed Delta tables.



Helps maintain efficient file layouts

Reduces manual maintenance effort

Only available for Unity Catalog managed Delta tables

Predictive Optimization

Why Delta Tables Need Maintenance



Small files accumulate over time

Table statistics may become stale

Old files increase storage overhead

Query performance may gradually degrade

Predictive Optimization



Predictive Optimization Automates



OPTIMIZE

Compact files and
improve layout



VACCUUM

Remove old files no
longer needed



ANALYZE

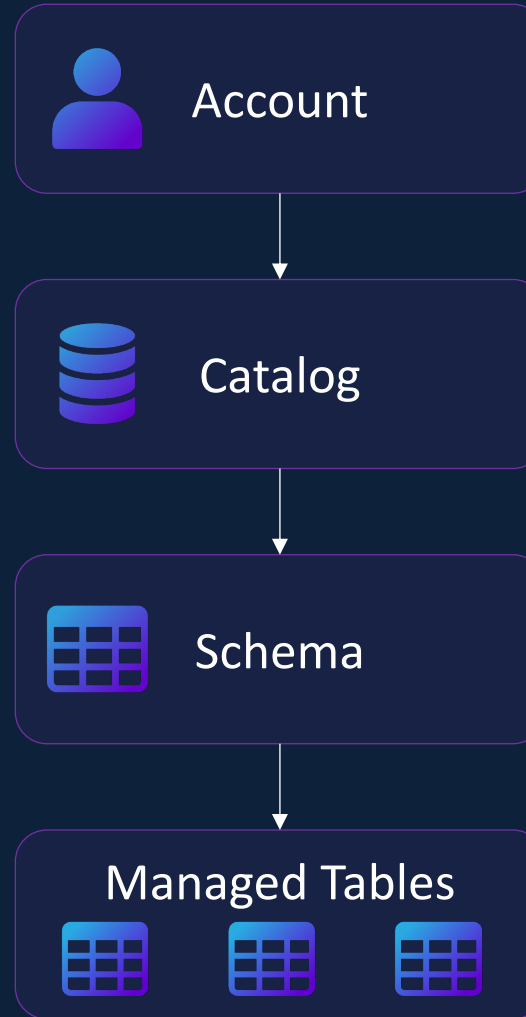
Collect statistics for
query optimizer

Predictive Optimization

Via Account Console ->
Settings / Feature Enablement

ALTER CATALOG catalog_name
ENABLE PREDICTIVE OPTIMIZATION

ALTER SCHEMA schema_name
ENABLE PREDICTIVE OPTIMIZATION



Settings are inherited

Delta Sharing

Delta Sharing

Delta Sharing is an open data sharing protocol that enables secure sharing of data across business units, customers, suppliers and partners.

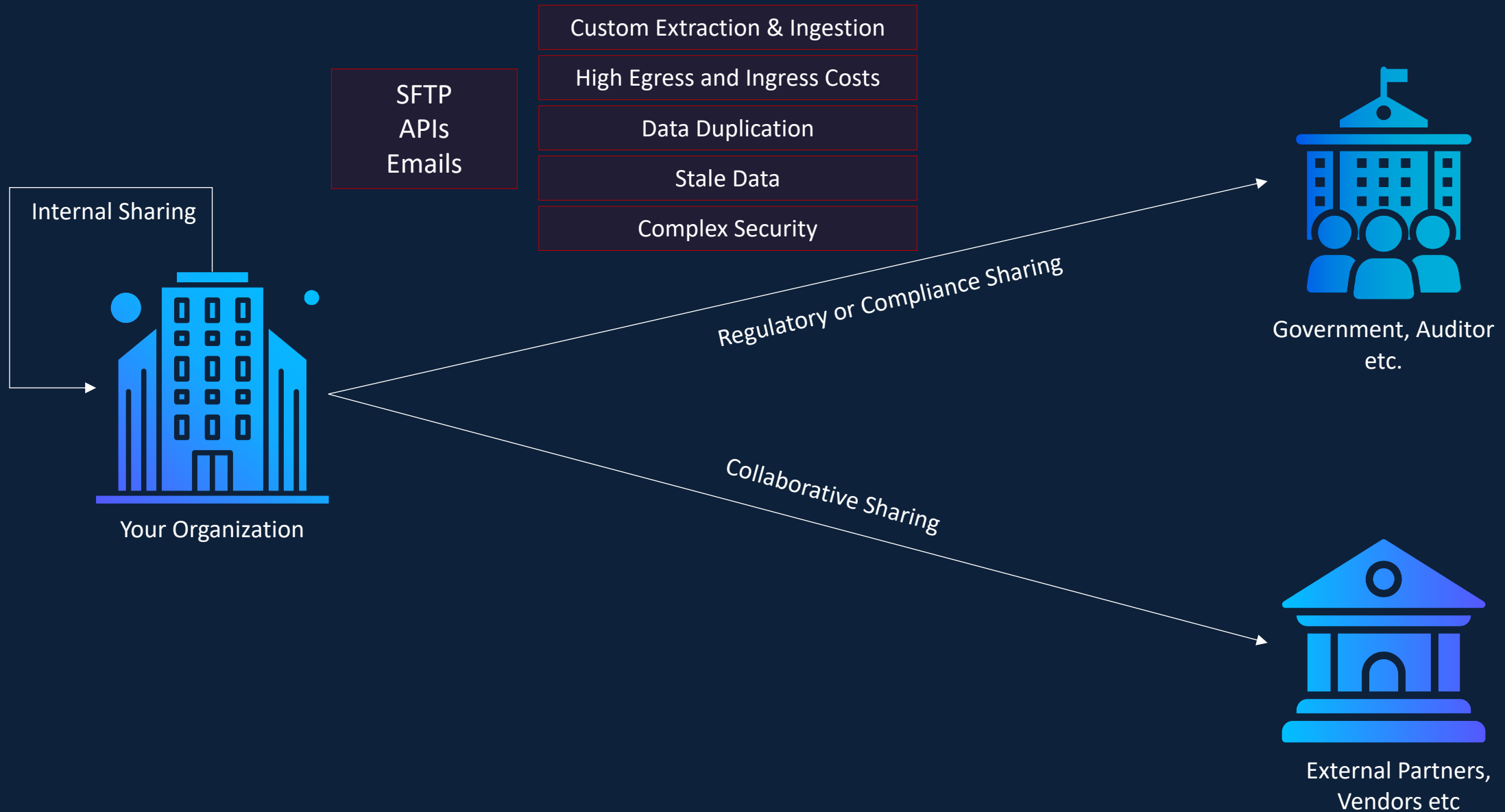
Developed by Databricks, contributed to open-source Delta Lake project & governed under Linux Foundation

Available as Fully managed inside the Databricks platform in workspaces enabled with Unity Catalog.

Native integration with Databricks Platform allows sharing of Notebooks, Dashboards, AI Models, Volumes etc.

Native integration with Unity Catalog for Governance and Security

Data Sharing

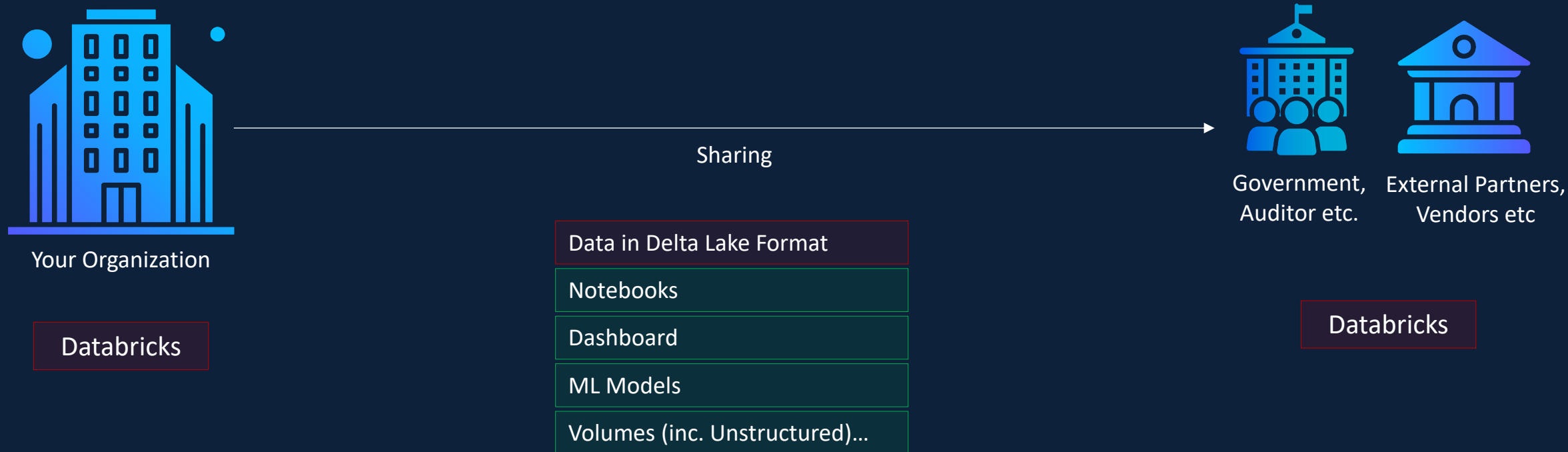


Delta Sharing



Delta Sharing Protocols

1. Databricks to Databricks Sharing Protocol



Delta Sharing Protocols

1. Databricks to Databricks Sharing Protocol

2. Databricks Open Sharing Protocol



Your Organization

Databricks

Sharing

Data in Delta Lake Format



Government,
Auditor etc.

External Partners,
Vendors etc

Non Databricks Tools

PowerBI

Tableau

Apache Spark

More...

Delta Sharing Protocols

1. Databricks to Databricks Sharing Protocol

2. Databricks Open Sharing Protocol

3. Open-source Delta Sharing Server – Customer Managed



Your Organization

Sharing



Government,
Auditor etc.

External Partners,
Vendors etc

Non Databricks Tools

PowerBI

Tableau

Apache Spark

More...

Data in Delta Lake Format

Non Databricks Tools

PowerBI

Tableau

Apache Spark

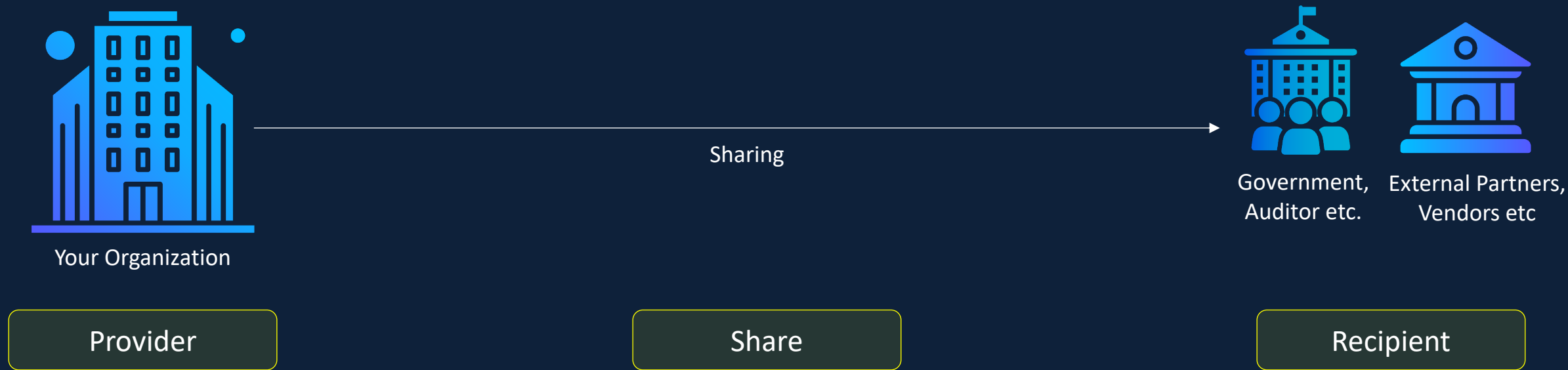
More...

Delta Sharing - Key Concepts

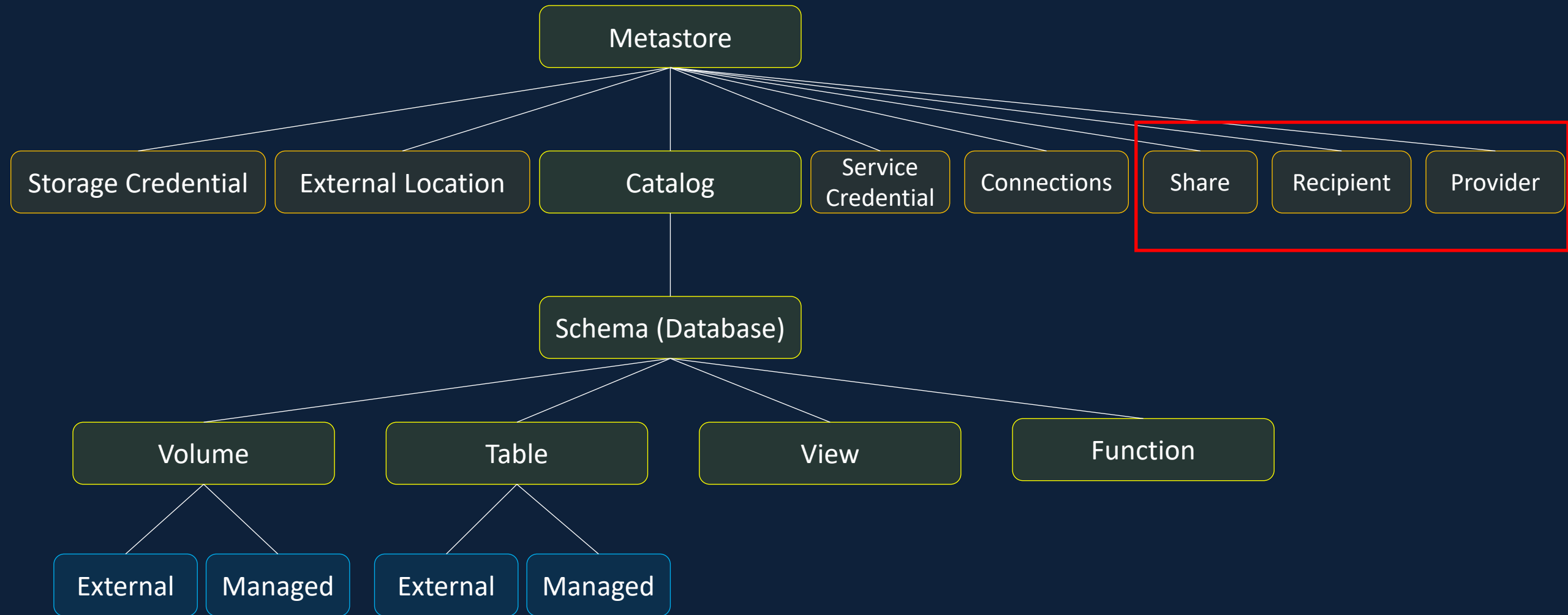
1. Databricks to Databricks Sharing Protocol

2. Databricks Open Sharing Protocol

3. Open-source Delta Sharing Server – Customer Managed

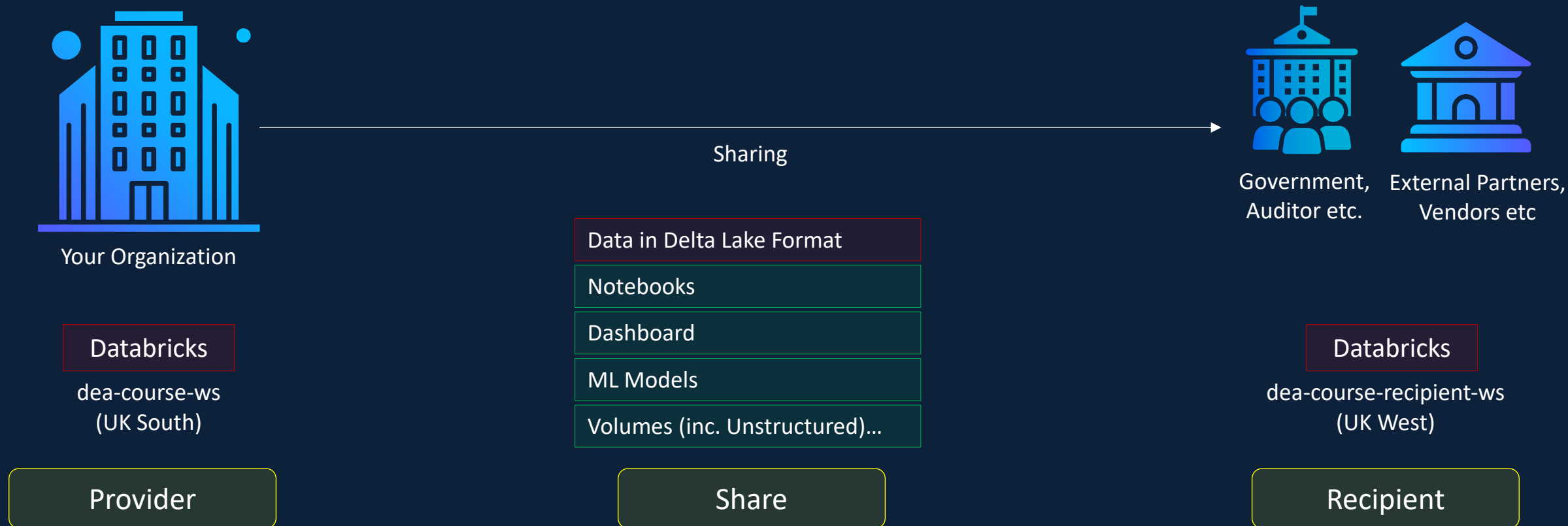


Unity Catalog Object Model

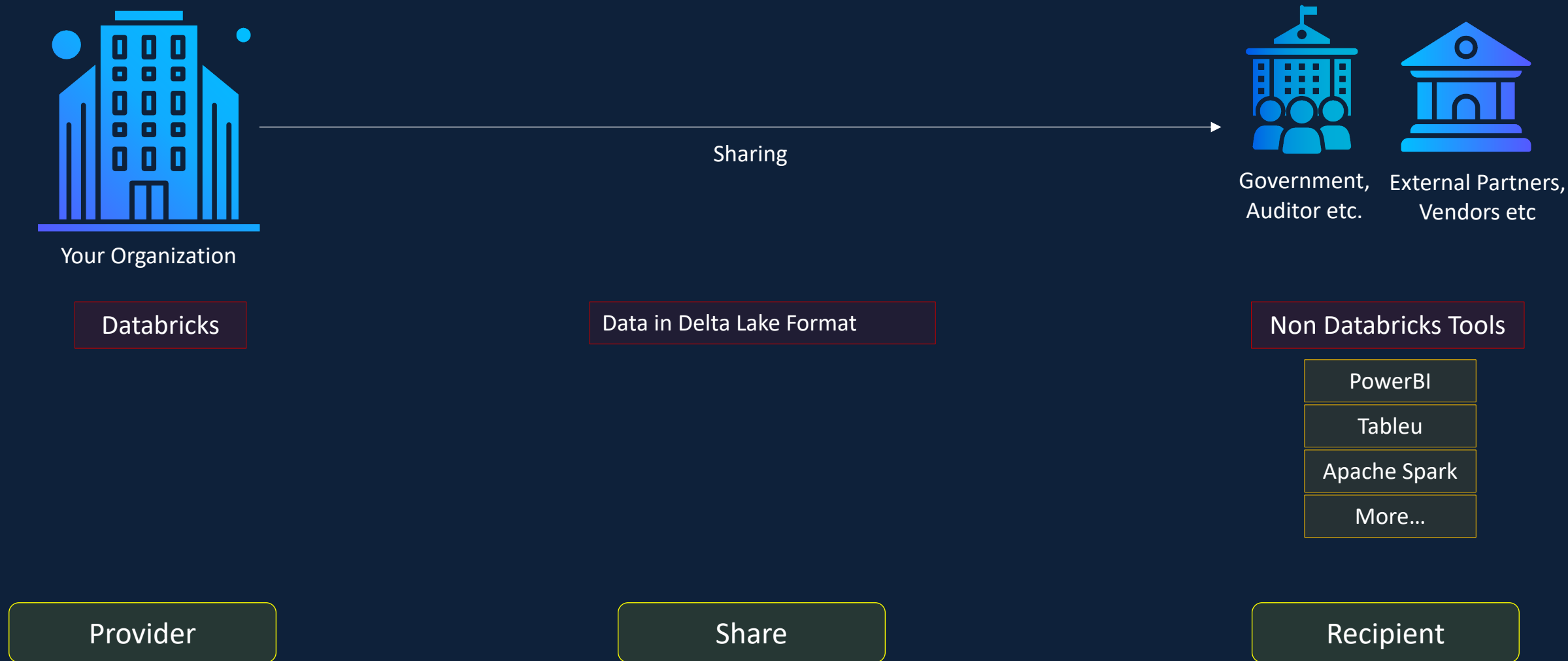


Databricks to Databricks Delta Sharing

1. Databricks to Databricks Sharing Protocol



Databricks Open Sharing Protocol

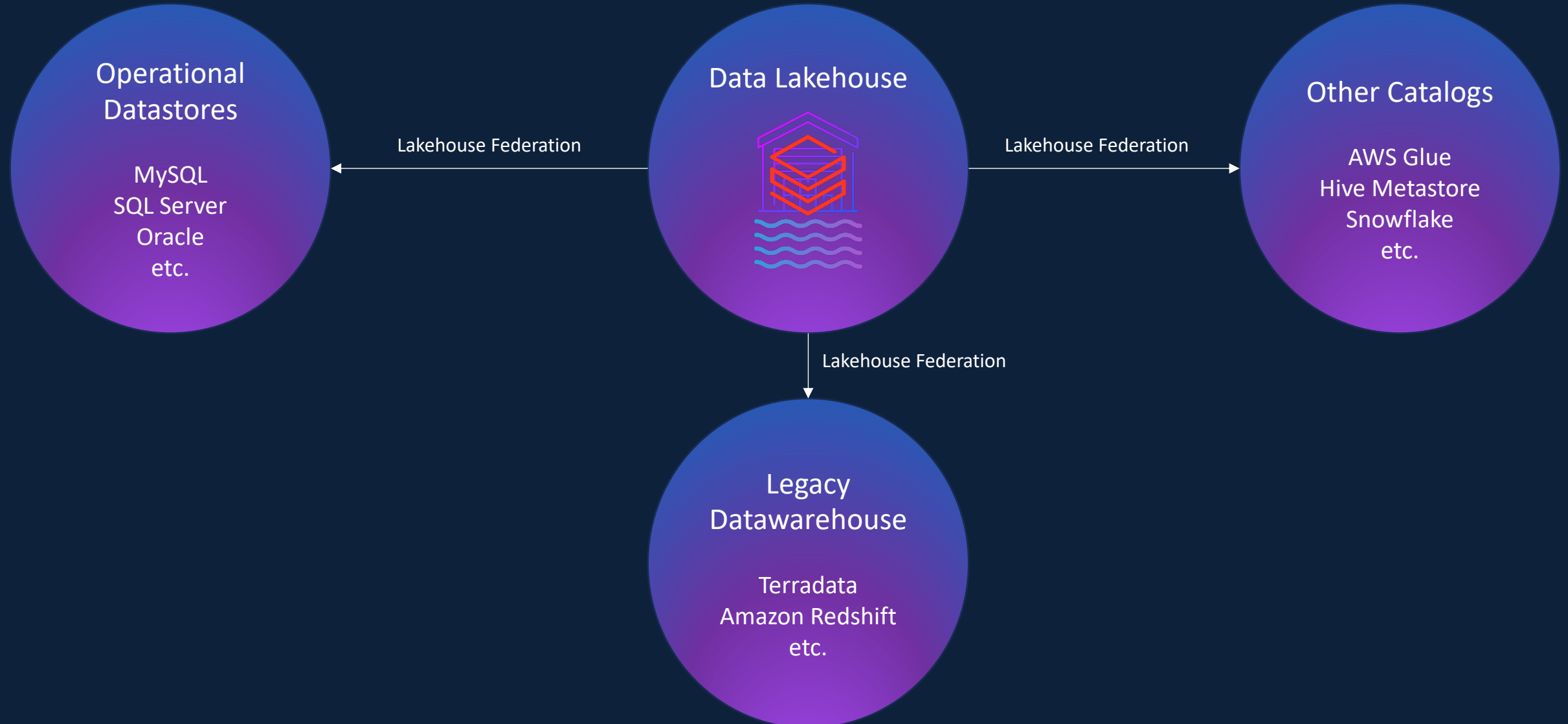


Lakehouse Federation

Lakehouse Federation

Lakehouse Federation is a capability in Databricks that allows you to access and govern data stored outside of the Lakehouse without physically moving or copying it into the Lakehouse.

Lakehouse Federation



Lakehouse Federation

Query Federation

Databases & Data Warehouses

MySQL
PostgreSQL
Teradata
Oracle
Amazon Redshift
Salesforce Data Cloud
Snowflake
Microsoft SQL Server
Azure Synapse
Google BigQuery

Data Lakehouse



Lakehouse Federation

Lakehouse Federation

Catalog Federation

Other Catalogs

Legacy Databricks Hive metastore
External Hive metastore
AWS Glue metastore
Salesforce Data Cloud
Snowflake

Lakehouse Federation Types

Query Federation

Direct JDBC connection to external databases (eg. MySQL, SQL Server, Redshift)

Query executed on the external database

Performance/cost depend on source system

Can join with Lakehouse data

Best for ad-hoc or PoC

Catalog Federation

Connects to external catalogs (AWS Glue, Hive Metastore, Snowflake)

Query executed on Databricks compute

More cost-effective and performance-optimized

Can join with Lakehouse data

Best for migration or hybrid models

Lakehouse Federation Implementation

Query Federation

Unity Catalog enabled

Create connection (JDBC + credentials)

Create foreign catalog

Grant privileges

Run queries → pushed down to external DB

Catalog Federation

Unity Catalog enabled

Create connection to external catalog

Create storage credential + external location

Create foreign catalog

Grant privileges

Run queries → executed on object storage

Lakehouse Federation Implementation

Query Federation

Unity Catalog enabled

Create connection (JDBC + credentials)

Create foreign catalog

Grant privileges

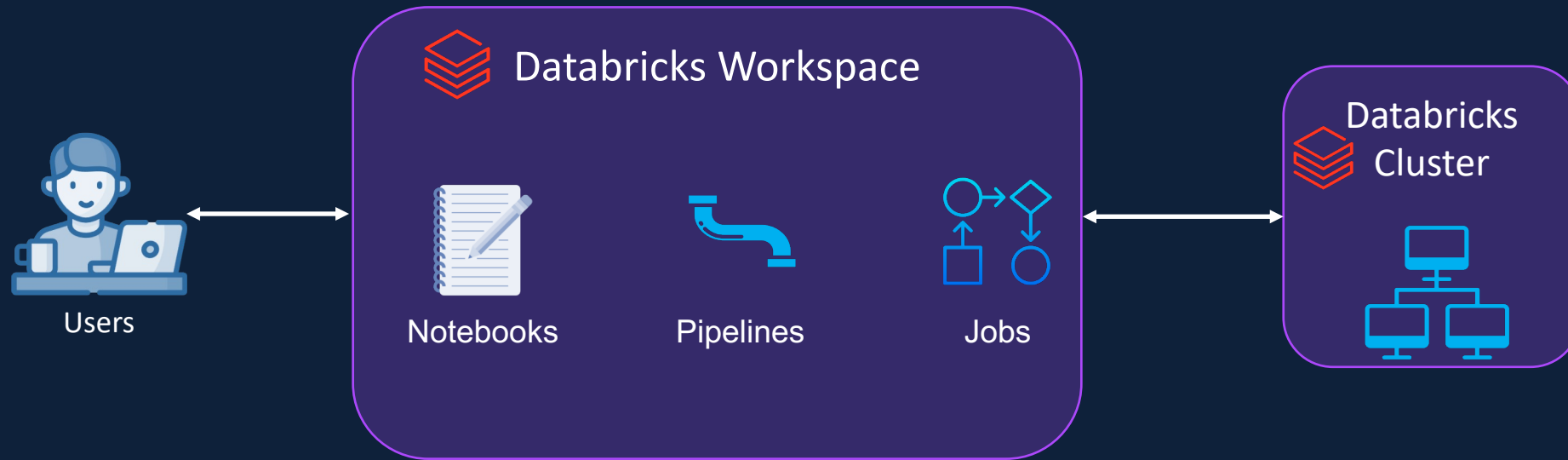
Run queries → pushed down to external DB

Catalog Explorer UI

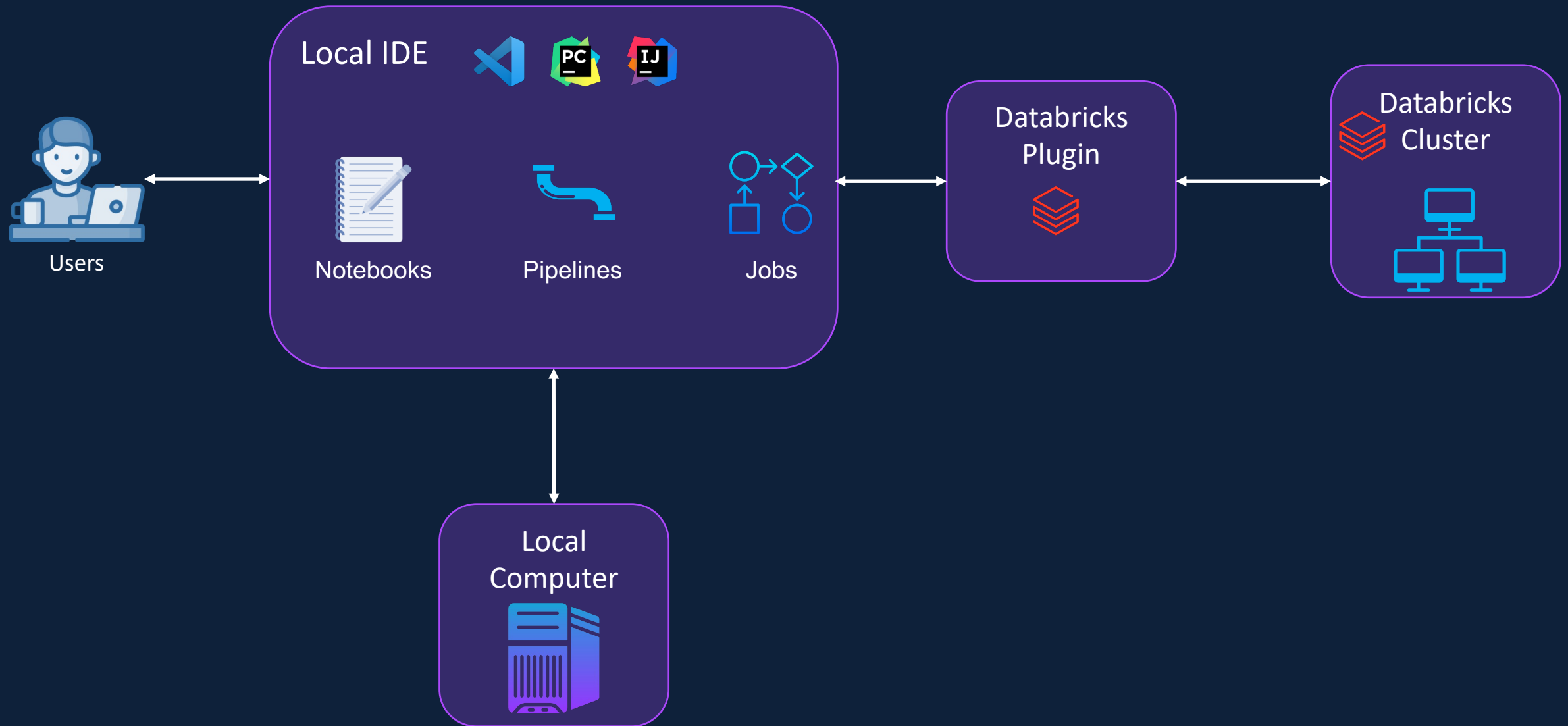
SQL Statements

Databricks Connect

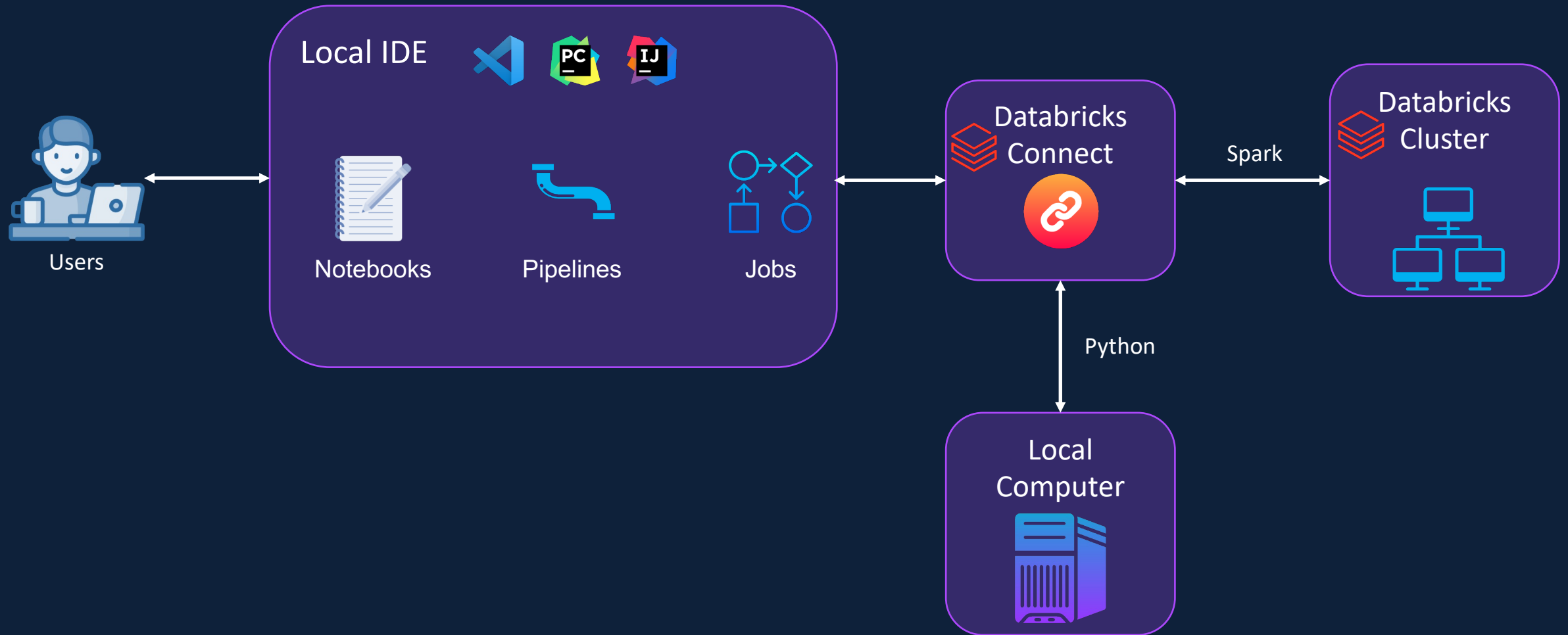
Local Development



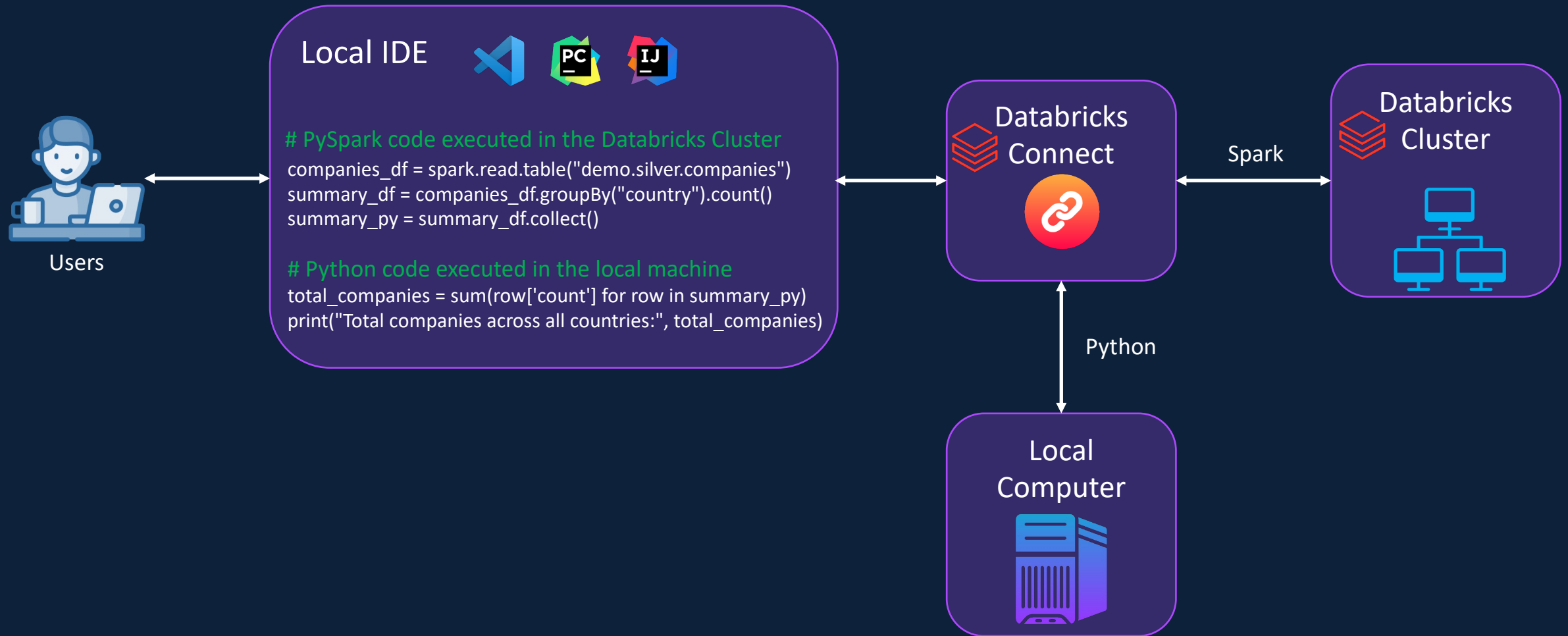
Local Development



Local Development



Local Development



Local Development Environment Set-up

Install Python

Install Visual Studio Code

Install Databricks Plugin

Create Databricks Project

Configure Access to Databricks

Run Code on Databricks Cluster

Databricks Connect Set-up

Create Python Virtual Environment

Configure Databricks Connect

Debug/ Run Code via Databricks Connect

Databricks Asset Bundles (DABs)

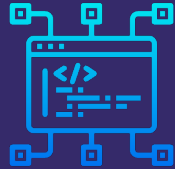
Databricks Asset Bundles (DABs)

Databricks Asset Bundles are a tool to facilitate the adoption of software engineering best practices, including source control, code review, testing, and continuous integration and delivery (CI/CD), for your data and AI projects.

- **Databricks**

Databricks Project

Environments



Development



Testing

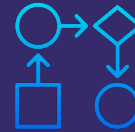


Production

Modern Deployment Approach



Pipelines



Jobs

Legacy Deployment Approach

Databricks REST APIs/ CLI

Terraform Provider

Databricks Labs - DBX

Databricks Asset Bundles



Notebooks



Scripts



Tests

Clusters



Databricks Asset Bundles

Collection of:

Code (notebooks, Python, SQL)

Configurations for Databricks resources (jobs, pipelines, clusters, MLflow, etc.)

Environment settings (dev, test, prod)

Represented in **YAML**

Deployed through **CI/CD tools** like GitHub Actions, Azure DevOps

Provides a standard, repeatable way to deliver Databricks projects

Structure of a Bundle

Structure of a Bundle

Defined in databricks.yml

Entry point of a Databricks Asset Bundle (DAB) Project

Uses YAML (Declarative Format, Human Readable)

Structure of a Bundle

bundle → project name & metadata

resources → jobs, pipelines, clusters, MLflow, etc.

targets → environment specific settings (dev, test, prod)

Structure of a Bundle

```
# Databricks Asset Bundle for dab_demo_project
bundle:
  name: dab_demo_project
resources:
  jobs:
    dab_demo_project_job:
      name: dab_demo_project_job
      tasks:
        - task_key: dab_demo_notebook
          notebook_task:
            notebook_path: src/dab_demo_notebook.ipynb
            job_cluster_key: job_cluster
      job_clusters:
        - job_cluster_key: job_cluster
          new_cluster:
            spark_version: 15.4.x-scala2.12
            node_type_id: Standard_D3_v2
            data_security_mode: SINGLE_USER
            num_workers: 1
  targets:
    dev:
      default: true
      mode: development
      workspace:
        host: https://adb-12345.14.azuredatabricks.net
    prod:
      workspace:
        host: https://adb-67890.14.azuredatabricks.net
```

Structure of a Bundle

variables → define reusable values (like parameters)

include → pull in configs from other files (for modular bundles)

run_as → specify the identity (user or service principal) to run jobs

artifacts → reference libraries, wheels, or other external assets

sync → control what gets synced between local and workspace



Exam Overview

Databricks Certified Data Engineer Associate

Exam Overview

Registration



Register via: <https://webassessor.com/databricks>

Registration Fee: \$200

Understand Computer/ Room Requirements

Provide one form of Identity document (e.g. Passport)

Delivery Method: Online Proctored

Test Aides Allowed: None

Exam Overview

Exam Format



Total Number of Questions: 45 [+5 to 10 Experimental]

Pass Percentage: 70% to 80% [Not Official]

Question Type: Multiple Choice

Time Limit: 90 minutes

Validity Period: 2 Years

Exam Overview

Exam Topics



Databricks Intelligence Platform

Data Ingestion and Loading

Data Transformation and Modelling

Working with Lakeflow Jobs

Implementing CI/CD

Troubleshooting, Monitoring, and Optimization

Governance and Security



Official Exam Guide - <https://www.databricks.com/learn/certification/data-engineer-associate>

Course Disclaimer

This course is **not affiliated with or endorsed by Databricks, Inc.**

Databricks® is a registered trademark of Databricks, Inc.

This course is independently created to help you prepare for the **Databricks Certified Data Engineer Associate** exam

It's based on publicly available resources and my own experience with the platform.

Practice Exam Disclaimer

All the questions in this course are **original**.

They are designed to mirror the format and style of the actual certification exam.

We do **not** use or share live exam questions.

It's built based on publicly available resources.

Use of Practice Exam

Get used to the structure of the exam

Test your understanding of the topics

Build Confidence before taking the actual exam

Not to replace the official Databricks Exam

Congratulations!
&
Thank you





Good Luck!



Ratings & Review



Thank you